Ingineria Programării

Curs 1 – 20 Februarie 2017

Adrian Iftene
adiftene@info.uaic.ro

Cuprins

- Context
- Motivație
- Erori celebre
- Statistici
- Definiții
- Etapele dezvoltării programelor
- Modele de dezvoltare
  - Cascadă
  - Spirală

Context

- **Aplicații de dimensiuni mari** (milioane de linii de cod) scrise pe durata a câteva luni, ani,…
- **Echipe de lucru**: Project manageri, Analiști, Arhitecți, Programatori, Testeri, Ingineri de suport (de ordinul 10, 100, 1.000 de persoane,…)
- Soluția acestor probleme este o soluție scrisă într-un limbaj de **programare orientat-obiect**

Motivație 1

- Din ce în ce mai multe sisteme sunt controlate de software: controlul traficului (aerian, feroviar, auto, naval, etc.), băncile, telefonia mobilă, Internet, supermarket, case inteligente
- Economiile tuturor statelor depind de software
- Ingineria programării propune teorii, metodologii și instrumente pentru dezvoltarea de software profesional

Motivație 2

- **1946** Goldstine și von Neumann: "**1.000 de instrucțiuni** reprezintă o limită superioară rezonabilă pentru complexitatea problemelor ce pot fi concepute ca rezolvabile cu ajutorul calculatorului"

- Sistemul de rezervare a biletelor pentru compania aerian a KLM conținea, în anul **1992**, **două milioane de linii** de cod în limbaj de asamblare

Motivație 3

- Sistemul de operare System V versiunea 4.0 (UNIX) a fost obținut prin compilarea a **3.700.000 linii de cod**
- Programele scrise pentru naveta spațială NASA au circa **40 de milioane de linii de cod** obiect
- Pentru realizarea sistemului de operare IBM OS360 au fost necesari **5.000 de ani-om**

Motivație – Erori celebre 1

- Facturi imense la energia electrică pentru pensionari, recalcularea pensiilor

- IBM OS360 conținea la fiecare relansare 1.000 de greșeli. *Resemnare...*

Motivație – Erori celebre 2

- *Programatorul ghinionist de la o bancă*: Banca dorea să trimită la clienți prin poștă o scrisoare care să le semnaleze diverse servicii
- Programatorul a scris un program ce selecta 2.000 de clienți și le scria o scrisoare personalizată
- În procesul de testare acesta a folosit un nume fictiv de client **Rich Bastard**
- Din păcate 2.000 de clienți au primit o scrisoare care începea "Dear **Rich Bastard**, ..."

Motivație – Erori celebre 3

- Sacramento: un dentist primește în căsuța poștală într-o săptămână 16.000 de formulare pentru plata taxelor – "*It was a computer problem*" a declarat un oficial

- "*Failure to convert English measures to metric values was the root cause of the loss of the Mars Climate Orbite…*"

Motivație – Erori celebre 4

- Pierdere **vehicul** explorare Venus. *Ah, era de fapt .,. in FOR!...*
- Sistem de **avertizare anti-rachetă activat.** *Atacăm sau nu?*
- Ariane 5 explodeaza. *Cost artificii: **500.000.000$***

Motivație – Statistici 1

- Acestea nu sunt doar “erori amuzante”

- Proiectele mari software pot fi cele mai complicate produse realizate de cineva
- Să ne uităm pe **statistici**
  - Vom considera că un proiect software are **succes**, dacă este realizat într-un timp rezonabil și cu un buget rezonabil
  - Un **eșec** al unui produs software are loc atunci când produsul nu este realizat sau când nu poate fi folosit

Motivație – Statistici 2

- **Studii de succes: USA'82 – Gibson & Singer**
- 18 proiecte
  - Succes: 17%
  - Parțial în folosință: 28%
  - Satisfacătoare 11%
  - Eșec: 22%
  - Neevaluate: 11%
- Motivele eșecurilor
  - Probleme de organizare
  - Noile metode de lucru și politicile salariale
  - Modificările neprevăzute în afacere

Motivație – Statistici 3

- **Studii de succes – ONNI'88 (Finland)**
- Din peste 100 proiecte
  - Succes: 33%
  - Cu probleme: 42%
  - Eșec: 25%
- Motivele eșecurilor
  - Slaba pregătire a inginerilor software
  - Resurse insuficiente
  - Probleme de management

Motivație – Statistici 4

- **The Robbins-Gioia Survey (2001),** Alexandria – Virginia, made a study over the perception by enterprises of their implementation of an E.R.P. (Enterprise Resource Planning) package.
- 232 survey respondents (36 % had, or were in the process of, implementing an ERP system)
- **51 % viewed their ERP implementation as unsuccessful**,
- **46 % did not understood how to use the system**
- 56 % of survey respondents noted their organization has a program management office (PMO) in place, and of these respondents, only 36 % felt their ERP implementation was unsuccessful

Motivație – Statistici 5

- **The Conference Board Survey (2001)**
- At 117 companies that attempted ERP implementations
- **34 % were very "satisfied"**
- **58 % were "somewhat satisfied"**
- **8 % were unhappy** with what they got.
- **40 % of the projects failed** to achieve their business case within one year of going live

- The companies that did achieve benefits said that achievement took **six months longer** than expected.
- Implementation costs were found to average 25 % over budget

IP – Definiție 1

- Prima definiție a ingineriei programării (NATO,1968): *Ingineria programării este stabilirea și utilizarea de principii inginerești solide pentru a obține în mod economic programe care sunt sigure și funcționează eficient pe mașini de calcul concrete*
- O definiție mai recentă (*IEEE Standard Glossary of Software Engineering Tehnology, 1983): Ingineria programării reprezintă abordarea sistematică a dezvoltării, funcționării, întreținerii, și retragerii din funcțiune a programelor*

IP – Definiție 2

- Se referă la două lucruri:
  - "software engineer" care a înlocuit termenul de "programmer”
  - "Software Engineering" care este folosit pentru a descrie "building of software systems which are so large or so complex that they are built by a team or teams of engineers“ – *Fundamentals of Software Engineering* (Ghezzi, Jazayeri, and Mandrioli)

IP – Definiții 3

- **FreeDictionary**: "The process of **manufacturing** software systems. A software system consists of executable computer code and the supporting documents needed to manufacture, use, and **maintain** the code."
- **Webopedia.com**: "The computer science discipline concerned with **developing large applications**. Software engineering covers not only the technical aspects of building software systems, but also **management issues**, such as **directing programming teams**, **scheduling**, and **budgeting**. "

IP – Definiții 3

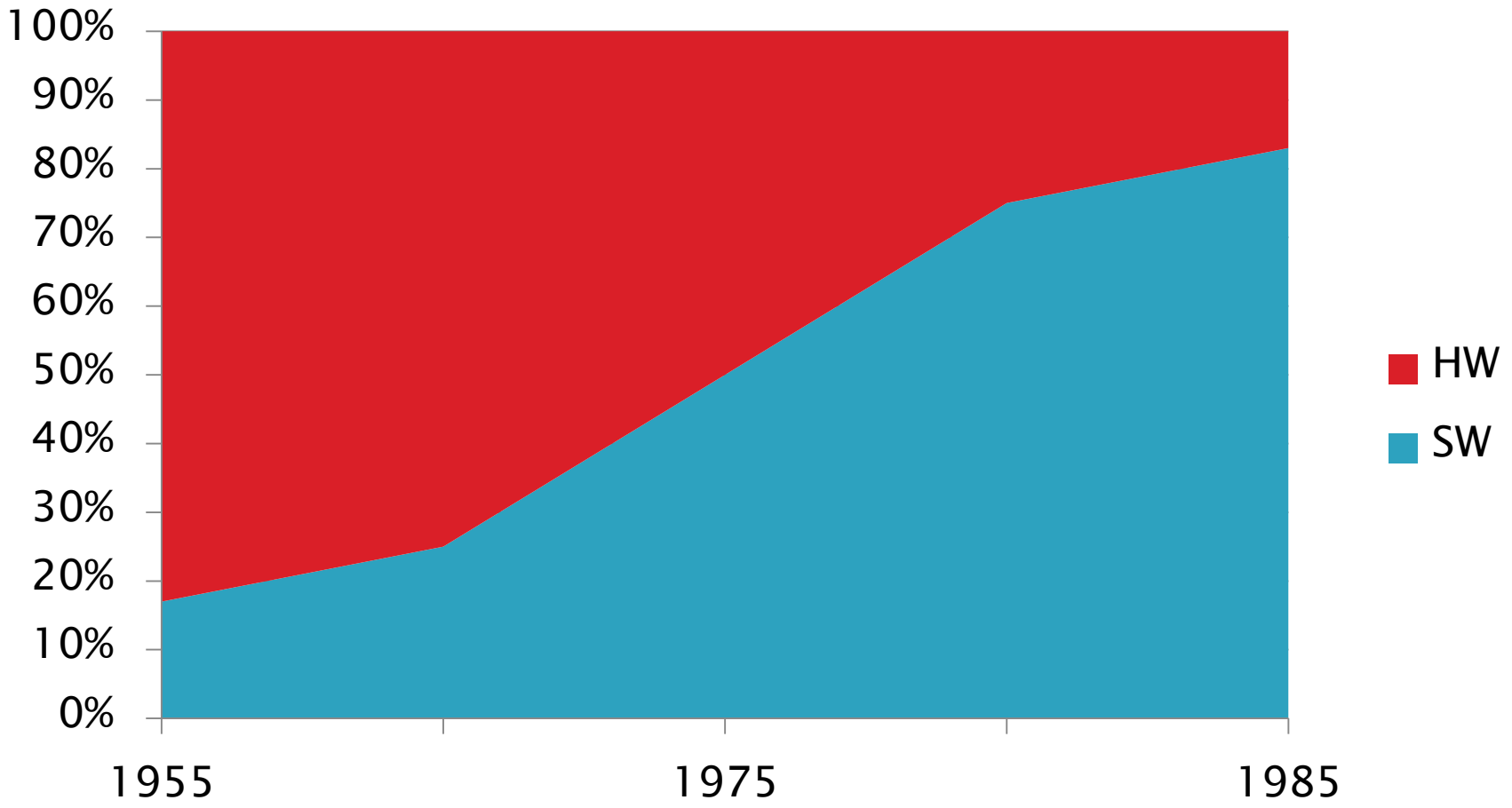
- **Wikipedia**: "**Software engineering** is the application of a systematic, disciplined, quantifiable approach to the **development**, operation, and **maintenance** of software, and the study of these approaches"

- **Answers.com**: "The **systematic** application of scientific and technological knowledge, through the medium of sound engineering principles, to the production of computer programs, and to the **requirements definition**, **functional specification**, **design description**, **program implementation**, and **test methods** that lead up to this code"

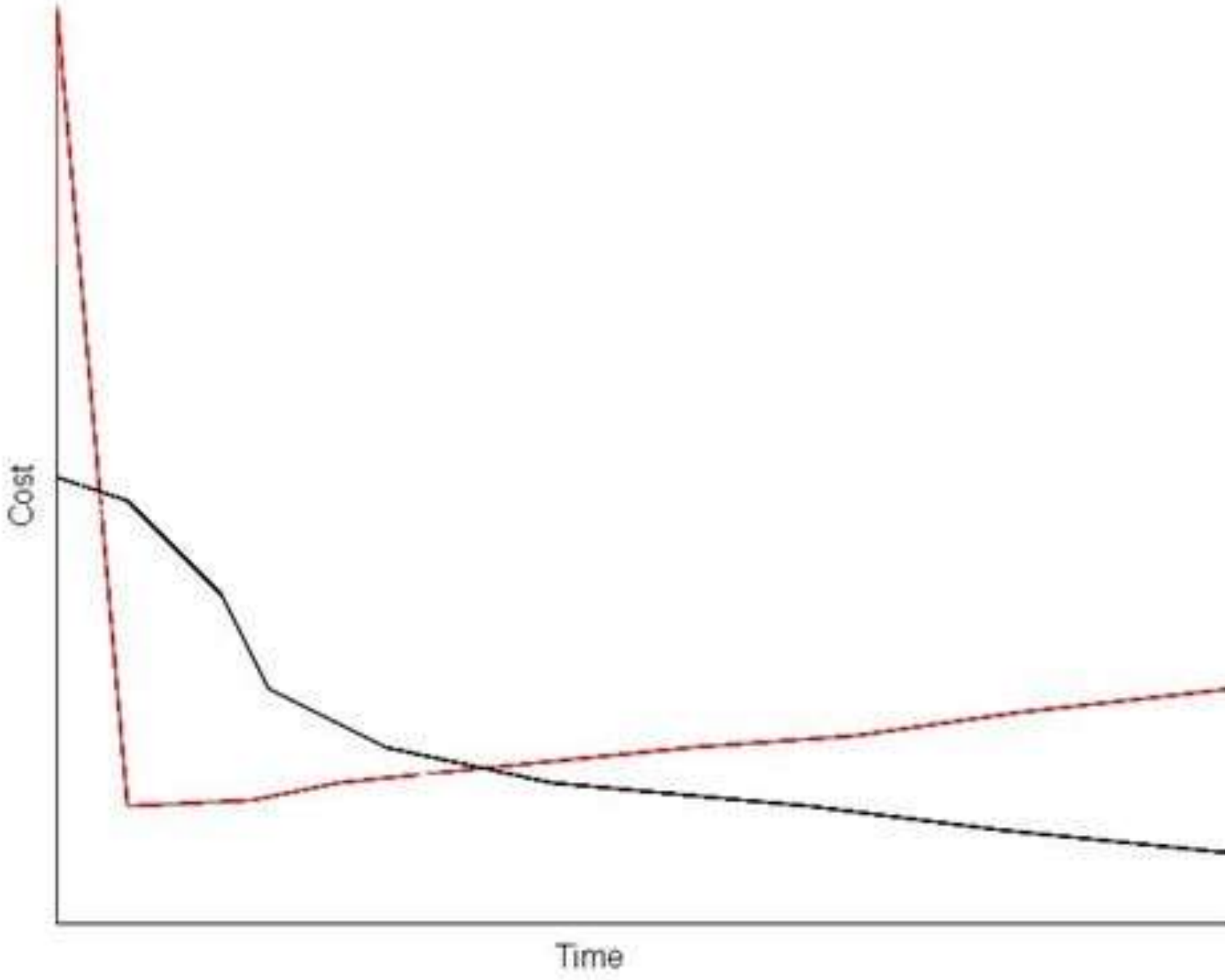
IP – Pe scurt

- Este o disciplină inginerească care se ocupă de toate aspectele dezvoltării unui program de dimensiuni mari, de către o echipă de dezvoltatori
- Propune adoptarea unei abordări sistematice și organizate a procesului de dezvoltare software
- Propune folosirea tehnicilor și instrumentelor adecvate având în vedere
  - problema care trebuie rezolvată
  - restricțiile impuse
  - resursele disponibile

Costurile produselor software



Software build (black) vs. buy (red)



Costurile produselor software

- În prezent costurile acestora sunt mai mari decât costurile rezervate componentelor hardware
- Costul întreținerii unui program este de regulă mai mare decât costul realizării unui program
- **Software** – reprezintă programele și documentația aferentă acestora

Atributele unui program bun

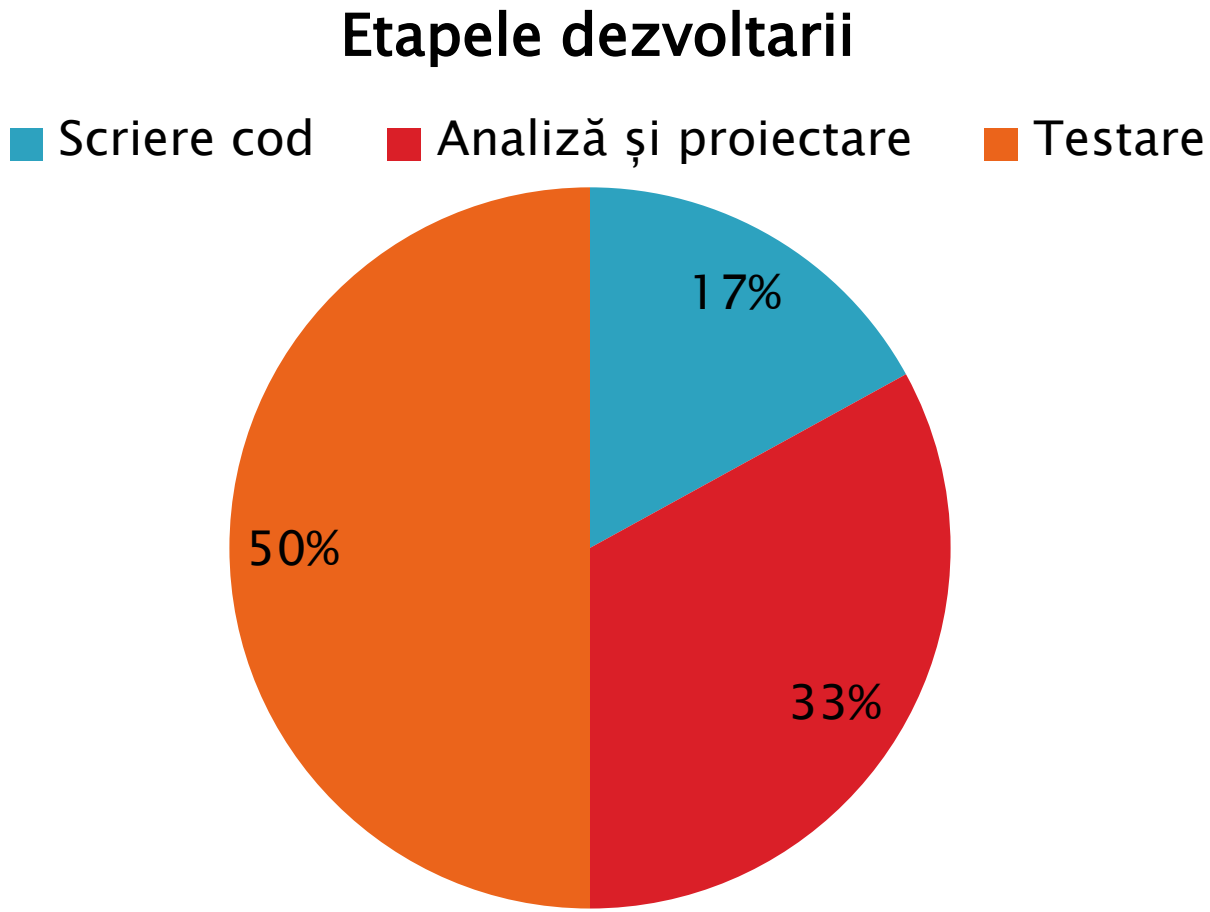
- Să ofere funcționalitățile cerute
- Să fie ușor de modificat, completat, schimbat
- Să fie sigur
- Să nu irosească resurse hardware
- Să fie ușor de folosit

Încadrarea IP

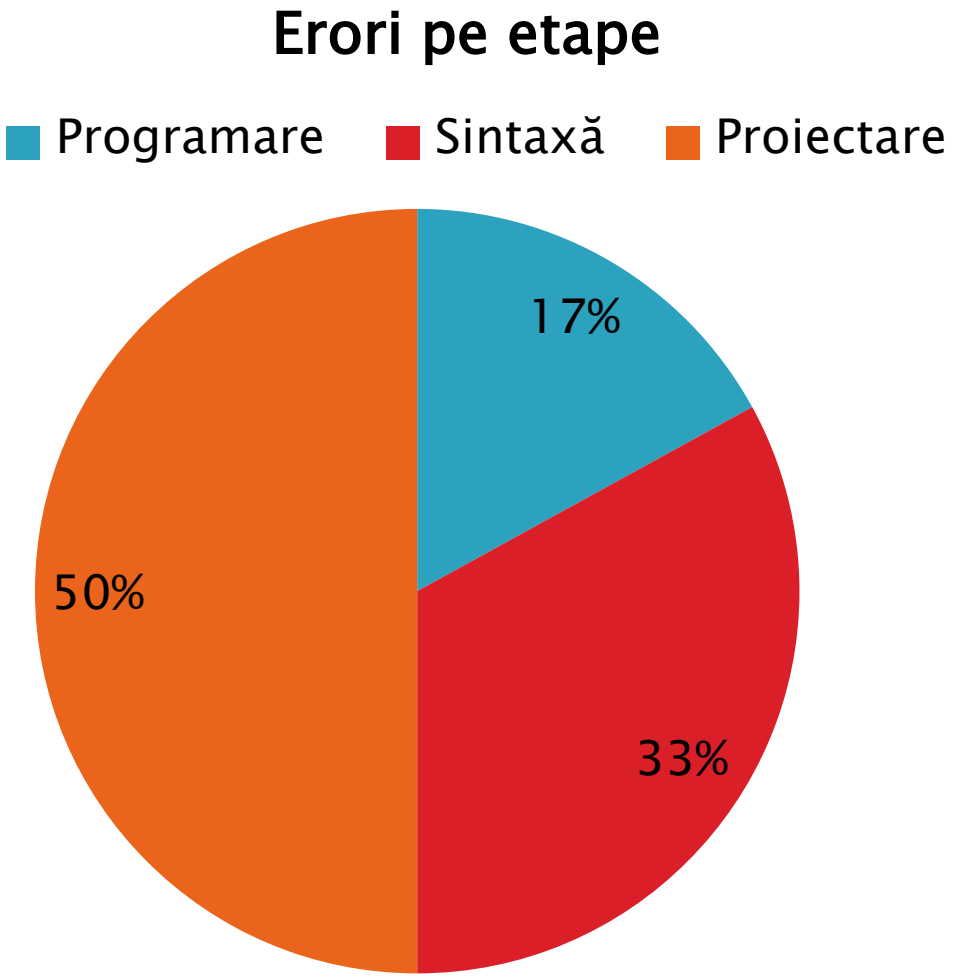
- **IP vs informatică**
  - Informatica se ocupă de **aspectele teoretice** ale dezvoltării software
  - IP se ocupa de **aspectele practice** ale dezvoltării software

- **IP vs ingineria sistemelor**
  - Ingineria sistemelor se ocupa de **toate aspectele dezvoltării** sistemelor de calcul (hardware, software, ingineria proceselor)
  - IP este o parte din ingineria sistemelor și se ocupă de:
    - Specificarea cerințelor
    - Proiectarea arhitecturală
    - Implementare
    - Testare
    - Integrare
    - Deployment
    - Întreținere

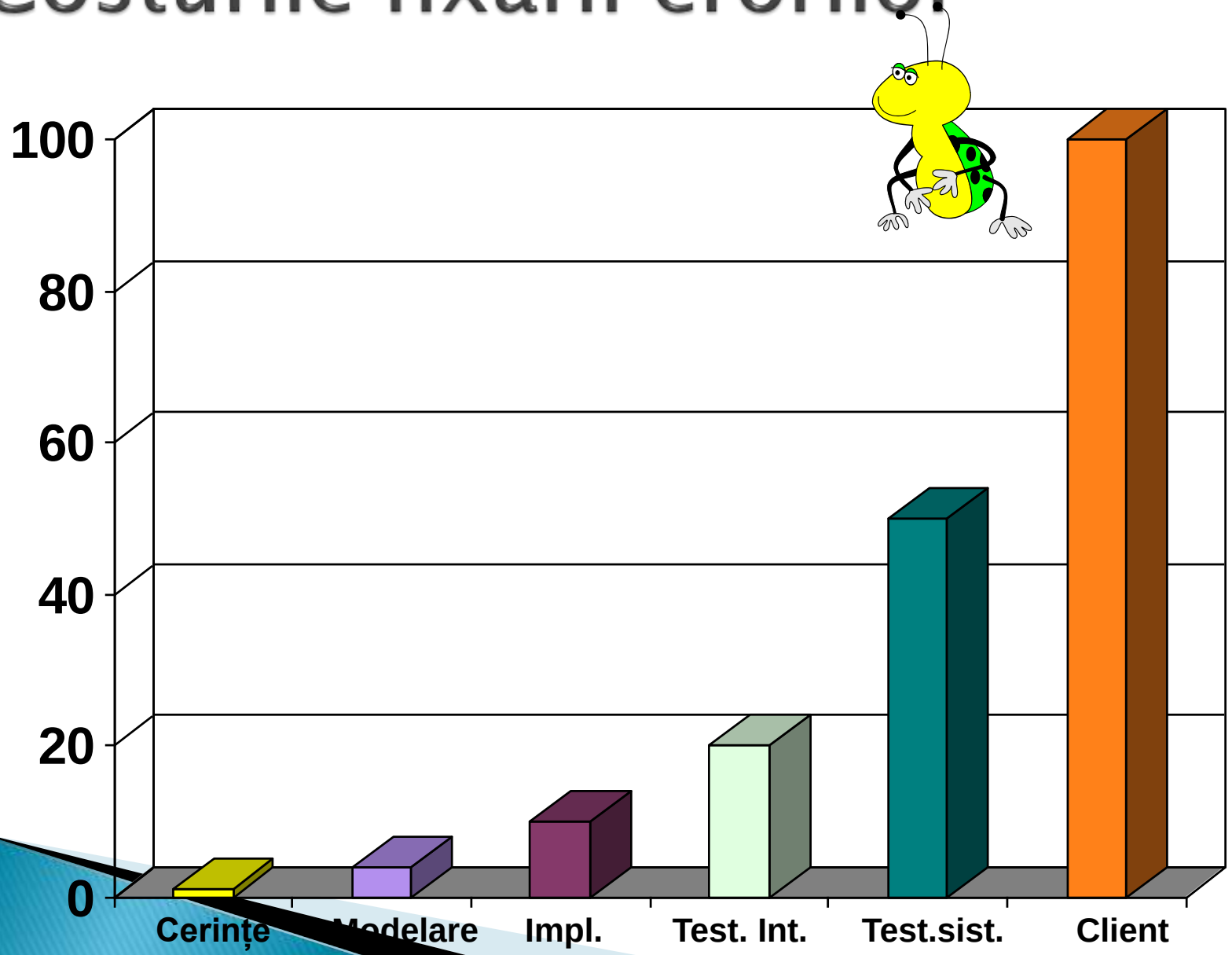
Repartizarea costurilor pe etape



Repartizarea erorilor pe etape



Costurile fixării erorilor



Dificultăți în IP

- Sistemele mai vechi care trebuie întreținute și actualizate
- Eterogenitatea sistemelor software/hardware
- Presiunea pentru a livra programul repede, cu un cost mic și cu o calitate bună

Modele de dezvoltare

- Pentru a dezvolta un program este nevoie de:
  - O înțelegere clară a ceea ce se cere
  - Un set de metode și instrumente de lucru
  - Un plan de acțiune

- Plan de acțiune = șablon = model de dezvoltare

Etapele dezvoltării programelor

- Ingineria cerințelor (Requirements engineering)
- Proiectarea arhitecturală (Arhitectural design)
- Proiectarea detaliată (Detailed design)
- Scrierea codului (Implementation)
- Integrarea componentelor (Integration)
- Validare (Validation)
- Verificare (Verification)
- Întreținere (Maintenance)

Ingineria cerințelor

- Se stabilește *ce anume vrea clientul ca programul să facă*
- Scopul este înregistrarea cerințelor într-o manieră cât mai clară și mai fidelă
- Probleme
  - Comunicare
  - Negociere
  - Sfătuirea clientului

Proiectarea

- Arhitecturală
  - Din motive de complexitate, programele mari nu pot fi concepute și implementate ca o singură bucată
  - Programul este împărțit în module sau componente mai simple, care pot fi abordate individual
- Detaliată
  - Se proiectează fiecare modul al aplicației, în cele mai mici detalii

Implementare + integrare

- Implementare
  - Proiectul detaliat este transpus într-un limbaj de programare
  - Acesta se realizează modular, pe structura rezultată la proiectarea arhitecturală
- Integrare
  - Modelul big-bang
  - Modelul incremental

Validare și verificare

- **Validare:** ne asigurăm că programul îndeplinește cerințele utilizatorului
  - *Construim produsul corect?*

- **Verificare:** ne asigurăm că programul este stabil și că funcționează corect din punctul de vedere al dezvoltatorilor.
  - *Construim corect produsul?*

Întreținere

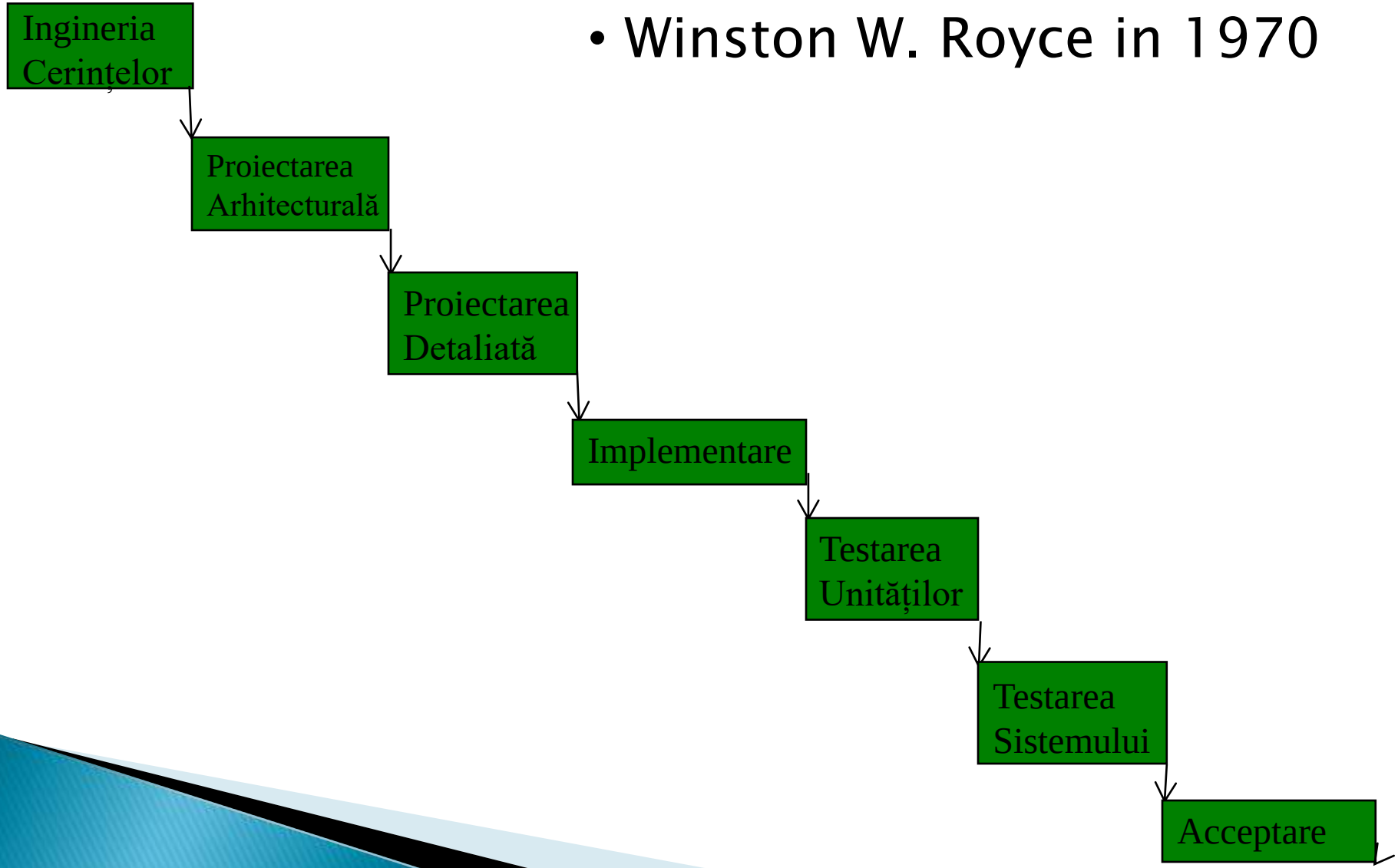
- După livrare
  - Sunt descoperite greșeli ce trebuie reparate
  - Pot apărea schimbări în specificații
  - Pot apărea noi cerințe
  - Instruirea celor ce vor folosi produsul
- Întreținere = gestionarea acestor tipuri de probleme

Modele de dezvoltare

- *Cum* efectuăm activitățile indicate de etapele dezvoltării programelor
- Exemple de modele de dezvoltare:
  - Ad-hoc: descurcă-te cum poți
  - Modelul în cascadă (cu feedback)
  - Prototipizare
  - Modelul în spirală
  - RUP (Rational Unified Process)
  - V-Model
  - XP (Extreme Programming)
  - Agile, Lean, Scrum
  - MDD, AMDD
  - TDD

Modelul în Cascadă (Waterfall)

- Winston W. Royce in 1970

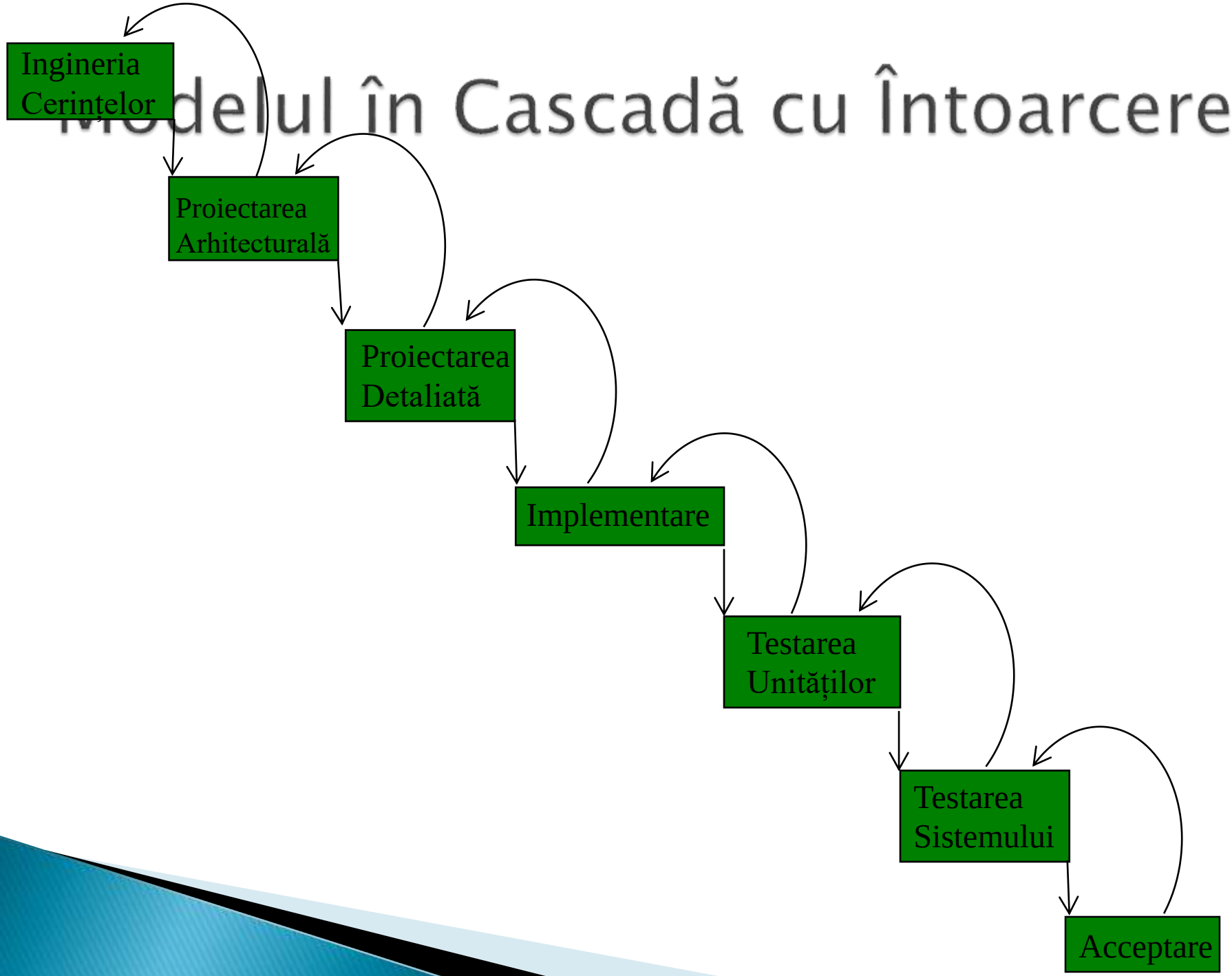


Modelul în Cascadă +-

- +: Împarte o sarcină complexă în pași mai mici
- +: Ușor de administrat și controlat
- +: Fiecare pas are ca rezultat un produs bine definit
- +: Tot timpul știm unde ce am făcut până în acel moment și știm ce mai avem de făcut

- -: Erorile se propagă între pași
- -: Nu exista mecanisme de reparare a erorilor

Modelul în Cascadă cu Întoarcere



Modelul în cascadă cu întoarcere +-

- +: Oferă cadrul pentru remedierea erorilor din pasul precedent

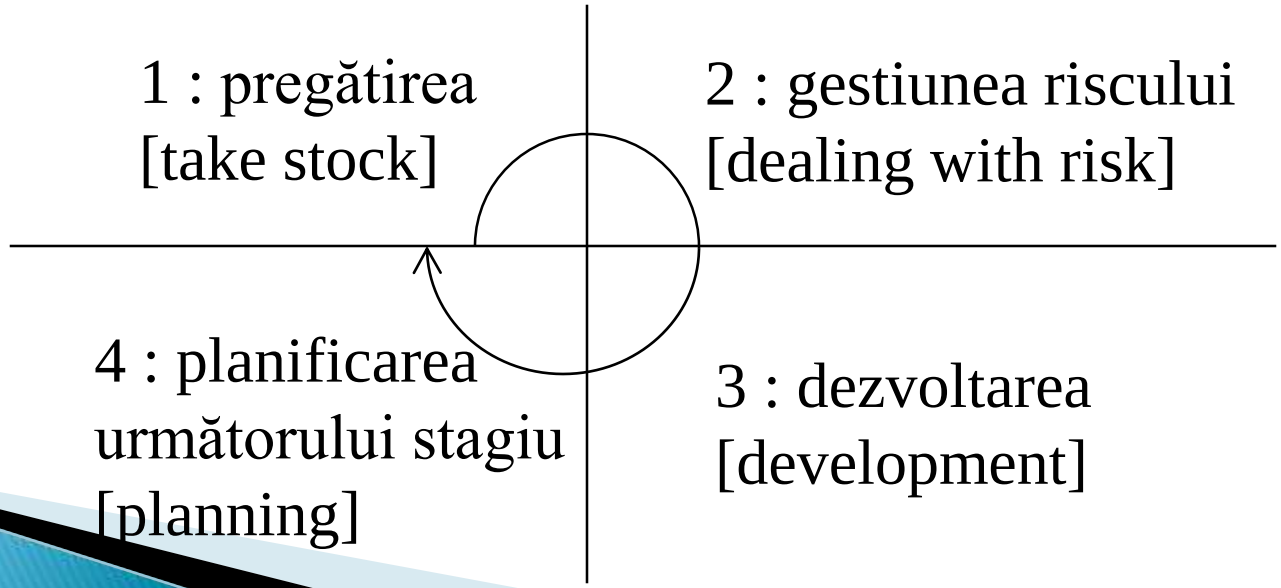
- -: Erorile la pasul i care sunt descoperite la pasul i + 2 nu sunt remediate

- -: Clientul vede produsul final abia la sfârșitul dezvoltării

Modelul în Spirală (Spiral)

- Studiul de fezabilitate
- Analiza cerințelor
- Proiectarea arhitecturii
- Implementarea

Pentru fiecare pas, se fac următoarele activități:



Modelul în Spirală +-

- +: Păstrează avantajele modelului în cascadă
- +: la în calcul noțiunea de risc

- Exemple de riscuri:
  - O firmă concurentă lansează un produs similar
  - Un arhitect părăsește echipa
  - Clientul schimbă cerințele
  - O echipă nu respectă termenele de livrare

Concluzii

- Statisticile ne demonstrează importanța folosirii unor tehnici inginerești în dezvoltarea de software
- Definițiile IP-ului folosesc cuvinte cheie precum: *metode inginerești*, *proiecte de dimensiuni mari care sunt implementate în echipă*, *programe sigure care funcționează eficient*, *dezvoltare, întreținere, planificare, buget*
- Etapele necesare dezvoltării proiectelor de dimensiuni mari
- Modele de dezvoltare: Ad-Hoc, Cascadă, Spirală

Bibliografie

- Ovidiu Gheorghieș: http://www.infoiasi.ro/~ogh/files/ip/curs-01.pdf
- Andy Kramek, New Software – Build or Buy? A Personal View: http://weblogs.foxite.com/andykramek /archive/2009/07/25/8674.aspx

Links

- Internet
- Wikipedia
- Failure rate: http://www.it-cortex.com/Stat_Failure_Rate.htm
- RUP in the dialogue with Scrum: http://www.ibm.com/developerworks/rational/library/feb05/krebs/

Ingineria Programării

Cursul 2 – 27 Februarie

adiftene@info.uaic.ro

Cuprins

- Din Cursul 1…
- Prototipizare
- RUP
- V-Model
- Extreme Programming
- Agile
- Scrum
- Lean, Kanban
- MDD, AMDD
- TDD
- Ingineria cerinţelor

Din Cursul 1 – 1

- Ingineria programării (Software engineering)
  - Se referă la metodologiile folosite în rezolvarea proiectelor mari care sunt rezolvate de echipe de oameni
  - Folosirea principiilor inginereşti în *analizarea, dezvoltarea*, *punerea în funcţiune*, *testarea*, *întreţinerea*, *retragerea* produselor software
  - Tot aici mai pot fi prinse: *gestionarea resurselor*, *coordonarea echipelor*, *planificare*, *buget*
- **Scop**: obţinerea de programe sigure şi care funcţionează eficient pe maşini de calcul concrete

Din Cursul 1 – Etapele dezvoltării programelor

- Analiza cerinţelor (Requirements analisys)
- Proiectarea architecturală (Arhitectural design)
- Proiectarea detaliata (Detailed design)
- Scrierea codului (Implementation)
- Integrarea componentelor (Integration)
- Validare (Validation)
- Verificare (Verification)
- Întreţinere (Maintenance)

Prototipizare (Prototyping)

- Tipuri de prototipuri
  - **De aruncat** (throw-away)
    - Scop: clarificarea specificațiilor
    - Se dezvoltă repede, orice altceva e secundar (quick-and-dirty)
    - Util în a rezolva “architecural/technology spikes”
    - Programul “adevărat” este scris apoi de la 0
  - **Evoluționar**
    - Scop: construire incrementală a produsului final
    - Se construiește un nucleu funcțional la care se adaugă apoi noi funcționalități

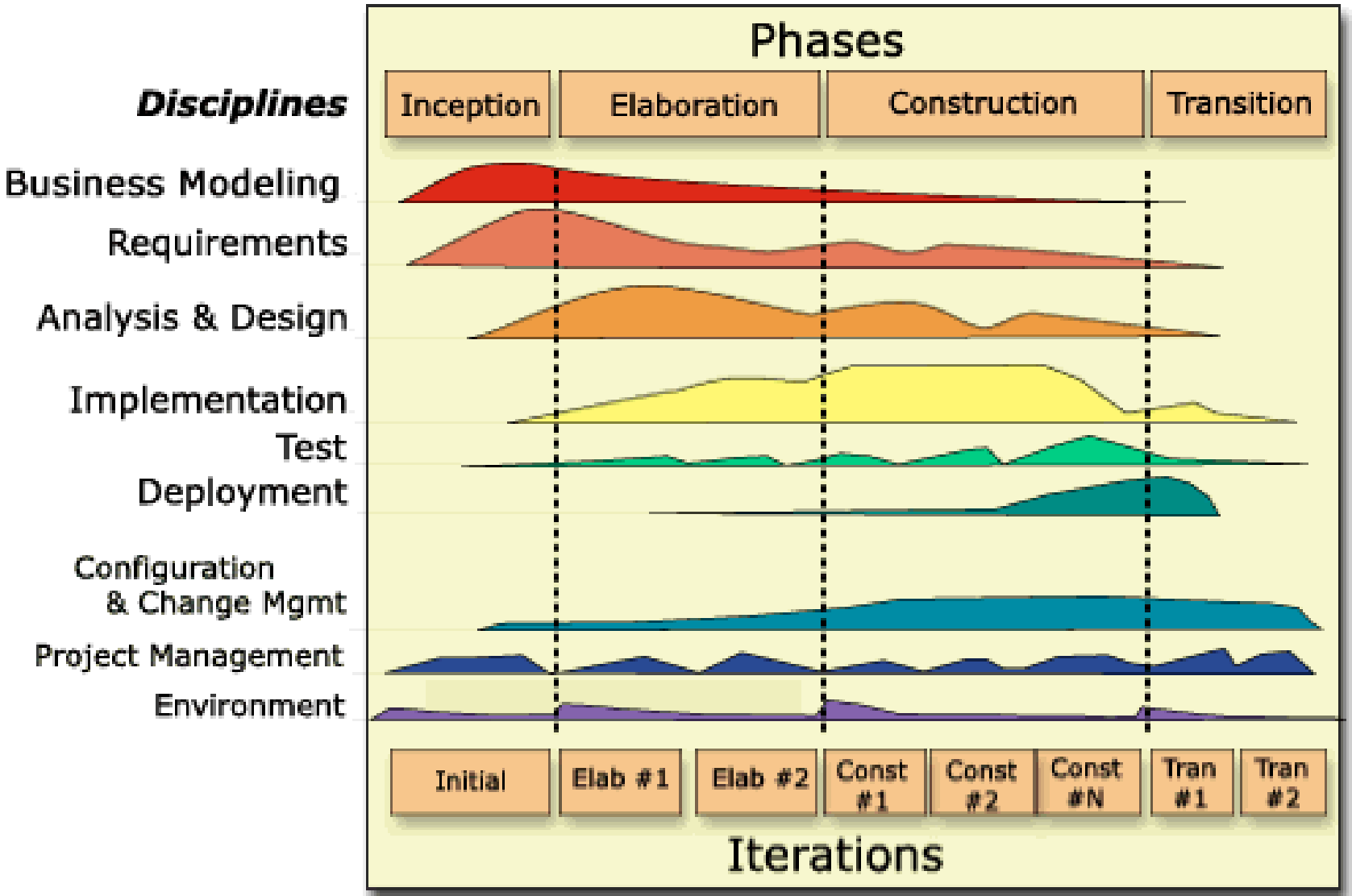
Prototipizare +-

- +: Se poate elimina lipsa de claritate a specificațiilor
- +: Clienții pot schimba cerințele (e ieftin de gestionat)
- +: Întreținere ieftină (verificare pe parcurs)
- +: Se poate facilita instruirea utilizatorilor

- -: Mediu artificial, probleme ascunse
- -: *Da' nu-i aproape gata?! De ce mai durează atât?*
- -: *Putem să schimbăm specificațiile? Pai aș vrea și...*
- -: *Adică munca mea este aruncată la gunoi?*

RUP (Rational Unified Process)

- Model iterativ folosit de IBM din 2003

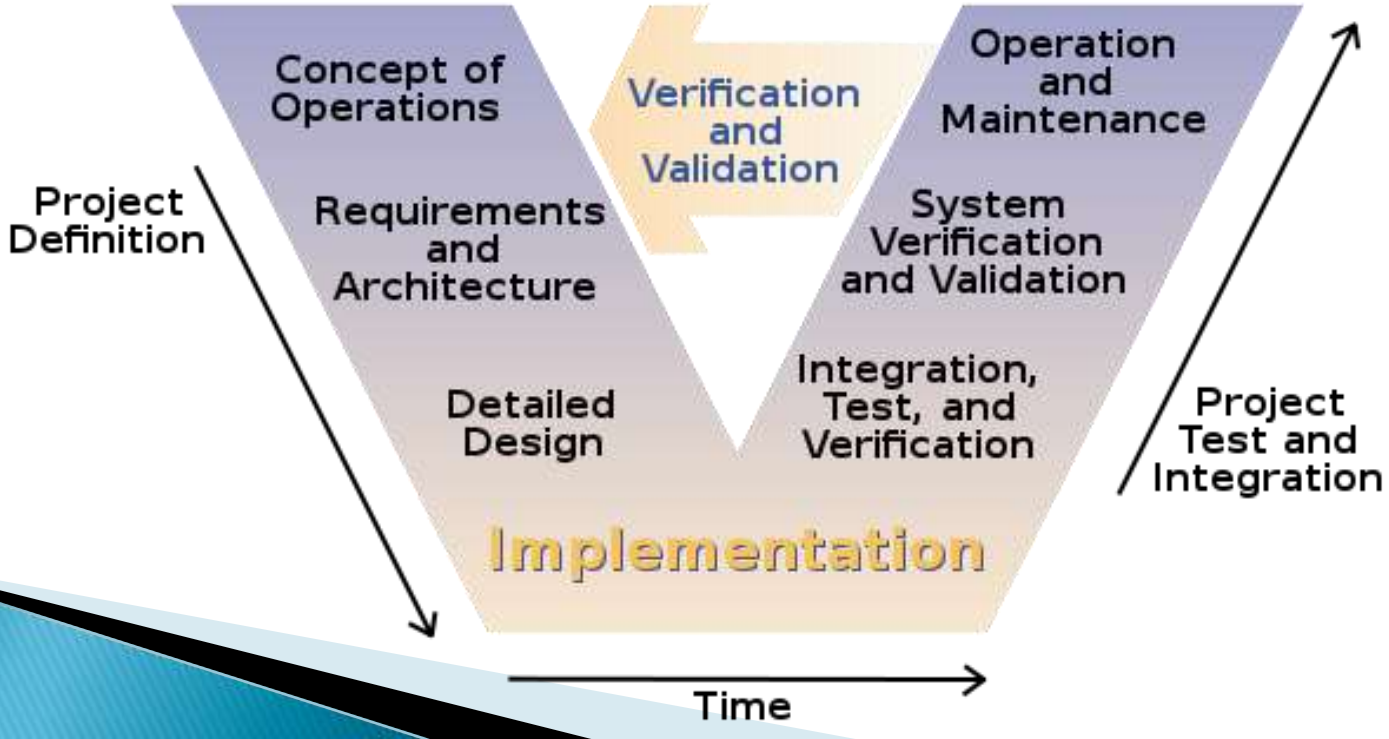


RUP

- Ingineria funcționalității. Sunt sintetizate necesitățile funcționale.
- Are la baza 4 etape:
  - **Inception**: pentru validarea costurilor și bugetului, studiu de risc, înțelegerea cerințelor
  - **Elaboration**: analiza domeniului problemei, arhitectura proiectului este stabilită
  - **Construction**: construcția sistemului, se obține prima versiune a sistemului
  - **Transition**: tranziția la sistemul din producție

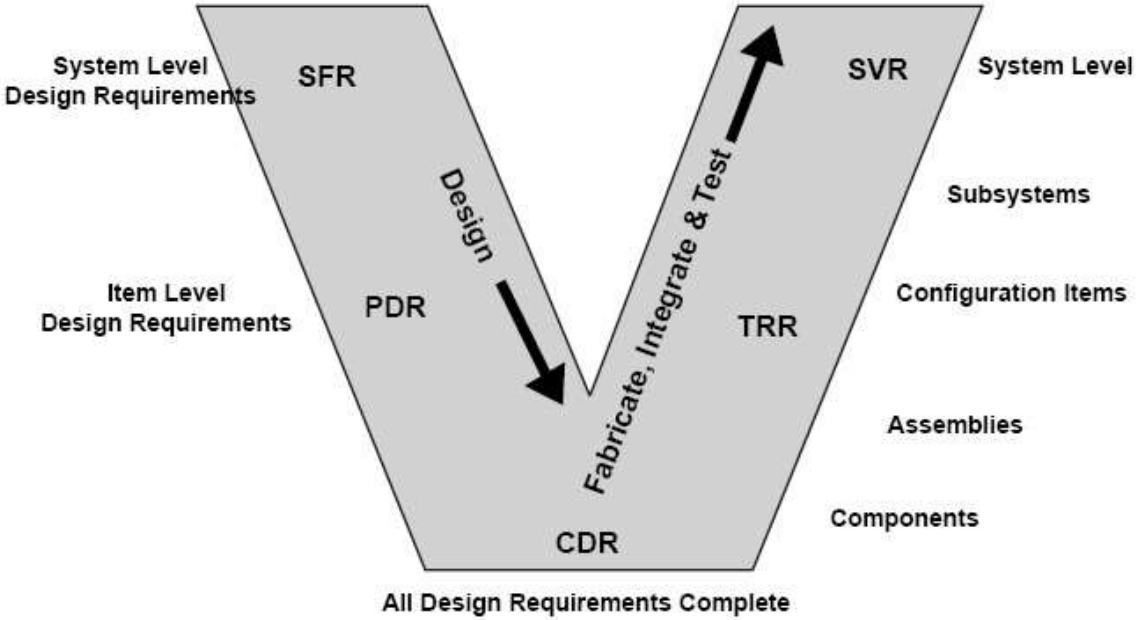
V-Model

- A fost utilizat în Germania și SUA în anii ’80
- **Partea stângă** – analiza cerințelor și crearea specificațiilor sistemului
- **Partea dreaptă** – integrarea părților și validarea lor
- **V** de la Verificare și Validare



V-Model – Obiective

- Minimizarea riscurilor
- Îmbunătățirea și garantarea calității
- Reducerea costurilor
- Îmbunătățirea comunicării



V-Model + -

- +: utilizatorii V-model participă la dezvoltare și la întreținere
- +: există un grup ce controlează modificările din specificații. Acesta se întâlnește odată pe an și hotărăște ce modificări sunt acceptate
- +: modelul asigură la fiecare etapă asistență și definește explicit ce avem de făcut

- -: nu se asigură întreținerea
- -: e folosit la proiecte de dimensiuni relativ mici

Extreme Programming

- **Why "Extreme"?**
  - "Extreme" means these practices get "turned up" to a much higher "volume" than on traditional projects.
- **What really matters?**
  - Listening, Testing, Coding, Designing

XP – Caracteristici

- XP este un model modern, uşor (lightweight), de dezvoltare, inspirat din RUP
- Dezvoltarea programelor nu înseamnă ierarhii, responsabilităţi şi termene limită, ci înseamnă colaborarea oamenilor din care este formată echipa
- **Membrii echipei sunt încurajaţi să-şi afirme personalitatea, să ofere şi să primească cunoaştere şi să devină programatori străluciţi**
- **XP consideră că dezvoltarea de programe înseamnă în primul rând scrierea de programe** (fişierele PowerPoint nu se pot compila)

Idei majore in XP

- Proiectul este în mintea tuturor programatorilor din echipa, nu în documentaţii, modele sau rapoarte
- La orice moment, un reprezentant al clientului este disponibil pentru clarificarea cerinţelor
- Codul se scrie cât mai simplu. **Se scrie cod de test întâi**
- Daca apare necesitatea re-scrierii sau aruncării de cod, aceasta se face fără milă
- Modificările aduse codului sunt integrate continuu (de câteva ori pe zi)
- **Se programează în echipă** (programare în perechi). **Echipele se schimbă la sfârşitul unei iteraţii** (1-2 săptămâni)
- **Se lucrează 40 de ore pe săptămână, fără lucru suplimentar**

XP Rules

Planning

- User stories are written.
- Release planning creates the schedule.
- Make frequent small releases.
- The Project Velocity is measured.
- The project is divided into iterations.
- Iteration planning starts each iteration.
- Move people around.
- A stand-up meeting starts each day.
- Fix XP when it breaks.

Designing

- Simplicity.
- Choose a system metaphor.
- Use CRC cards for design sessions.
- Create spike solutions to reduce risk.
- No functionality is added early.
- Refactor whenever and wherever possible.

Coding

- The customer is always available.
- Code must be written to agreed standards.
- Code the unit test first.
- All production code is pair programmed.
- Only one pair integrates code at a time.
- Integrate often.
- Use collective code ownership.
- Leave optimization till last.
- No overtime.

Testing

- All code must have unit tests.
- All code must pass all unit tests before it can be released.
- When a bug is found tests are created.
- Acceptance tests are run often and the score is published.

Agile

- Satisfacerea rapidă a clientului prin oferirea continuă de software util (săptămânal dacă e posibil)
- Progresul se măsoară în funcţie de partea funcţională a proiectului
- **Chiar şi modificările târzii în cerinţe sunt binevenite**
- **O cooperare foarte apropiată între client şi programatori**
- Discuţiile face-to-face constituie cea mai bună formă de comunicare
- **Adaptare continuă la modificările care apar**
- Dezvoltarea unui spirit de evidenţiere şi rezolvare a problemelor, nu de ascundere sau 'neobservare' a lor

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

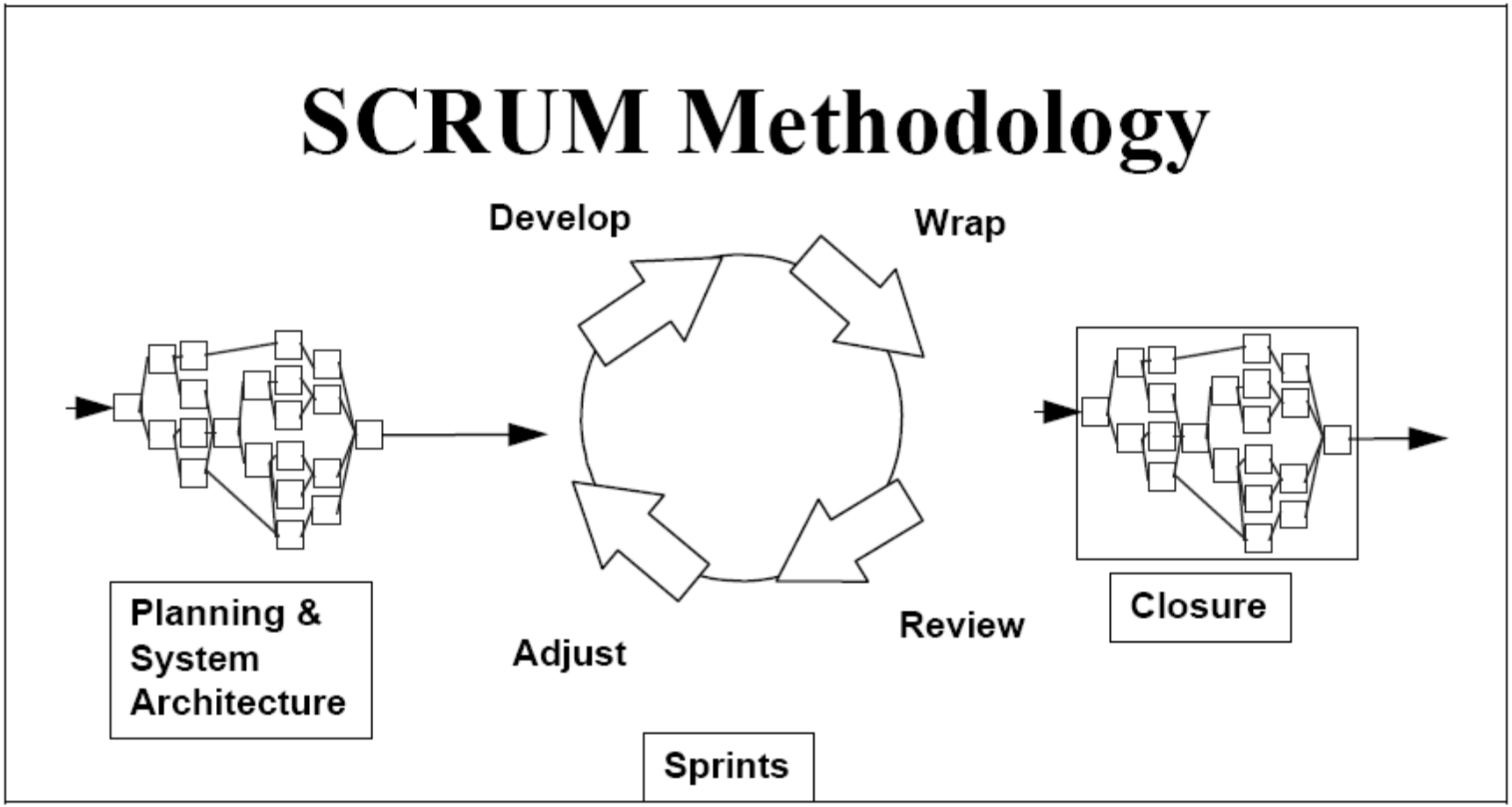
Agile +-

- -:
  - Imposibilitatea realizării documentaţiei necesare
  - Se lucrează doar cu dezvoltatori “senior-level”
  - Insuficientă structurare a modelării software
  - Poate duce la negocieri de contract dificile
- +:
  - **Companiile care au adoptat metoda de lucru Toyota și-au îmbunătățit cu 83% productivitatea, cu 93% timpul de producție, cu 91% calitatea produselor și au redus la jumătate overtime-ul** – după cum arată un studiu oficial U.S., realizat în urmă cu câțiva ani pe companii din industria auto

Scrum (“grămadă”)

- **Clientul devine parte a echipei de dezvoltare**
- Frecvente distribuiri intermediare a părţii software, cu verificări şi validări imediate
- Discuţii zilnice:
  - Ce ai făcut ieri? (realizări)
  - Ce ai de gând să faci până mâine? (de realizat)
  - Care sunt problemele care te-ar putea încurca? (probleme/riscuri)
- Transparenţă în planificare şi dezvoltare
- Întâlniri frecvente pentru a monitoriza progresul
- Nu sunt probleme ţinute sub covor
- Eficienţa muncii: “să lucrezi mai multe ore" nu înseamnă neapărat “obţinerea mai multor rezultate"

Metodologia Scrum



Lean (“sprijin”)

- Principiile Lean Software Development (LSD)

1. **Eliminarea lucrurilor nefolositoare**
2. **Amplificarea învăţării**
3. Decide cât mai târziu posibil
4. Termină cât mai curând posibil
5. Oferă **responsabilităţi** membrilor echipei
6. Construieşte un **proiect integru**
7. Construieşte văzând tot **proiectul în ansamblu**

Kanban

- A scheduling system for lean and **just-in-time** (JIT) production
- Taiichi Ohno at Toyota in 1953
- A system to control the logistical chain from a production point of view, and is an **inventory** control system
- Aligns inventory levels with actual consumption. A signal tells a supplier to produce and deliver a new shipment when material is consumed
- An approach where the "pull" comes from demand

Kanban – Toyota's Six Rules

- Later process picks up the number of items indicated by the kanban at the earlier process.
- Earlier process produces items in the quantity and sequence indicated by the kanban.
- No items are made or transported without a kanban.
- Always attach a kanban to the goods.
- Defective products are not sent on to the subsequent process. The result is 100% defect-free goods.
- Reducing the number of kanban increases the sensitivity.

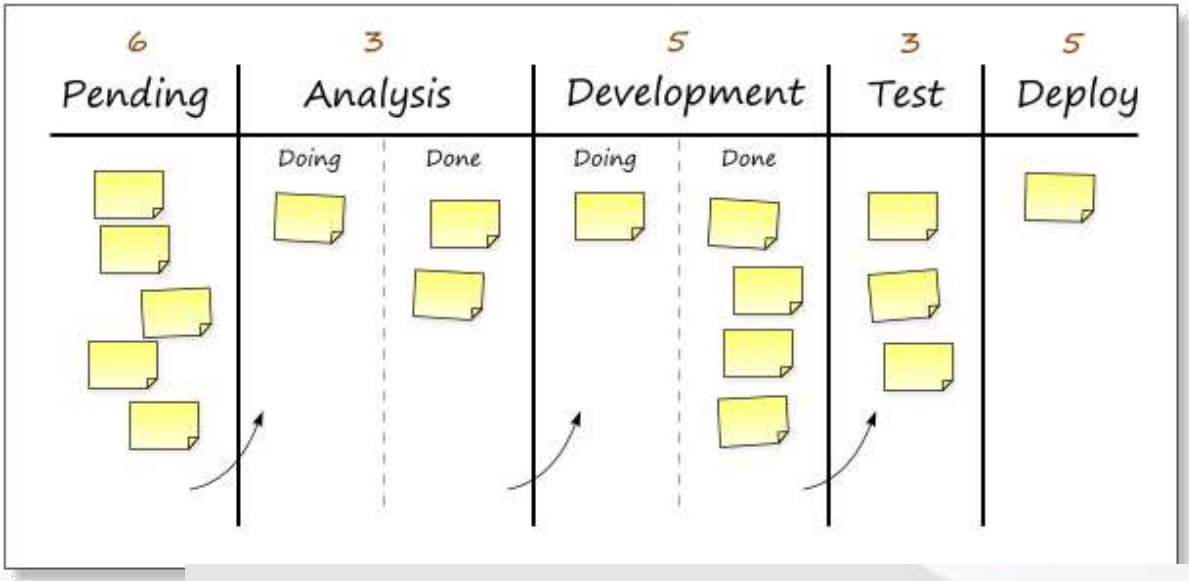
Kanban methodology

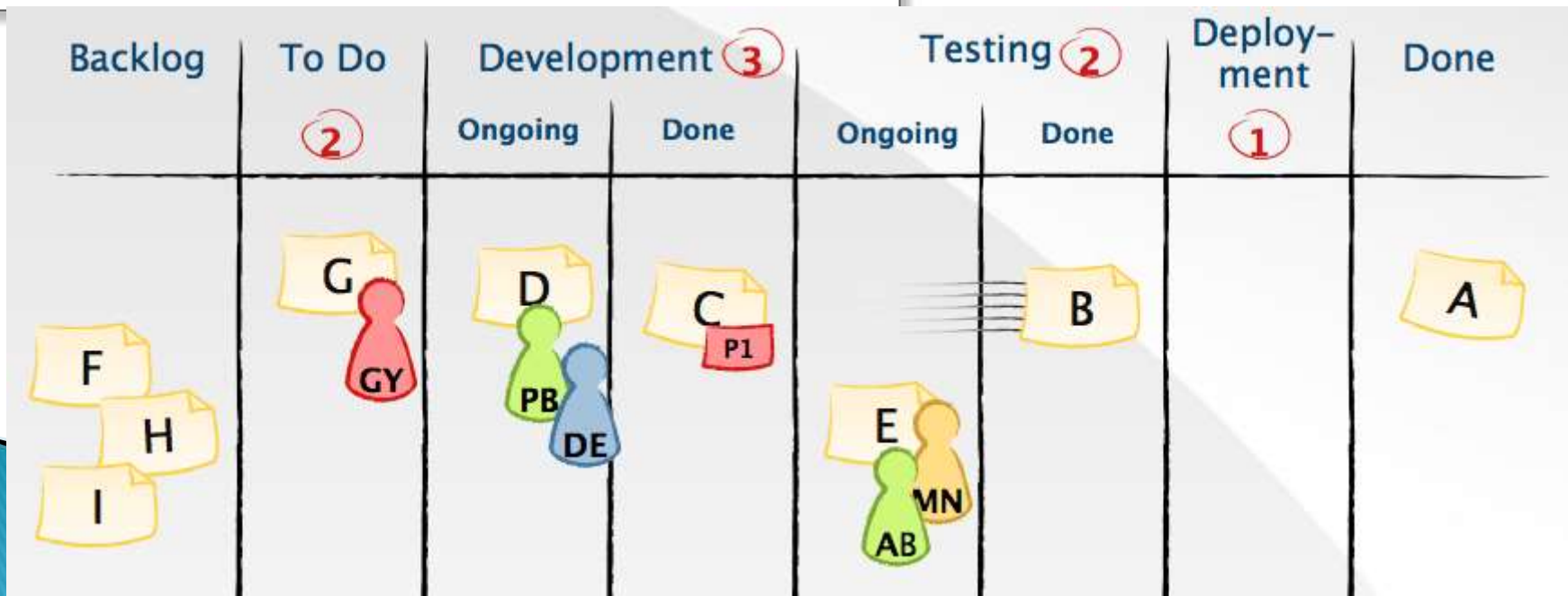
- 2009, Corey Ladas, Scrumban, 2010, David Anderson
- A method for managing knowledge work with an emphasis on **just-in-time delivery** while not **overloading the team members**
- A visual process-management system that tells what to produce, when to produce it, and how much to produce
- Visualisation is an important aspect that allows to understand the work and the workflow

Kanban principles

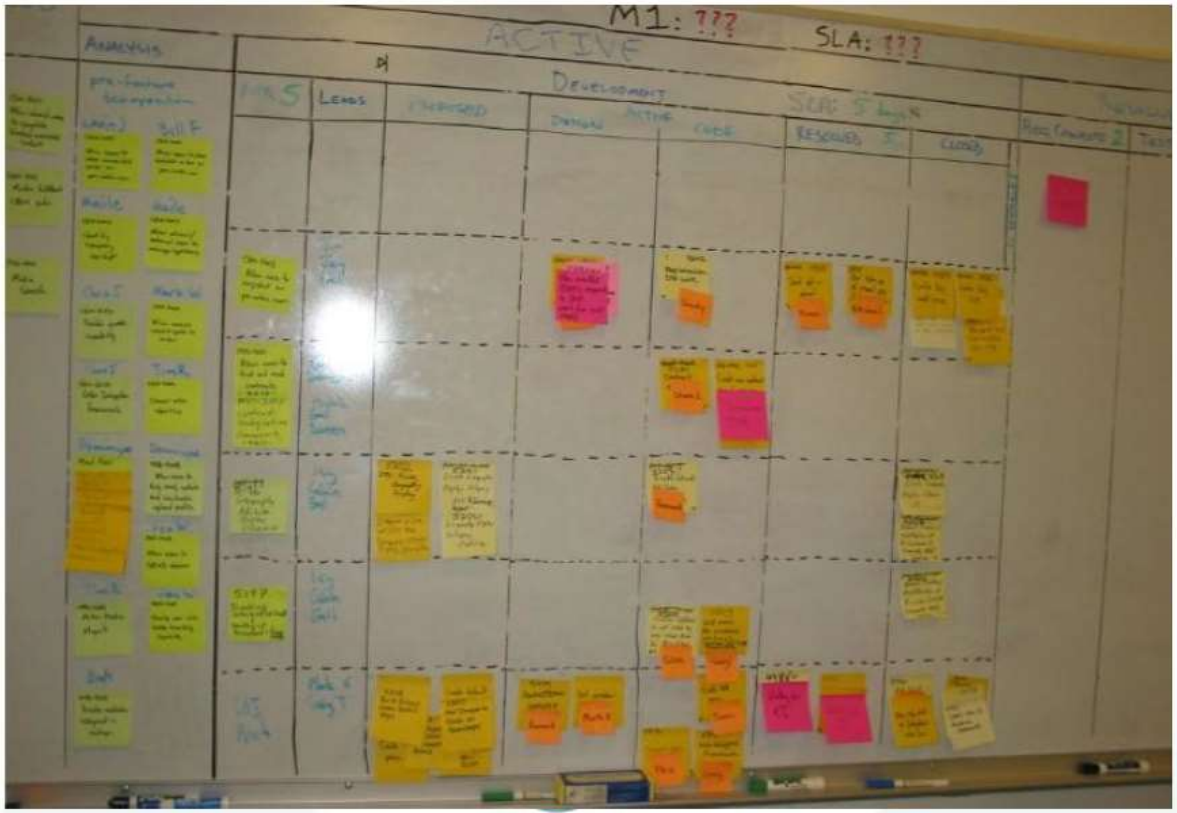
- **Start with existing process** – The Kanban method does not prescribe a specific set of roles or process steps
- **Agree to pursue incremental, evolutionary change** – continuous, incremental and evolutionary change is the way to make system improvements and make them stick
- **Respect the current process, roles, responsibilities and titles** – agreeing to respect current roles, responsibilities and job titles with the goal of gaining broader support
- **Leadership at all levels** – Acts of leadership at all levels in the organization are encouraged

Kanban – Visual Example





Kanban – Examples

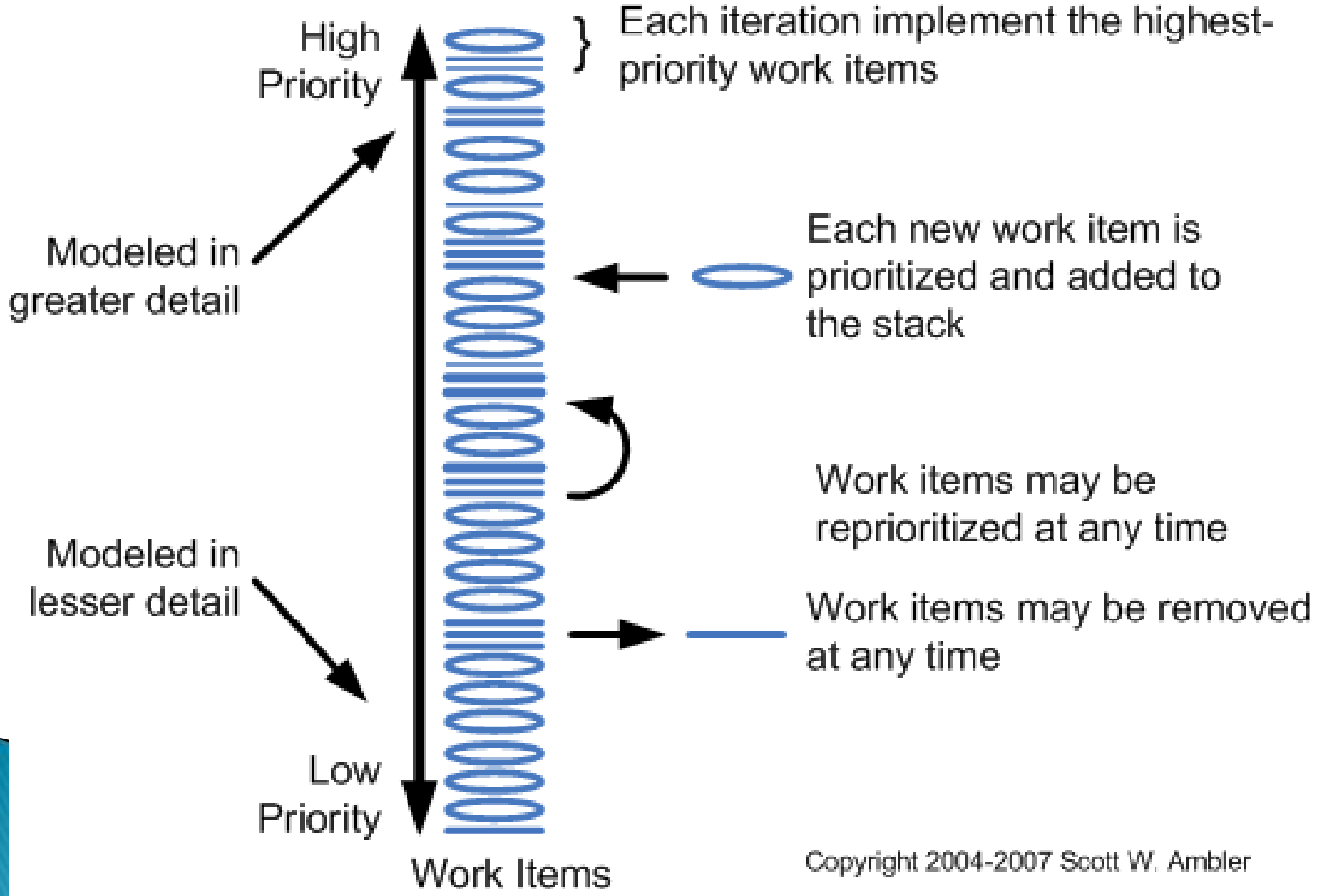


Model Driven Development

- *Model Driven Development* (MDD) is a paradigm for writing and implementing computer programs **quickly, effectively and at minimum cost**
- MDD is an approach to software development where **extensive models are created before source code is written**
- A primary example of MDD is the Object Management Group (OMG)'s Model Driven Architecture (MDA) standard

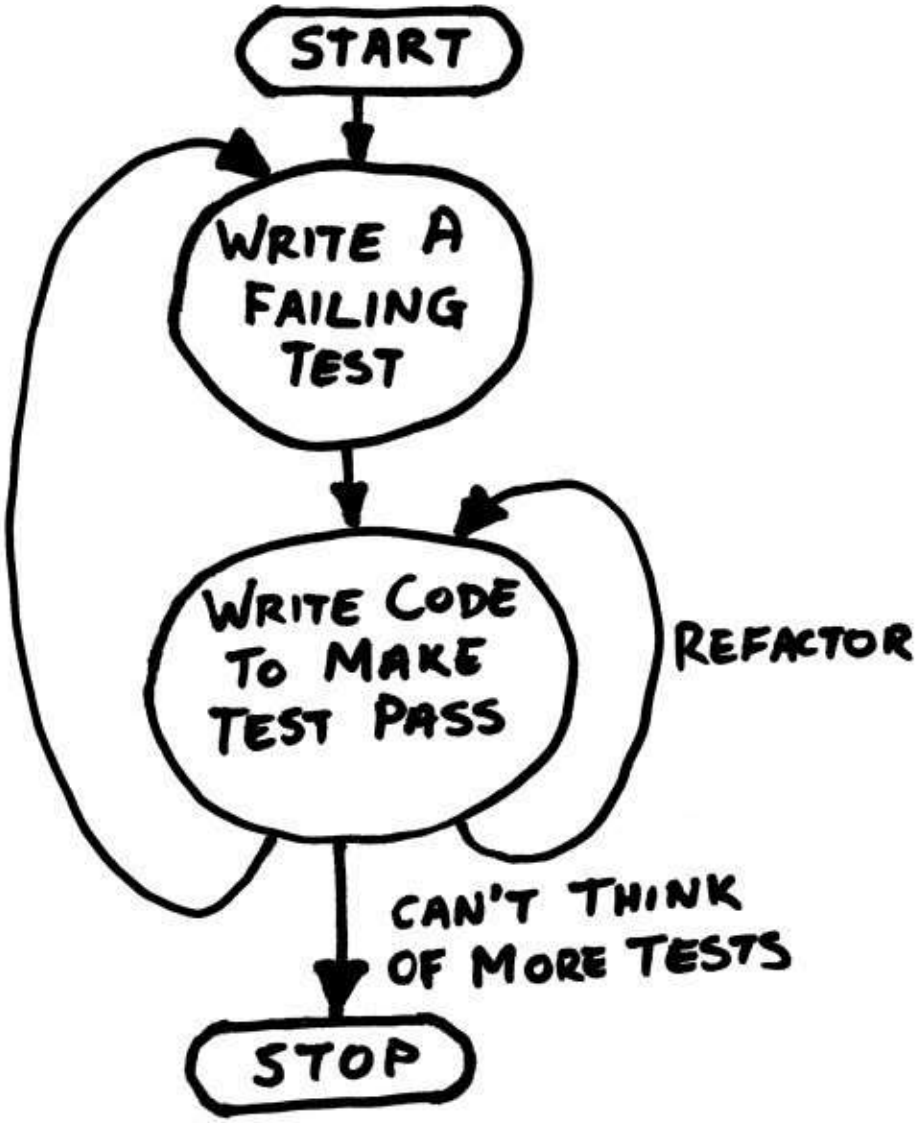
Agile MDD – Iteration Modeling

- Thinking Through What You'll Do This Iteration



Copyright 2004-2007 Scott W. Ambler

Test Driven Development



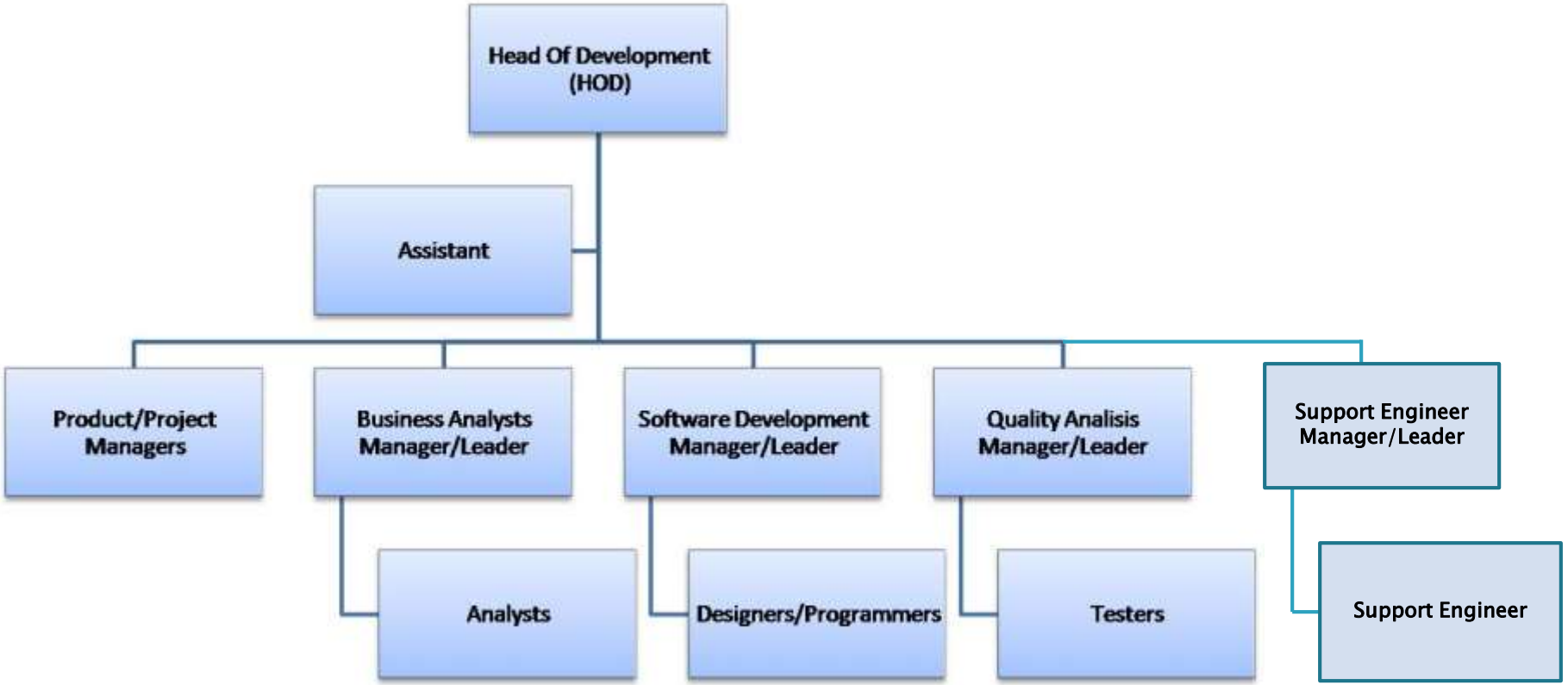
Myths

- **Clients**
  - A general description of the objectives is sufficient to begin writing program
  - Requirements are constantly changing, but the software is flexible and can easy adapts
- **Developers**
  - Once the program is written and it is functional, our role has ended
  - Until the program doesn't work, we can not assess the quality
  - The only good product is the functional program
  - Software Engineering will create voluminous and unnecessary documentation and will cause delays

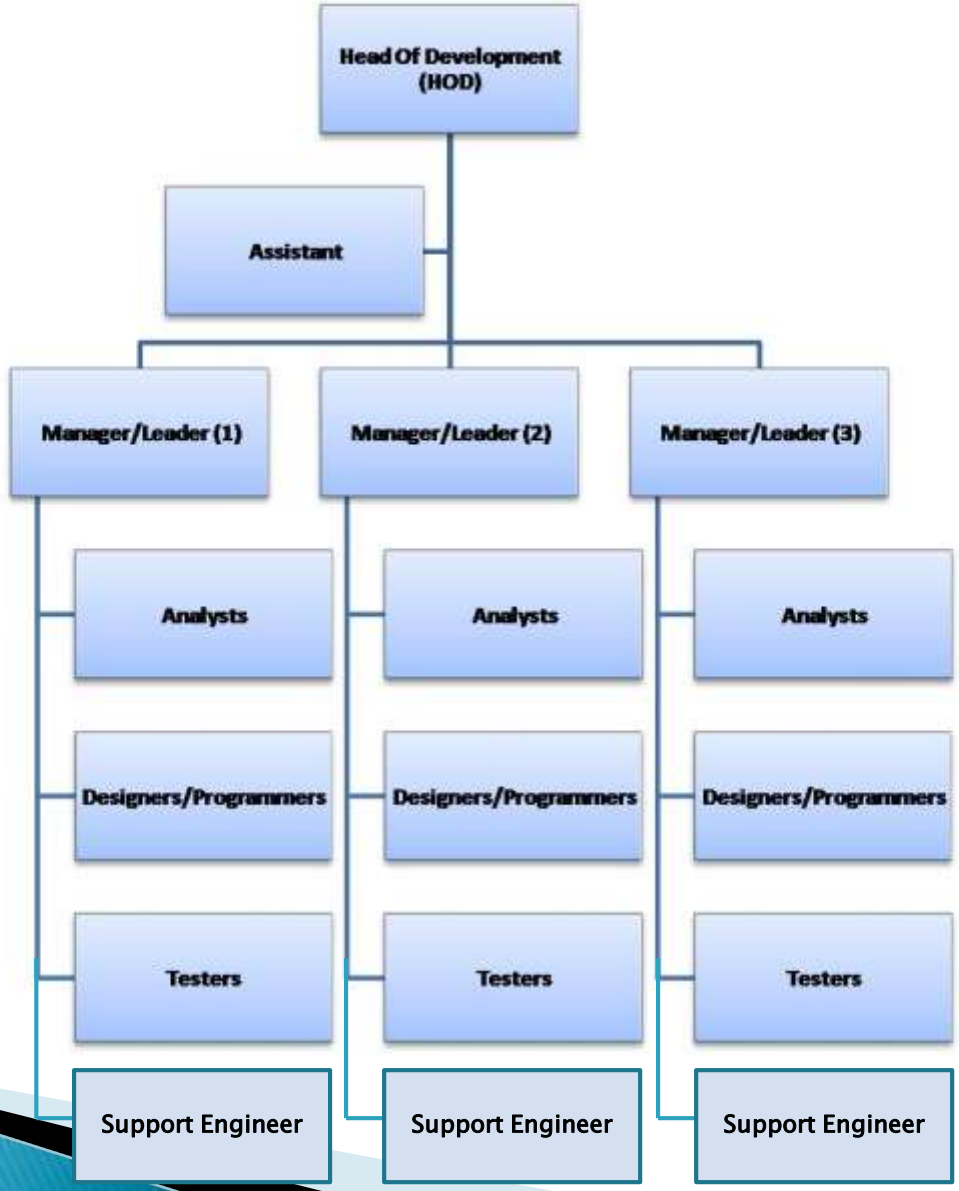
Cum vom face noi?

- O comunicare foarte bună cu CLIENTUL care face parte din echipă (SCRUM)
- După fiecare etapă veţi obţine un produs finit care de regulă nu va putea fi refăcut la paşii următori (Cascadă)
- Ca membru al “echipei” vă voi sprijini cât mai mult posibil (LEAN)
- Fiecare va fi încurajat să facă ce îi place mai mult (XP)
- NU am să aduc modificări continue în cerinţele mele (NU AGILE)
- (NU) vom face un studiu de risc ((NU) MODEL ÎN SPIRALĂ)

Ierarhia dintr-o firmă de software – Compartimente



Ierarhia dintr-o firmă de software - Proiecte



Etapele dezvoltării programelor

- În engleză: **Software Development Life Cycle**

- Analiza cerinţelor (Requirements analisys)
- Proiectarea arhitecturală (Architectural design)
- Proiectarea detaliată (Detailed design)
- Scrierea codului (Implementation)
- Integrarea componentelor (Integration)
- Validare (Validation)
- Verificare (Verification)
- Întreținere (Maintenance)

Cum începe un proiect?

- Un client doreşte să-şi
  - Îmbunătăţească productivitatea
  - Rezolve o problemă de personal
  - Facă reclamă la produsele pe care le vinde
  - Gestioneze mai uşor activitatea sucursalelor din ţară
- Un proiect interesant
- O idee, nevoia de a-mi gestiona cheltuielile zilnice, etc.
- **Din acest punct urmează Ingineria Cerinţelor!**

Ingineria cerinţelor (Requirement analysis) – Definiţie

- Procesul înţelegerii nevoilor clientului şi a aşteptărilor acestuia de la aplicaţia noastră
- O etapă bine definită din ciclul de viaţă al dezvoltării unui produs (Software Development Life Cycle)
- La ce ne aşteptăm de la o aplicaţie să facă
- Cum ar trebui sistemul să se comporte şi care sunt caracteristicile acestuia

Ingineria cerinţelor – Exemplu

- Realizaţi un program C++ care să realizeze suma a două matrici citite din fişier.
- +:
  - Se specifică limbajul
  - Ştim că citirea se face din fişier
- –:
  - Nu ştim ce să facem cu două matrici care nu au aceleaşi dimensiuni
  - Ce facem cu rezultatul?

Ingineria cerinţelor – Cine?

- Datorită multitudinii de tipuri de interacţiuni care pot exista între utilizatori, procese de business, dispozitive hardware, etc., pot exista diverse tipuri de cerinţe, de la aplicaţii simple, la aplicaţii complexe
- Procesul de analiză a cerinţelor presupune alegerea şi documentarea acestor tipuri de cerinţe, şi construirea documentelor ce vor constitui baza construirii sistemului

- **Cine se ocupă?** Project Manager, Program Manager sau Business Analyst

Ingineria cerinţelor – De ce?

- Studiile făcute demonstrează că atenţia insuficientă acordată analizei cerinţelor este **cea mai des întâlnită cauză în cadrul proiectelor vulnerabile**
- Foarte multe organizaţii au cheltuit sume imense pe proiecte software care în final nu făceau ceea ce se dorea iniţial de la ele
- În momentul de faţă foarte multe companii investesc timp şi bani pentru a face o analiză a cerinţelor eficientă

Ingineria cerinţelor – Paşi

1. Stabilirea limitelor aplicaţiei
2. Găsirea clientului
3. Identificarea cerinţelor
4. Procesul de analiză a cerinţelor
5. Specificarea cerinţelor
6. Gestionarea cerinţelor

IC – Limitele aplicaţiei (1)

- Ca prim pas, are ca scop identificarea modulului în care această nouă aplicaţie **se va integra în mediul** pentru care va fi concepută
- Care va fi **scopul** aplicaţiei
- Care vor fi **limitele** aplicaţiei

IC– Găsirea clientului (2)

- **Obiectivul** ultimilor ani: *Cine este utilizatorul (clientul) care va folosi efectiv aplicaţia ?*
- **Ca rezultat**, vom şti exact ce persoane vor fi direct sau indirect afectate de realizarea acestui produs
- Vom şti pe cine să întrebăm pentru eventuale clarificări

IC – Identificarea cerinţelor (3)

- Cerinţele se colectează de la mai multe grupuri ce au fost identificate în etapa anterioară
- Se identifică ce anume doresc aceştia ca aplicaţia să realizeze
- **Nivelul de detaliere** depinde de:
  - Numărul şi de dimensiunea grupurilor
  - Complexitatea procesului de business
  - Dimensiunea aplicaţiei
- **Probleme întâlnite** în această etapă
  - Ambiguităţi în înţelegerea proceselor
  - Inconsistenţă în înţelegerea aceluiaşi proces
  - Date insuficiente
  - Modificări în cerinţe după începerea proiectului

IC – Cine face identificarea cerinţelor ? (3)

- Această persoană trebuie să interacţioneze direct cu multe grupuri de lucru
- Are de a face cu idei contradictorii
- Trebuie să aibă abilităţi de comunicare şi de lucru cu oamenii
- Trebuie să aibă cunoştinţe de programare
- În final trebuie să cadă de acord cu clientul în privinţa cerinţelor

IC – Metode folosite în identificarea cerinţelor (3)

- Interviuri cu viitorii utilizatori şi cu grupuri de utilizatori
- Folosirea documentaţiei existente (manuale de utilizare, diagrame ale organizaţiei, specificaţii de sistem, etc.)
- Metode:
  - Prototipuri
  - Diagrame “Use case”
  - Diagrame de flux a datelor şi a proceselor
  - Interfeţe utilizator

IC – Procesul de analiză a cerinţelor (4)

- Se face o analiză structurată care foloseşte tehnici specifice:
  - “animarea” cerinţelor
  - Raţionament automat
  - Privire critică din punct de vedere al cunoaşterii
  - Verificarea consistenţei
  - Raţionament analogic şi bazat pe exemple

IC – Specificarea cerinţelor (5) 1

- Se face într-un mod **clar, neambiguu**
- **Scrierea unui document** în care se specifică cerinţele este **obligatoriu**!
- Acest document va circula între toate persoanele implicate în această fază: *client, grupuri de utilizatori, echipele de dezvoltare şi de testare*
- Documentul va fi folosit la:
  - Validarea cerinţelor de către client
  - Contractul dintre client şi echipa de dezvoltare
  - Bază pentru proiectarea sistemului de către dezvoltatori
  - Bază pentru planificări
  - Sursă pentru realizarea scenariilor de testare

IC – Specificarea cerinţelor (5) 2

- Trebuie să surprindă **viziunea clientului** despre produs
- Reprezintă rezultatul colaborării dintre **utilizator** (care nu e un expert) şi **analistul de sistem** (care surprinde situaţia în termeni tehnici)
- E posibil ca specificarea cerinţelor să se facă în două documente separate:
  - **Cerinţele utilizator** – scrise în clar folosind cazuri de utilizare (pentru utilizator)
  - **Cerinţele sistemului** – descrise folosind un model matematic sau programatic (pentru dezvoltatori şi pentru testeri)

IC – Specificarea cerinţelor (5) 3

- În cerinţele utilizatorului **nu trebuie** să apară noţiuni tehnice (protocol de comunicare, criptarea folosind MD5, http, IP, etc)

- În cerinţele sistemului **trebuie** să apară formatul de export al datelor (XML), adresa serverului de pe care se fac citiri, locul în care se depozitează fişierele log

IC – Specificarea cerinţelor (5) 4

- Nivelul de detaliere:
  - **Ridicat** – presuspune multă muncă, uneori inutilă (este mai precis şi mai clar)
  - **Scăzut** – poate fi vag (nu ajută în procesul de dezvoltare şi testare)
- Exemplu:
  - Realizaţi un program care să facă suma a două matrici.
  - Realizaţi un program C# care să aibă clasa **Matrice** cu atributele **n,m** de tip **int** reprezentând numărul de linii şi de coloane şi **matrice** de tip **int[3][3]** reprezentând elementele matricii. Metodele disponibile în clasa Matrice sunt .....

IC – Specificarea cerinţelor (5) 5

- Tipuri de cerinţe:
  - **Cerinţe utilizator**: legate de locul unde va fi folosit sistemul, eficienţă, durata de viaţă a produsului (*produsul va fi folosit de compartimentul financiar*)
  - **Cerinţe funcţionale:** despre modul în care se fac anumite calcule, modul în care se manipulează datele (*impozitul pe salar este de 16 %*)
  - **Cerinţe de performanţă**: modul în care anumite funcţii sunt apelate cantitativ, calitativ (*sistemul va permite 1000 de interogări pe secundă*)
  - **Constrângeri**: nu se va permite ca două persoane să introducă simultan date în tabele

IC – Gestionarea cerinţelor (6)

- Este un proces continuu care surprinde toate aspectele identificării cerinţelor şi în plus asigură verificarea, validarea acestora

- Pentru a fi utilă trebuie să asigure neambiguitatea cerinţelor, eliminarea erorilor şi completarea omisiunilor

Scenarii de utilizare 1

- Folosesc **actori** (elemente cu care programul interacţionează):
  - **Utilizatori umani**
  - **Elemente software (Ex: program care prelucrează informaţiile colectate de pe Internet)**
  - **Elemente hardware (Ex: cititor de coduri de bare, telefoane mobile, etc.)**
- Folosesc **scenarii (use case)**
  - Acestea descriu cum interacţionează actorul cu sistemul
  - Cum reacţionează sistemul în urma acestor acţiuni
  - Care e rezultatul vizibil pentru actori

Scenarii de utilizare 2

- Ce nu conţin acestea:
  - Diagrame de clase
  - Structura modulară a programului
  - Tipul datelor de intrare şi de ieşire

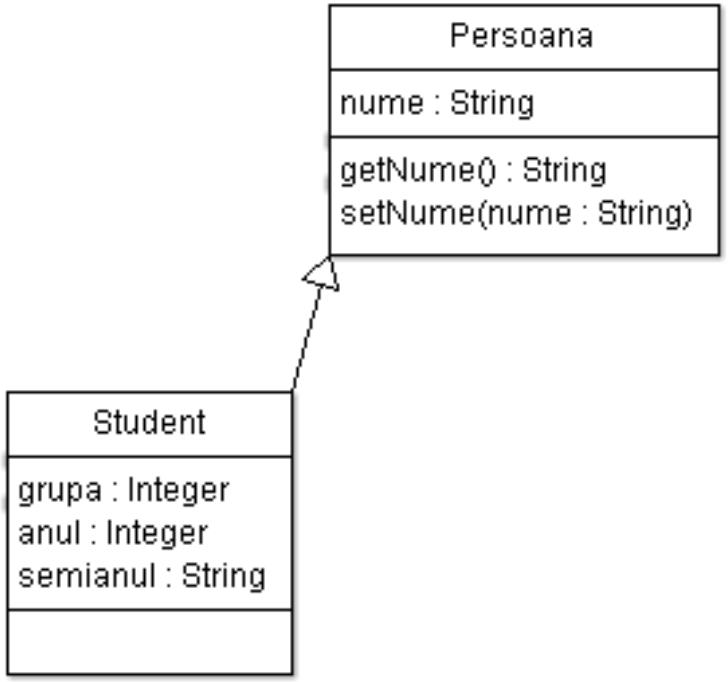
- Use Case – Tipuri de conţinut:
  - **Pe scurt** – descrie principalul caz de succes
  - **Cazual** – conţine ce ar trebui făcut în caz că se întâmplă ceva
  - **Detaliat** – se prezintă pe larg toate situaţiile posibile

Use Case – Exemple

- **Pe scurt**: Programul trebuie să poată aduna 2 matrici
- **Cazual**: Programul trebuie să poată aduna 2 matrici dacă au acelaşi număr de linii şi de coloane, **altfel** se va afişa un mesaj de eroare corespunzător
- **Detaliat**: Programul trebuie să poată aduna două matrici de numere întregi citite de la tastatură, **dacă** au acelaşi număr de linii şi de coloane, iar matricea rezultată se va afişa într-un fişier "rezultat.txt" câte o linie pe rând. **Altfel** se va afişa un mesaj de eroare corespunzător într-un fişier "mesaj.txt" aflat în directorul curent. (*Mai trebuie specificat ceva?*)

UML – Diagrame de clase

- Relaţia de generalizare

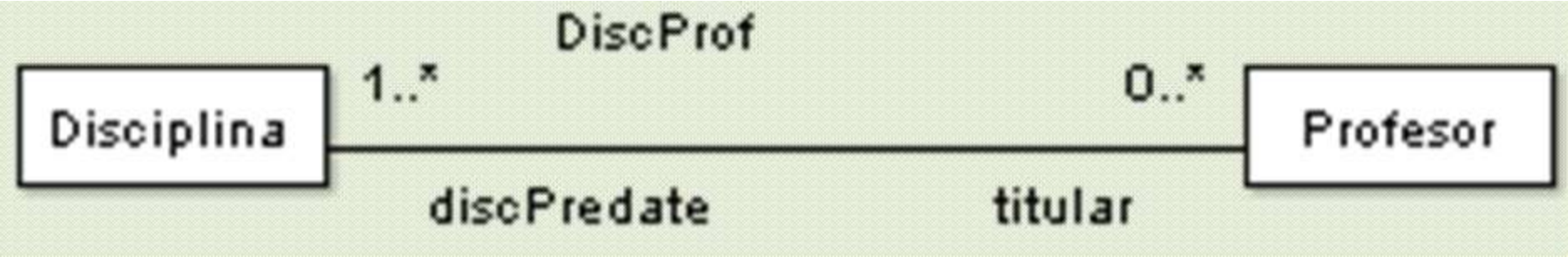


Relaţia de asociere 1

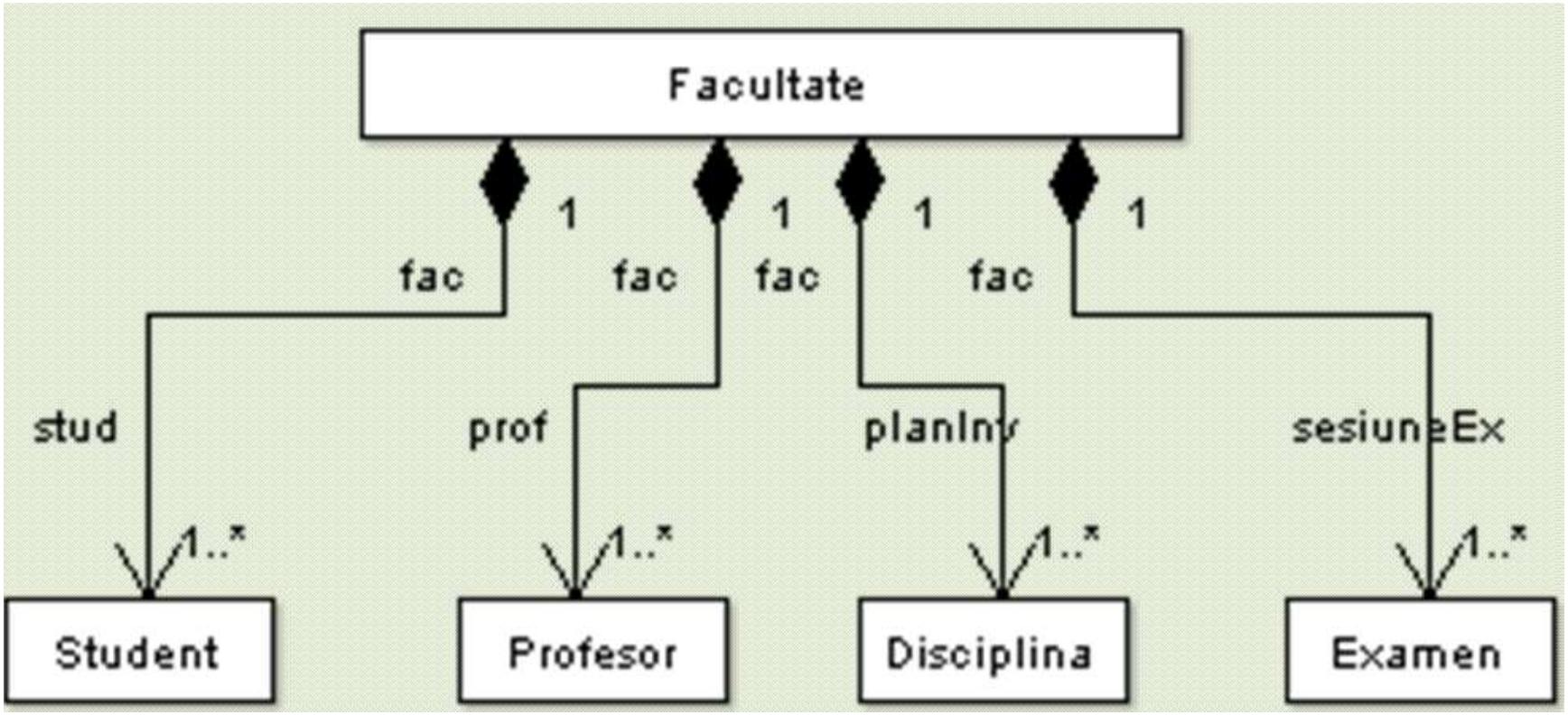
- Relaţia Student – Disciplină
  - **Student**: urmez 0 sau mai multe discipline, cunosc disciplinele pe care le urmez;
  - **Disciplină**: pot fi urmată de mai mulţi studenţi, **nu** cunosc studenţii care mă urmează

Relaţia de asociere 2

- Relaţia Disciplină – Profesor
  - **Disciplină**: sunt predată de un profesor, îmi cunosc titularul
  - **Profesor**: pot preda mai multe discipline, cunosc disciplinele pe care le predau



Relaţia de Compoziţie (“hasA”)



ArgoUML

- Link: http://argouml-downloads.tigris.org/argouml-0.34/
- Varianta “zip” trebuie doar dezarhivată
- Trebuie să aveţi instalat Java
  - În Path sa aveti c:\Program Files\Java\jdk1.6.0_03\bin
  - Variabila

  JAVA_HOME=c:\Program Files\Java\jdk1.6.0_03\

Concluzii

- Metodologii
- Ingineria cerințelor
- POO

- Sugestii:
  - Lucrați la laborator cu ArgoUML (care permite să faceți forward and reverse engineering)
  - Verificați în avans tematica laboratorului și exersați acasă realizarea pașilor introductivi (instalare, găsire opțiuni, implementarea unui exemplu foarte simplu,...)

Bibliografie

- Anil Hemrajani, Agile Java Development with Spring, Hibernate and Eclipse, 2006

- Dorel Lucanu, Principii POO

Links

- XP: http://www.extremeprogramming.org/rules.html
- Agile: http://agilemanifesto.org/
- Scrum: http://jeffsutherland.com/oopsla/schwapub.pdf
- Lean: http://www.projectperfect.com.au/info_lean_development.php
- V-model: http://en.wikipedia.org/wiki/V-Model, http://en.wikipedia.org/wiki/V-Model_%28software_development%29
- Project Management White Paper Index: http://www.projectperfect.com.au/wp_index.php
- Requirements analysis process: http://www.outsource2india.com/software/RequirementAnalysis.asp
- ImageCup 2009: http://fiistudent.wordpress.com/2008/12/10/imagine-cup-2009-ce-ar-fi-daca-intr-o-zi-am-ajunge-toti-la-muzeu/
- Curs 2 IP – Ovidiu Gheorghieş: http://www.infoiasi.ro/~ogh/files/ip/curs-02.pdf
- Software house: http://en.wikipedia.org/wiki/Software_house

Ingineria Programării

Cursul 3 – 6 Martie

adiftene@info.uaic.ro

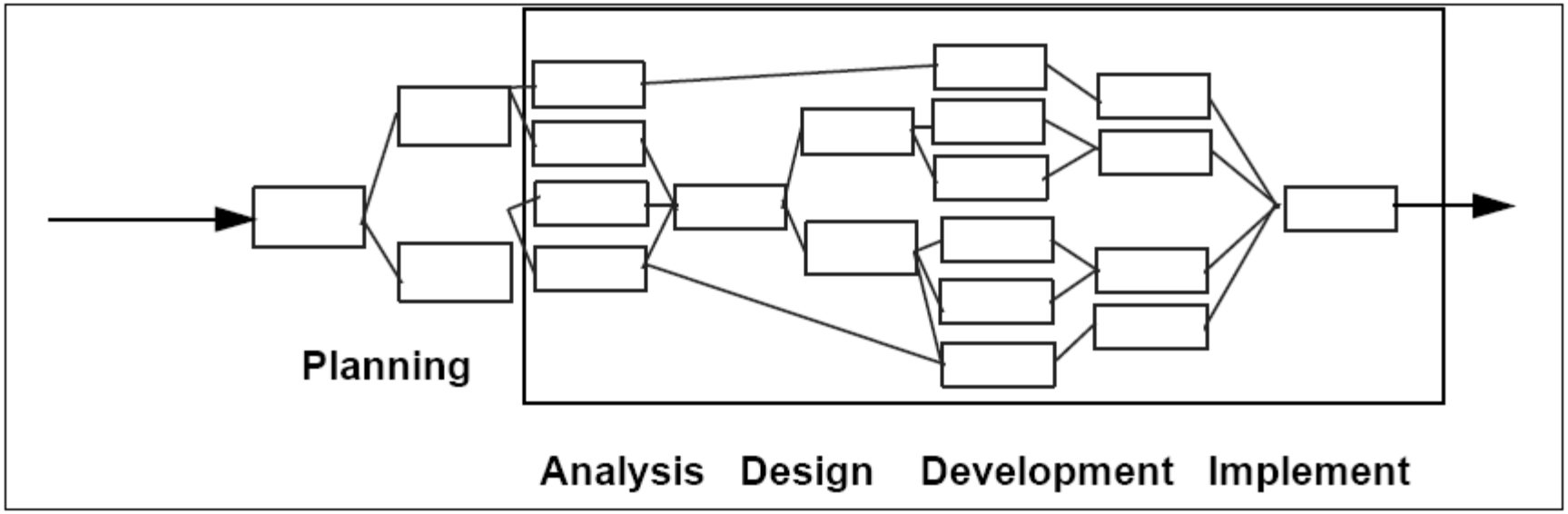
Cuprins

- Din Cursurile 1, 2…
- Modelarea
- Limbaje de Modelare
  - Limbaje Grafice
- UML – Istoric
- UML – Definiţie
- UML – Tipuri de Diagrame
- UML – Diagrame Use Case
  - Actori
  - Use Case
- UML – Diagrame de Clase

Din Cursurile 1, 2

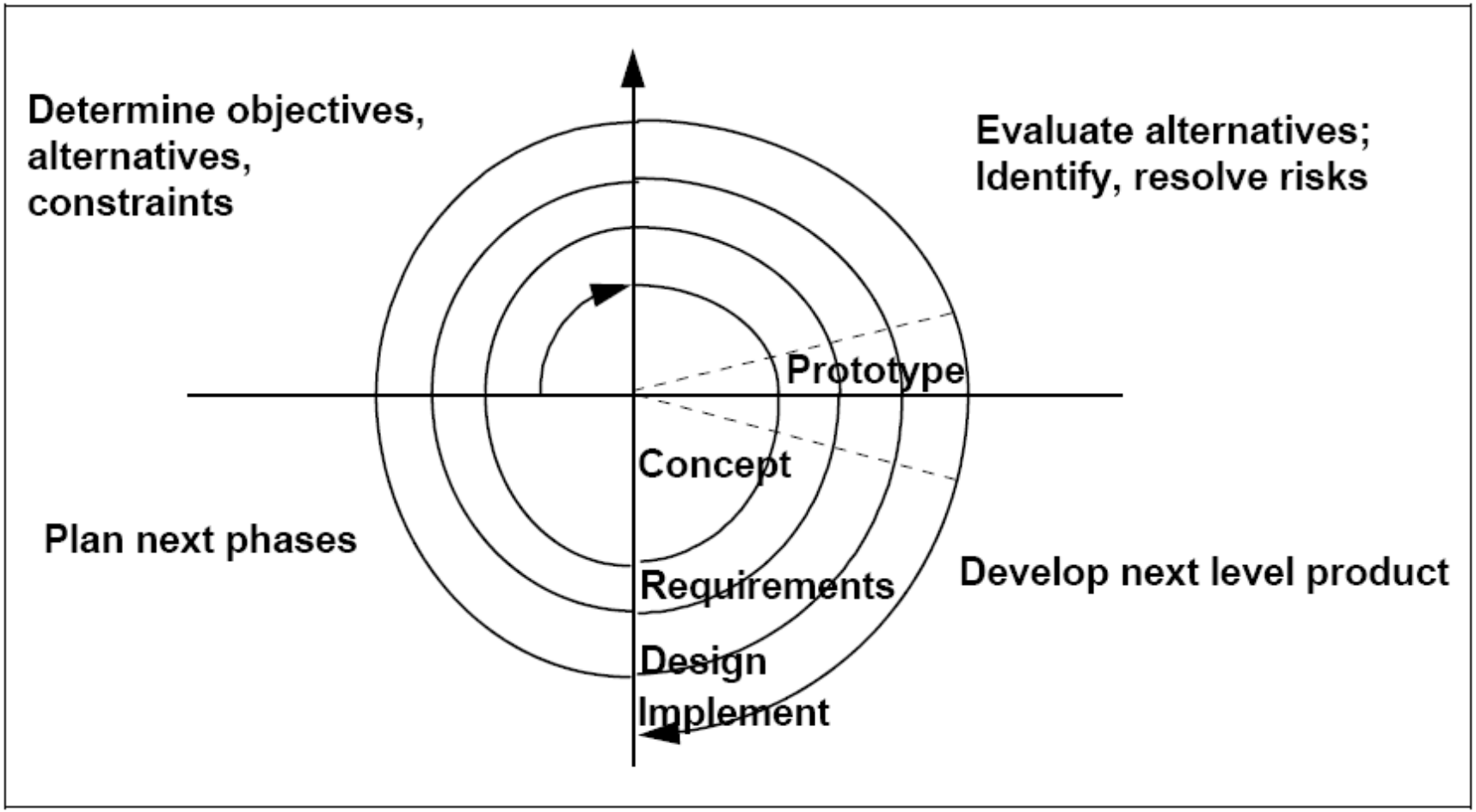
- Ingineria programării (Software engineering)
- Etapele dezvoltării programelor
- Modele de dezvoltare
- Ingineria cerinţelor

Din cursul 1



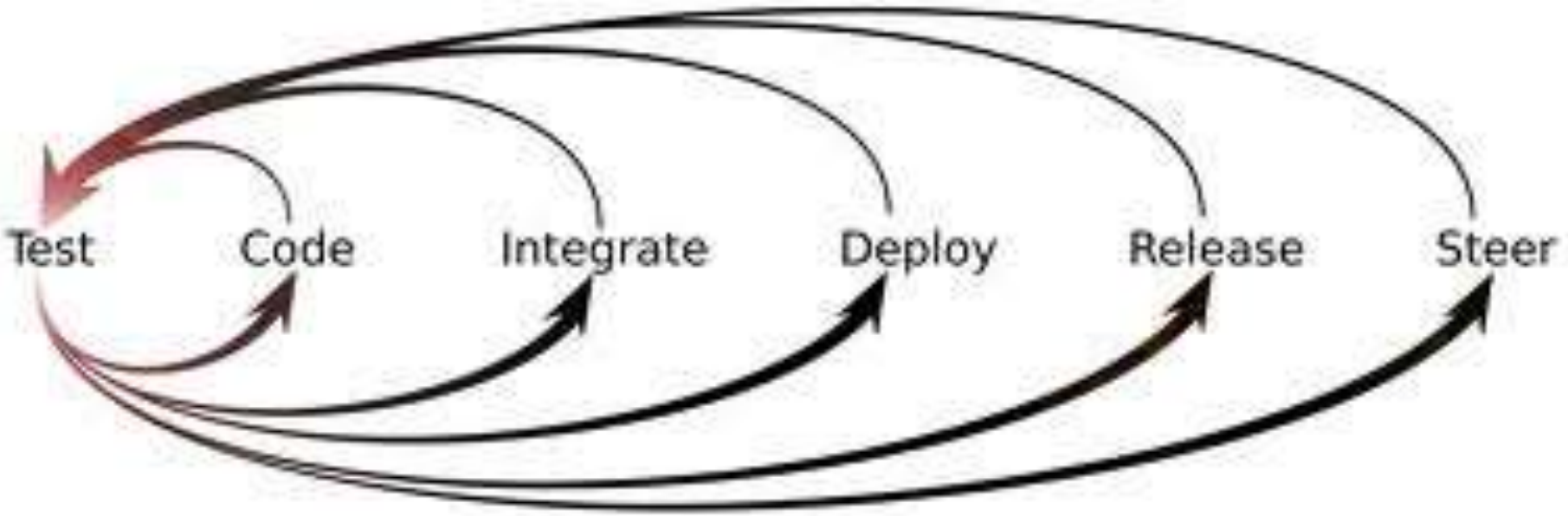
Modelul în cascadă...

Din cursul 1



Modelul în spirală

Din cursul 2



- XP, TDD…

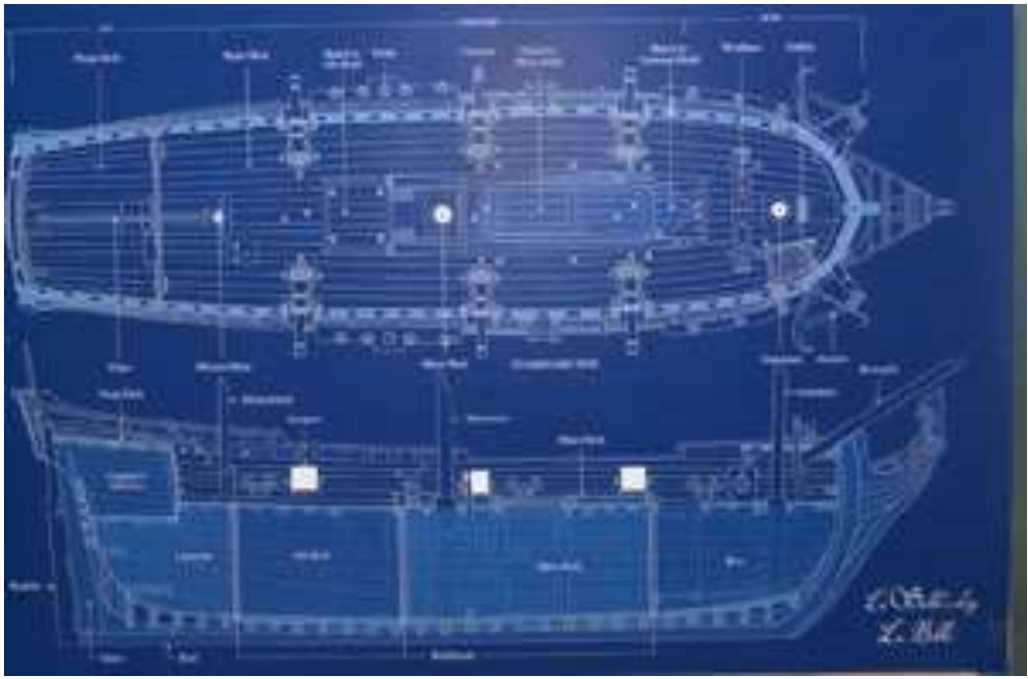
Din cursul 2

- Ingineria cerințelor:
  - Actori
  - Scenarii

- Studiu de caz
  - Identificare actori
  - Identificare scenarii (Obiectiv, Pași, Extensii)
  - Construire diagrame de clasă

Modelarea – De ce?

- Ce este un model?
  - Simplificarea realităţii
  - Planul detaliat al unui sistem (blueprints)



- De ce modelăm?
  - Pentru a înţelege mai bine ce avem de făcut
  - Pentru a ne concentra pe un aspect la un moment dat
- Unde folosim modelarea?

Scopurile Modelării

- Vizualizarea unui sistem
- Specificarea structurii sale şi/sau a comportării
- Oferirea unui şablon care să ajute la construcţie
- Documentarea deciziilor luate

Modelarea Arhitecturii

- Cu ajutorul **Use case**-urilor: pentru a prezenta cerinţele
- Cu ajutorul **Design**-ului: surprindem vocabularul şi domeniul problemei
- Cu ajutorul **Proceselor**: surprindem procesele şi thread-urile
- Cu ajutorul **Implementării**: avem modelarea aplicaţiei
- Cu ajutorul **Deployment**: surprindem sistemul din punct de vedere ingineresc

Principiile Modelării

- Modelele influenţează soluţia finală
- Se pot folosi diferite niveluri de precizie
- Modelele bune au corespondent în realitate
- Nu e suficient un singur model

Limbaje de Modelare

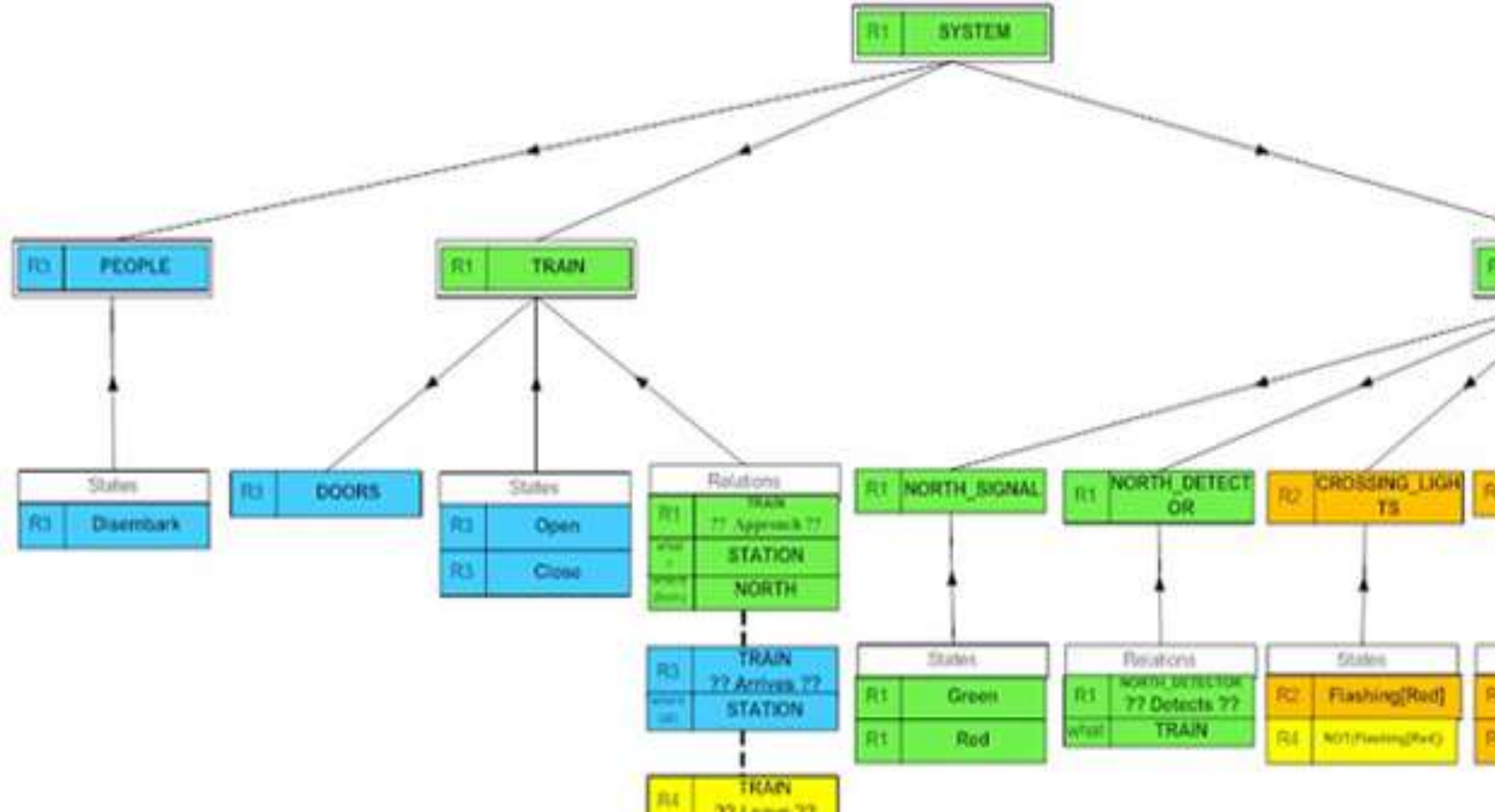
- Analiza şi proiectarea unui proiect trebuie făcute înainte de realizarea codului
- În prezent, se acordă o atenţie deosebită acestei etape, deoarece de ele depind **producerea** şi **refolosirea** de software
- Pentru analiza și proiectarea programelor s-au creat **limbajele de modelare**
- **Limbaj de modelare este** un limbaj artificial care poate fi folosit să exprime informații sau cunoaștere sau sisteme

Tipuri de Limbaje de Modelare

- **Limbaje Grafice**: arbori comportamentali, modelarea proceselor de business, EXPRESS (modelarea datelor), flowchart, ORM (modelarea rolurilor), reţele Petri, **diagrame UML**
- **Limbaje Specifice**: modelare algebrică (AML) (pentru descrierea şi rezolvarea problemelor de matematică ce necesită putere computaţională mare), modelarea domeniilor specifice (DSL), modelarea arhitecturilor specifice (FSML), modelarea obiectelor (object modeling language), modelarea realităţii virtuale (VRML)

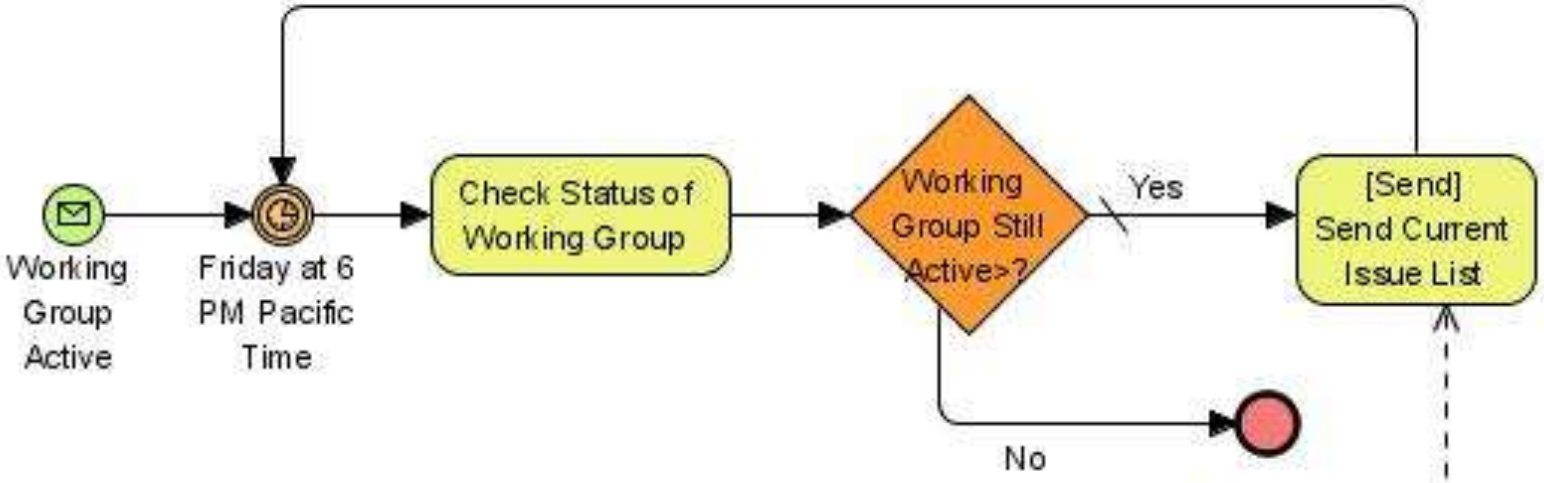
Limbaje Grafice 1

- Arbori comportamentali

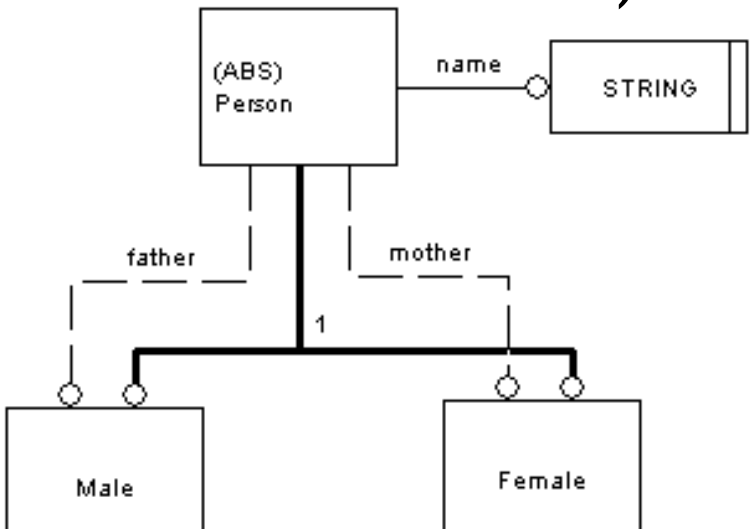


Limbaje Grafice 2

- Modelarea proceselor de business

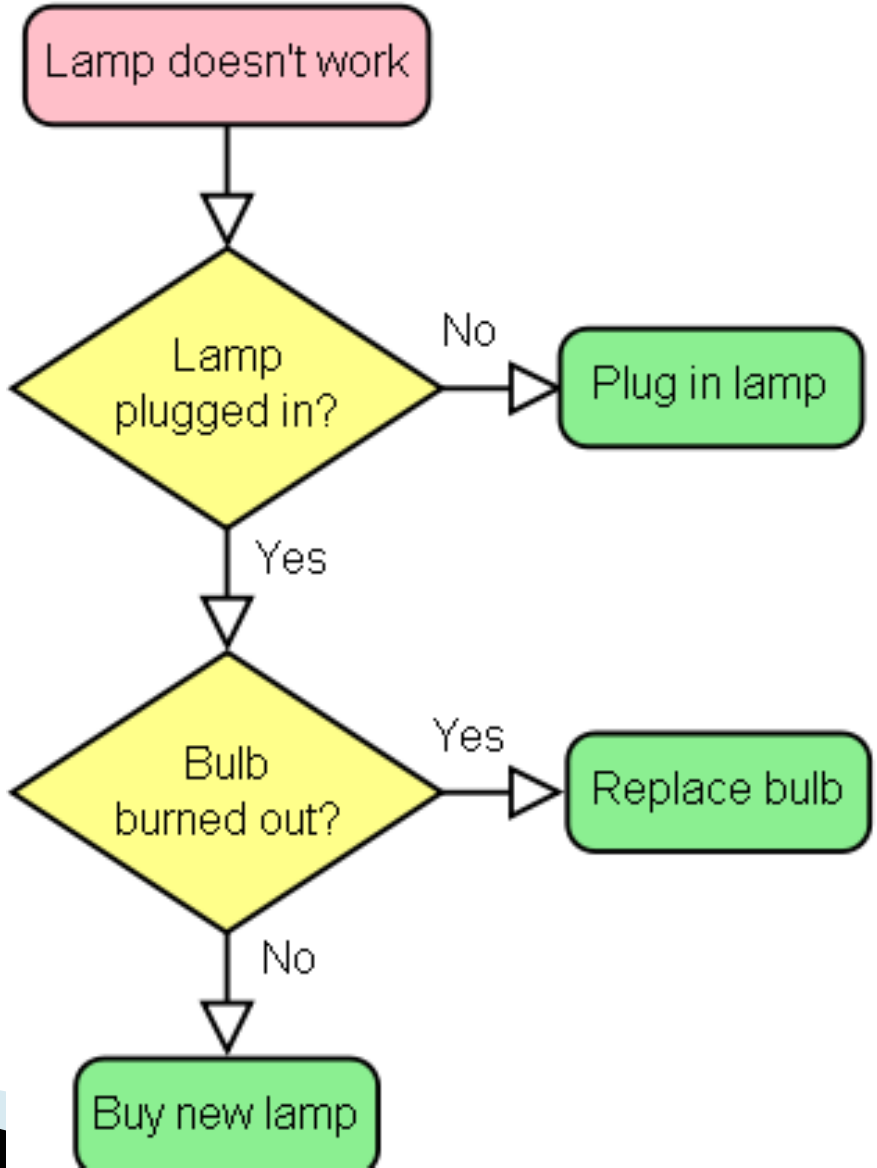


- EXPRESS (modelarea datelor)



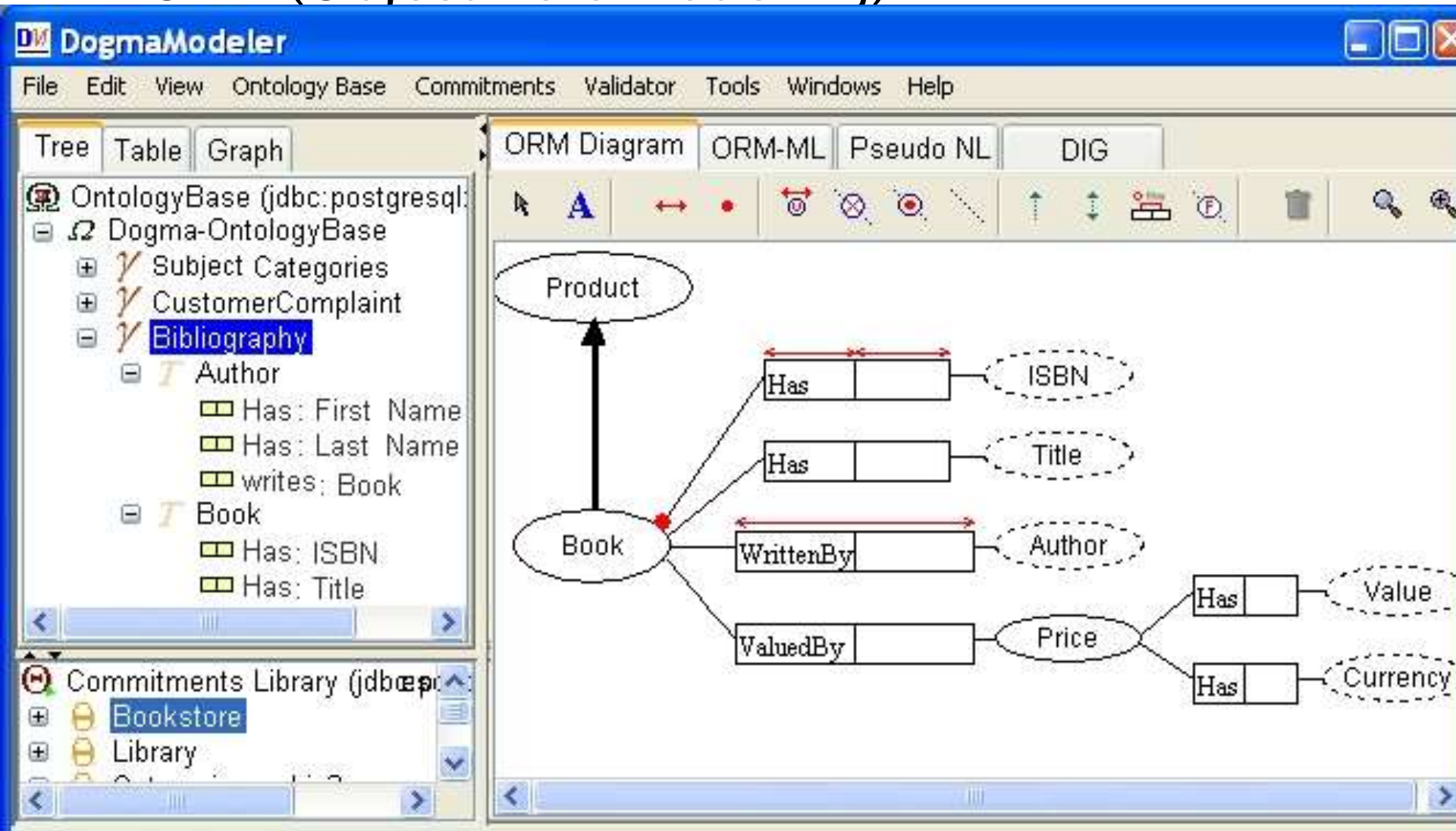
Limbaje Grafice 3

- Flowchart



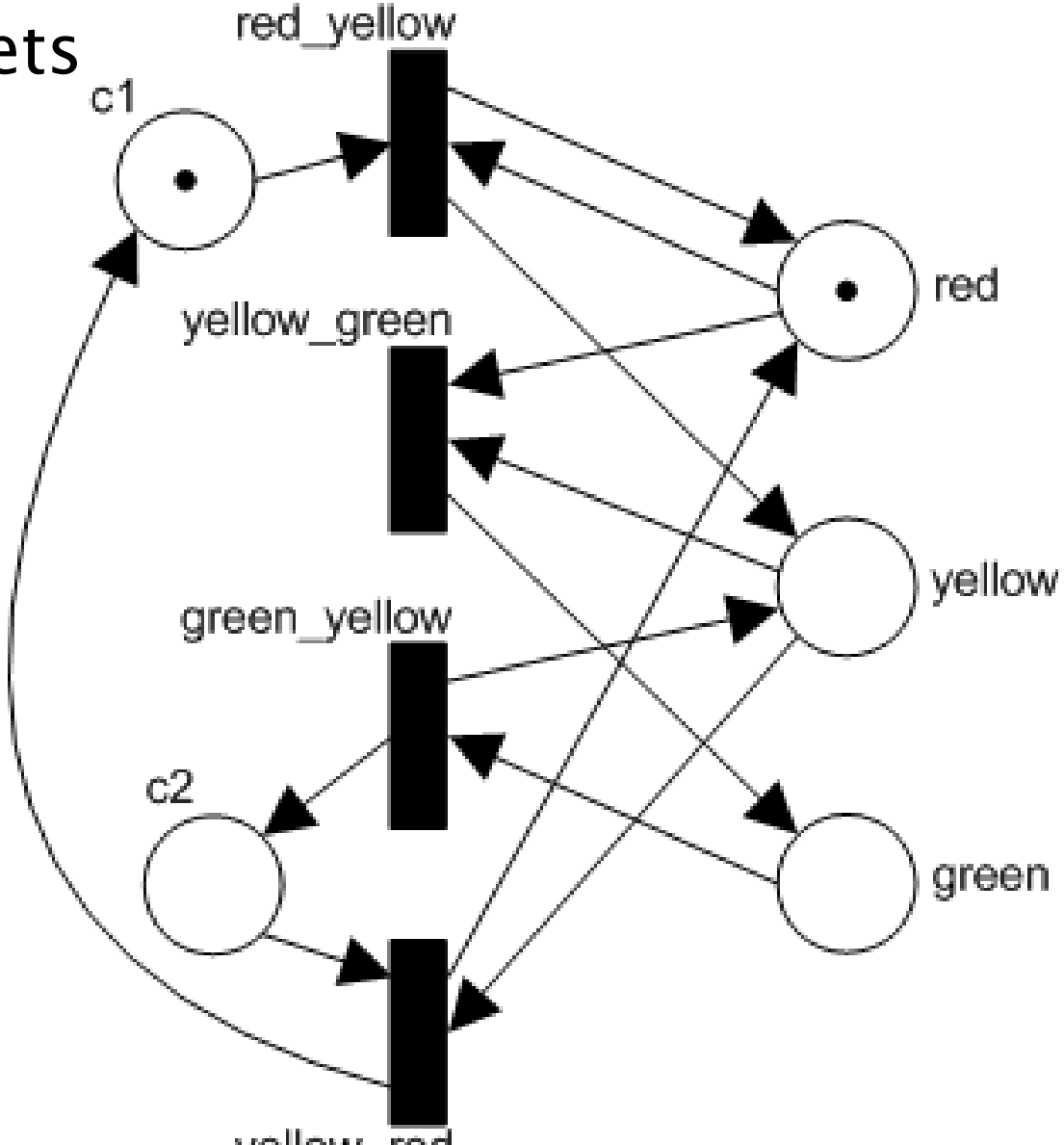
Limbaje Grafice 4

- ORM (Object Role Modeling)



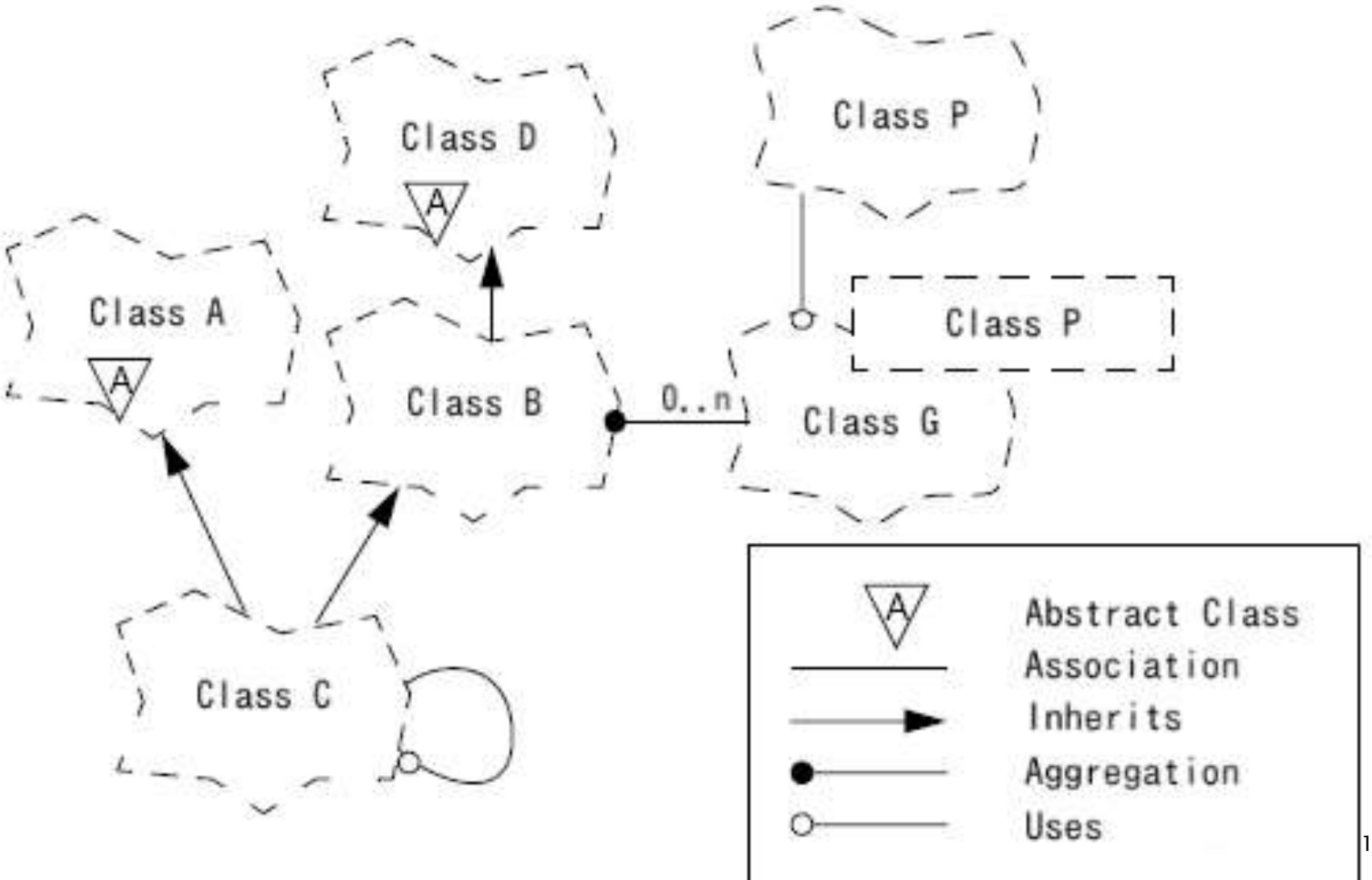
Limbaje Grafice 5

- Petri Nets



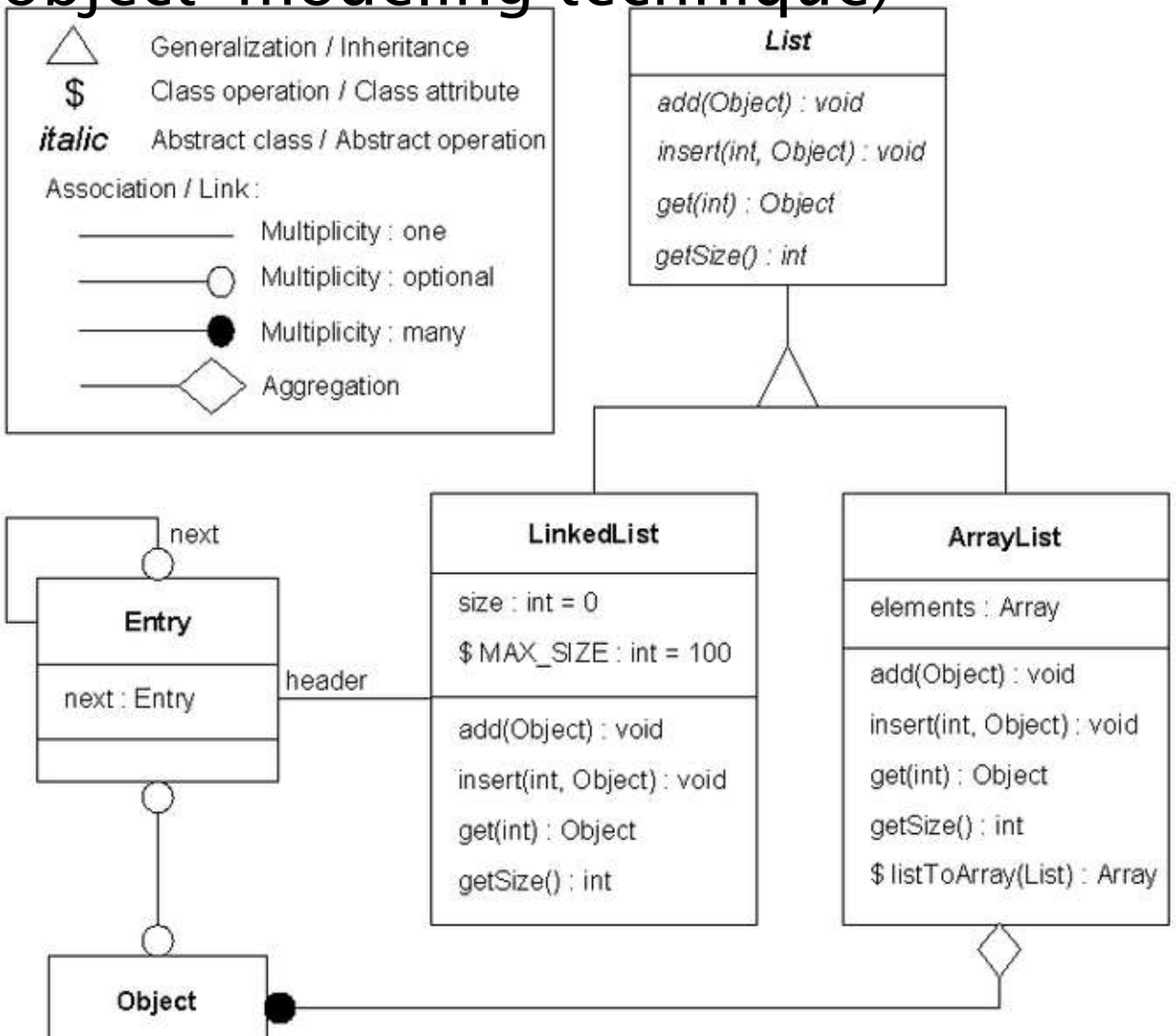
Limbaje Grafice 6

- Metoda Booch (Grady Booch) – analiza şi design oo



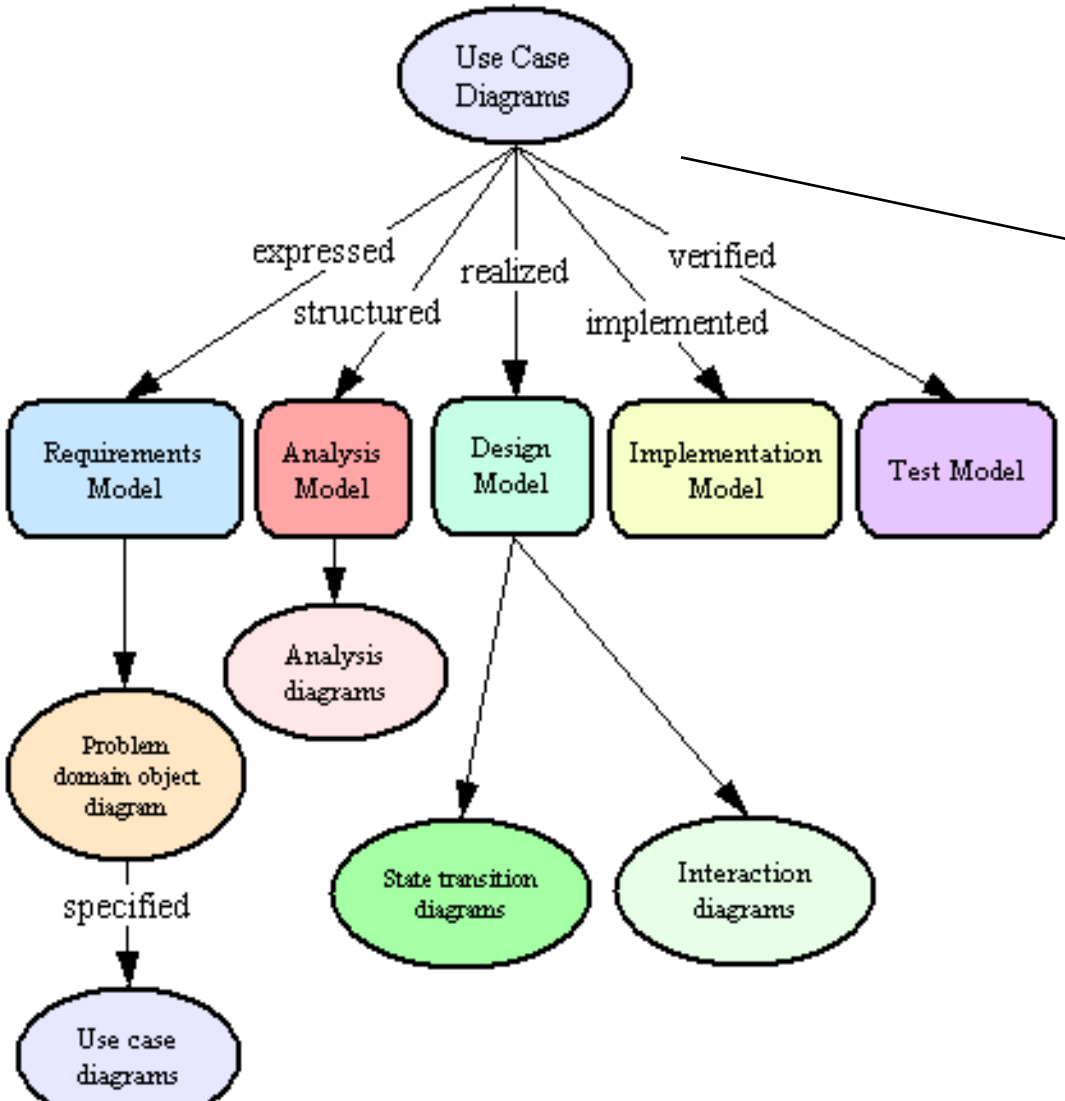
Limbaje Grafice 7

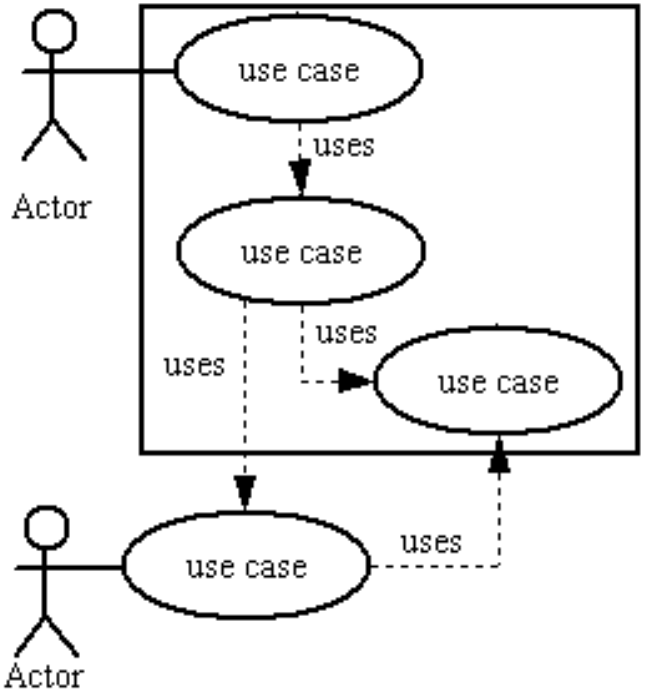
- OMT (object-modeling technique)



Limbaje Grafice 8

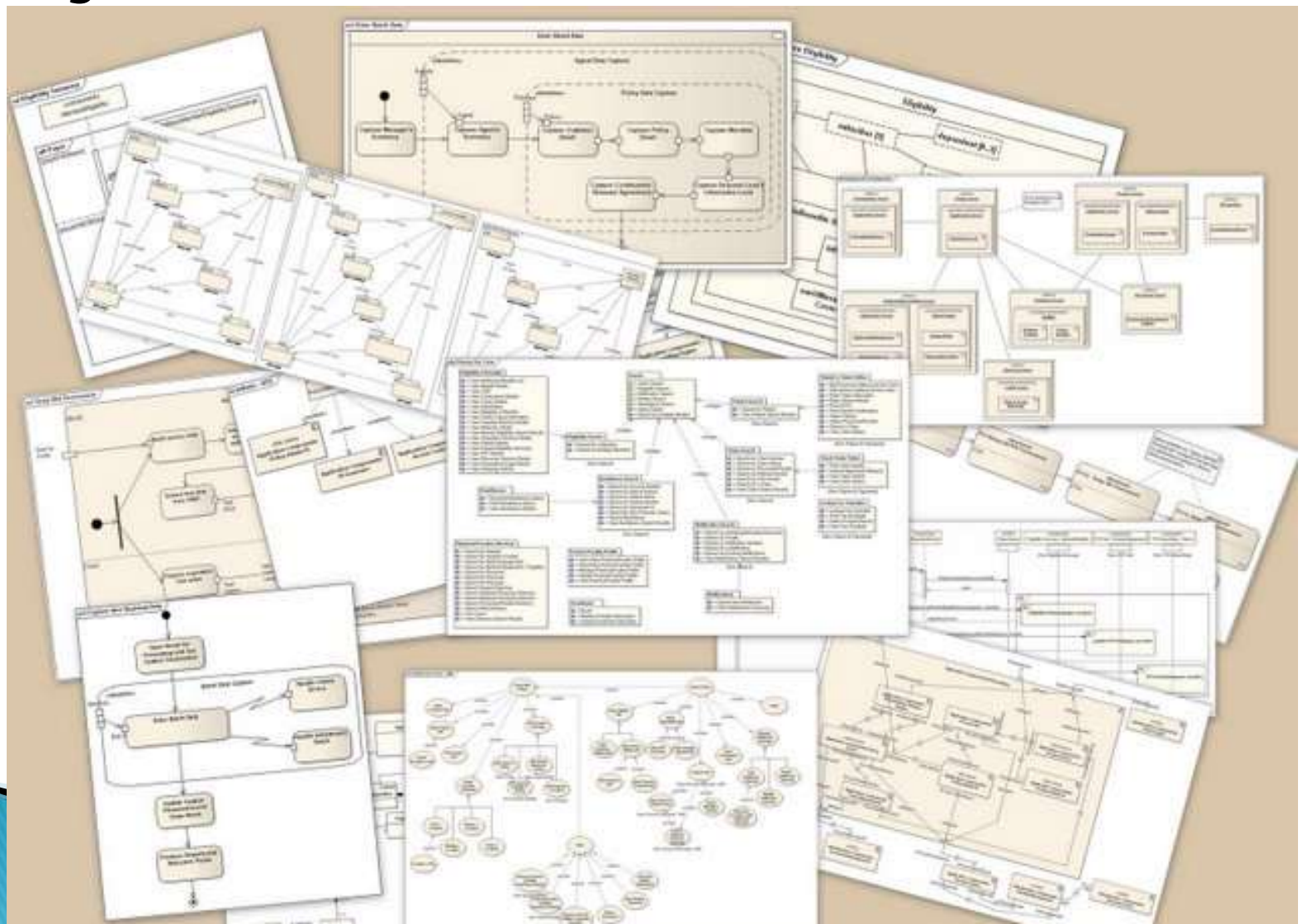
- OOSE (Object-oriented software engineering)





Limbaje Grafice 9

- Diagrame UML

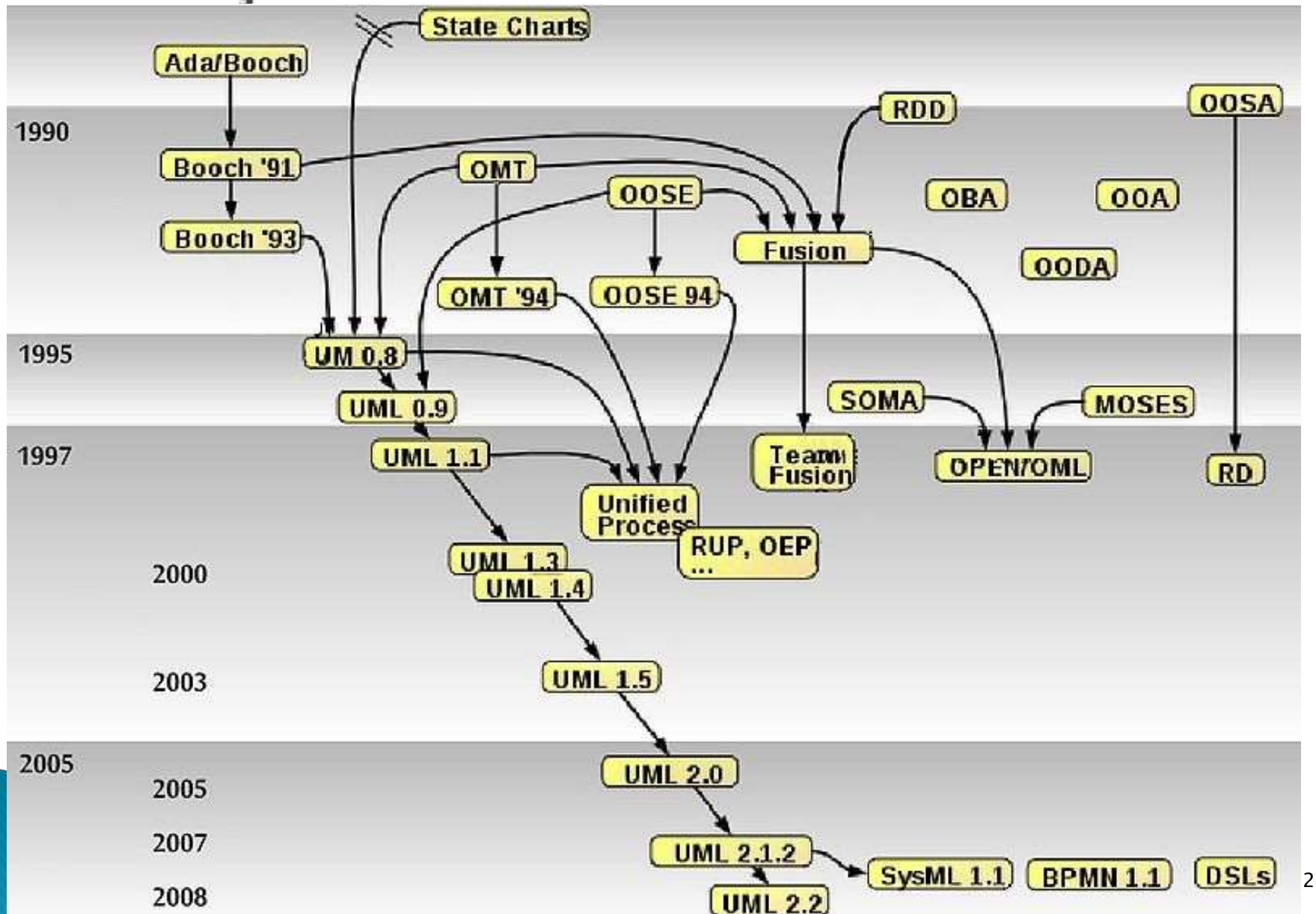


UML – Introducere

- UML (Unified Modeling Language) este succesorul celor mai bune trei limbaje OO de modelare anterioare:
  - Booch (Grady Booch)
  - OMT (Ivar Jacobson)
  - OOSE (James Rumbaugh)

- UML se constituie din unirea acestor limbaje de modelare şi în plus are o expresivitate mai mare

Evoluţie UML

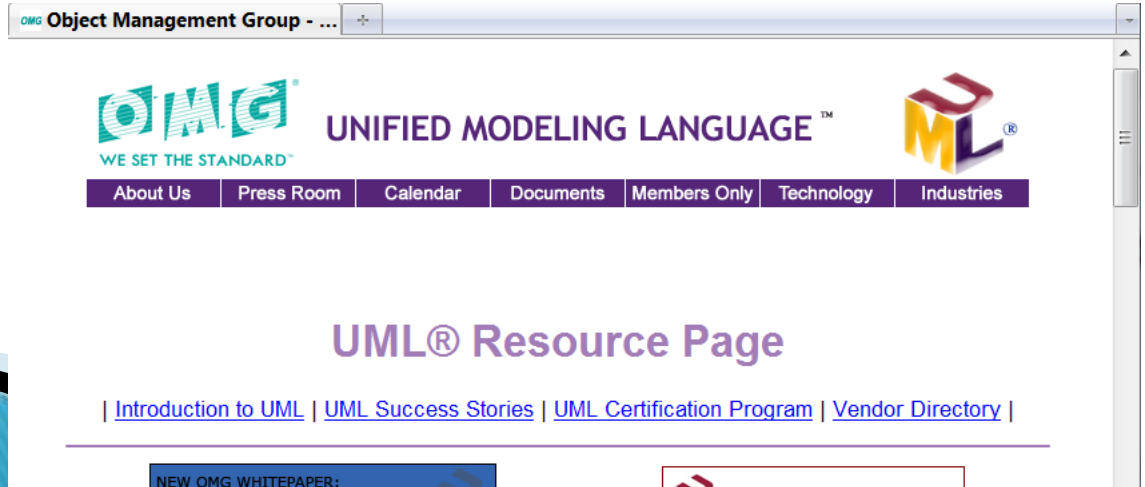


UML – Definiţie (OMG)

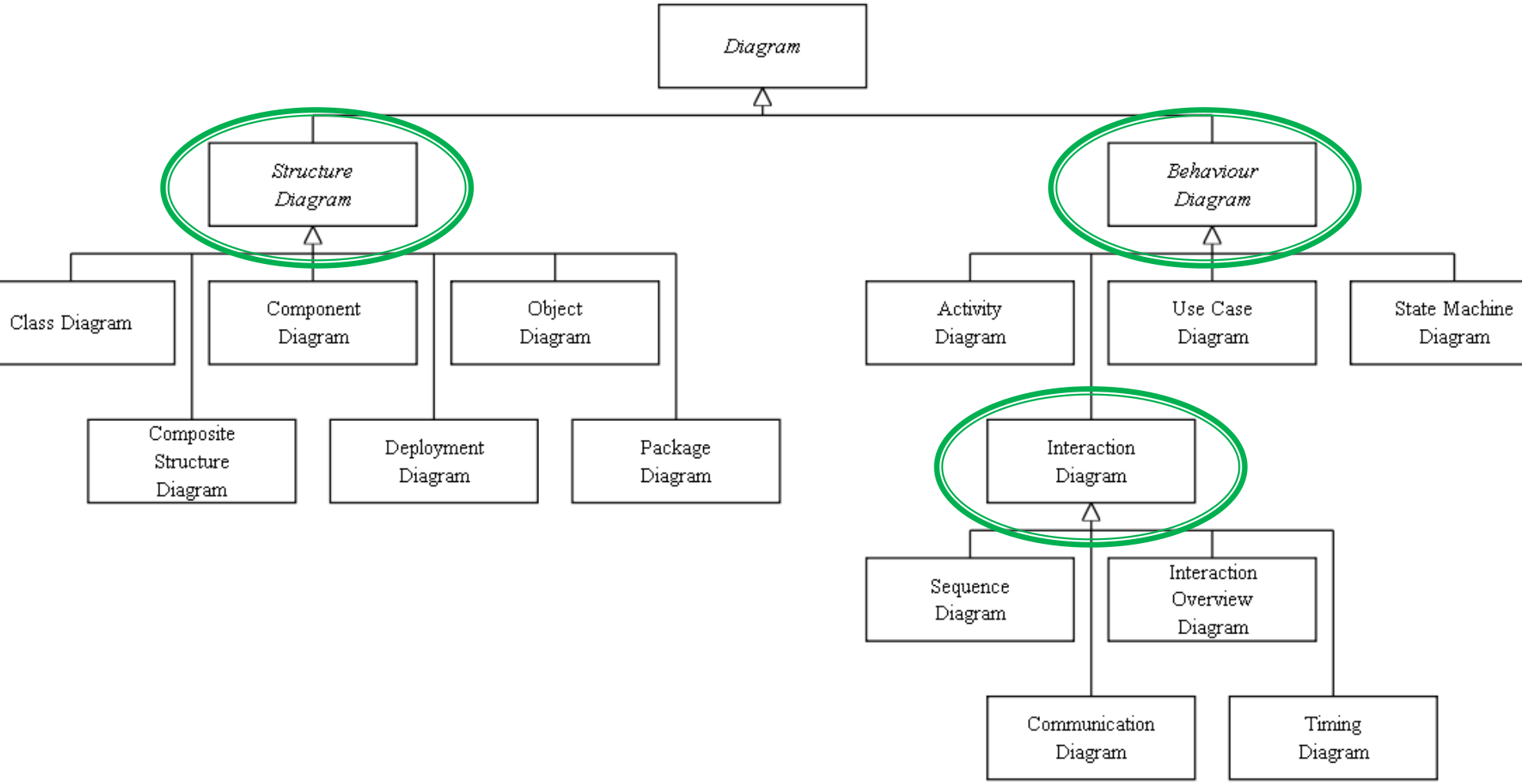
- *"The Unified Modeling Language (UML) is a* ***graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system.***
- *The UML offers a* ***standard way to write a system's blueprints****, including conceptual things such as* ***business processes*** *and* ***system functions*** *as well as concrete things such as* ***programming language statements****,* ***database schemas****, and* ***reusable software components****."*

UML – Standard Internaţional

- Ianuarie 1997 – UML 1.0 a fost propus spre standardizare în cadrul OMG (Object Management Group)
- Noiembrie 1997 – Versiunea UML 1.1 a fost adoptată ca standard de către OMG
- Ultima versiune este UML 2.4.1
- Site-ul oficial: http://www.uml.org

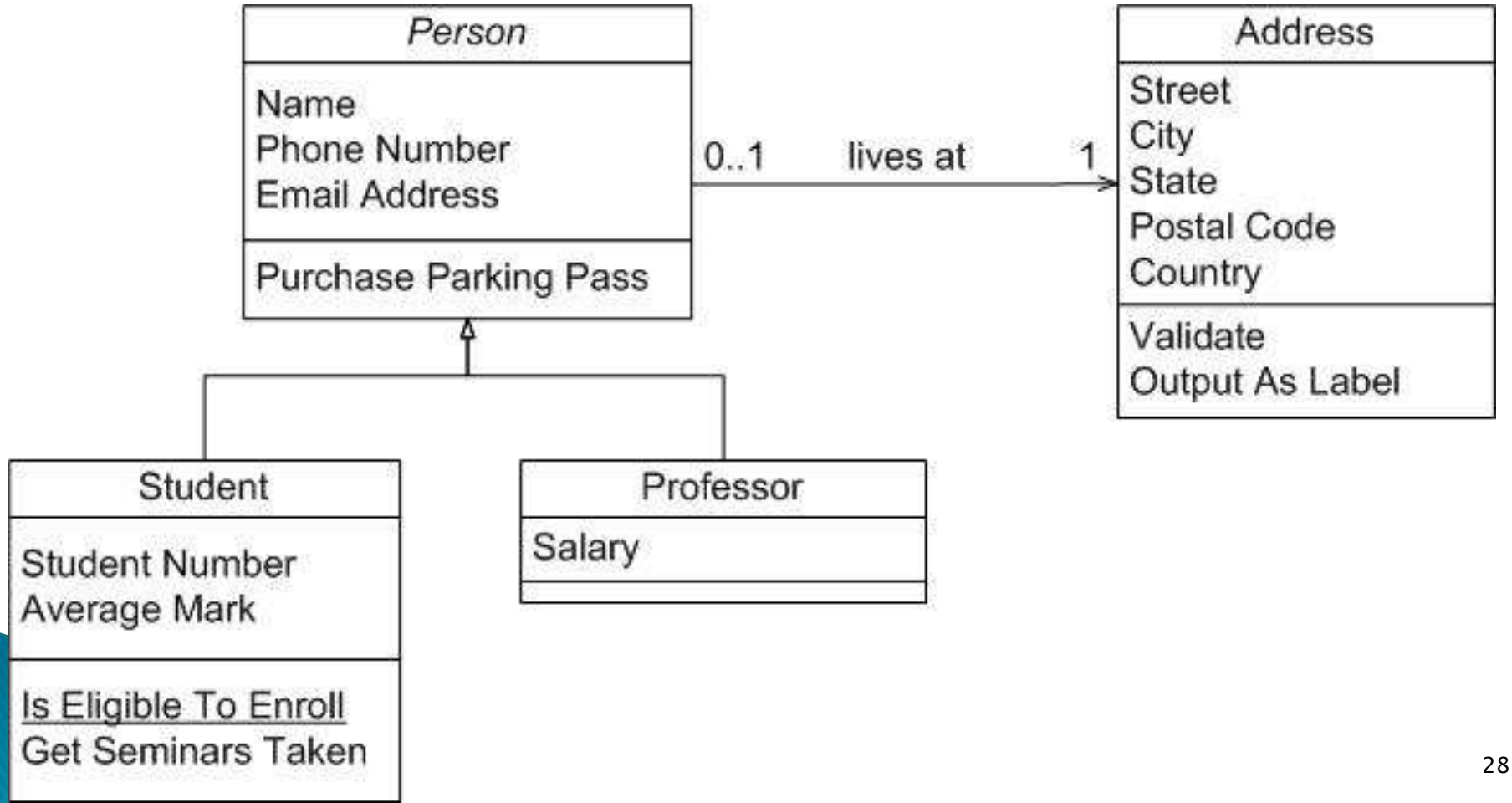


UML2.0 – 13 Tipuri de Diagrame



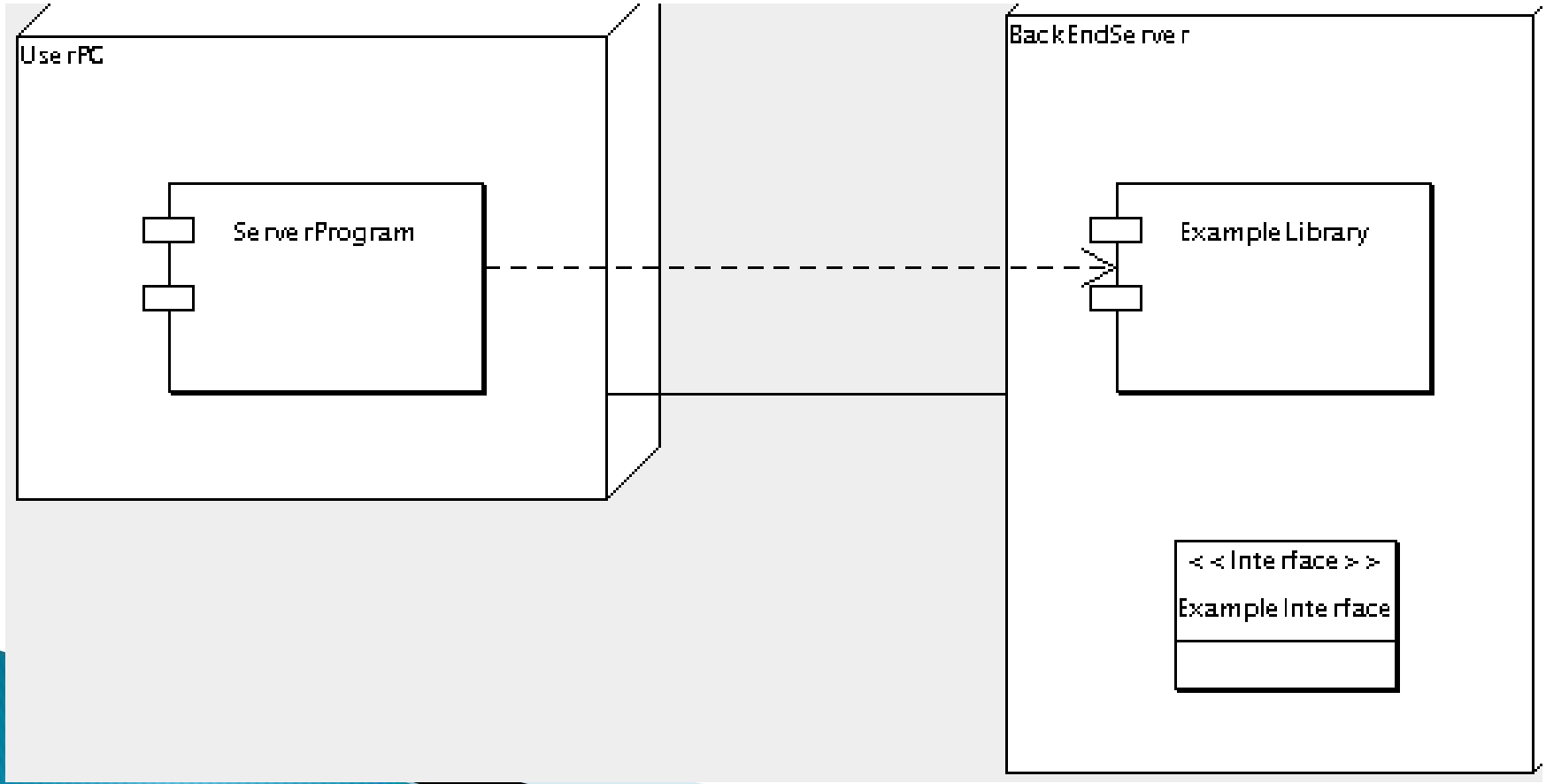
UML2.0 – Diagrame de Structură 1

- **Diagrame de Clasă**: clasele (atributele, metodele) şi relaţiile dintre clase



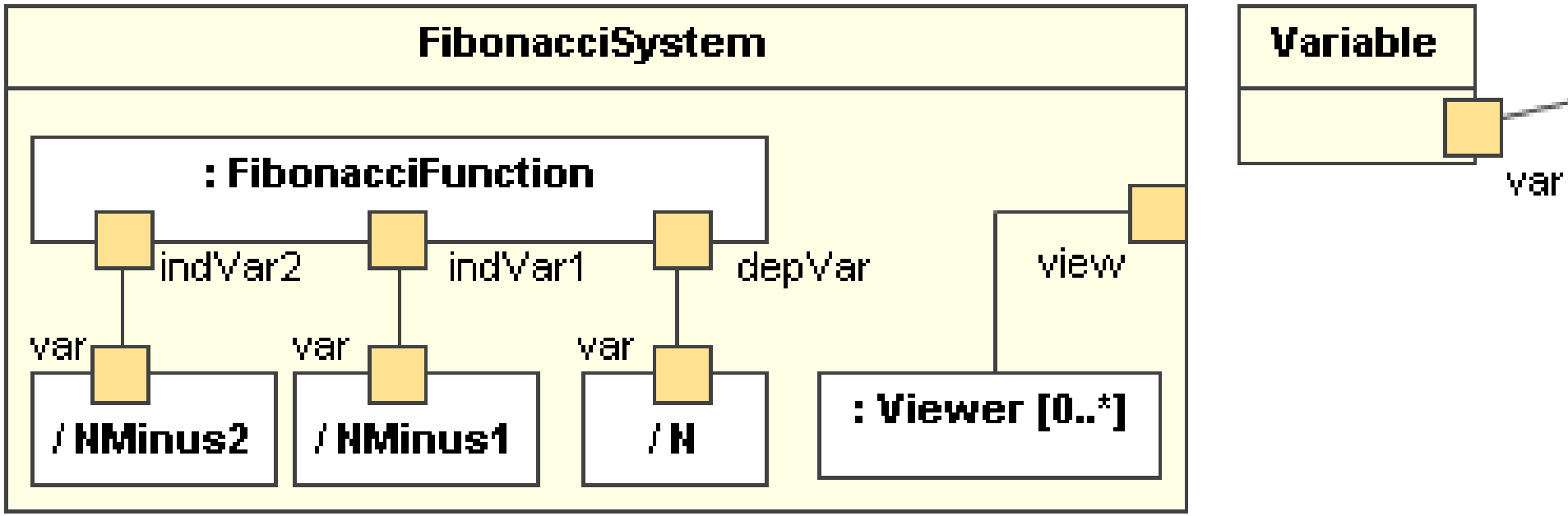
UML2.0 – Diagrame de Structură 2

- **Diagramă de Componente**: componentele sistemului şi legăturile între componente



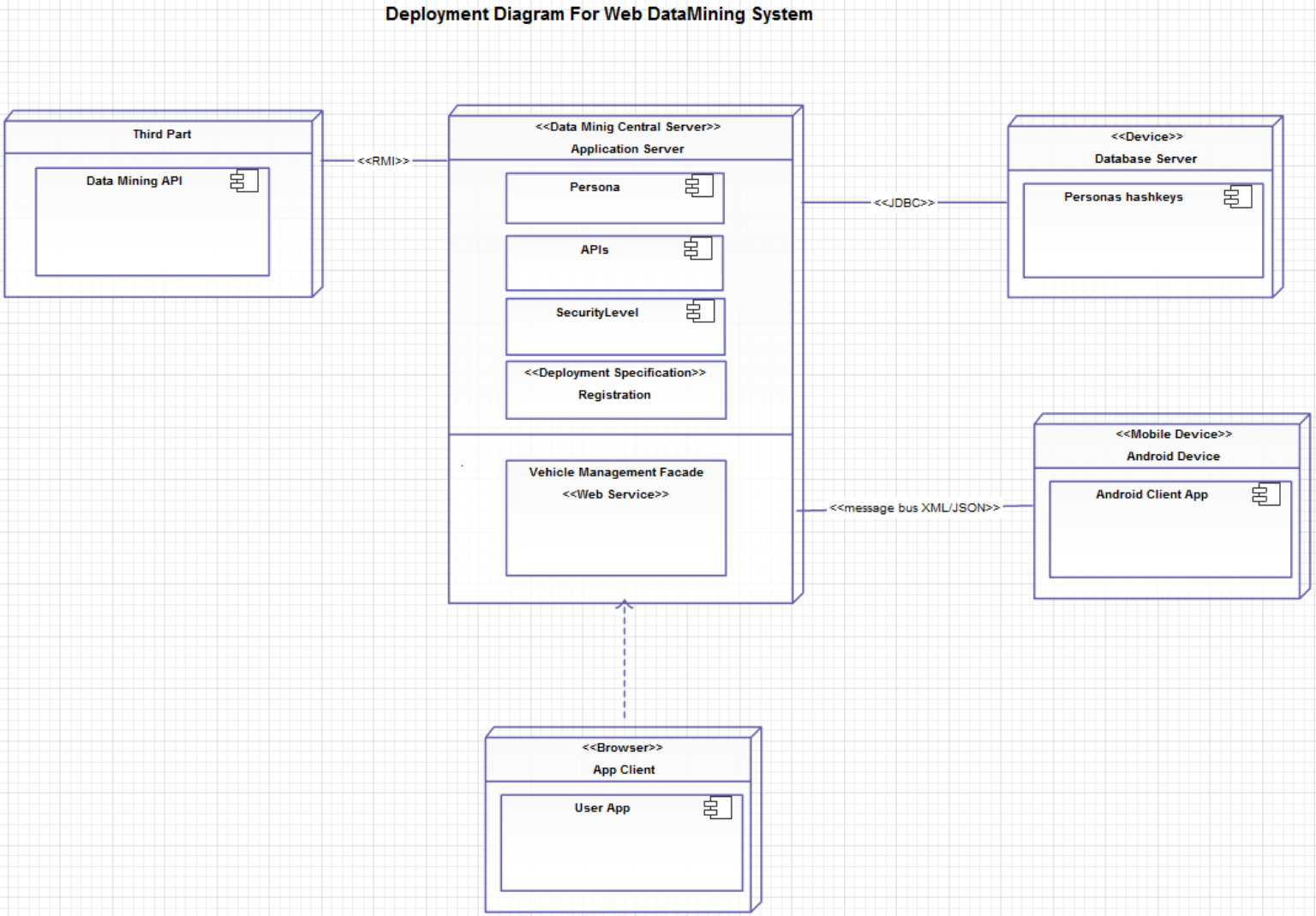
UML2.0 – Diagrame de Structură 3

- **Diagrame structură composită**: structura internă



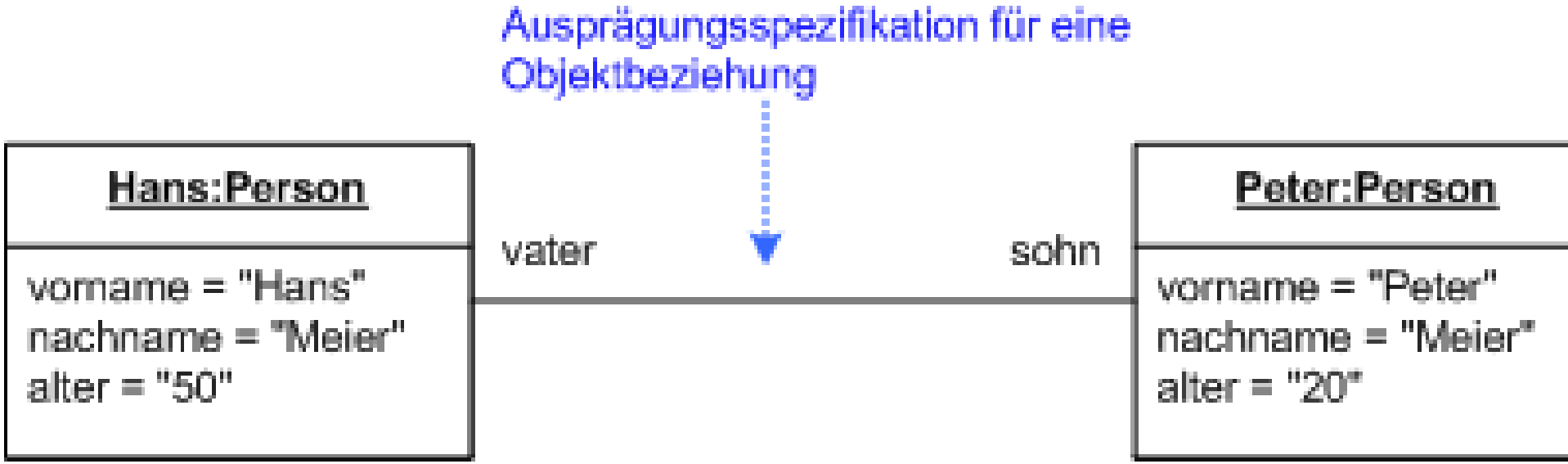
UML2.0 – Diagrame de Structură 4

- **Diagramă de Deployment**: modelarea structurii hardware



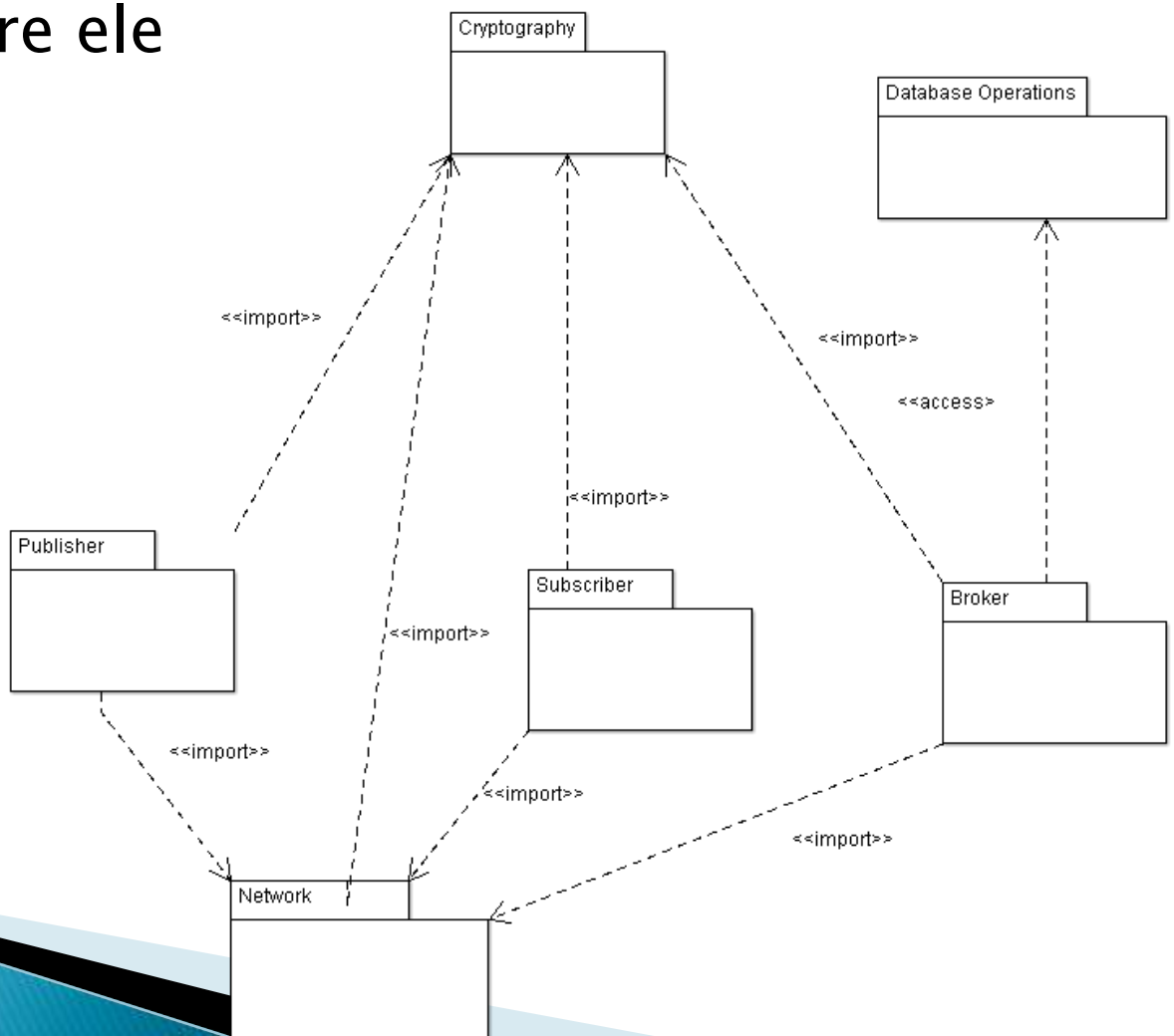
UML2.0 – Diagrame de Structură 5

- **Diagramă de obiecte**: structura sistemului la un moment dat



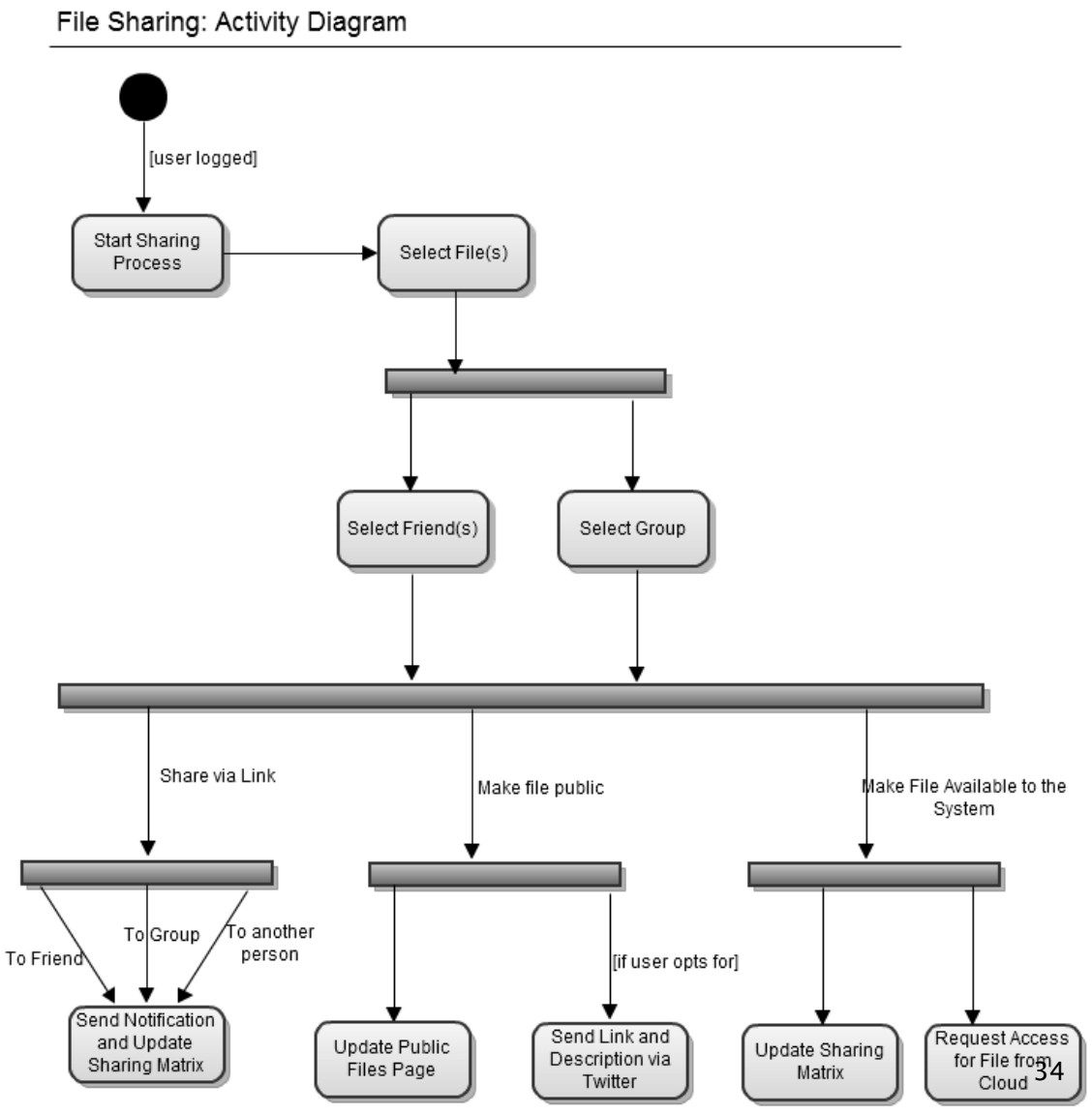
UML2.0 – Diagrame de Structură 6

- **Diagramă de pachete:** împărţirea sistemului în pachete şi relaţiile dintre ele



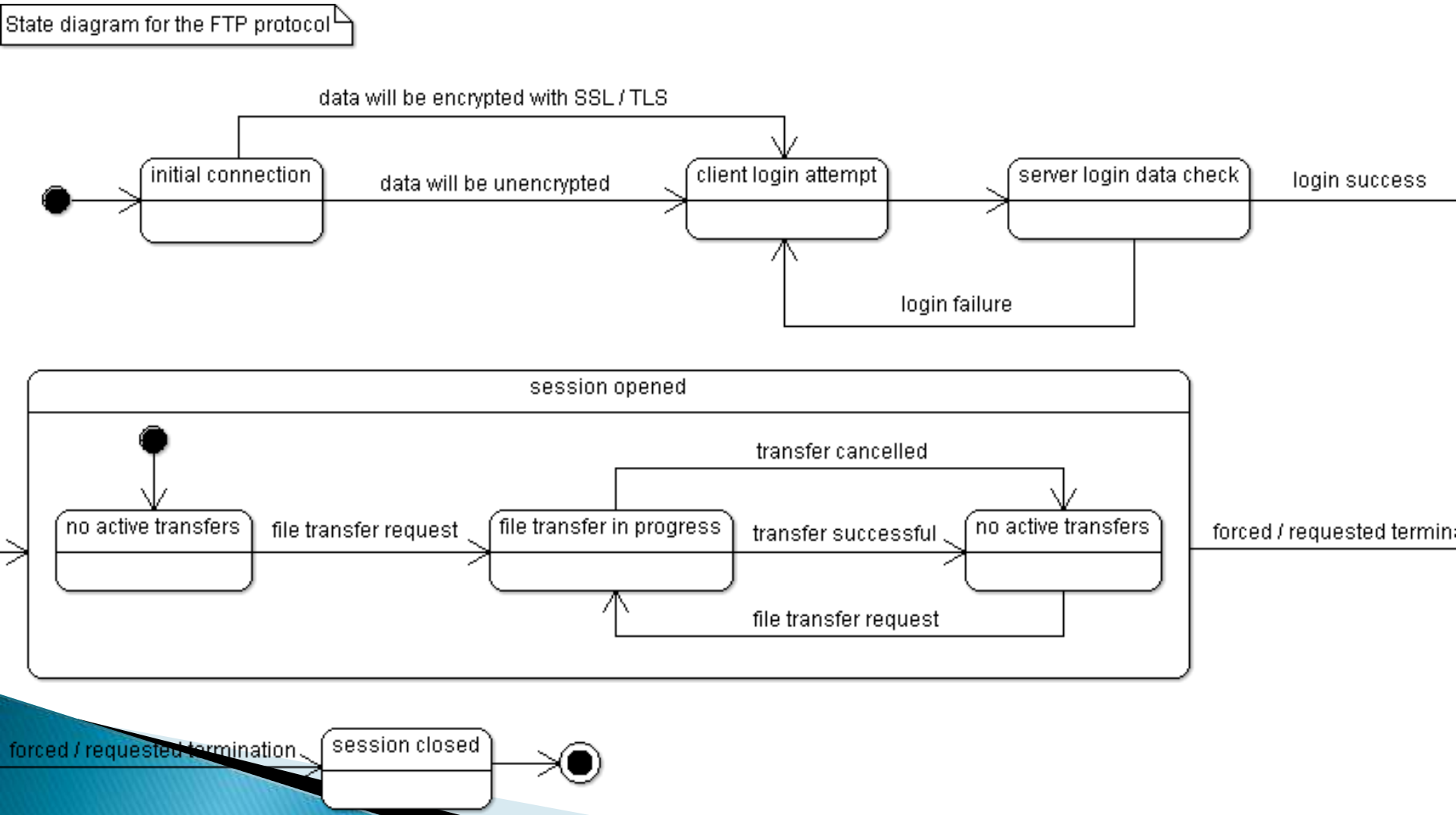
UML 2.0 – Diagrame Comportamentale 1

- **Diagrame de activitate**: prezentare business şi a fluxului de activităţi



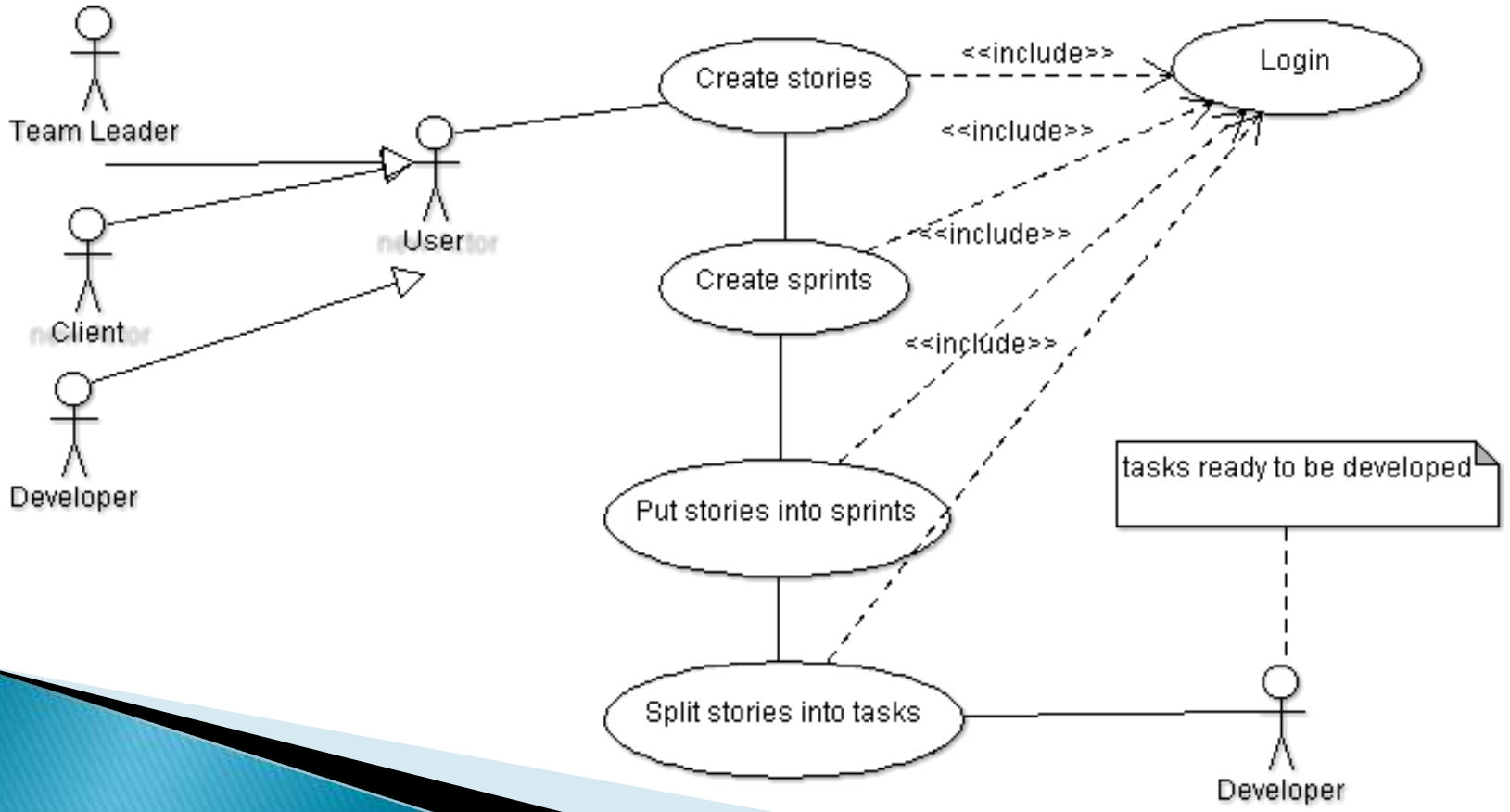
UML 2.0 – Diagrame Comportamentale 2

- **Diagrame de stare**: pentru a prezenta stările obiectelor



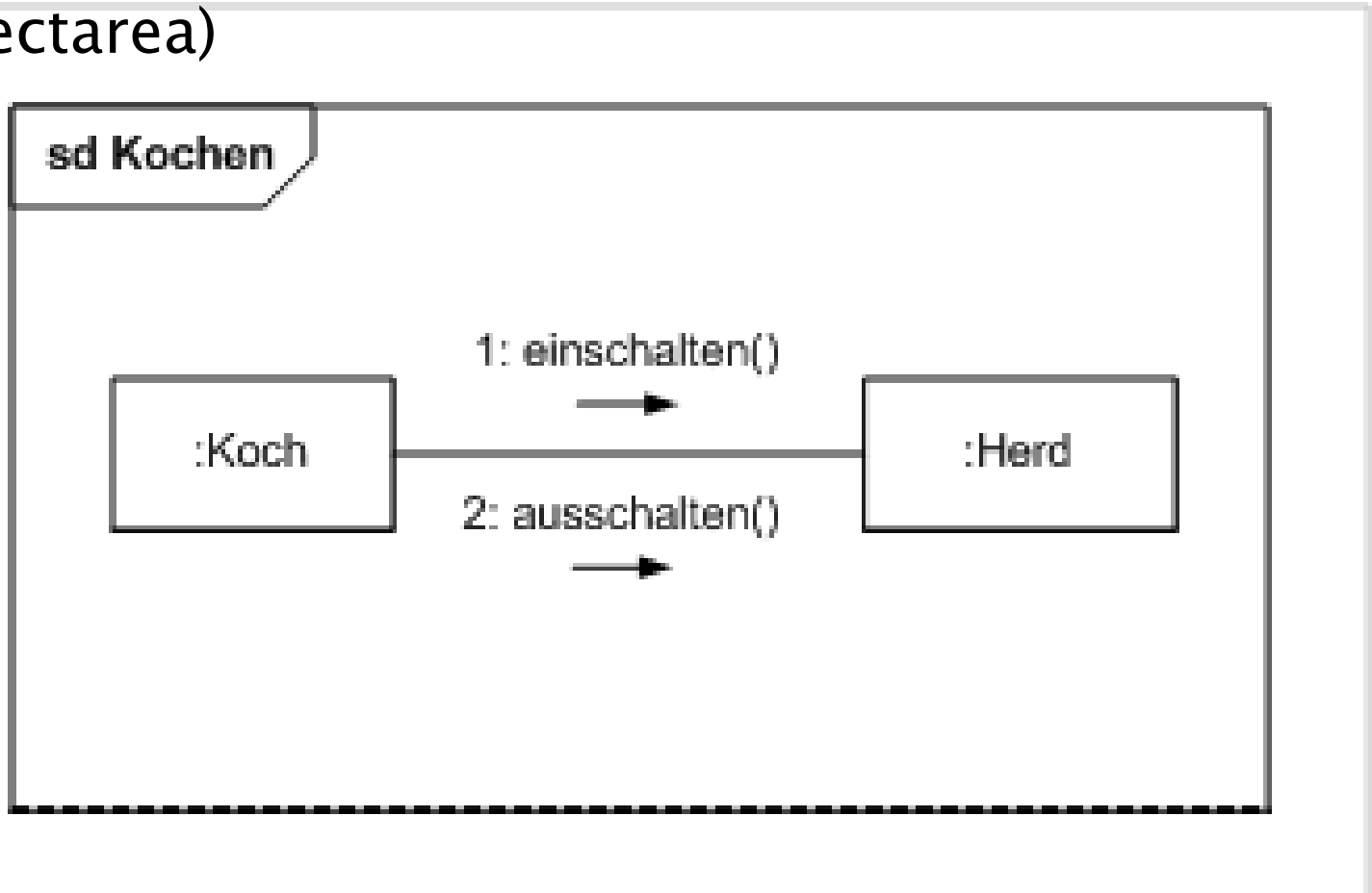
UML 2.0 – Diagrame Comportamentale 3

- **Diagrame Use Case**: prezintă funţionalităţile sistemului folosind actori, use case-uri şi dependenţe între ele



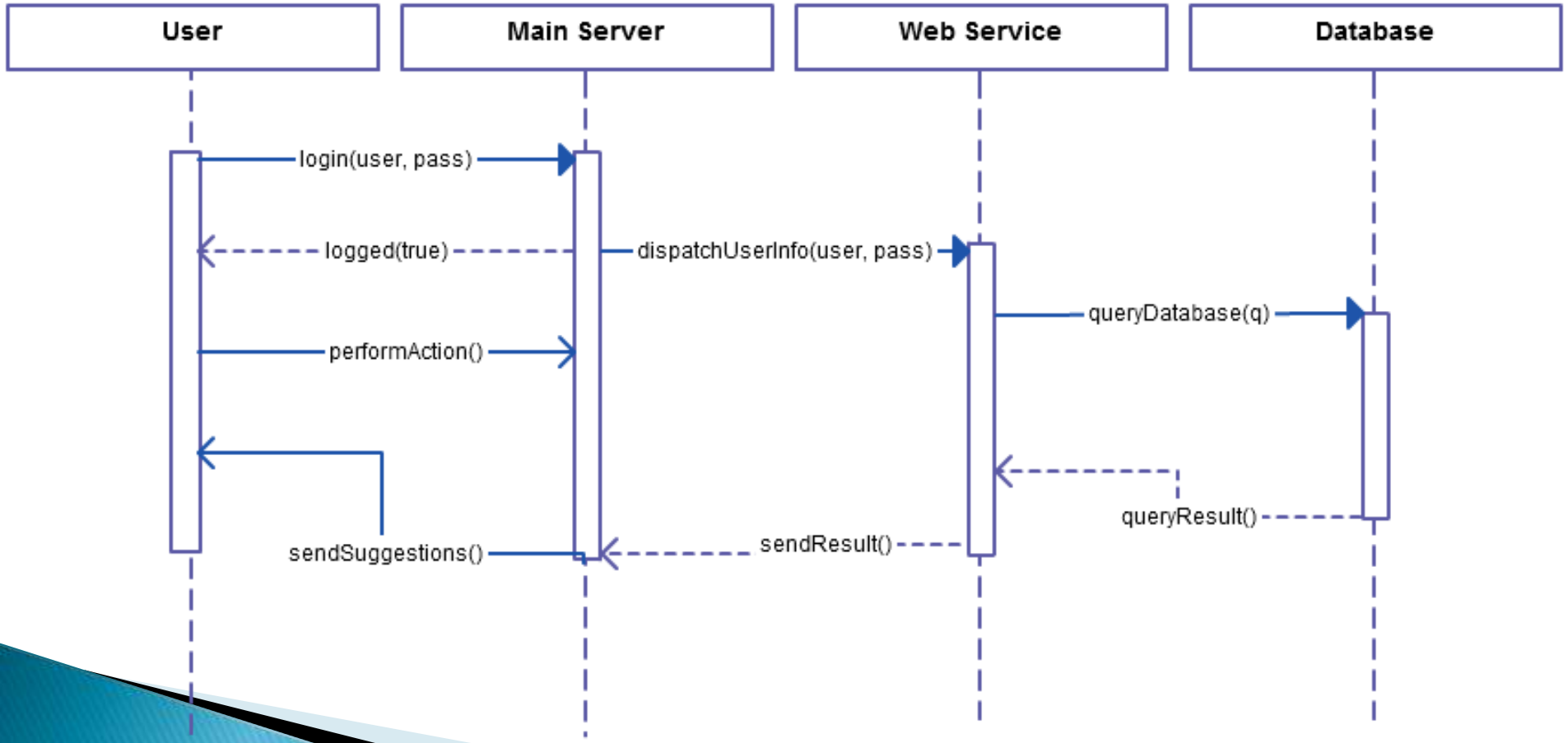
UML 2.0 – Diagrame de interacţiuni 1

- **Diagrama de comunicare**: arată interacţiunile între obiecte (comportamentul dinamic al sistemului) (**actori**: bucătar, aragaz, **acțiuni**: gătirea, aprinderea, deconectarea)



UML 2.0 – Diagrame de interacţiuni 2

- **Diagramă de secvenţă**: prezintă modul în care obiectele comunică între ele din punct de vedere al trimiterii de mesaje



[online diagramming & design] creately.com

Diagrama cazurilor de utilizare (*Use Case Diagram*)

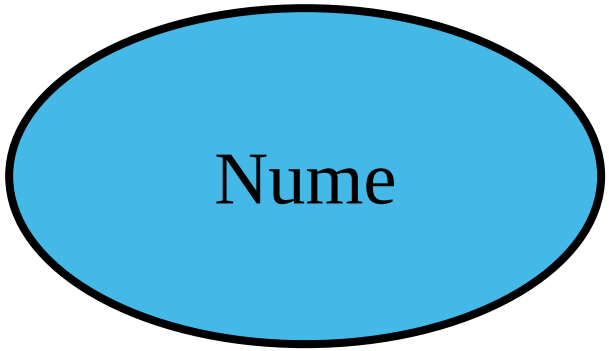
- Este o diagramă comportamentală care captează cerinţele sistemului
- Delimitează graniţele sistemului
- Punctul de plecare îl constituie scenariile de folosire a sistemului din fişa cerinţelor
- Poate prezenta:
  - specificarea cerinţelor (externe) din punctul de vedere al utilizatorului
  - specificarea funcţionalităţii sistemului din punctul de vedere al sistemului
- Conţine:
  - **UseCase**-uri = funcţionalităţi ale sistemului
  - **Actori** = entităţi externe cu care sistemul interacţionează
  - **Relaţii**

UseCase

- **Este o descriere a unei mulţimi de secvenţe de acţiuni** (incluzând variante) pe care un program le execută atunci când interacţionează cu entităţile din afara lui (*actori*) şi care conduc la obţinerea unui rezultat observabil
- Poate fi un sistem, un subsistem, o clasă, o metodă
- **Reprezintă o funcţionalitate a programului**
- Precizează ce face un program sau subprogram
- **Nu precizează cum se implementează o funcţionalitate**
- Identificarea UseCase-urilor se face pornind de la cerinţele clientului şi analizând descrierea problemei.

UseCase – Reprezentare

- Notaţie
- Atribute
  - Nume = **fraza verbală ce denumeşte o operaţie sau un comportament** din domeniul problemei.
- Restricţii
  - Numele este unic

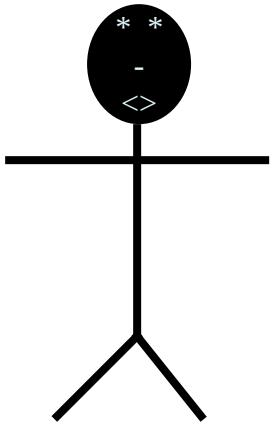


Actor

- Reprezintă **un rol** pe care utilizatorii unui UseCase îl joacă atunci când interacţionează cu acesta
- Este o entitate exterioară sistemului
- Interacţionează cu sistemul:
  - Iniţiază execuţia unor cazuri de utilizare
  - Oferă funcţionalitate pentru realizarea unor cazuri de utilizare
- Poate fi:
  - Utilizator (uman)
  - Sistem software
  - Sistem hardware

Actor – Reprezentare

- Notaţie
- Atribute
- Nume = **indică rolul** pe care actorul îl joacă în interacţiunea cu un UseCase
- Restricţii
  - Numele este unic



Relaţii

- Se stabilesc între două elemente

- Tipuri de relaţii:
  - **Asociere**: Actor – UseCase, UseCase – UseCase
  - **Generalizare**: Actor – Actor, UseCase – UseCase
  - **Dependenţă**: UseCase – UseCase (<<include>>, <<extend>>)

Relaţia de Asociere

- Modelează o comunicare între elementele pe care le conectează

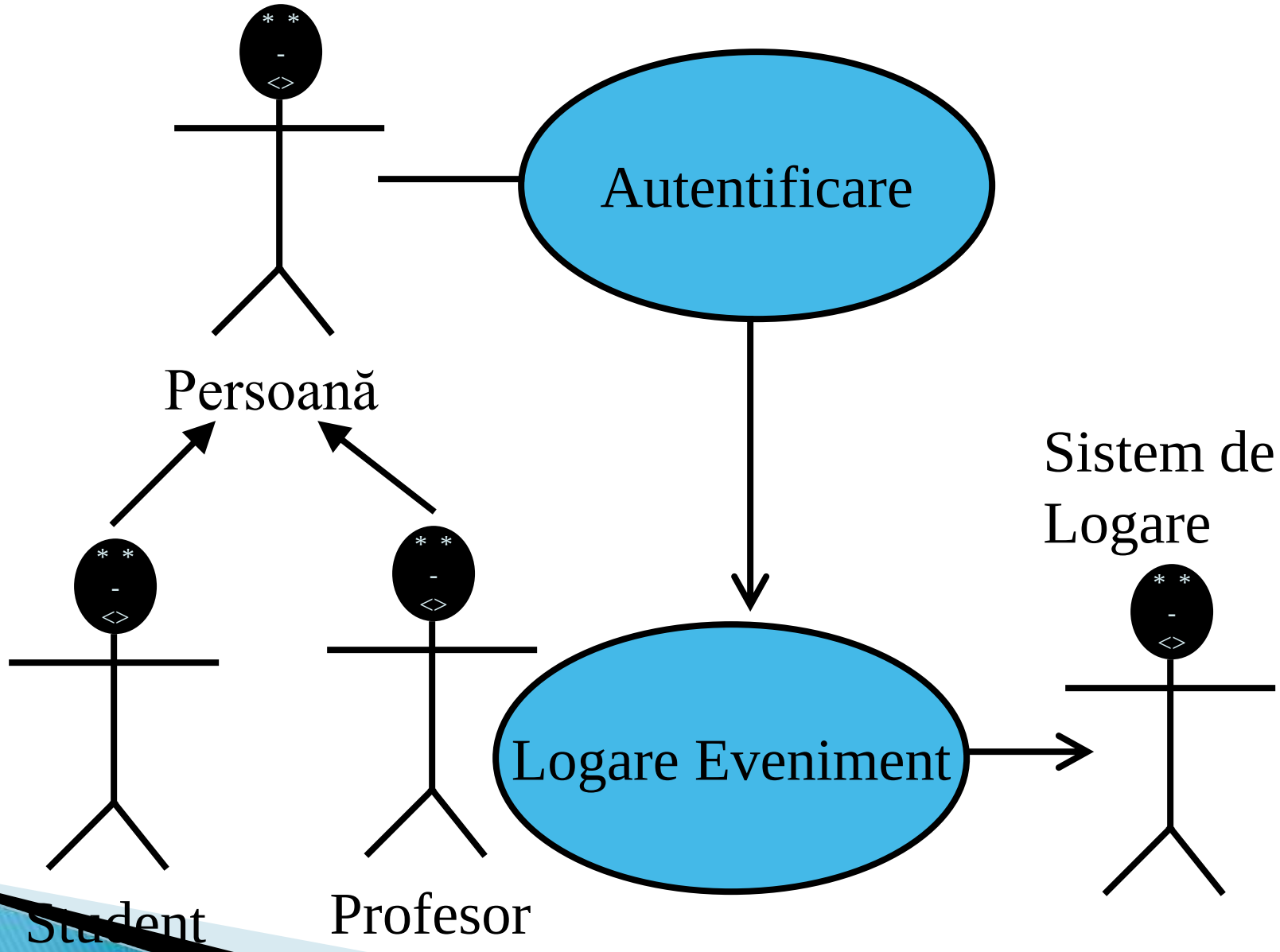
- Poate sa apară între
  - **un actor şi un UseCase** (actorul iniţiază execuţia cazului de utilizare sau oferă funcţionalitate pentru realizarea acestuia)
  - **două UseCase-uri** (transfer de date, trimitere de mesaje/semnale)

- **Notaţie**

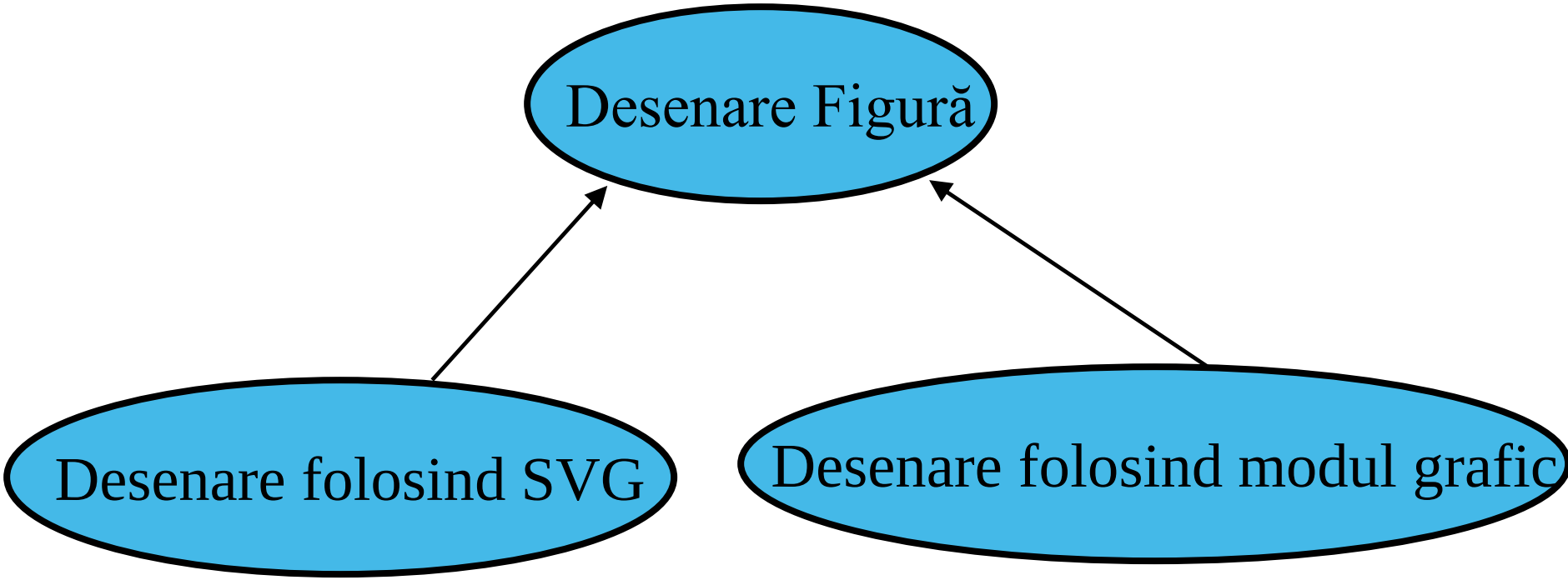
Relaţia de Generalizare

- **Se realizează între elemente de acelaşi tip ⇒ ierarhii**
- Modelează situaţii în care un element este un caz particular al altui element
- Elementul particular moşteneşte relaţiile în care este implicat elementul general
- **Notaţie:**

Exemplu 1


Autentificare
Persoană
Sistem de Logare
Logare Eveniment
Student
Profesor

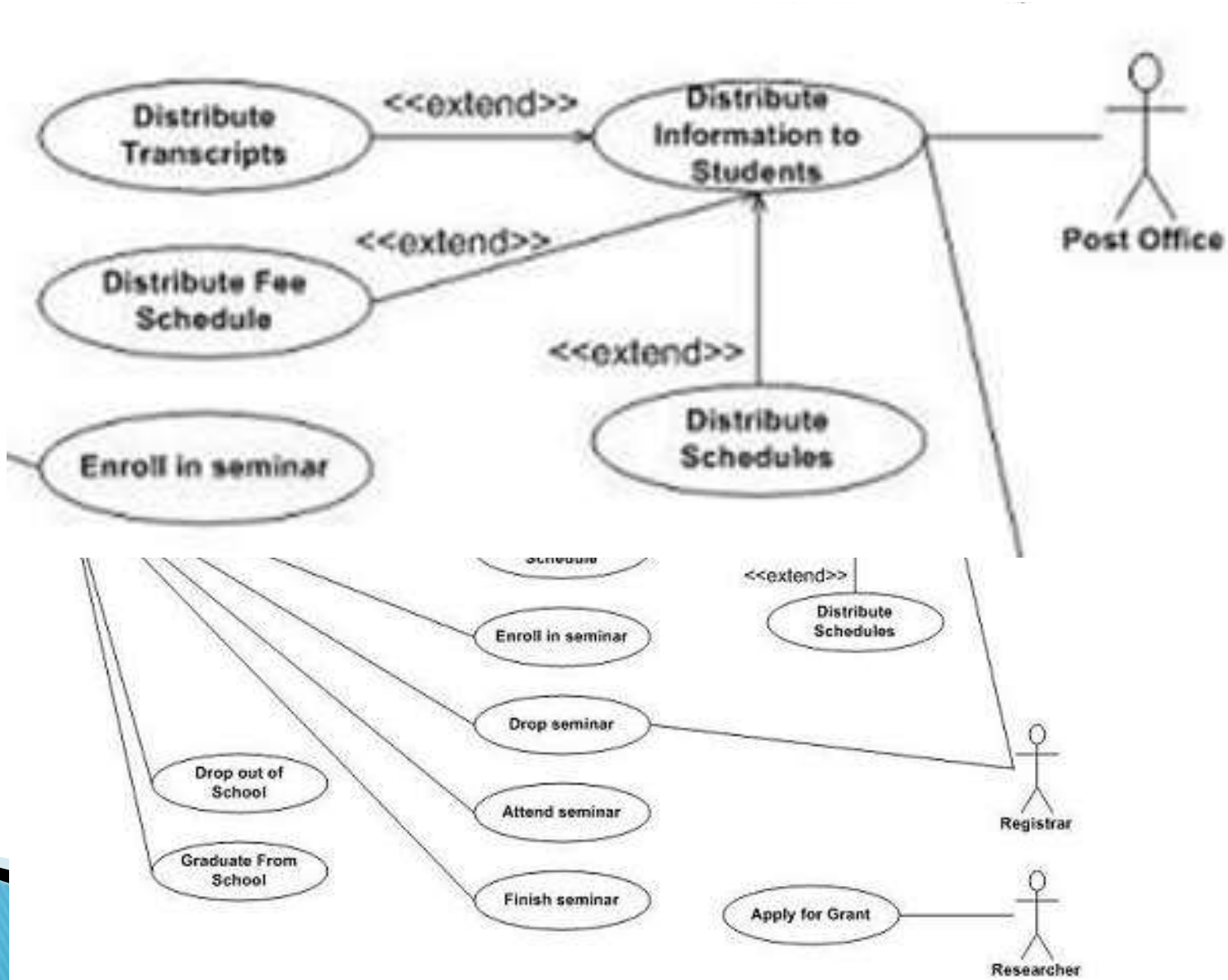
Relaţia de Generalizare



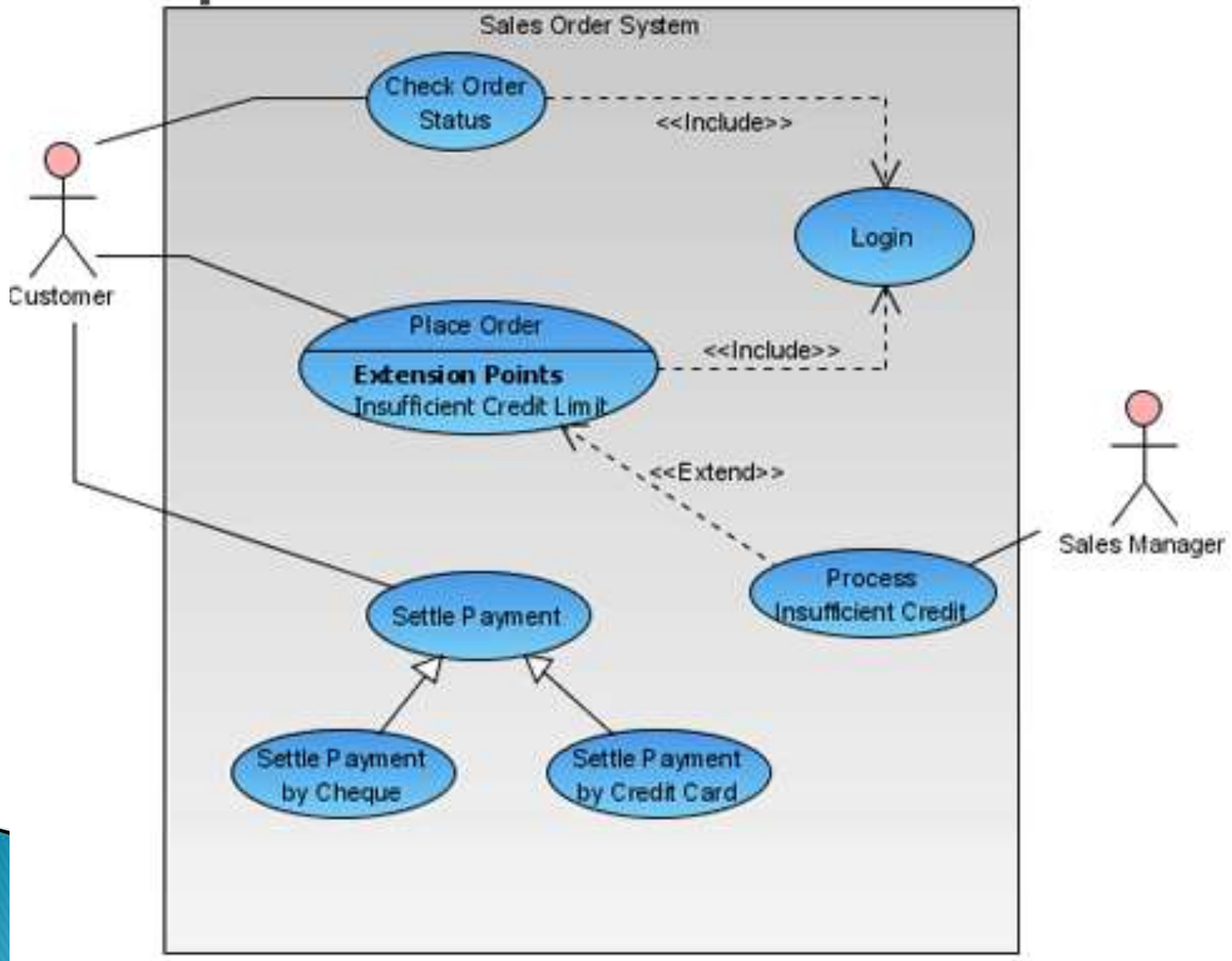
Relaţia de Dependenţă

- Apare între două UseCase-uri
- Modelează situaţiile în care
  - Un UseCase **foloseşte** comportamentul definit în alt UseCase (<<include>>)
  - Comportamentul unui UseCase **poate fi extins** de către un alt UseCase (<<extend>>)
- **Notaţie** <- - - - - - - - - - - - - -

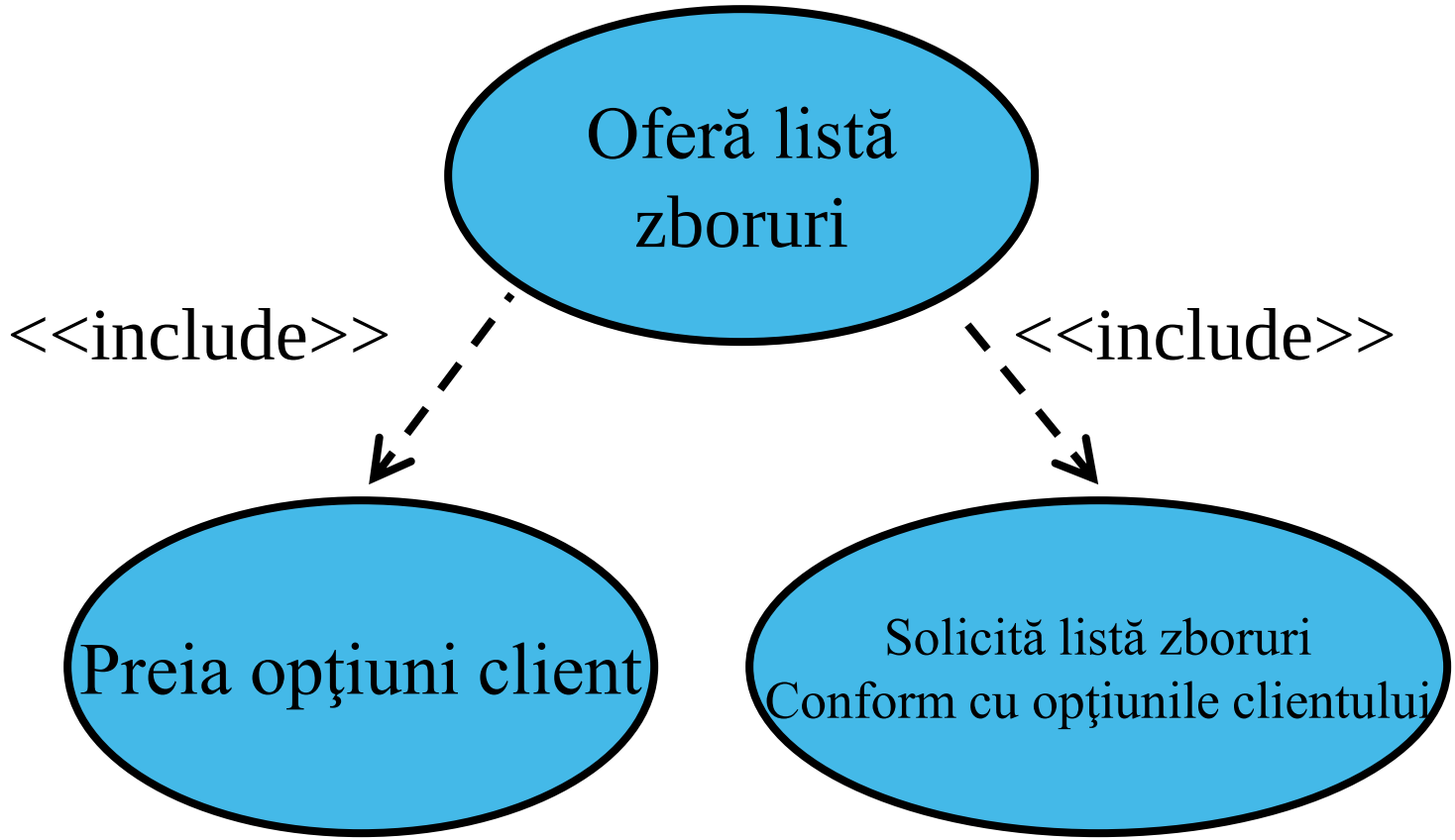
Exemplul 2


Distribute Transcripts
<<extend>>
Distribute Information to Students
Post Office
<<extend>>
Distribute Fee Schedule
<<extend>>
Distribute Schedules
Enroll in seminar
<<extend>>
Distribute Schedules
Enroll in seminar
Drop seminar
Drop out of School
Attend seminar
Registrar
Graduate From School
Finish seminar
Apply for Grant
Researcher

Exemplul 3


Sales Order System
Check Order Status
<<Include>>
Login
Customer
Place Order
Extension Points
Insufficient Credit Limit
<<Include>>
<<Extend>>
Sales Manager
Process Insufficient Credit
Settle Payment
Settle Payment by Cheque
Settle Payment by Credit Card

Exemplu 4


Oferă listă zboruri
<<include>>
<<include>>
Preia opţiuni client
Solicită listă zboruri
Conform cu opţiunile clientului

Concepte OO – Recapitulare

- **Obiect, Clasă, Instanţă**
- **Obiect:** Entitate care are: identitate, stare, comportament
  - Exemplu: Mingea mea galbena de tenis, cu diametrul de 10 cm, care sare
- **Clasă:** Descriere a unei mulţimi de obiecte cu aceleaşi caracteristici structurale, aceleaşi caracteristici comportamentale
  - Exemplu: mingi care au culoare, diametru, întrebuinţare, sar
- **Instanţă**: un obiect care aparţine unei clase
  - Exemplu: Popescu Viorel este un Student

Orientat Obiect 1

- Este orice abordare ce cuprinde
  - **Încapsularea datelor**
  - **Moştenire**
  - **Polimorfism**

- **Încapsularea datelor** (exemplu clasa Punct)
  - Înseamnă punerea la un loc a datelor (atributelor) şi a codului (metodelor)
  - Datele pot modificate (doar) prin intermediul metodelor
  - Data hiding: nu ne interesează cum se oferă serviciile, ci doar ca se oferă
  - Dacă se schimba structura, sau modul de realizare, interfaţa rămâne neschimbată

Orientat Obiect 2

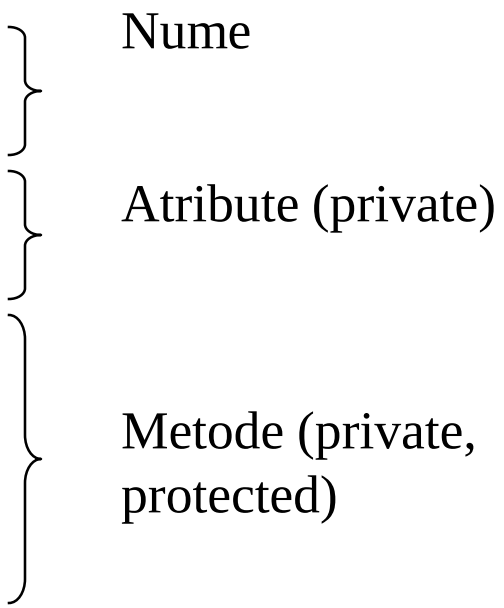
- **Moştenire**:
  - Anumite clase sunt specializări (particularizări) ale altor clase
  - O subclasa are (moşteneşte) caracteristicile super-clasei, pe care le poate extinde într-un anume fel
  - O instanţă a unei clase derivate este în mod automat şi o instanţă a clasei de bază
  - Exemplu (Student – Persoana)
- **Polimorfism**
  - Interpretarea semanticii unui apel de metoda se face de către cel care primeşte apelul
  - Exemplu: Eu spun unei forme: DESENEAZĂ-TE. Ea, daca e pătrat trage 4 linii, daca e cerc, face nişte puncte de jur împrejurul centrului
  - De altfel, nu mă interesează cine, cum face

Diagrama de clase – Class Diagram

- **Scop**:
  - Modelează vocabularul sistemului ce trebuie dezvoltat
  - Surprinde conexiunile semantice sau interacţiunile care se stabilesc între elementele componente
  - Folosită pentru a modela structura unui program
- **Conţine**
  - Clase/Interfeţe
  - Obiecte
  - Relaţii (Asociere, Agregare, Generalizare, Dependenţă)

Clase

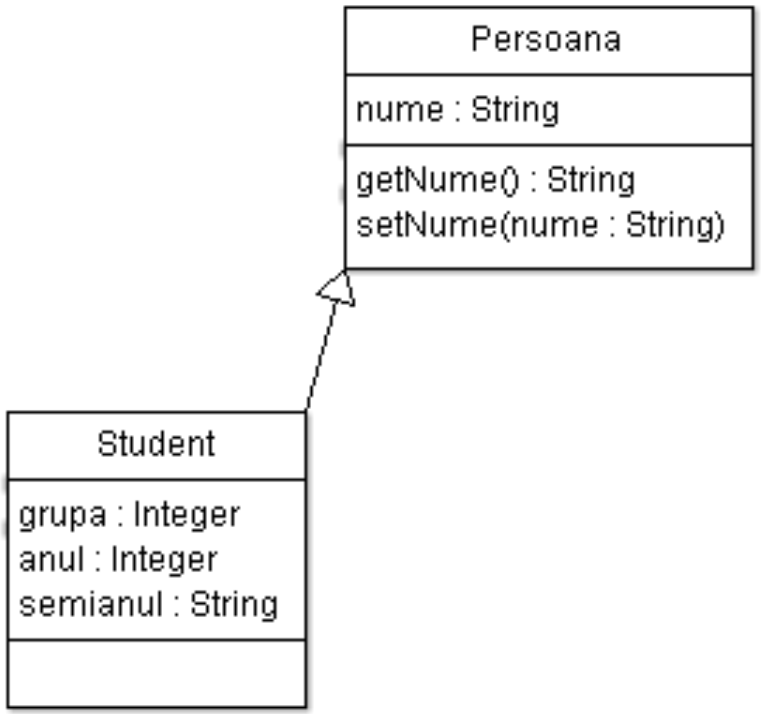
- Modelează vocabularul = identifică conceptele pe care clientul sau programatorul le foloseşte pentru a descrie soluţia problemei
- Elementele unei clase:
  - Nume: identica o clasa
  - Atribute: proprietăţi ale clasei
  - Metode: implementarea unui serviciu care poate cerut oricărei instanţe a clasei



Relaţii – Generalizare – C#

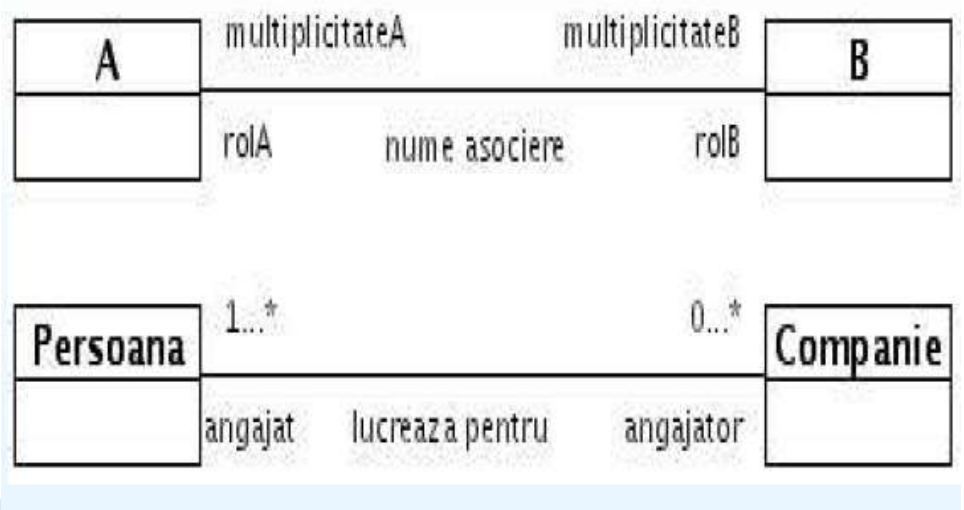
- Modelează conceptul de moştenire între clase
- Mai poartă denumirea de relaţie de tip *is a* (este un/este o)

ArgoUML – Relaţia de generalizare



Relaţii – Asociere

- Exprimă o conexiune semantică sau o interacţiune între obiecte aparţinând diferitelor clase
- Pe măsura ce sistemul evoluează noi legaturi între obiecte pot fi create, sau legături existente pot fi distruse
- O asociere interacţionează cu obiectele sale prin intermediul capetelor de asociere
- Elemente:
  - Nume: descrie relaţia
  - Capete de asociere
    - Nume = rolul jucat de obiect în relaţie
    - Multiplicitate = câte instanţe ale unei clase corespund unei singure instanţe ale celeilalte clase

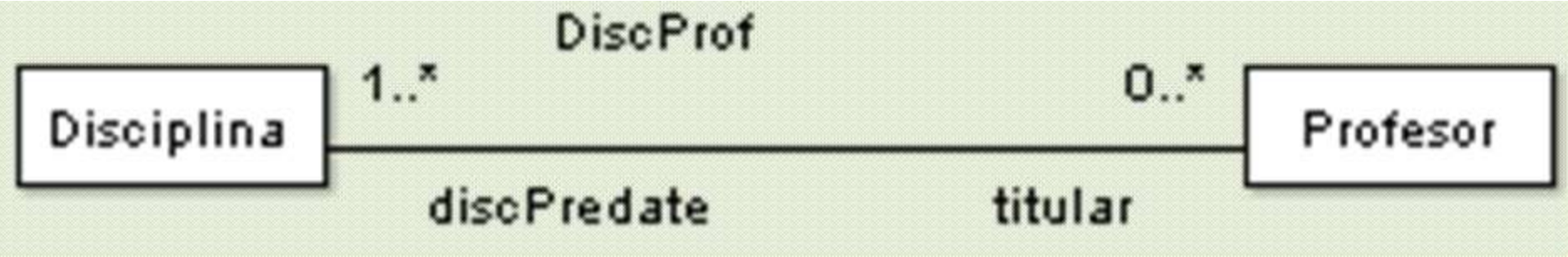


Relaţia de asociere 1

- Relaţia Student – Disciplină
  - **Student**: urmez 0 sau mai multe discipline, cunosc disciplinele pe care le urmez;
  - **Disciplină**: pot fi urmată de mai mulţi studenţi, **nu** cunosc studenţii care mă urmează

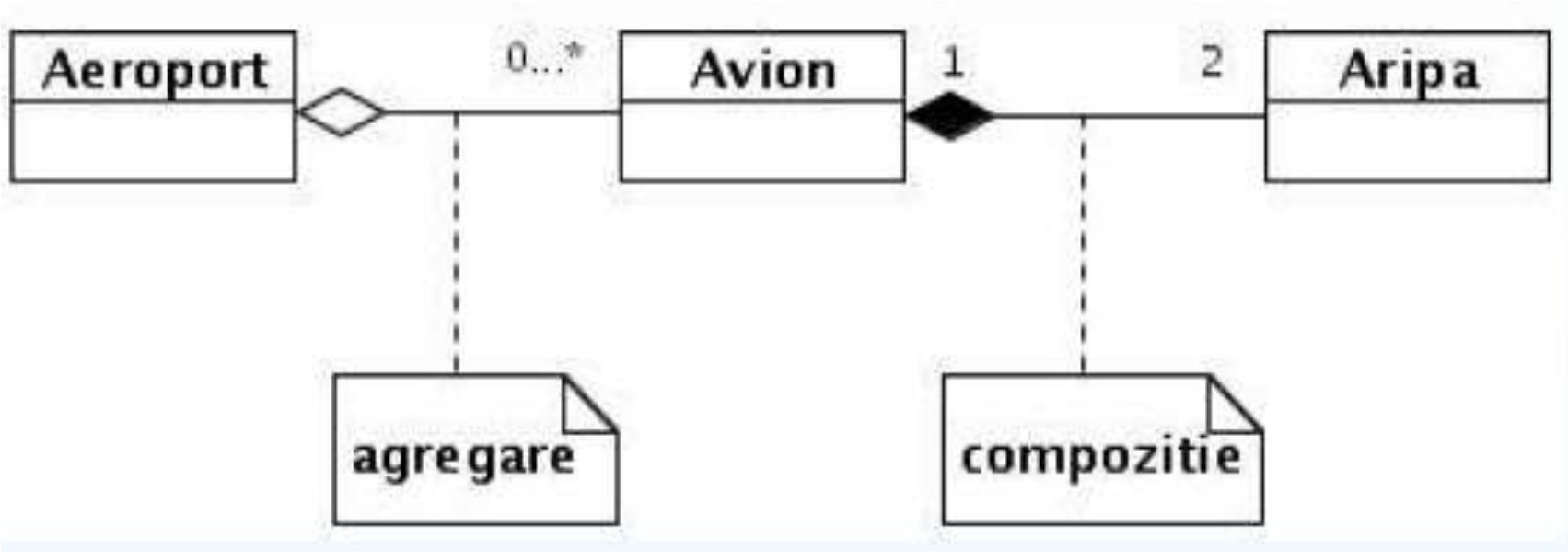
Relaţia de asociere 2

- Relaţia Disciplină – Profesor
  - **Disciplină**: sunt predată de un profesor, îmi cunosc titularul
  - **Profesor**: pot preda mai multe discipline, cunosc disciplinele pe care le predau

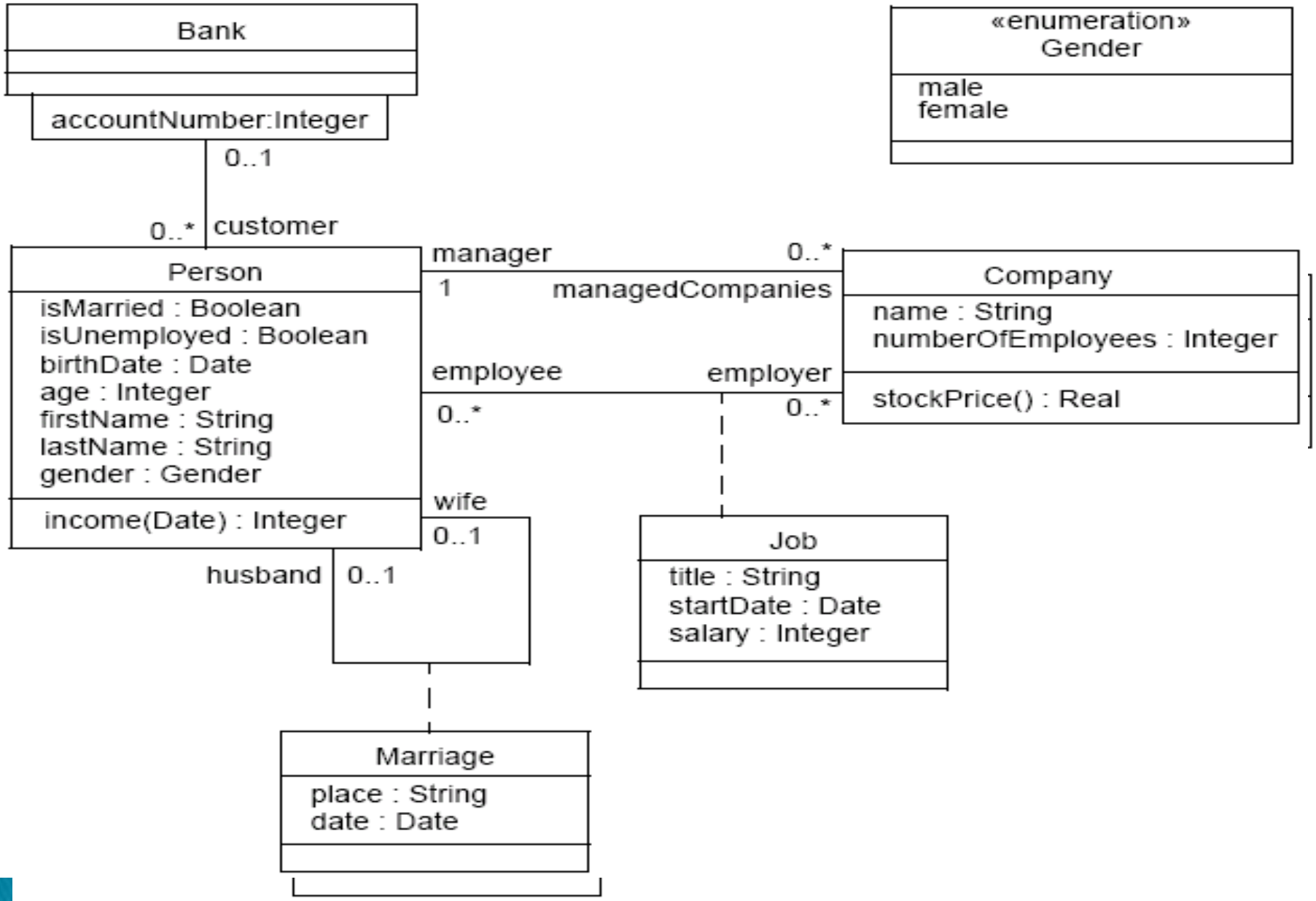


Relaţii – Agregare

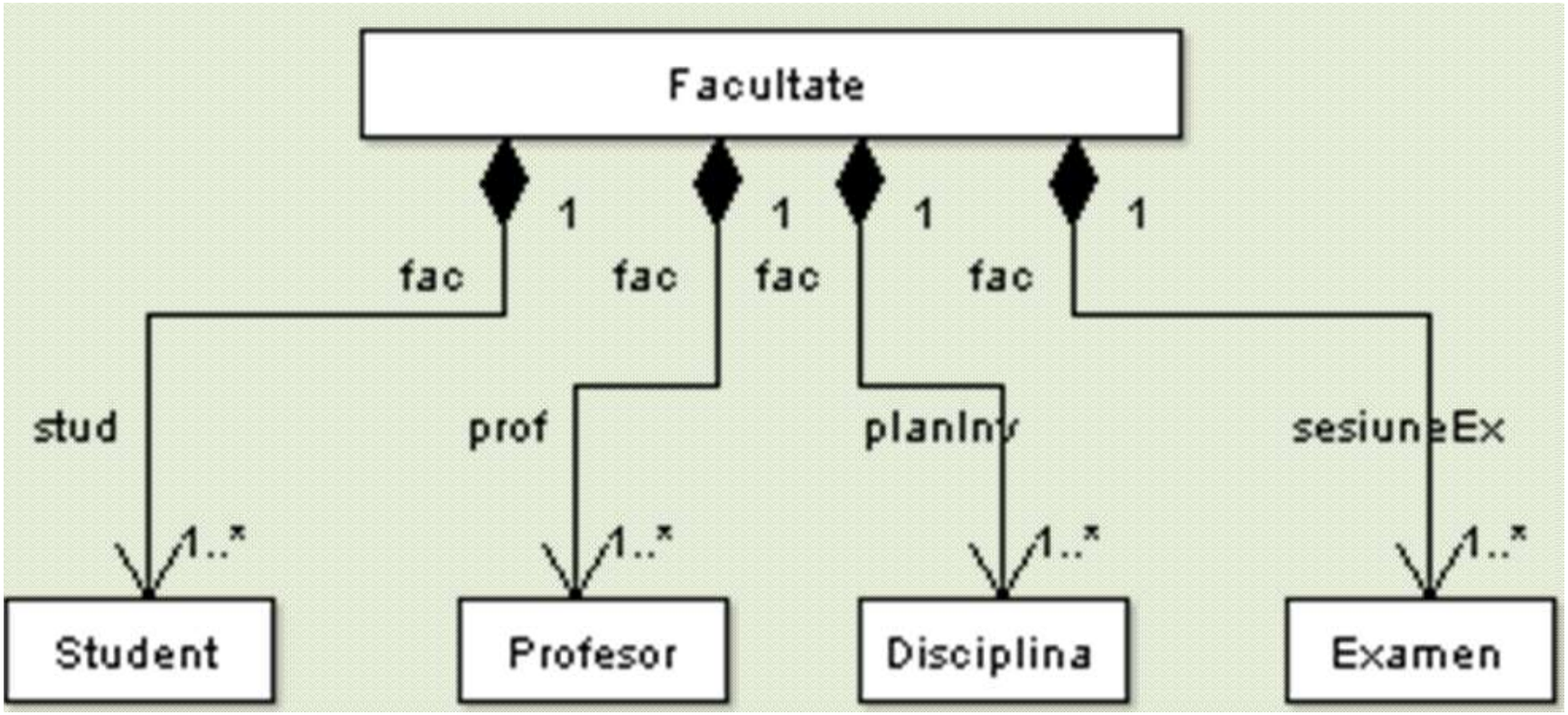
- Este un caz particular al relaţiei de asociere
- Modelează o relaţie de tip parte-întreg
- Poate avea toate elementele unei relaţii de asociere, însă în general se specifică numai multiplicitatea
- Se foloseşte pentru a modela situaţiile între care un obiect este format din mai multe componente.



Exemplul


Bank
accountNumber:Integer
0..1
0..*
customer
«enumeration»
Gender
male
female
Person
isMarried : Boolean
isUnemployed : Boolean
birthDate : Date
age : Integer
firstName : String
lastName : String
gender : Gender
income(Date) : Integer
manager
1
managedCompanies
0..*
Company
name : String
numberOfEmployees : Integer
stockPrice() : Real
employee
0..*
employer
0..*
wife
0..1
husband
0..1
Job
title : String
startDate : Date
salary : Integer
Marriage
place : String
date : Date

Relaţia de Compoziţie (“hasA”)



Studiu de Caz

- Obţinerea Studenţilor Bursieri
  - Actori
  - Scenarii de utilizare

ArgoUML

- Link: http://argouml-downloads.tigris.org/argouml-0.34/
- Varianta “zip” trebuie doar dezarhivată
- Trebuie să aveţi instalat Java
  - În Path sa aveti c:\Program Files\Java\jdk1.6.0_03\bin
  - Variabila JAVA_HOME=c:\Program Files\Java\jdk1.6.0_03\

Concluzii

- Modelare – De ce?
- Limbaje grafice
- UML
  - Structurale: clase
  - Comportamentale: use-case
  - De interacțiuni

Bibliografie

- OMG Unified Modeling Language**TM (OMG UML), Infrastructure,** Version 2.2, May 2008, http://www.omg.org/docs/ptc/08-05-04.pdf
- **ArgoUML User Manual,** A tutorial and reference description, http://argouml-stats.tigris.org/documentation/printablehtml/manual/argomanual.html
- Ovidiu Gheorghieş, Curs IP, Cursurile 3, 4
- Diagrame UML, Regie.ro

Links

- OOSE: http://cs-exhibitions.uni-klu.ac.at/index.php?id=448
- ArgoUML: http://argouml-stats.tigris.org/nonav/documentation/manual-0.22/
- Wikipedia

Ingineria Programării

Cursul 4 – 13 Martie 2017

adiftene@info.uaic.ro

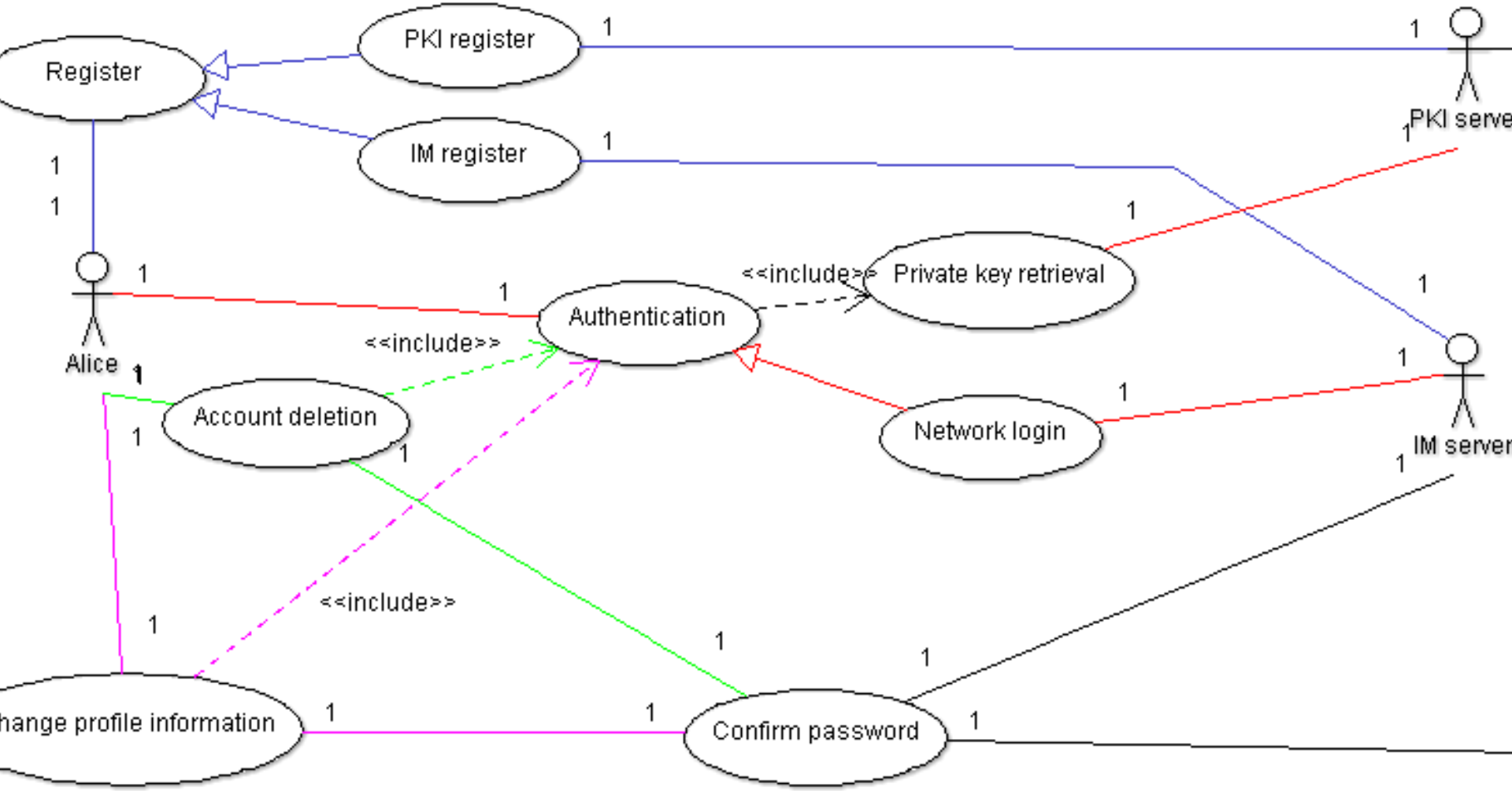
Cuprins

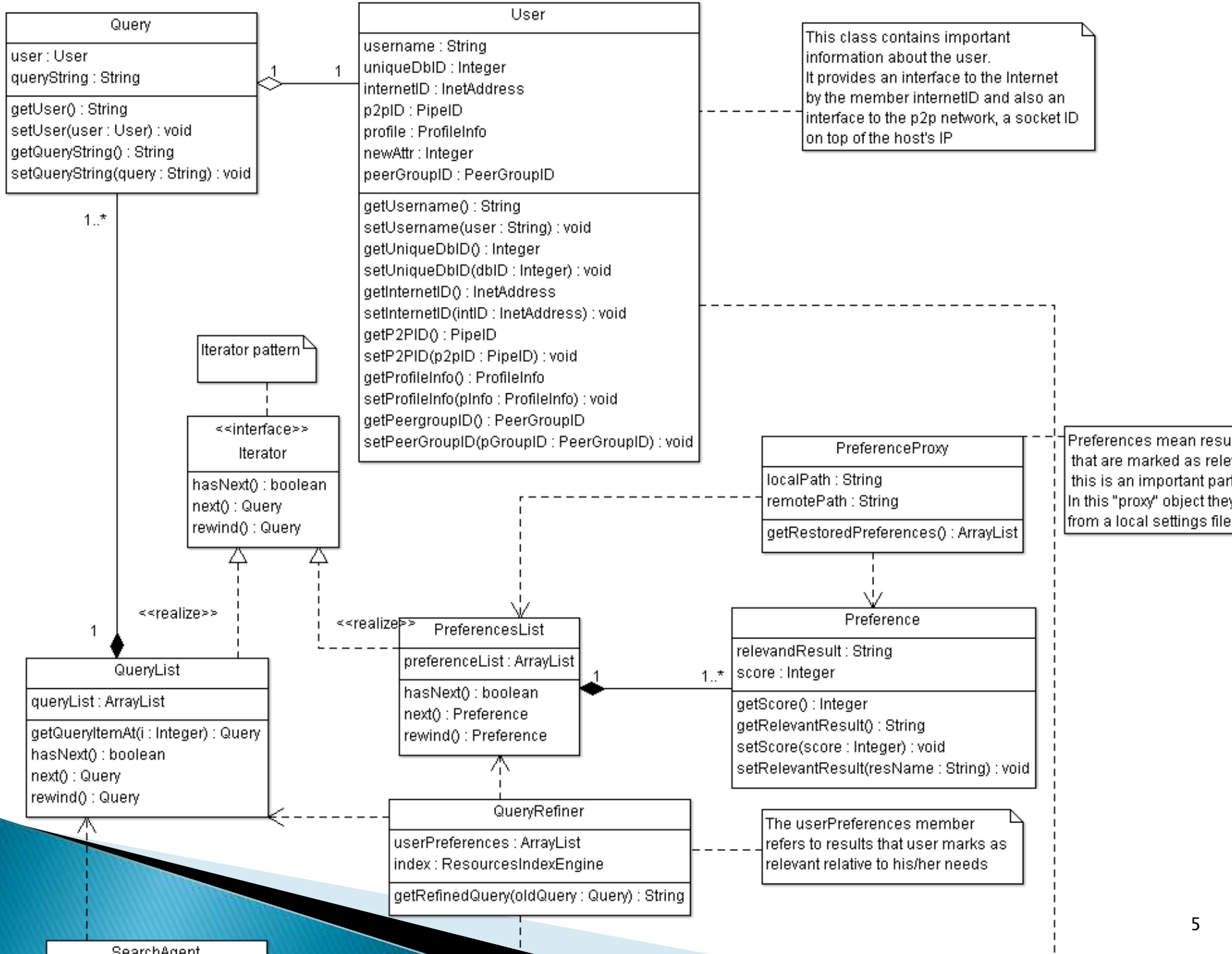
- Din Cursurile trecute…

- Diagrame UML:
  - Interacțiuni (secvență, colaborare)
  - Comportamentale (Stări, Activități)
  - Structură (Deployment)

Din cursurile trecute…

- Diagrame
- Diagrame UML
- Diagrame Use Case
- Diagrame de Clase


Register
PKI register
IM register
PKI serve
Alice
Authentication
<<include>>
Private key retrieval
Account deletion
Network login
IM server
<<include>>
hange profile information
Confirm password
1


Query
user : User
queryString : String
getUser() : String
setUser(user : User) : void
getQueryString() : String
setQueryString(query : String) : void
1
1
User
username : String
uniqueDbID : Integer
internetID : InetAddress
p2pID : PipeID
profile : ProfileInfo
newAttr : Integer
peerGroupID : PeerGroupID
getUsername() : String
setUsername(user : String) : void
getUniqueDbID() : Integer
setUniqueDbID(dbID : Integer) : void
getInternetID() : InetAddress
setInternetID(intID : InetAddress) : void
getP2PID() : PipeID
setP2PID(p2pID : PipeID) : void
getProfileInfo() : ProfileInfo
setProfileInfo(pInfo : ProfileInfo) : void
getPeergroupID() : PeerGroupID
setPeerGroupID(pGroupID : PeerGroupID) : void
This class contains important information about the user.
It provides an interface to the Internet by the member internetID and also an interface to the p2p network, a socket ID on top of the host's IP
1..*
Iterator pattern
<<interface>>
Iterator
hasNext() : boolean
next() : Query
rewind() : Query
PreferenceProxy
localPath : String
remotePath : String
getRestoredPreferences() : ArrayList
Preferences mean resu
that are marked as rele
this is an important part
In this "proxy" object they
from a local settings file
<<realize>>
<<realize>>
1
PreferencesList
preferenceList : ArrayList
hasNext() : boolean
next() : Preference
rewind() : Preference
1
1..*
Preference
relevandResult : String
score : Integer
getScore() : Integer
getRelevantResult() : String
setScore(score : Integer) : void
setRelevantResult(resName : String) : void
QueryList
queryList : ArrayList
getQueryItemAt(i : Integer) : Query
hasNext() : boolean
next() : Query
rewind() : Query
QueryRefiner
userPreferences : ArrayList
index : ResourcesIndexEngine
getRefinedQuery(oldQuery : Query) : String
The userPreferences member refers to results that user marks as relevant relative to his/her needs
SearchAgent

UML2.0 – 13 Tipuri de Diagrame

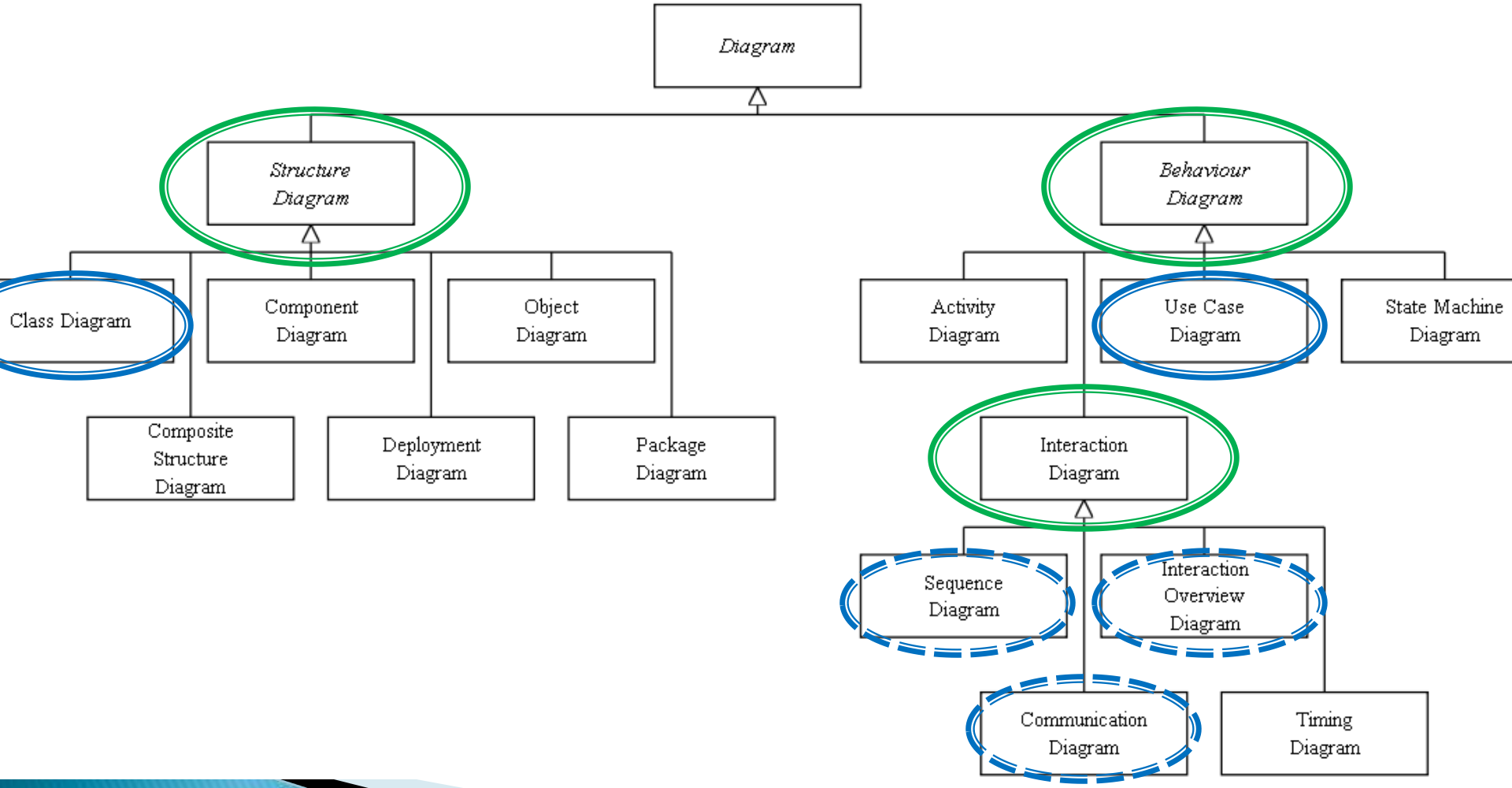
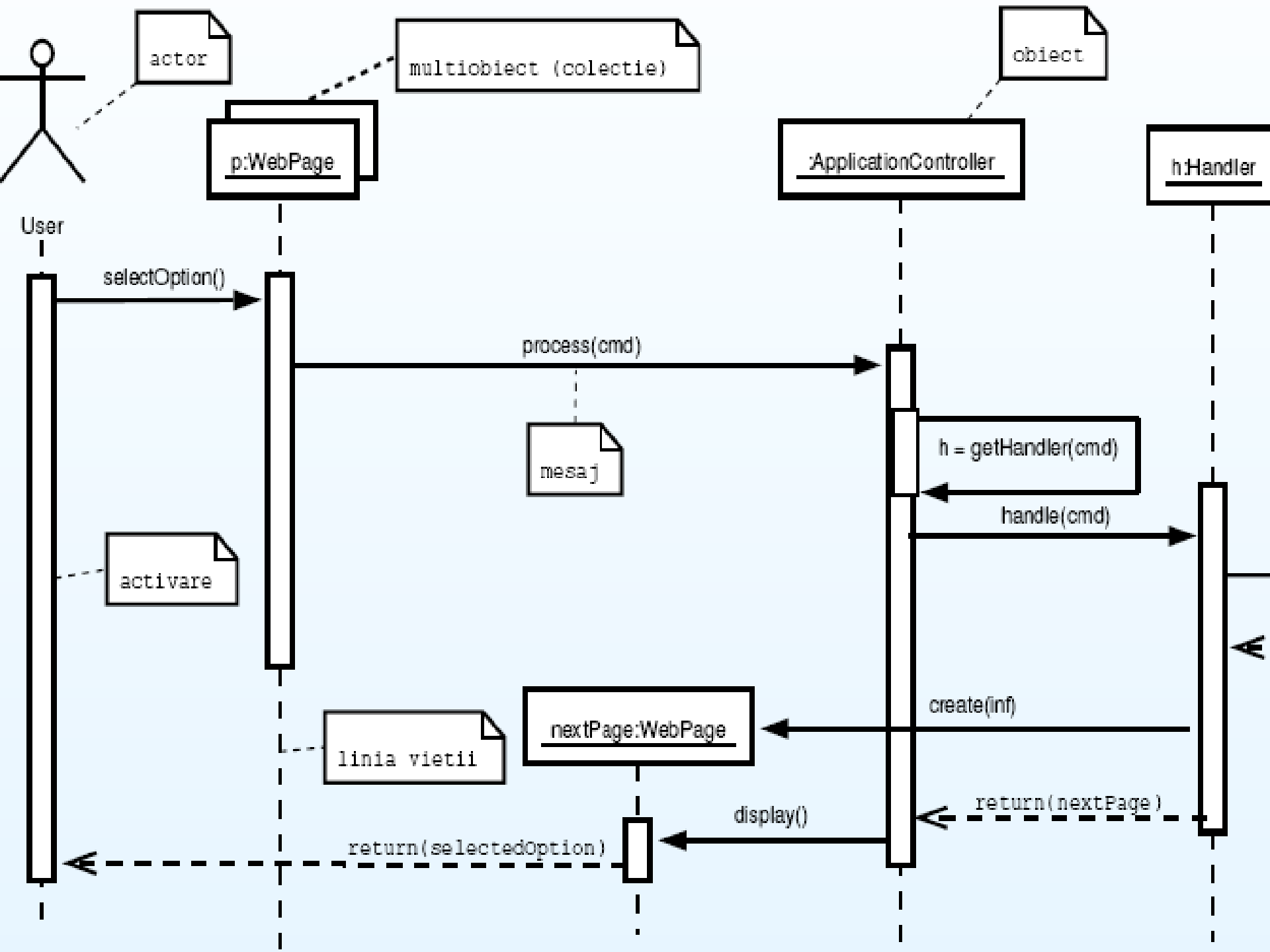
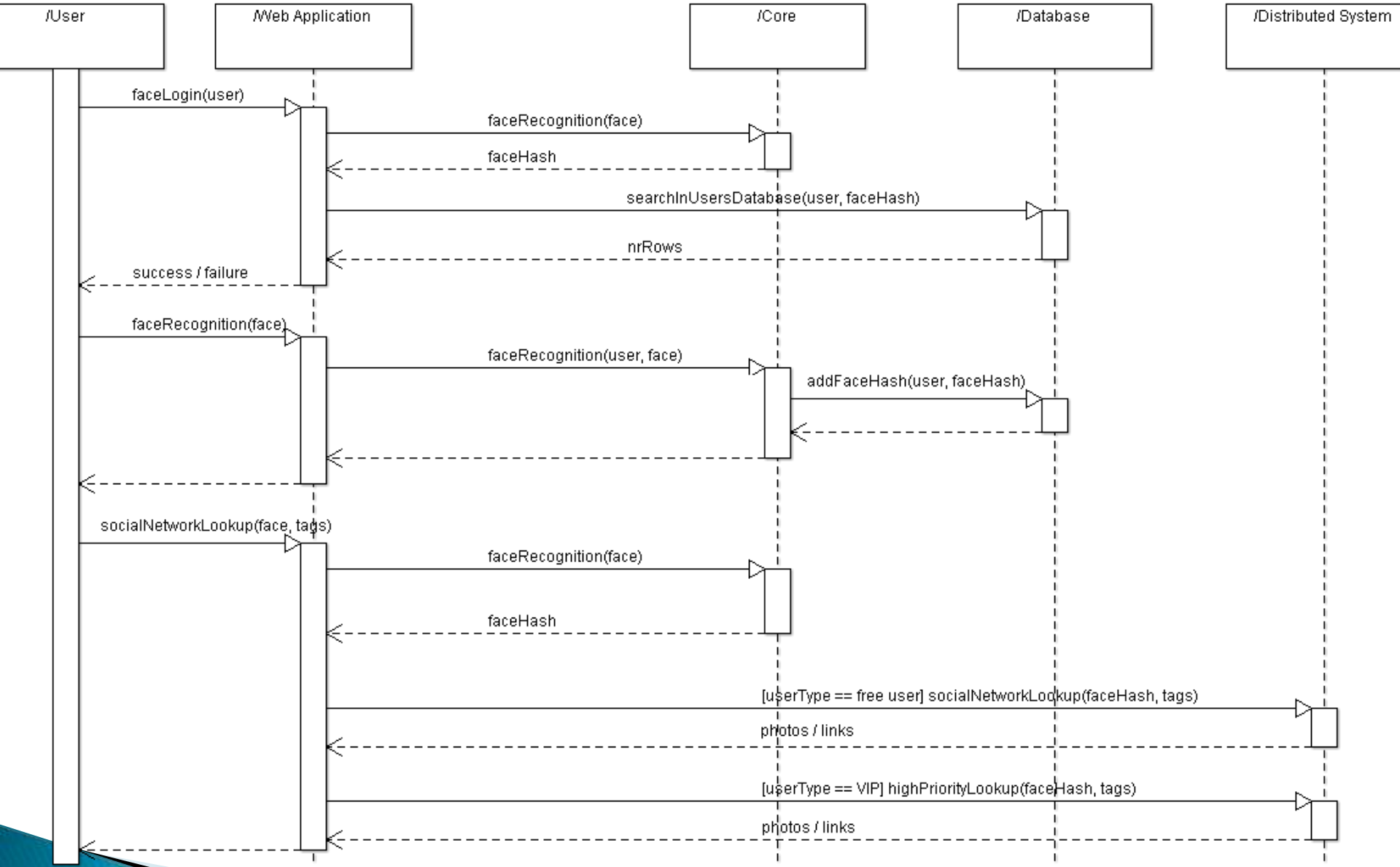


Diagrama de Secvenţă

- **Diagrama de secvenţă** curprinde secvenţa acţiunilor care au loc în sistem, invocarea metodelor fiecărui obiect ca şi ordinea în timp în care aceste invocări au loc

- O diagramă de secvenţă este bidimensională
  - Pe axa verticală se prezintă viaţa obiectului
    - linia vieţii obiectelor (grafic: linie punctată)
    - perioada de activare în care un obiect preia controlul execuţiei (grafic: dreptunghi pe linia vieţii)
  - Pe axa orizontală se arată secvenţa creării sau invocărilor
    - mesaje ordonate în timp (grafic: săgeţi)

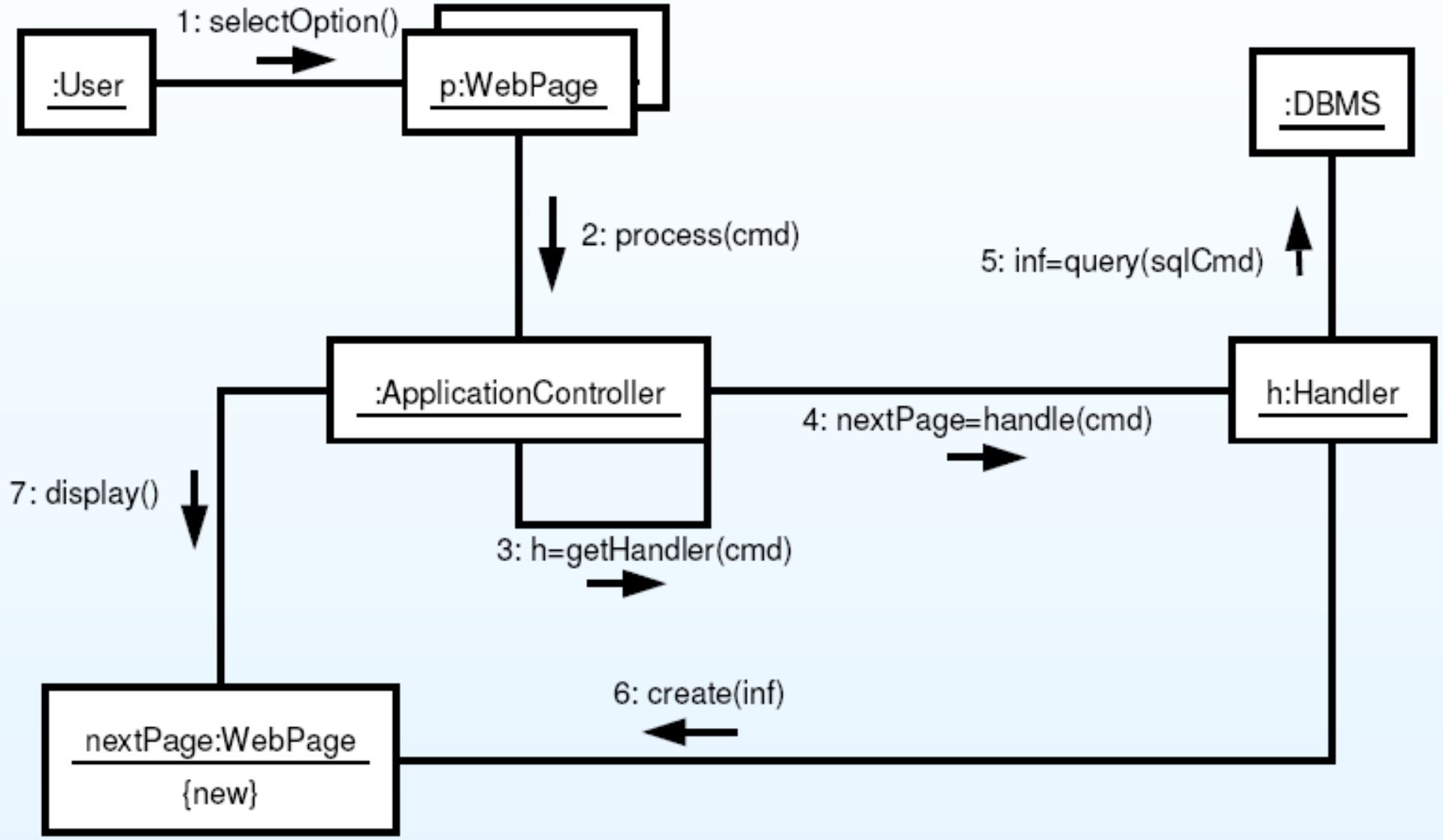

actor
multiobiect (colectie)
obiect
p:WebPage
:ApplicationController
h:Handler
User
selectOption()
process(cmd)
mesaj
h = getHandler(cmd)
handle(cmd)
activare
create(inf)
nextPage:WebPage
linia vietii
return(nextPage)
display()
return(selectedOption)


/User
/Web Application
/Core
/Database
/Distributed System
faceLogin(user)
faceRecognition(face)
faceHash
searchInUsersDatabase(user, faceHash)
nrRows
success / failure
faceRecognition(face)
faceRecognition(user, face)
addFaceHash(user, faceHash)
socialNetworkLookup(face, tags)
faceRecognition(face)
faceHash
[userType == free user] socialNetworkLookup(faceHash, tags)
photos / links
[userType == VIP] highPriorityLookup(faceHash, tags)
photos / links

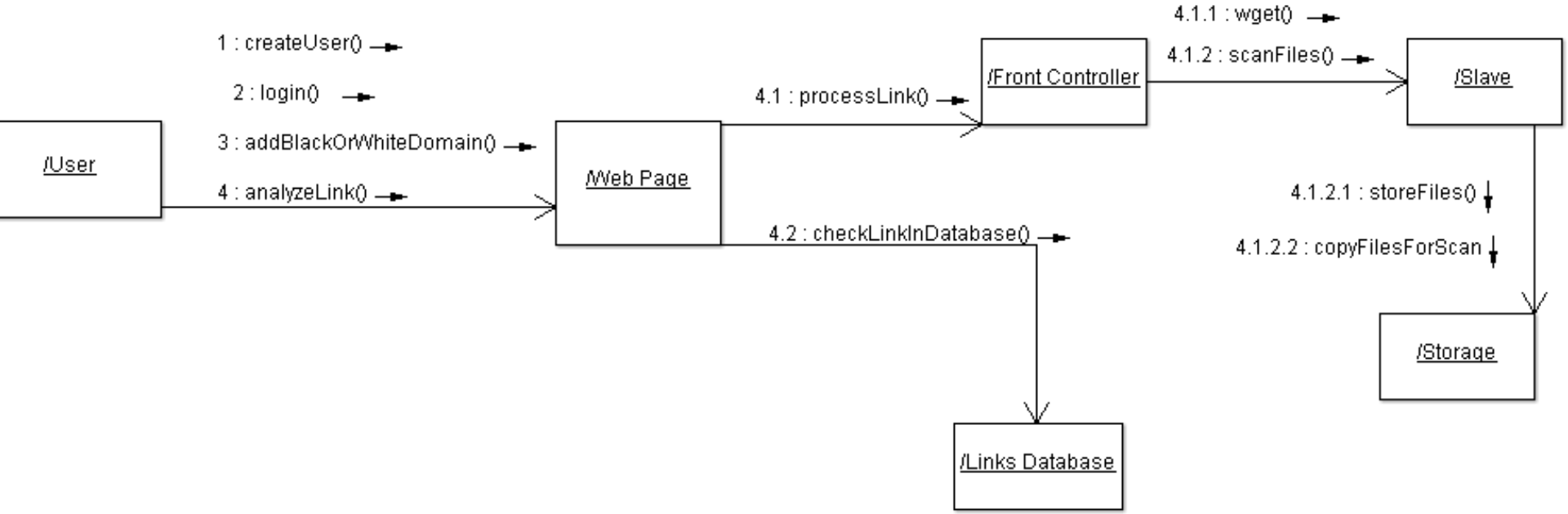
Diagramă de Colaborare

- Pune accentul pe organizarea structurală a obiectelor care participă la interacţiune
- Ilustrează mai bine ramificări complexe, iteraţii şi comportament concurent
- Poate conţine:
  - Obiecte, clase, actori
  - Legături între acestea
  - Mesaje

Exemplul 1


1: selectOption()
:User
p:WebPage
:DBMS
2: process(cmd)
5: inf=query(sqlCmd)
:ApplicationController
h:Handler
4: nextPage=handle(cmd)
7: display()
3: h=getHandler(cmd)
6: create(inf)
nextPage:WebPage
{new}

Exemplul 3

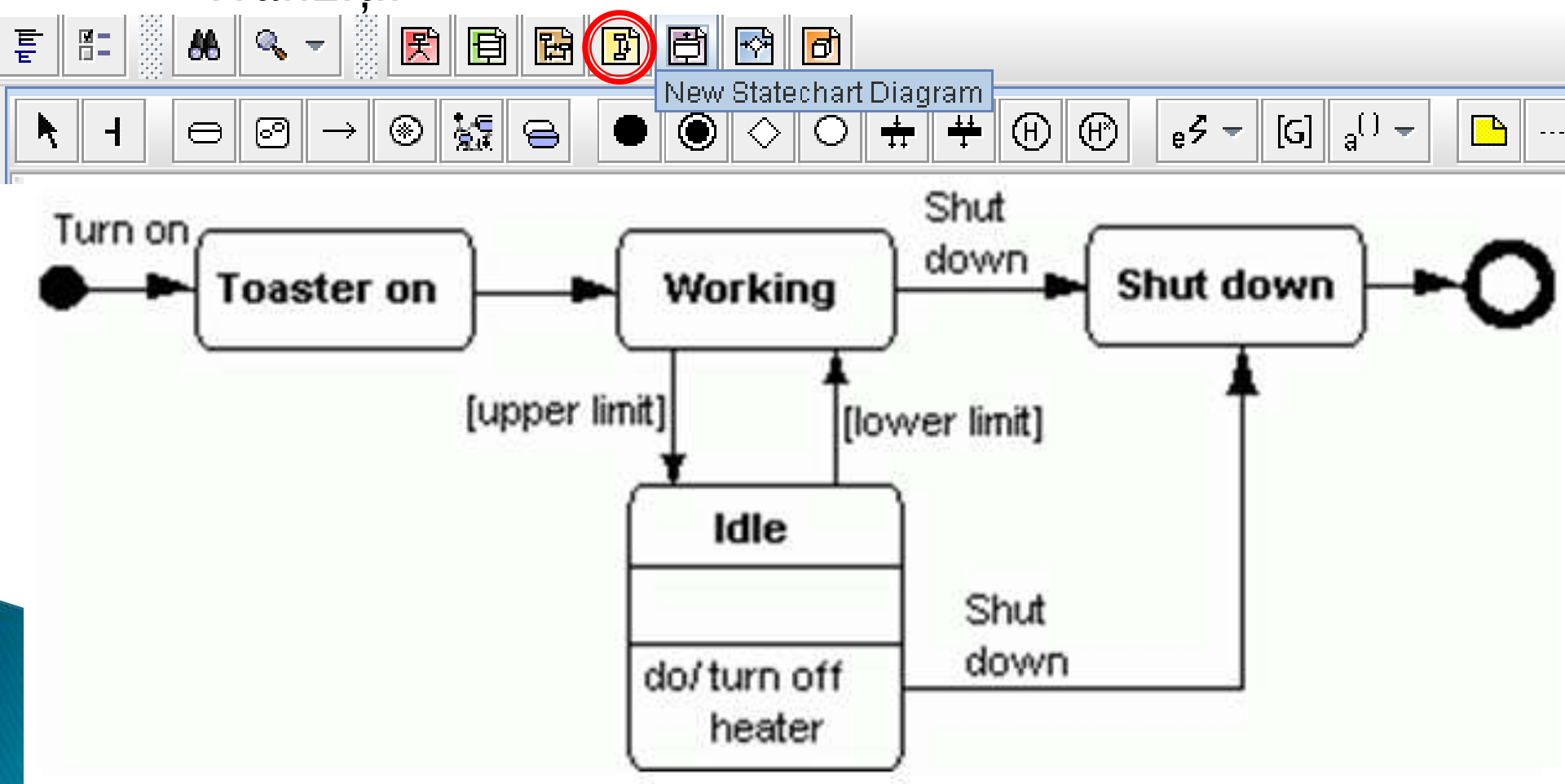

1 : createUser()
2 : login()
3 : addBlackOrWhiteDomain()
4 : analyzeLink()
/User
/Web Page
4.1 : processLink()
/Front Controller
4.1.1 : wget()
4.1.2 : scanFiles()
/Slave
4.1.2.1 : storeFiles()
4.1.2.2 : copyFilesForScan
/Storage
4.2 : checkLinkInDatabase()
/Links Database

Diagrame comportamentale

- Diagrame de stări, diagrame de activități
- Elemente de bază
  - Eveniment
  - Acțiune
  - Activitate

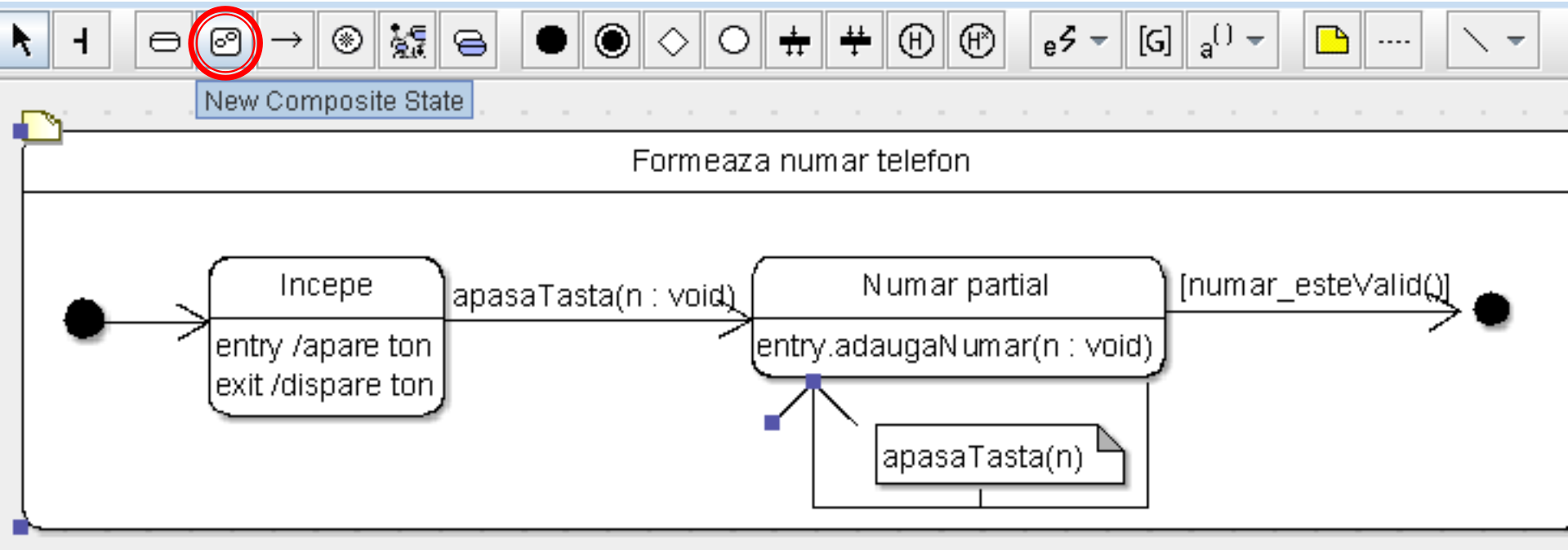
Diagramă de Stări

- Conține:
  - Stări
  - Tranziții



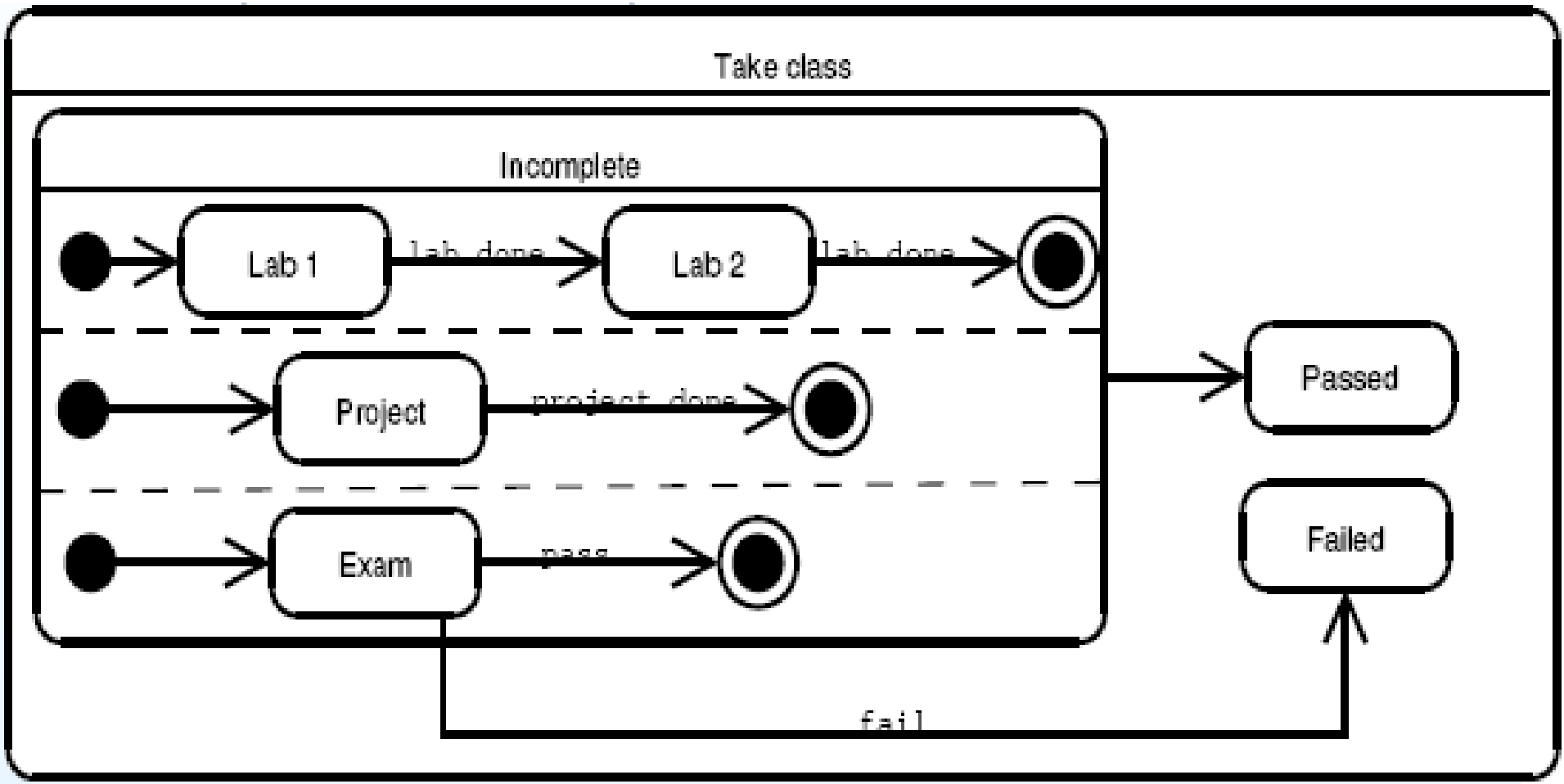
Exemplu de Stare compusă 1

- Stare compusă cu substări **secvențial** active:



Exemplu de Stare compusă 2

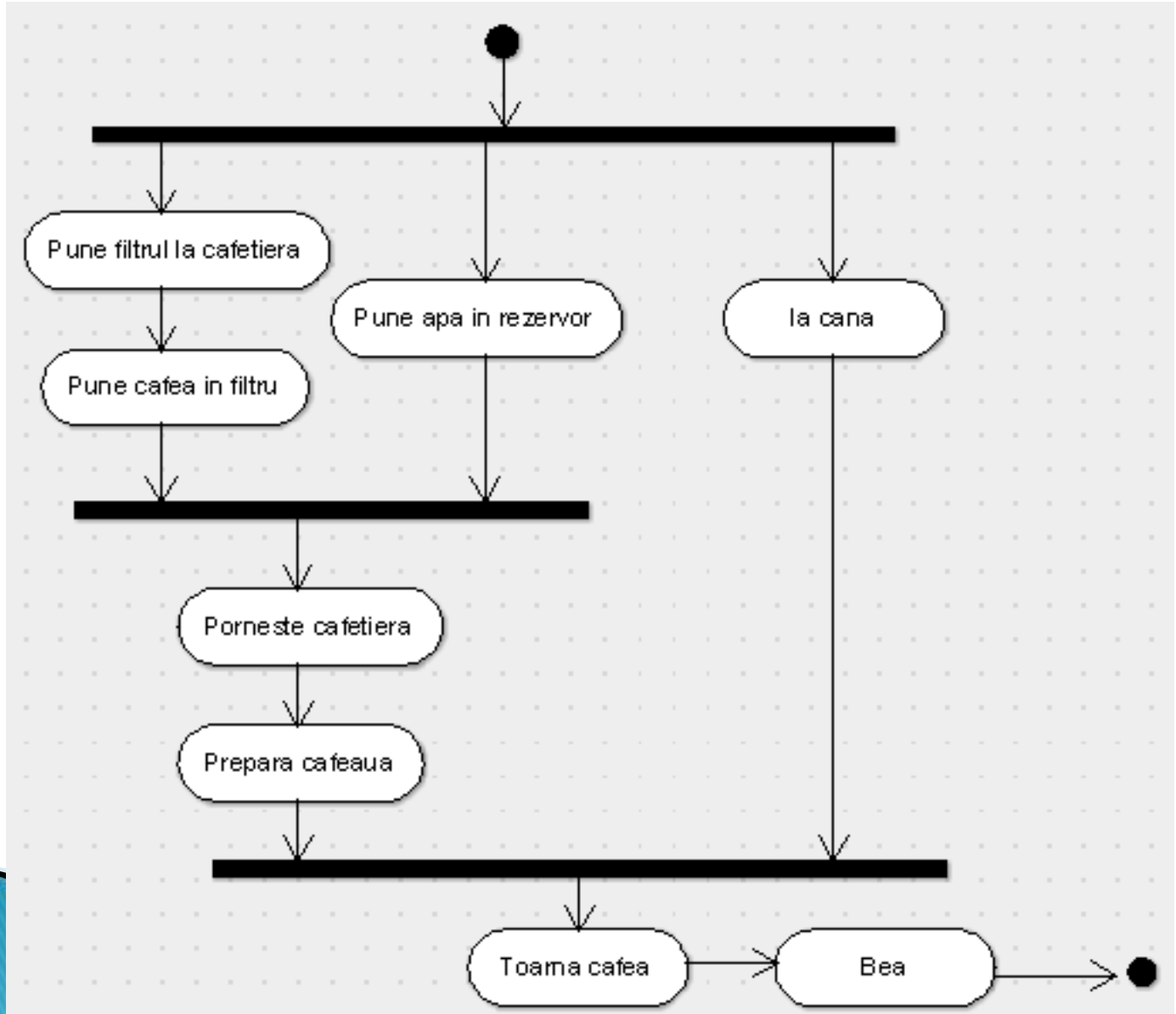
- Stare compusă cu substări **paralel** active:

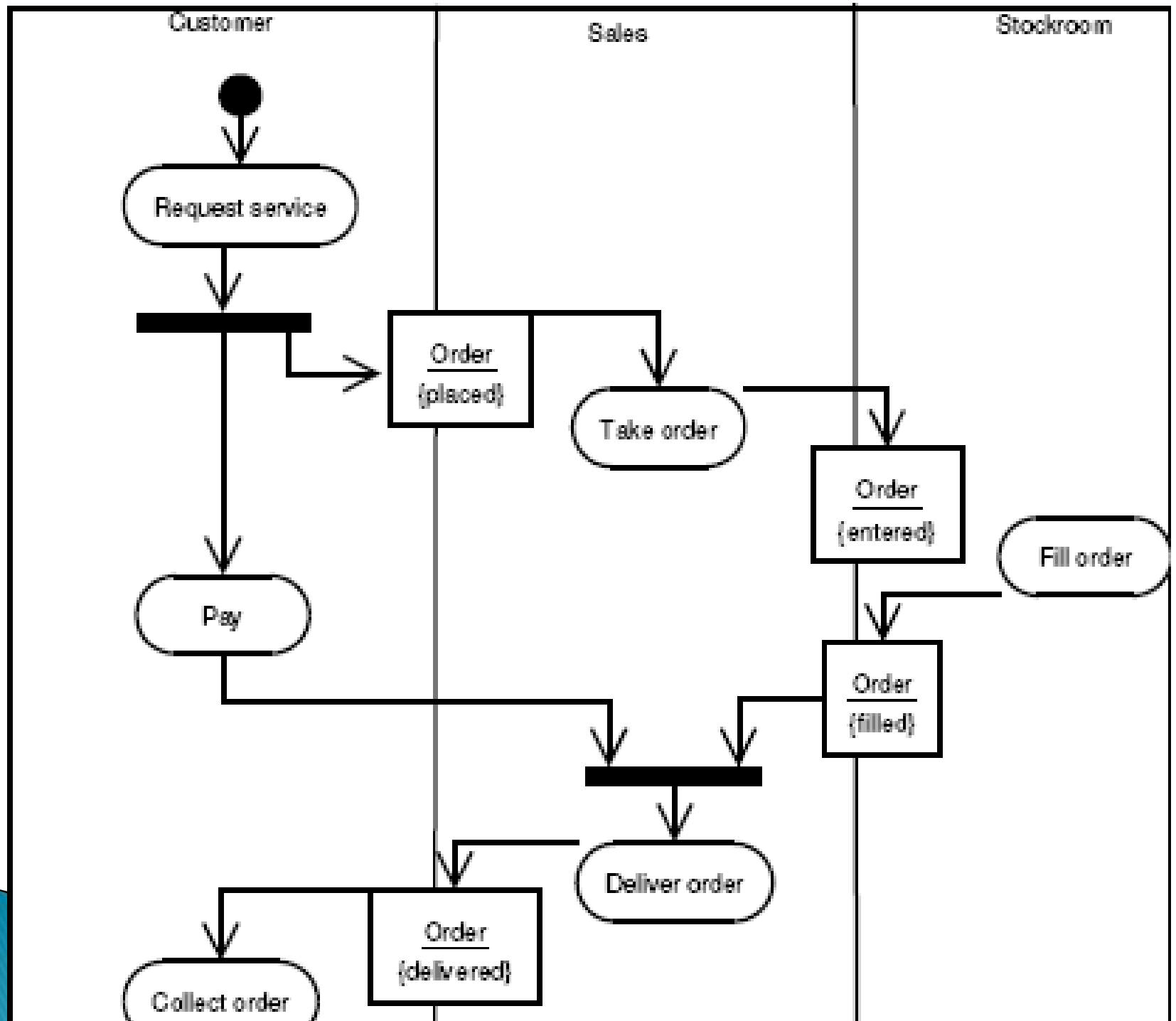


Diagramă de Activități (Activity Diagram)

- Folosită pentru **a modela dinamica unui proces sau a unei operații**
- Evidențiază controlul execuției de la o activitate la alta
- Se atașează:
  - Unei clase (modelează un caz de utilizare)
  - Unui pachet
  - Implementării unei operații

Exemplu de DA (Sincronizare)

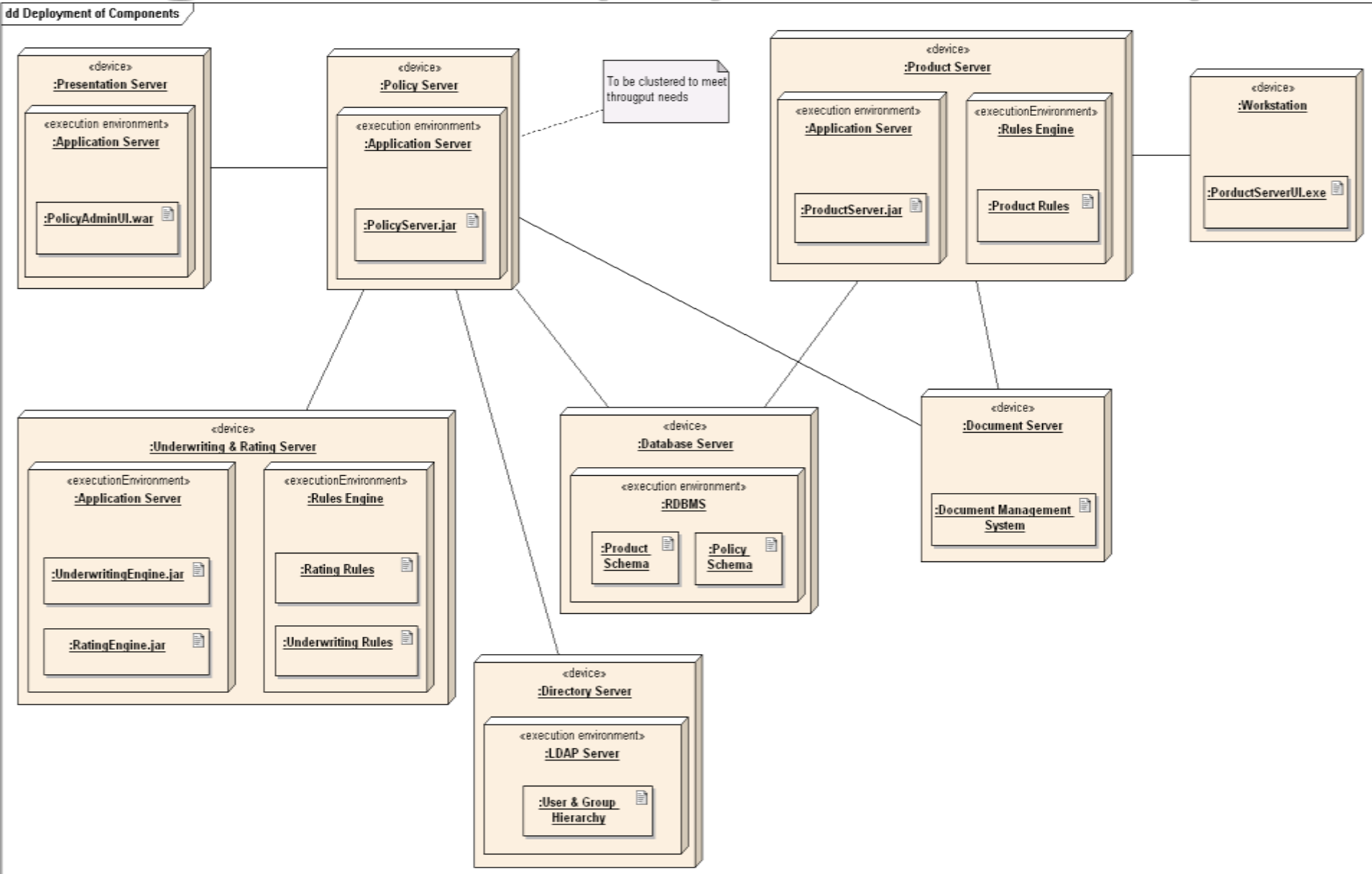



Customer
Sales
Stockroom
Request service
Order
{placed}
Take order
Order
{entered}
Fill order
Pay
Order
{filled}
Deliver order
Order
{delivered}
Collect order

Diagrame de deployment

- Modelează mediul hardware în care va funcționa proiectul
- Exemplu: pentru a descrie un site web o diagramă de deployment va conține **componentele hardware**
  - server-ul web,
  - server-ul de aplicații,
  - server-ul de baze de date
- **Componentele software** de pe fiecare din acestea
  - Aplicația web
  - Baza de date
- Modul în care acestea sunt conectate:
  - JDBC, REST, RMI

Diagramă de deployment – Exemplu 1



Diagrame de Pachete (Package Diagram)

- Pachetul:
  - Este un container logic pentru elemente între care se stabilesc legături
  - Definește un spațiu de nume
  - Toate elementele UML pot fi grupate în pachete (cel mai des pachetele sunt folosite pentru a grupa clase)
  - Un pachet poate conține subpachete => se creează o structură arborescentă (similară cu organizarea fișierele/directoarelor)

Diagrame de Pachete 2

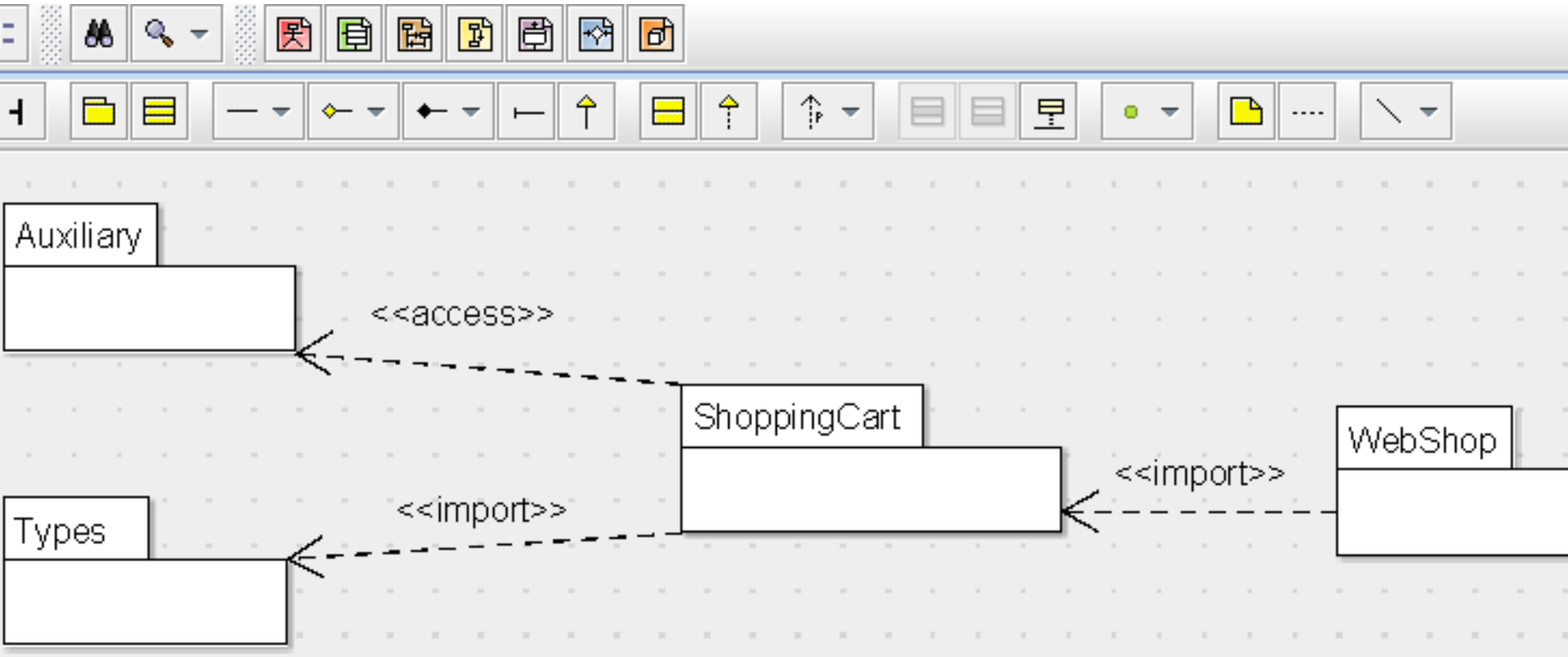
- Relații:
  - dependență <<access>> = import privat
  - dependență <<import>> = import public

- Ambele relații permit folosirea elementelor aflate în pachetul destinație de către elementele aflate în pachetul sursă fără a fi necesară calificarea numelor elementelor din pachetul destinație (similar directivei **import** din java)

- Aceste tipuri de diagrame se realizează în cadrul diagramelor de clasă

Exemplu de Diagramă de Pachete

- Elementele din Types sunt importate în ShoppingCart și apoi sunt importate mai departe de către WebShop
- Elementele din Auxiliary pot fi accesate însă doar din ShoppingCart și nu pot fi referite folosind nume necalificate din WebShop



Utilitatea diagramelor de pachete

- Împart sisteme mari în subsisteme mai mici și mai ușor de gestionat
- Permit dezvoltare paralelă iterativă
- Definirea unor interfețe clare între pachete promovează refolosirea codului (ex. pachet care oferă funcții grafice, pachet care oferă posibilitatea conectării la BD, etc...)

Recomandări în realizarea diagramelor UML

- Diagramele să nu fie nici prea complicate, dar nici prea simple: scopul este comunicarea eficientă
- Dați nume sugestive elementelor componente
- Aranjați elementele astfel încât liniile să nu se intersecteze
- Încercați să nu arătați prea multe tipuri de relații odată (evitați diagramele foarte complicate)
- Dacă este nevoie, realizați mai multe diagrame de același tip

Concluzii

- Diagrame UML:
  - Interacțiuni
  - Comportamentale
  - Structură

Bibliografie

- Ovidiu Gheorghieş, Curs 5 IP
- www.uml.org

Ingineria Programării

Cursul 5 – 20 Martie 2017

adiftene@info.uaic.ro

Cuprins

- Din Cursurile trecute…
- Forward and Reverse Engineering
- GRASP
  - Information Expert
  - Creator
  - Low coupling
  - High cohesion
  - Controller

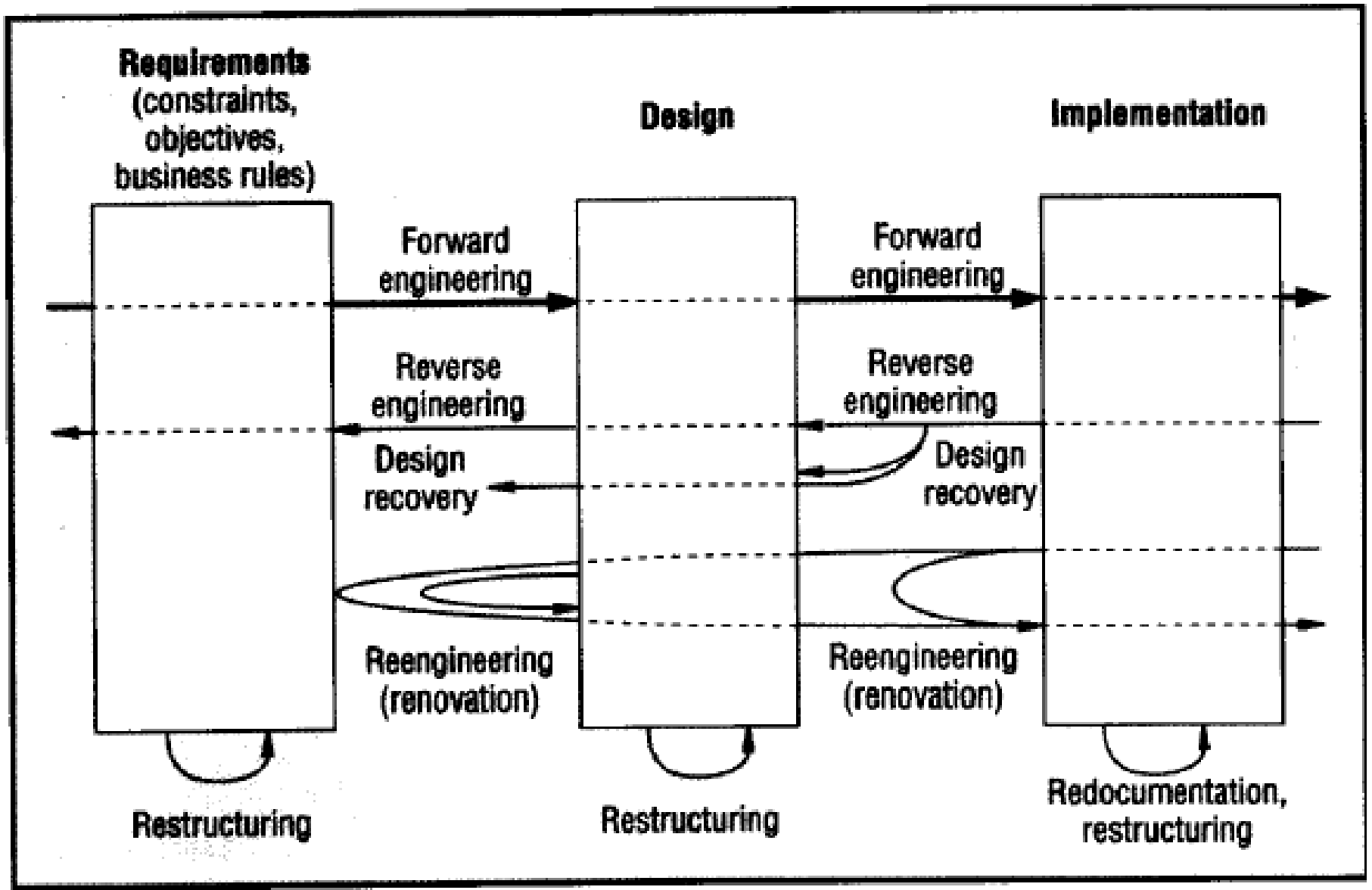
Atenție

- Săptămâna 7-a e termenul limită pentru alegerea proiectului
- După care urmează: documentare, înțelegere, knowledge transfer, diagrame use case, diagrame de clasă, implementare, unit testing, etc.
- Săptămâna a 7-a începe efectiv lucrul la proiect, iar evaluarea se încheie în săptămâna a 14-a
- În săptămâna a 8-a *nu se fac ore*…

RE

- De ce avem nevoie de modelare?
- Cum putem modela un proiect?
- SCRUM – roles, values, artifacts, events, rules

Forward and Reverse Engineering



Forward Engineering

- A traditional process of moving from high-level abstractions and logical to the implementation-independent designs to the physical implementation of a system

- FE follows a sequence of going **from requirements through designing its implementation**

Reverse Engineering

- **Reverse engineering** (RE) is the process of discovering the technological principles of a device, object or system through analysis of its structure, function and operation
- *To try to make a new device or program that does the same thing without copying anything from the original*
- Reverse engineering has its origins in the analysis of hardware for commercial or military advantage

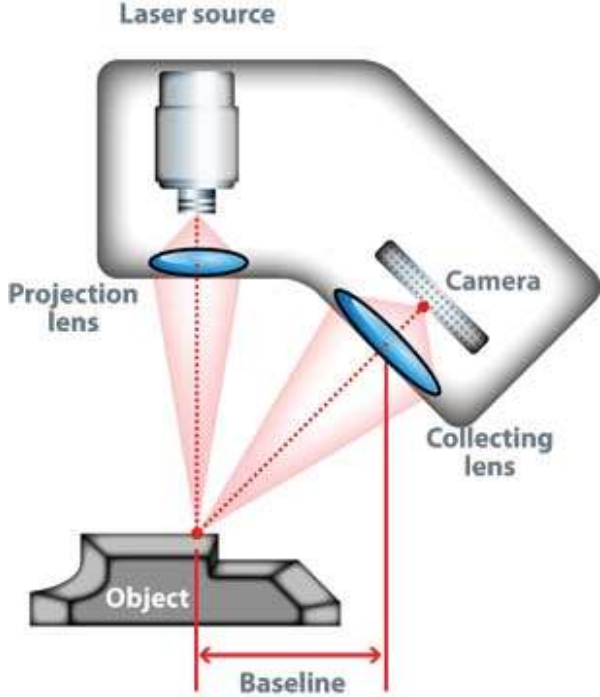
RE Motivation

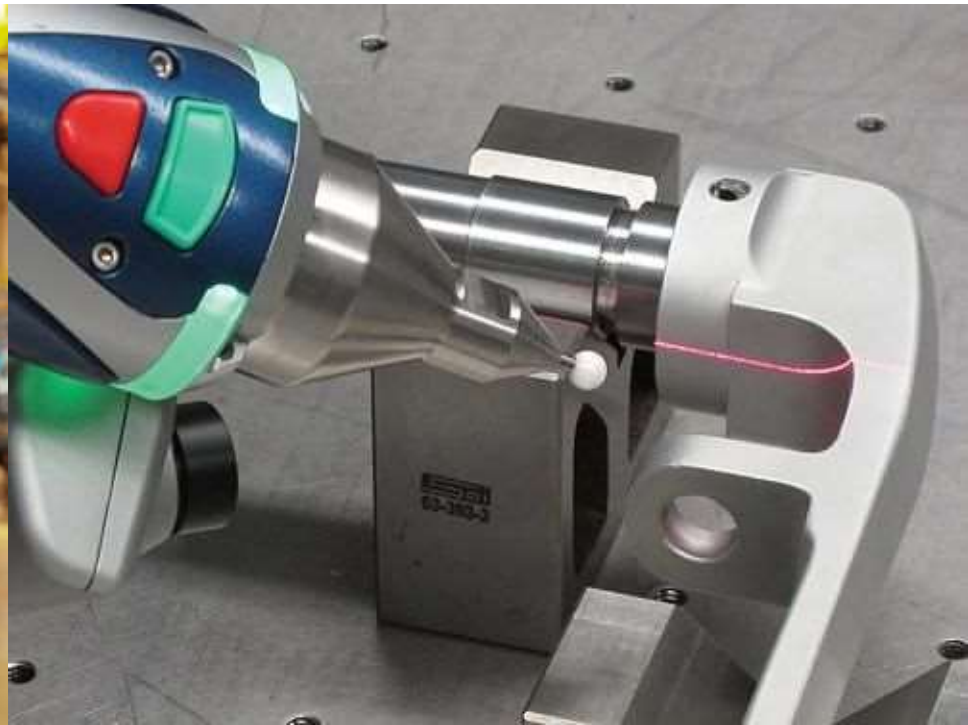
- Interoperability
- Lost documentation
- Product analysis
- Security auditing
- Removal of copy protection, circumvention of access restrictions
- Creation of unlicensed/unapproved duplicates
- Academic/learning purposes
- Curiosity
- Competitive technical intelligence (understand what your competitor is actually doing versus what they say they are doing)
- Learning: Learn from others mistakes

Types of RE

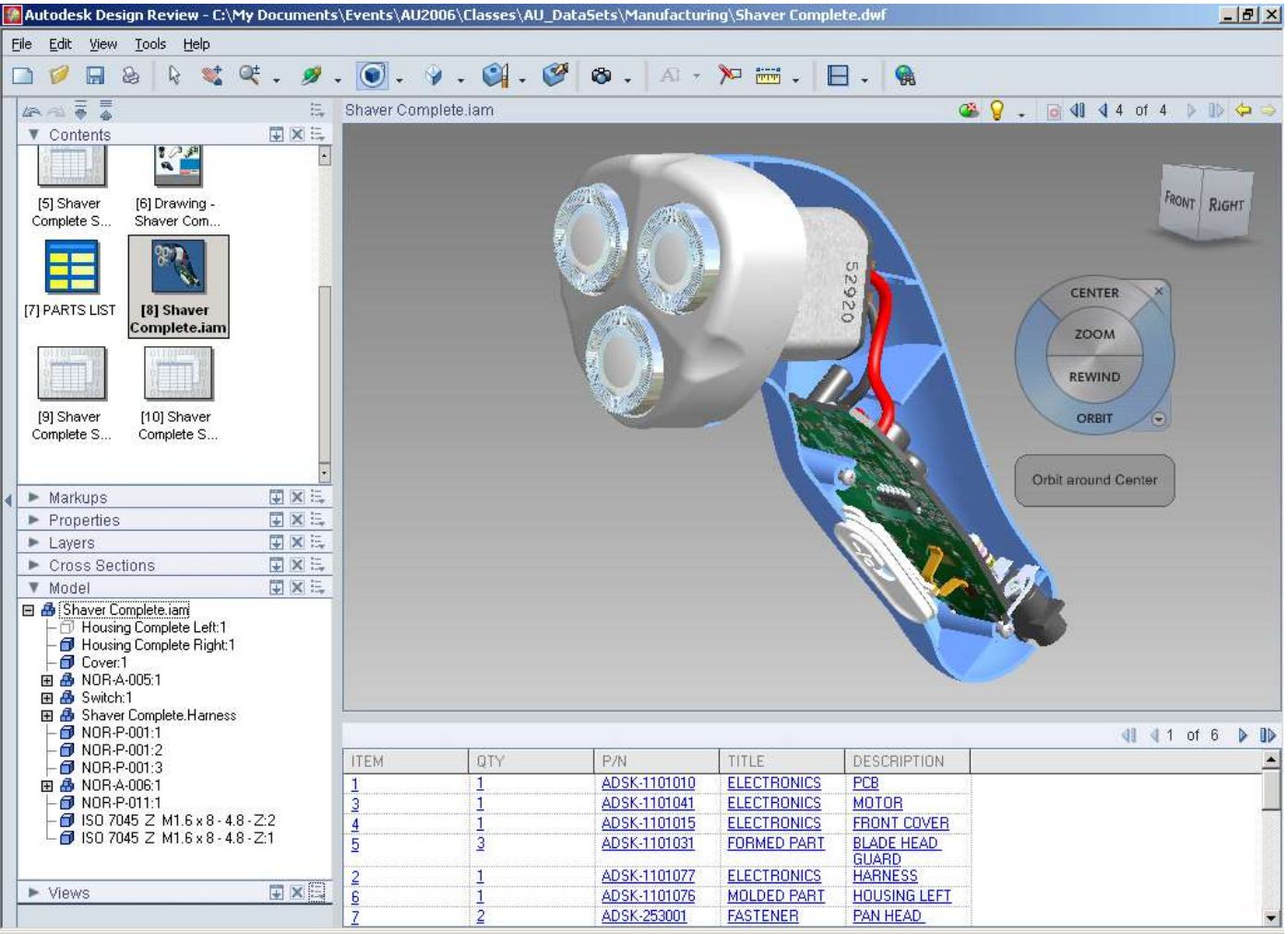
- RE1: Reverse engineering of mechanical devices
- RE2: Reverse engineering of integrated circuits/smart cards
- RE3: Reverse engineering for military applications
- RE4: Reverse engineering of software

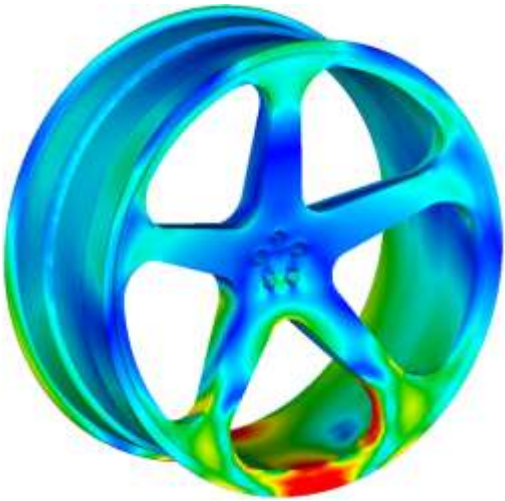
RE1: Scanere laser 3D

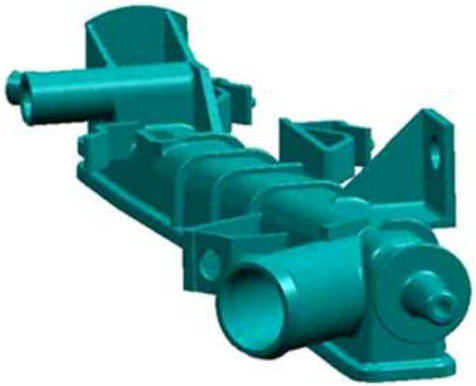




RE1: Servicii de modelare 3D CAD





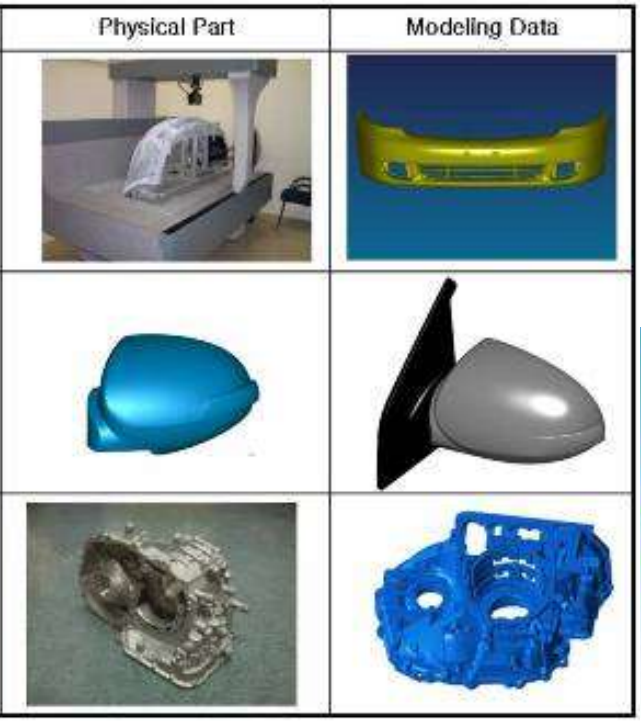


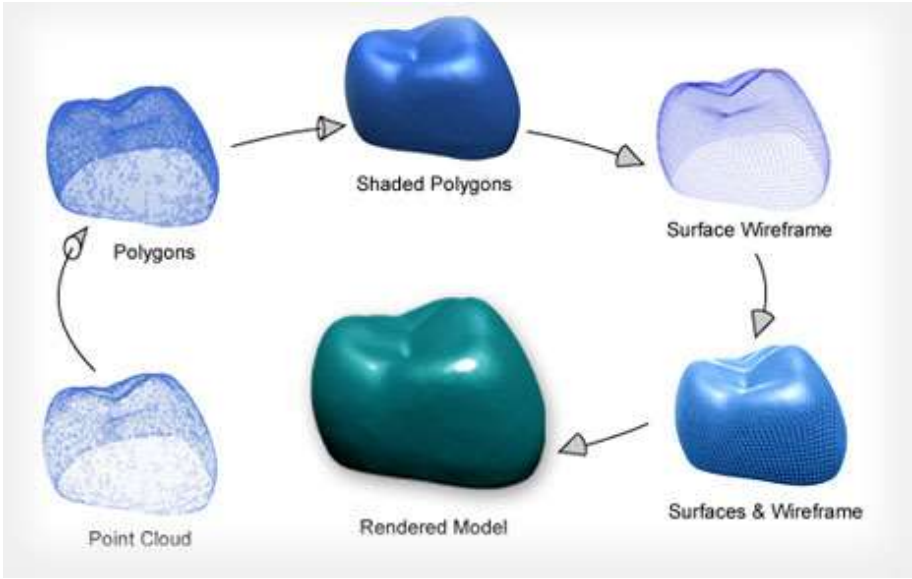
RE1: Servicii de imprimare 3D

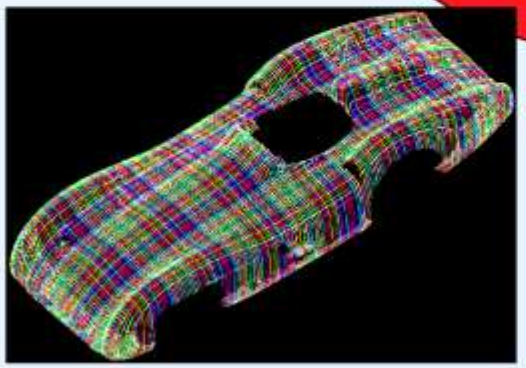
- Rapid prototyping

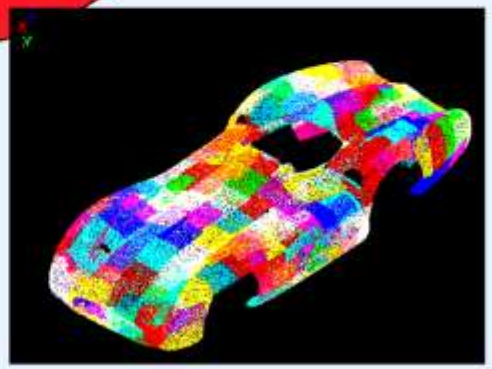
- FullCure materials

RE1: Domenii


Physical Part
Modeling Data


Shaded Polygons
Polygons
Surface Wireframe
Point Cloud
Rendered Model
Surfaces & Wireframe



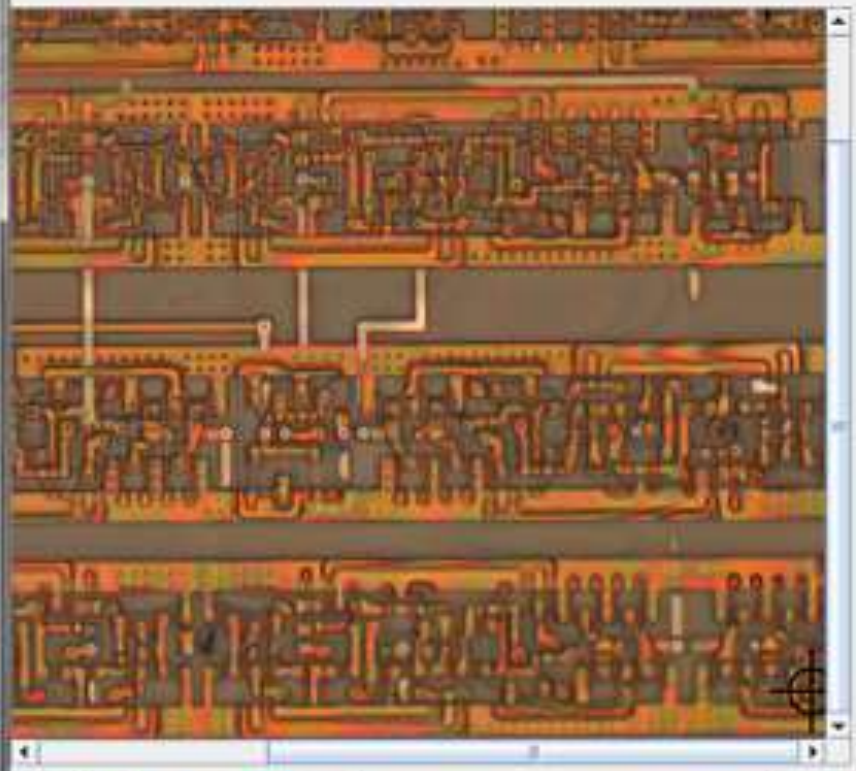


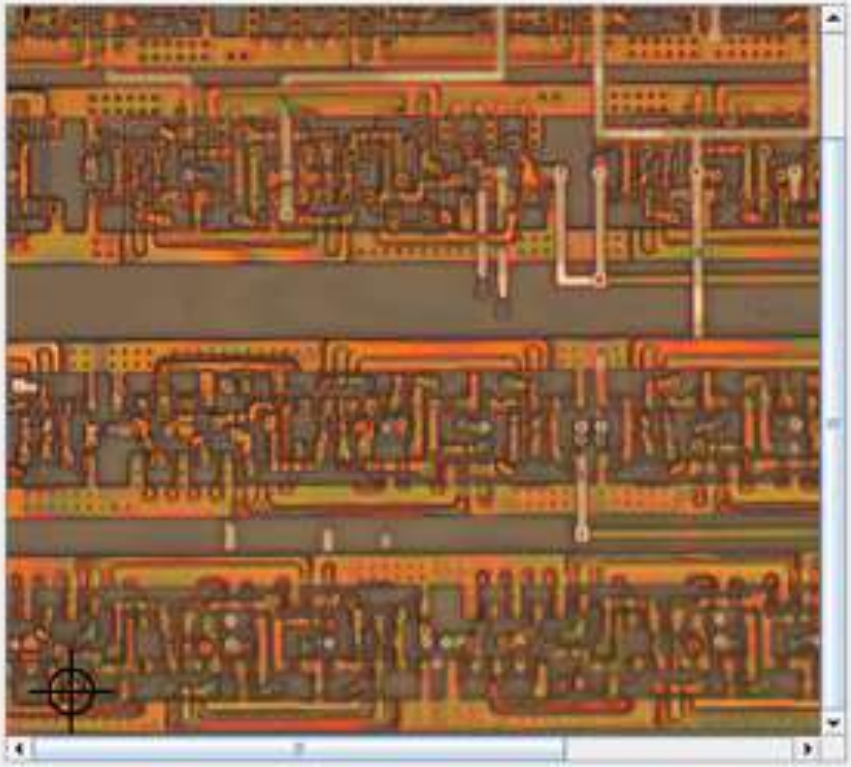
Reverse engineering of integrated circuits/smart cards

- RE is an invasive and destructive form of analyzing a smart card
- The attacker grinds away layer by layer of the smart card and takes pictures with an electron microscope
- Engineers employ sensors to detect and prevent this attack

Image Stitching Tool

3. Schritt: Setzen der Kontrollpunkte





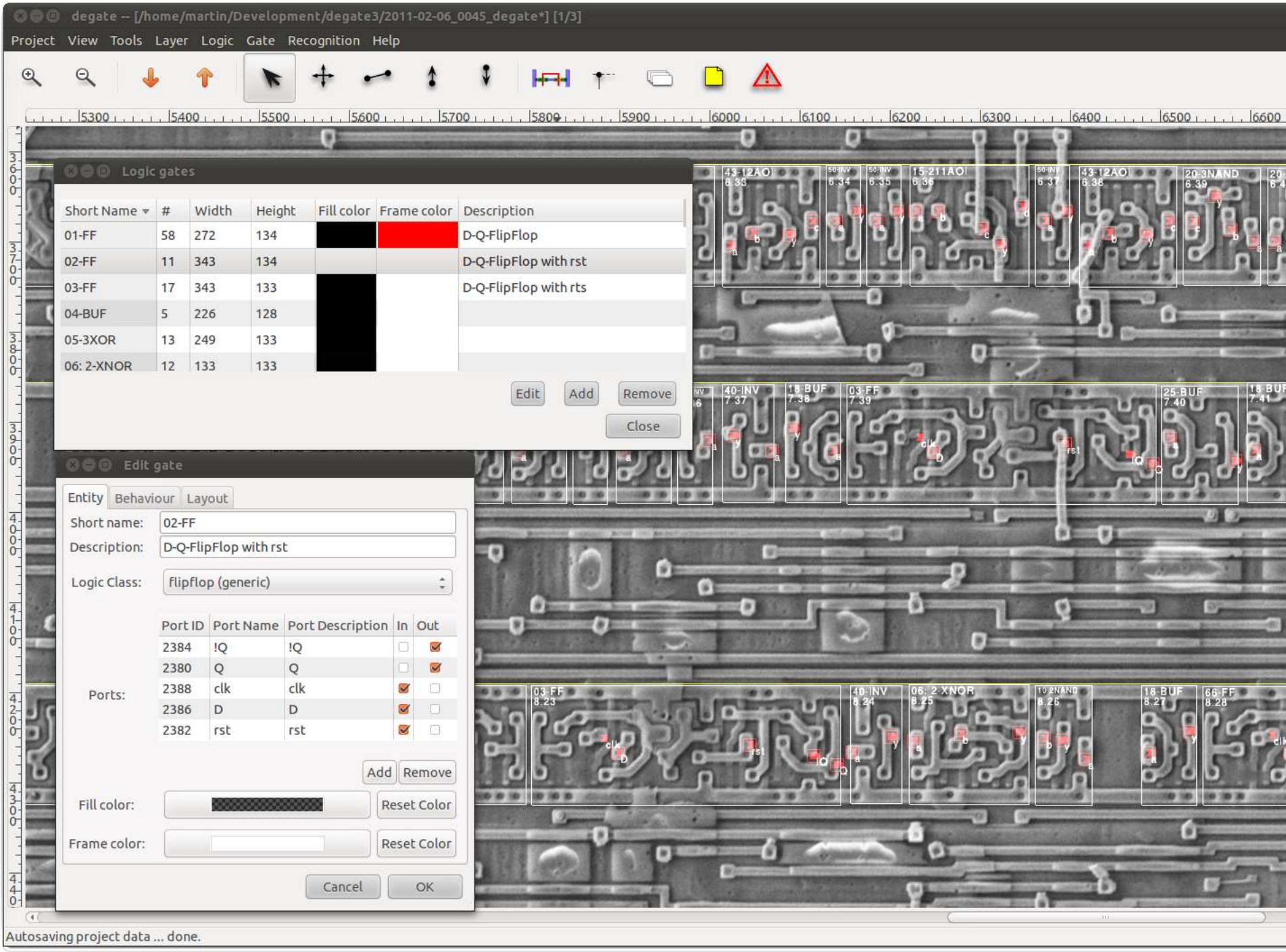
654 - 63

680 - 484

12 - 64

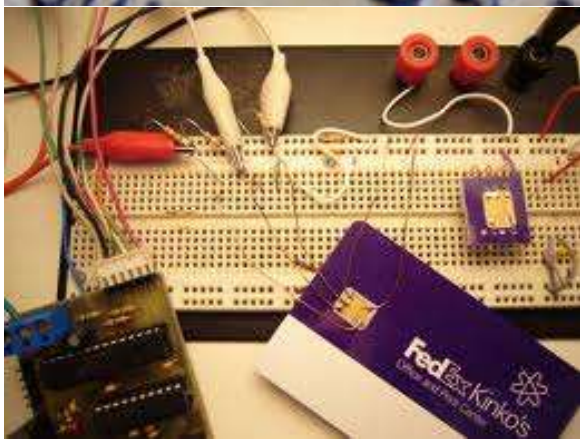
38 - 486

Kontrollpunkt speichern • Weiter


degate -- [/home/martin/Development/degate3/2011-02-06_0045_degate*] [1/3]
Project View Tools Layer Logic Gate Recognition Help
Logic gates
Short Name # Width Height Fill color Frame color Description
01-FF 58 272 134 D-Q-FlipFlop
02-FF 11 343 134 D-Q-FlipFlop with rst
03-FF 17 343 133 D-Q-FlipFlop with rts
04-BUF 5 226 128
05-3XOR 13 249 133
06: 2-XNOR 12 133 133
Edit Add Remove
Close
Edit gate
Entity Behaviour Layout
Short name: 02-FF
Description: D-Q-FlipFlop with rst
Logic Class: flipflop (generic)
Ports:
Port ID Port Name Port Description In Out
2384 !Q !Q
2380 Q Q
2388 clk clk
2386 D D
2382 rst rst
Add Remove
Fill color: Reset Color
Frame color: Reset Color
Cancel OK
Autosaving project data ... done.

RE2: Smart cards

- Satellite TV
- Security card
- Phone card
- Ticket card
- Bank card



Reverse engineering for military applications

- Reverse engineering is often used by militaries in order to copy other nations' technologies, devices or information that have been obtained by regular troops in the fields or by intelligence operations

- It was often used during the Second World War and the Cold War

- Well-known examples from WWII and later include: rocket, missile, bombers, China has reversed many examples of US and Russian hardware, from fighter aircraft to missiles and HMMWV cars

RE3: Avioane

- US – B-29

URSS – Tupolev Tu-4

RE3: Avioane (2)

- Chinese J-20, Black Eagle US F-22, Russian Sukhoi T-50

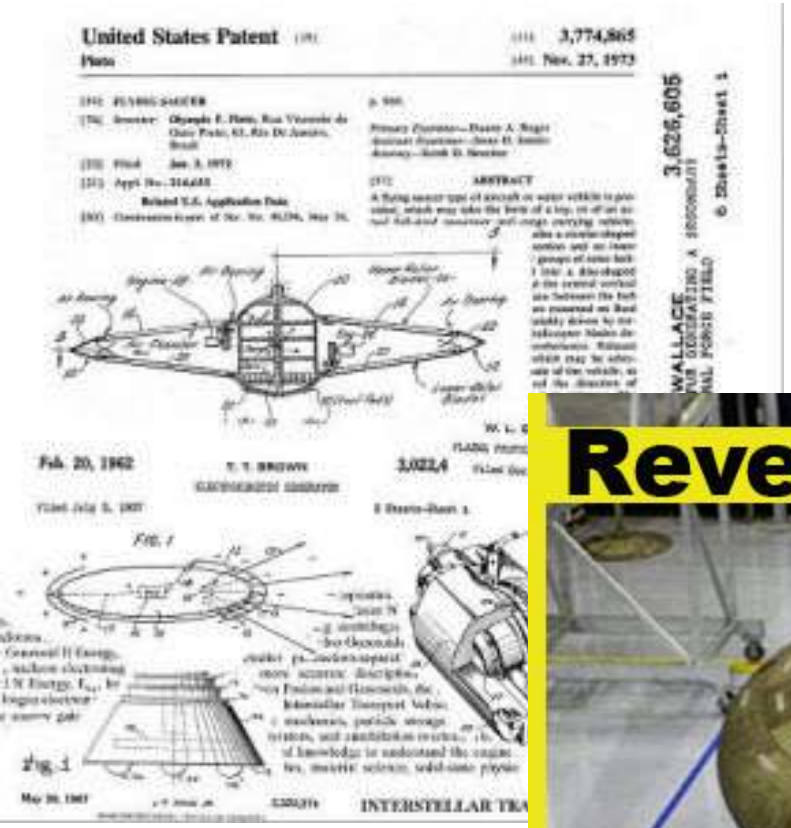
RE3: Rachete

- US –AIM-9 Sidewinder Soviet – Vympel K-13

RE3: Submar

Russia's new ballistic missile submarine
OLD
Typhoon class
16 x SLBM's
557 ft 9in
NEW
Borei class
20 x SLBM's
574 ft 2in
Yury Dolgoruky First Borei-class submarine is undergoing sea trials, two others are being built
Royal Navy
Vanguard class
16 x SLBM's
491 ft 10in

RE3: UFOs


United States Patent [19]
Plata
[11] 3,774,865
[45] Nov. 27, 1973
3,626,605
Feb. 20, 1962
Fig. 1
INTERSTELLAR TRA

Reverse engineering of software

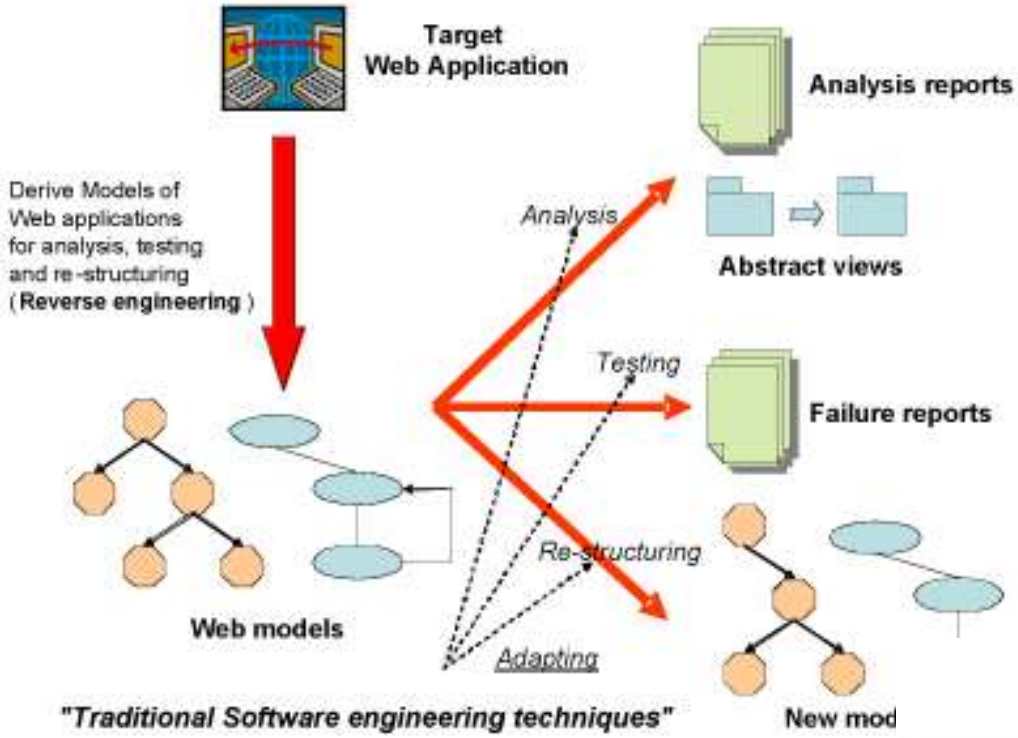
- Reverse engineering is the process of analyzing a subject system to create representations of the system at a higher level of abstraction

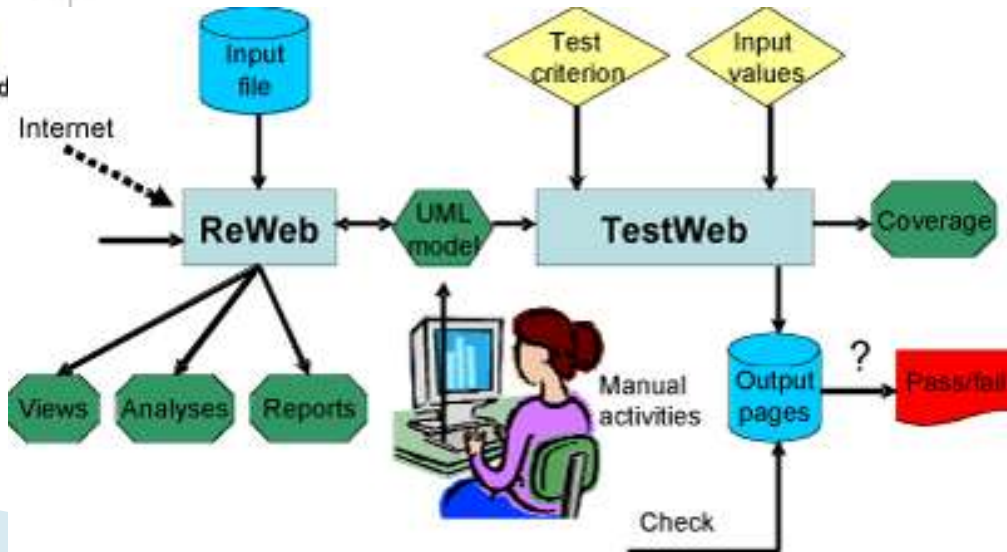
- In practice, two main types of RE emerge:
  - **Source code is available** (but it is poorly documented)
  - **There is no source code available for the software**

- **Black box testing** in software engineering has a lot in common with reverse engineering

RE4: Smart phones

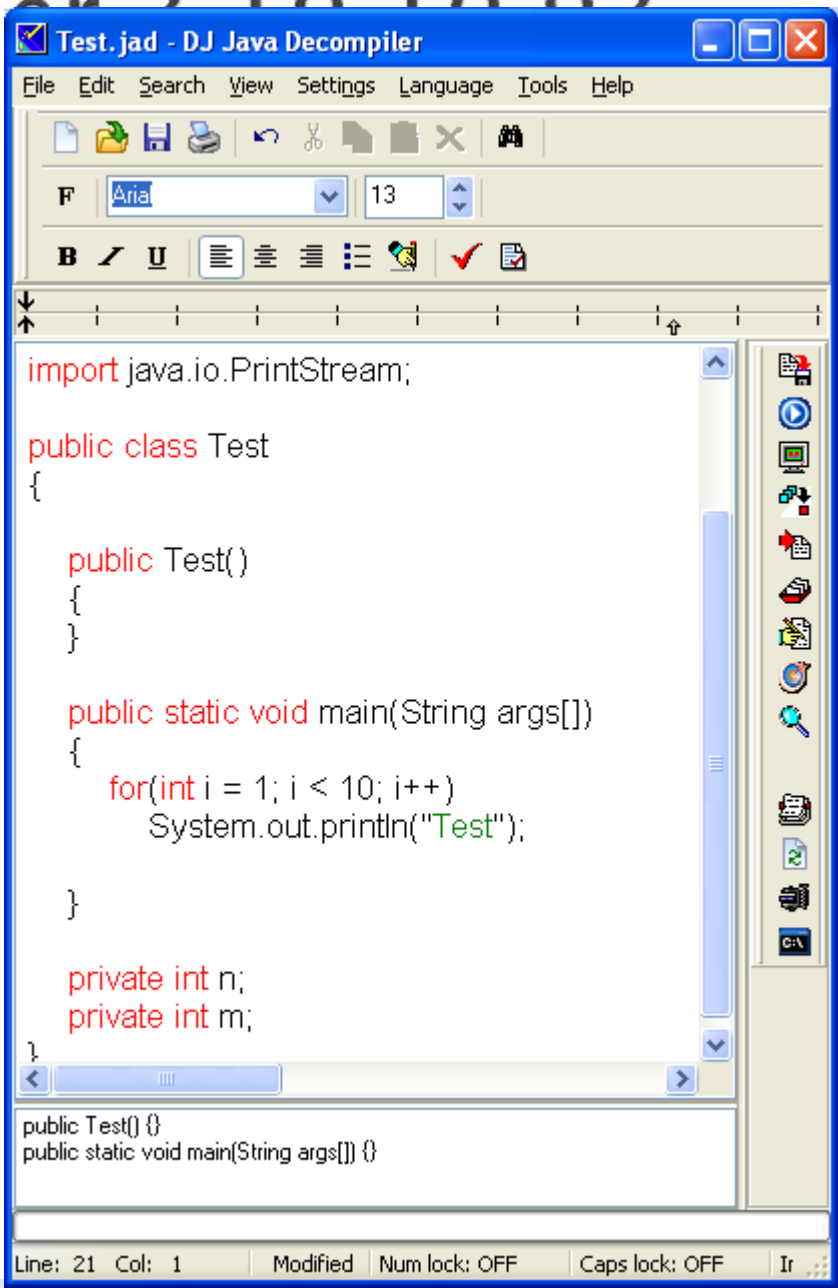
RE4 of Web Applications





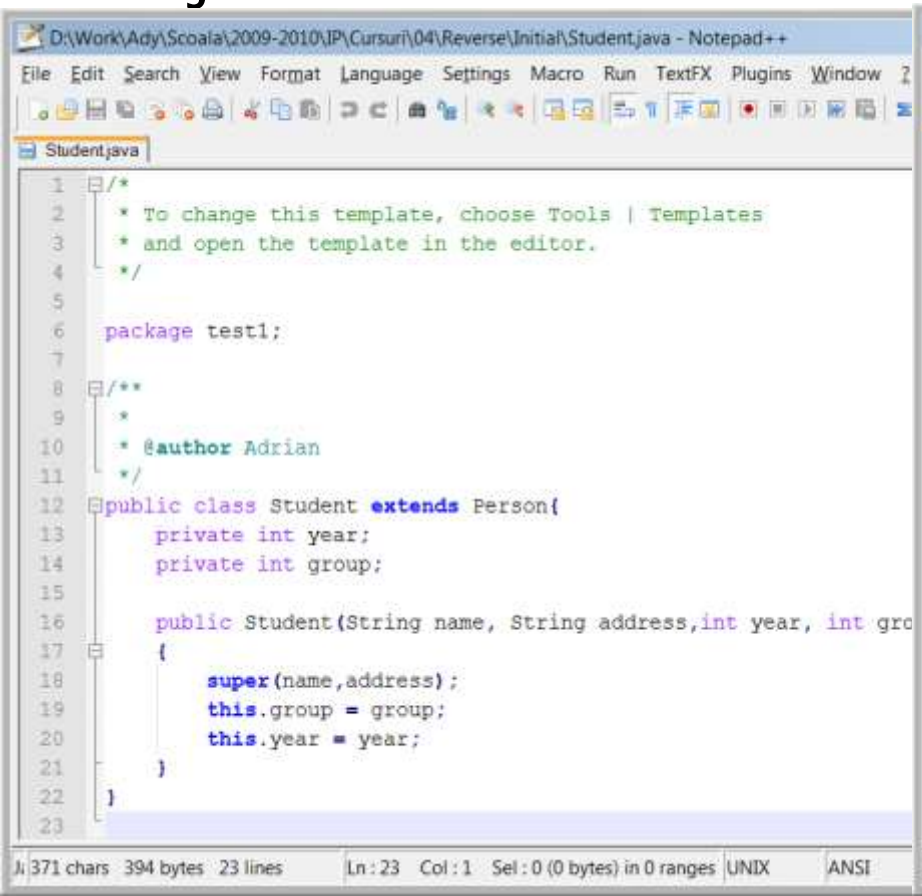
RE4: DJ Java Decompiler 3.10.10.93

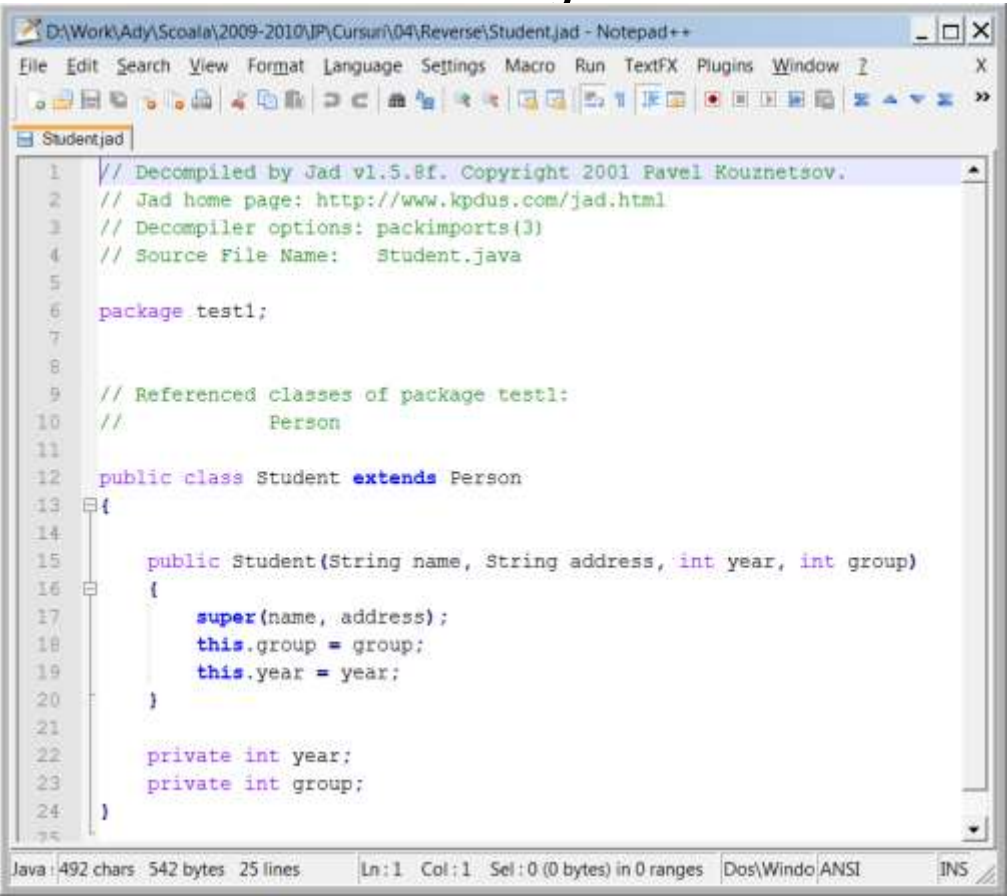
```
public class Test
{
   private int n;
   private int m;

   public static void main(String
   args[])
   {
        for(int i=1;i<10;i++)
        System.out.println("Test");
   }
}
```



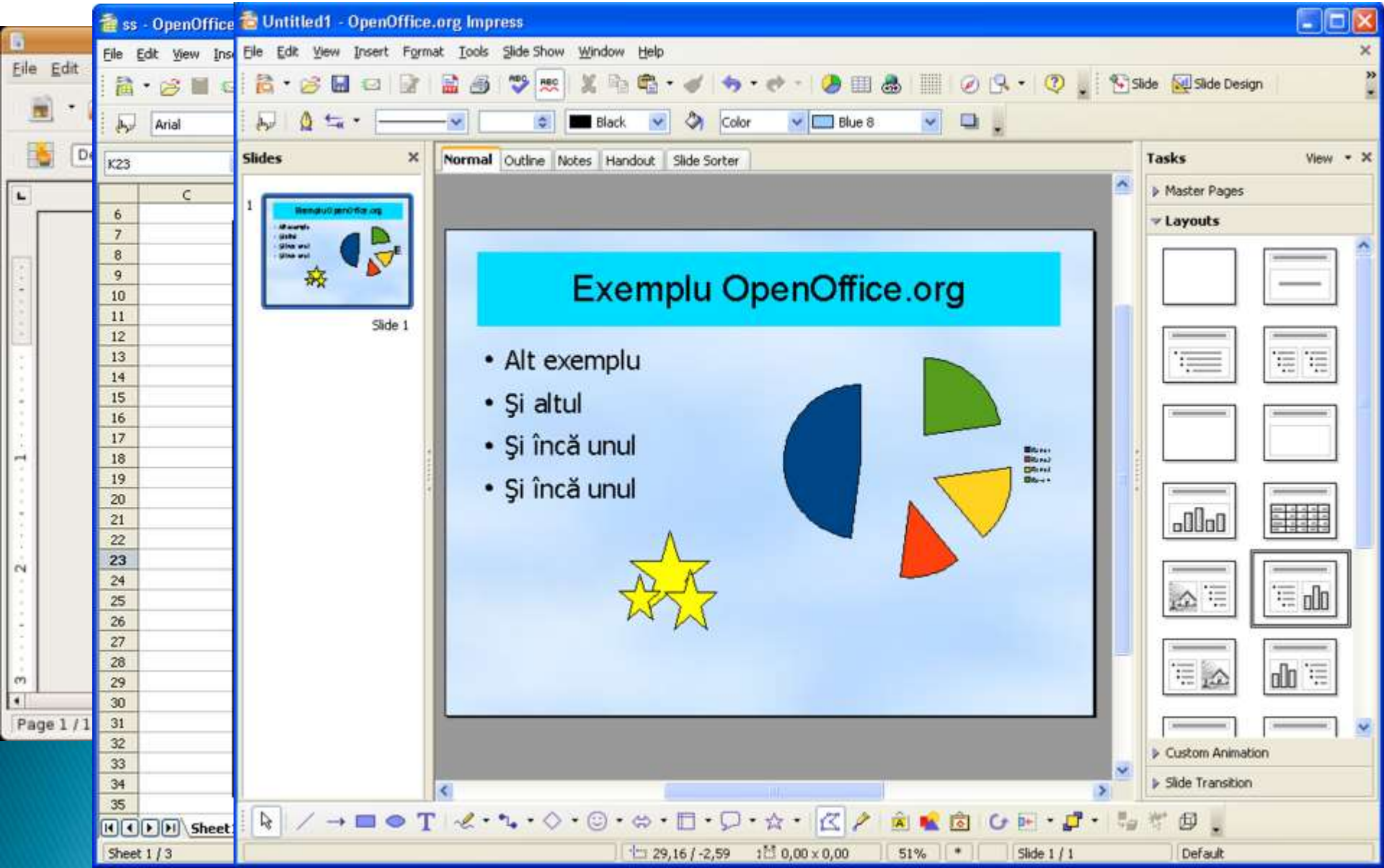
RE4: JAD

- Link: http://www.steike.com/code/java-reverse-engineering/
- jad.exe NumeFisier.class => NumeFisier.jad

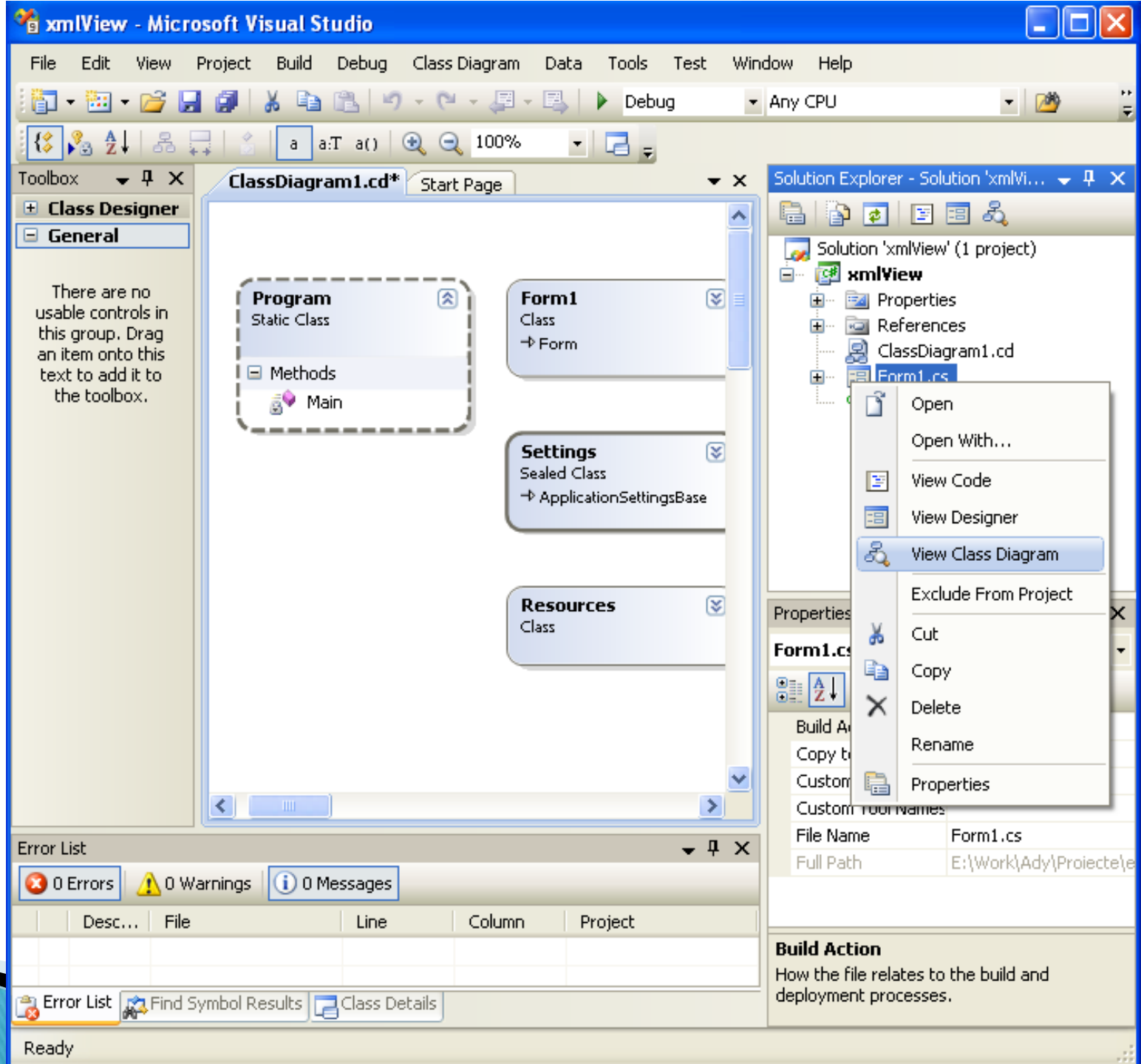




RE4: Open Office

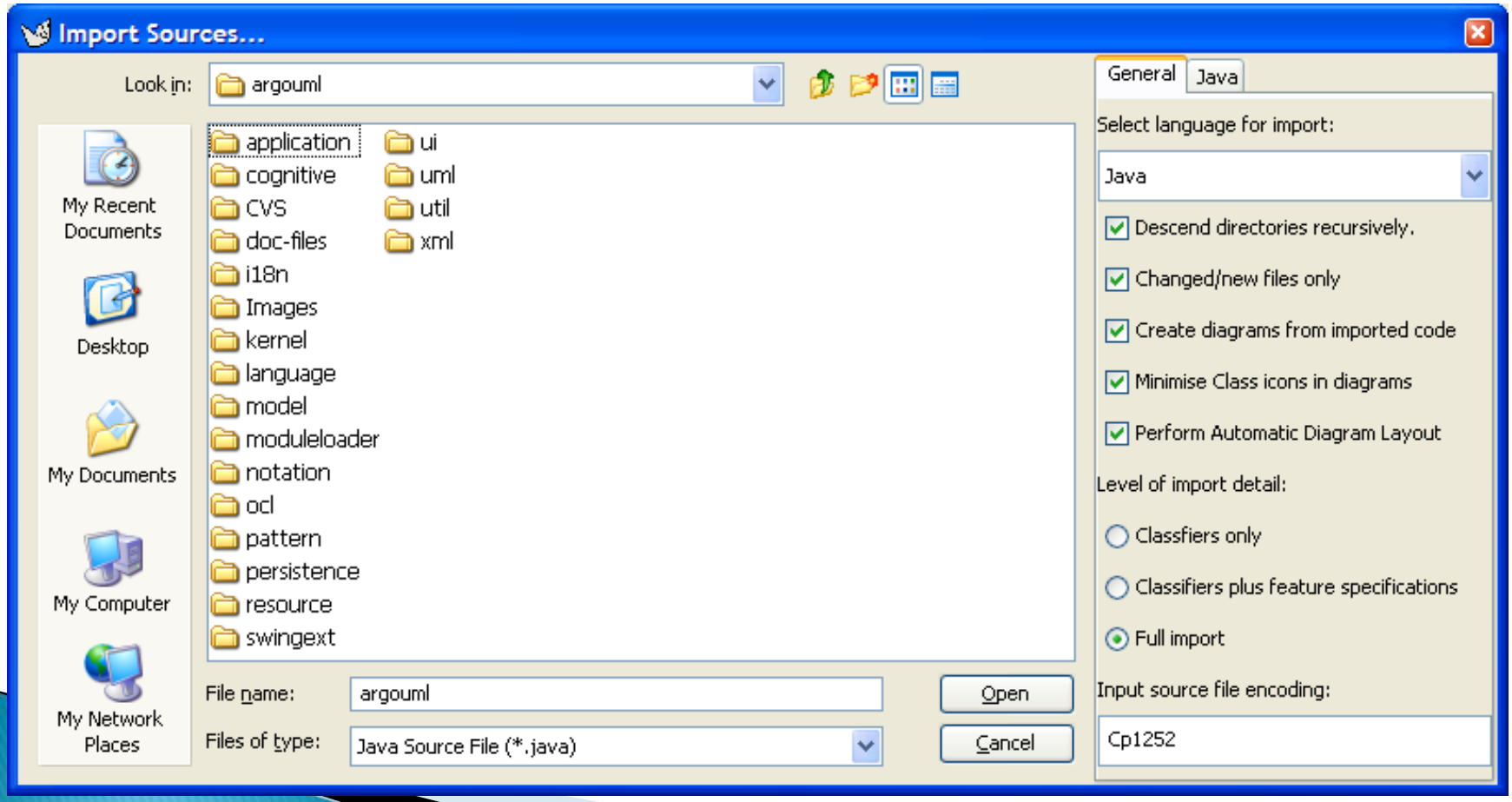


C#

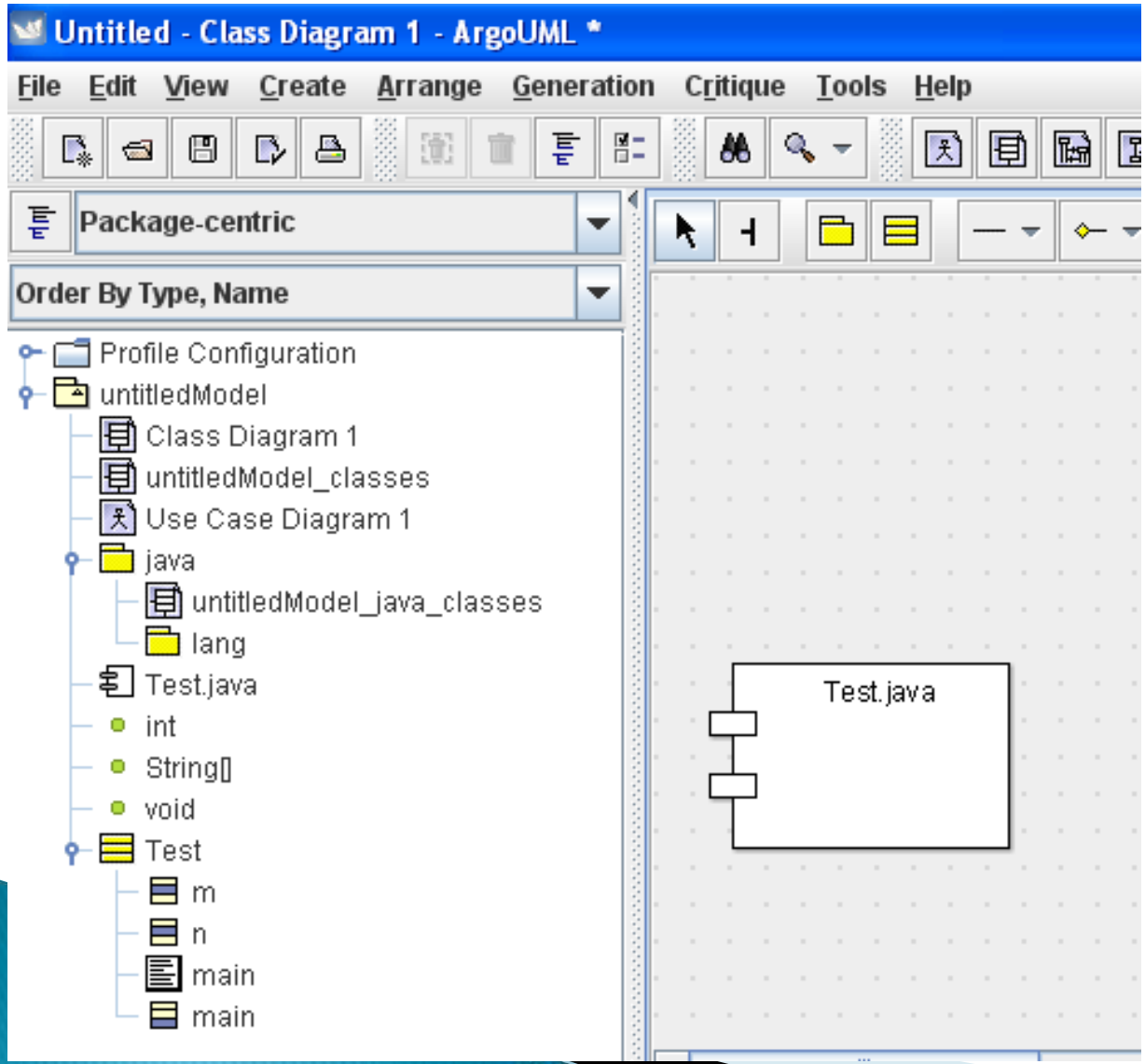

xmlView - Microsoft Visual Studio
File Edit View Project Build Debug Class Diagram Data Tools Test Window Help
Debug
Any CPU
100%
Toolbox
Class Designer
General
There are no usable controls in this group. Drag an item onto this text to add it to the toolbox.
ClassDiagram1.cd*
Start Page
Program
Static Class
Methods
Main
Form1
Class
Form
Settings
Sealed Class
ApplicationSettingsBase
Resources
Class
Solution Explorer - Solution 'xmlVi...
Solution 'xmlView' (1 project)
xmlView
Properties
References
ClassDiagram1.cd
Form1.cs
Open
Open With...
View Code
View Designer
View Class Diagram
Exclude From Project
Cut
Copy
Delete
Rename
Properties
Properties
Form1.cs
File Name
Form1.cs
Full Path
E:\Work\Ady\Proiecte\e
Build Action
How the file relates to the build and deployment processes.
Error List
0 Errors
0 Warnings
0 Messages
Desc...
File
Line
Column
Project
Error List
Find Symbol Results
Class Details
Ready

RE în ArgoUML

- File -> **Import Sources...**



Pentru exemplul anterior...



GRASP

- GRASP = General Responsibility Assignement Software Patterns (Principles)
- Descrise de Craig Larman în cartea *Applying UML and Patterns. An Introduction to Object Oriented Analysis and Design*
- **Ne ajută să alocăm responsabilități claselor și obiectelor în cel mai elegant mod posibil**
- Exemple de principii folosite în GRASP: *Information Expert* (sau Expert), *Creator*, *High Cohesion*, *Low Couplig*, *Controller* Polymorphism, Pure Fabrication, Indirection, Protected Variations

Ce responsabilități?

- Să facă:
  - Să facă ceva el însuși, precum crearea unui obiect sau să facă un calcul
  - Inițializarea unei acțiuni în alte obiecte
  - Controlarea și coordonarea activităților altor obiecte
- Să cunoască:
  - Atributele private
  - Obiectele proprii
  - Lucrurile pe care le poate face sau le poate apela

Pattern

- Traducere: șablon, model
- Este o soluție generală la o problemă comună
- Fiecare pattern are un nume sugestiv și ușor de reținut (ex. composite, observer, iterator, singleton, etc.)

Information Expert 1

- **Problemă**: dat un anumit comportament (operație), cărei clase trebuie să-i fie atribuit?

- O alocare bună a operațiilor conduce la sisteme care sunt:
  - Ușor de înțeles
  - Mai ușor de extins
  - Refolosibile
  - Mai robuste

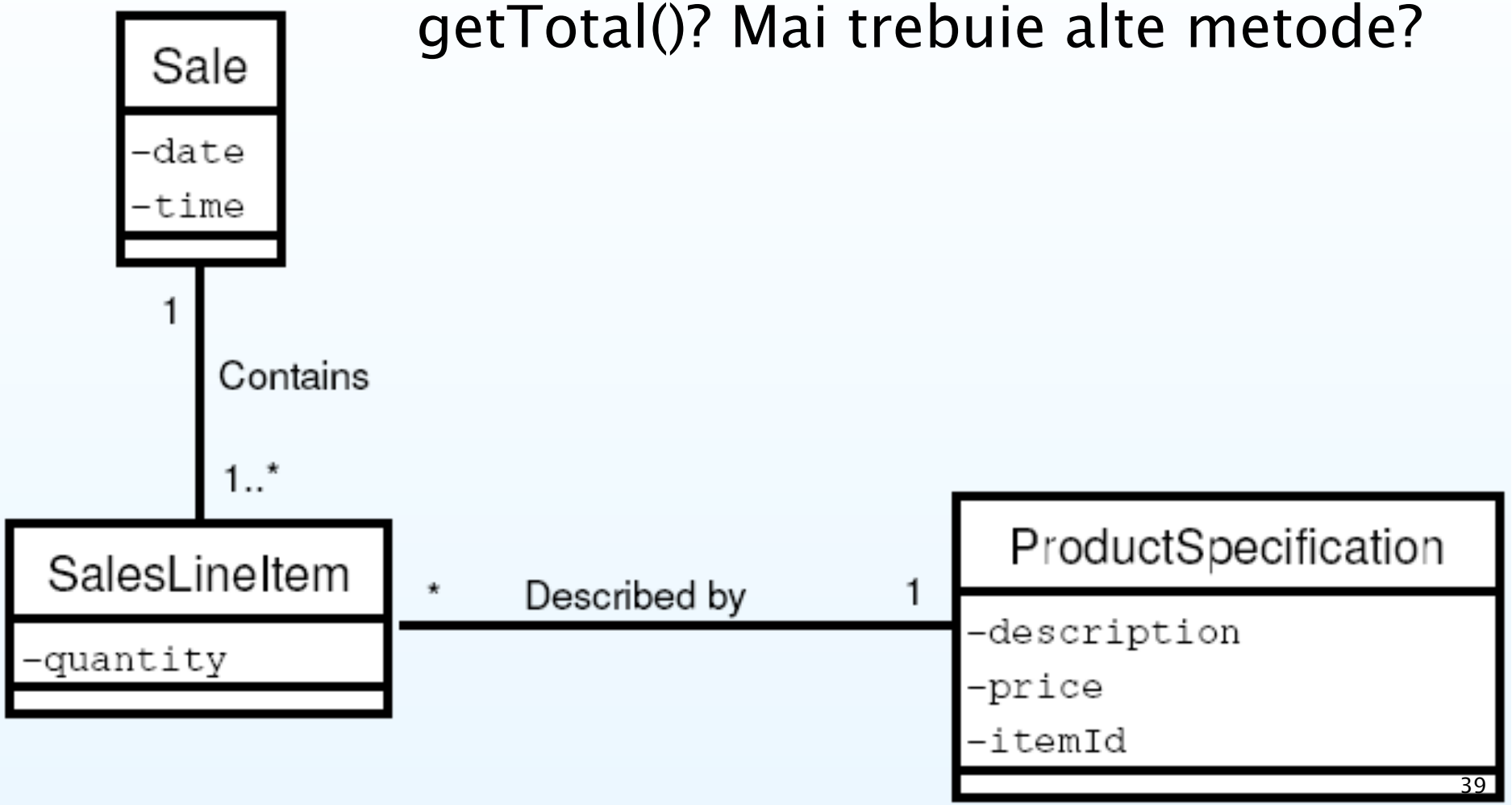
Information Expert 2

- **Soluție**:
- asignez o responsabilitate clasei care are *informațiile necesare* pentru îndeplinirea acelei responsabilități

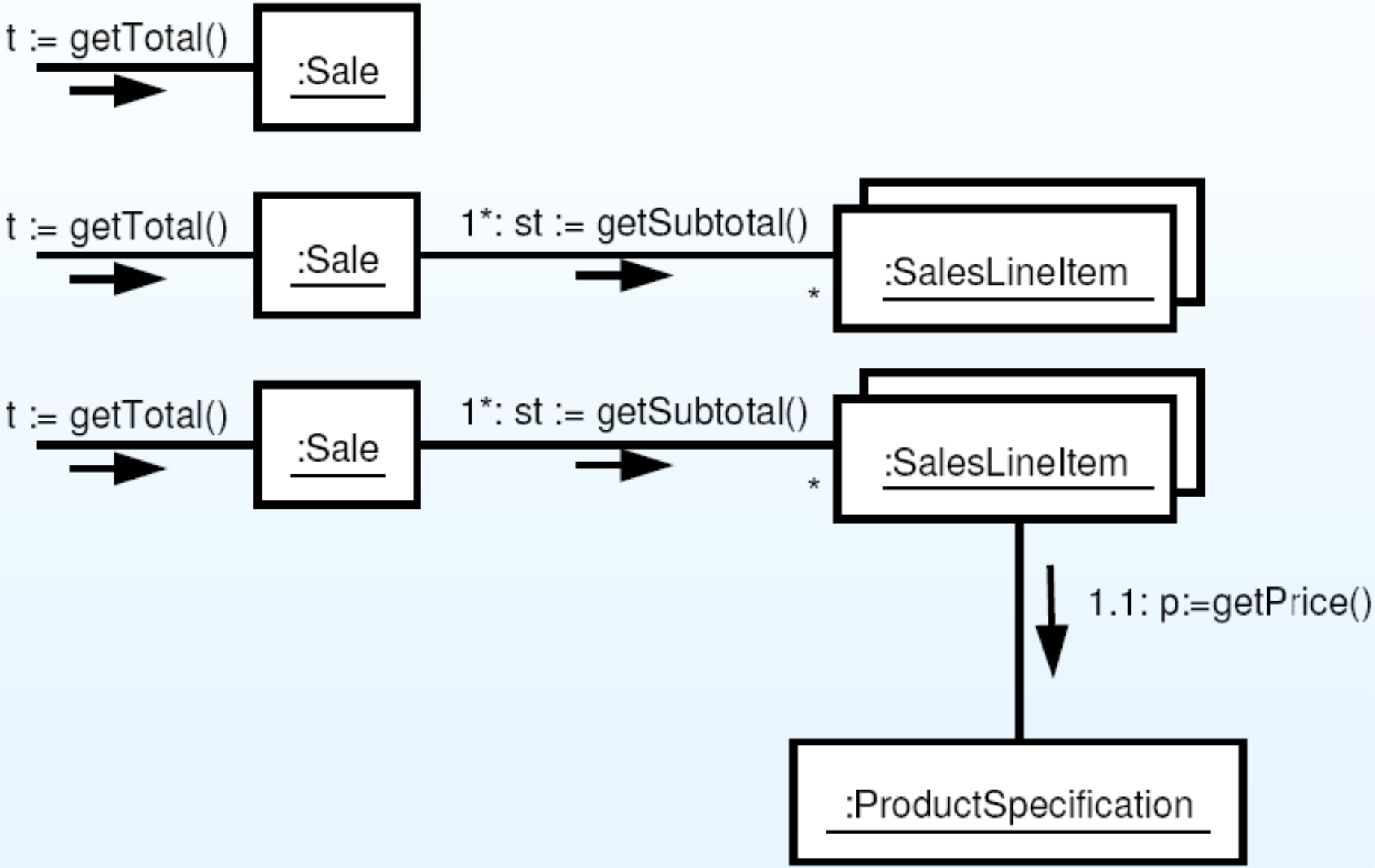
- **Recomandare**:
- începeți asignarea responsabilităților evidențiind clar care sunt responsabilitățile

Exemplul 1

- Carei clase trebuie sa-i fie asignată metoda getTotal()? Mai trebuie alte metode?

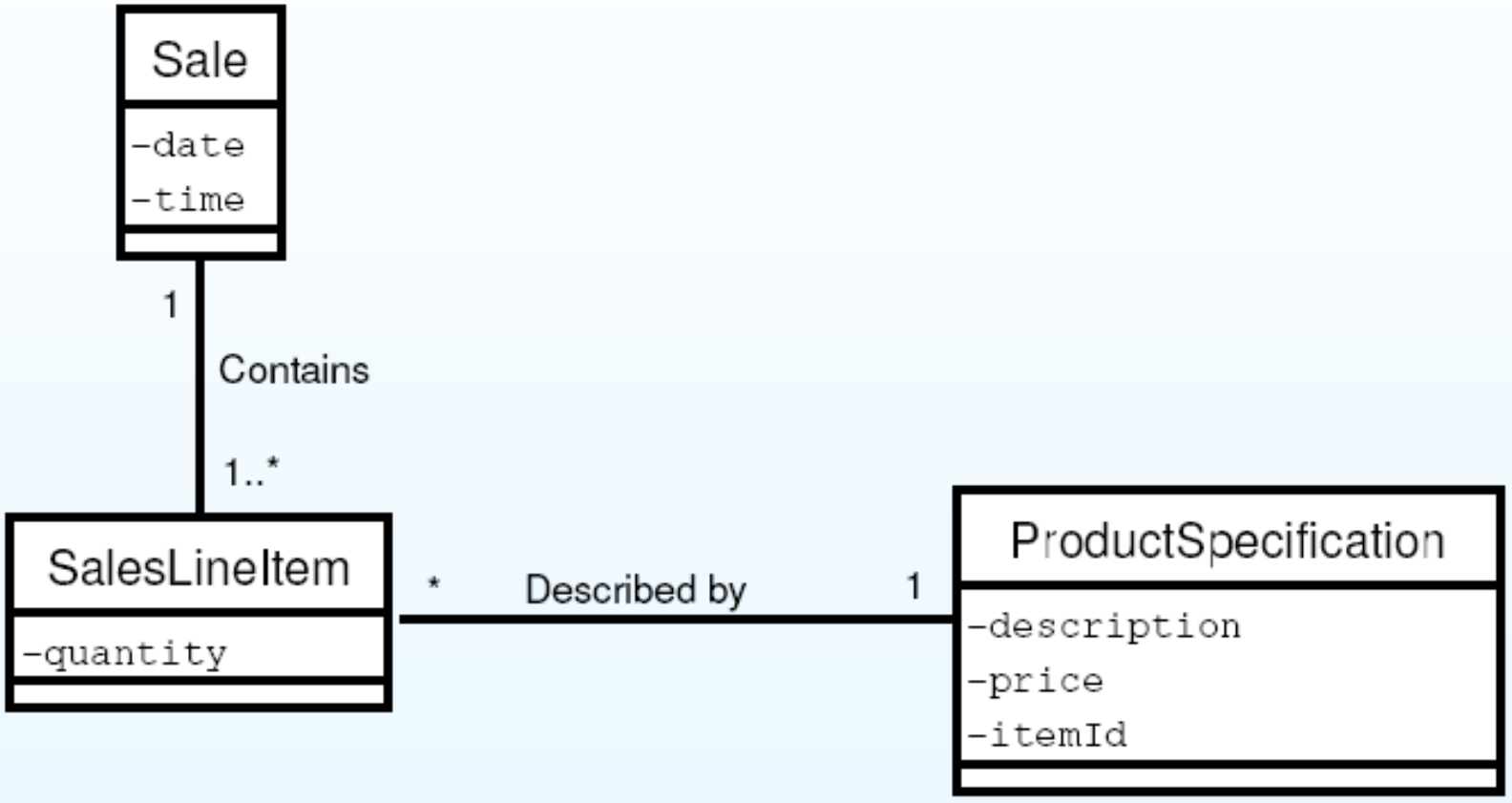


Exemplul 2

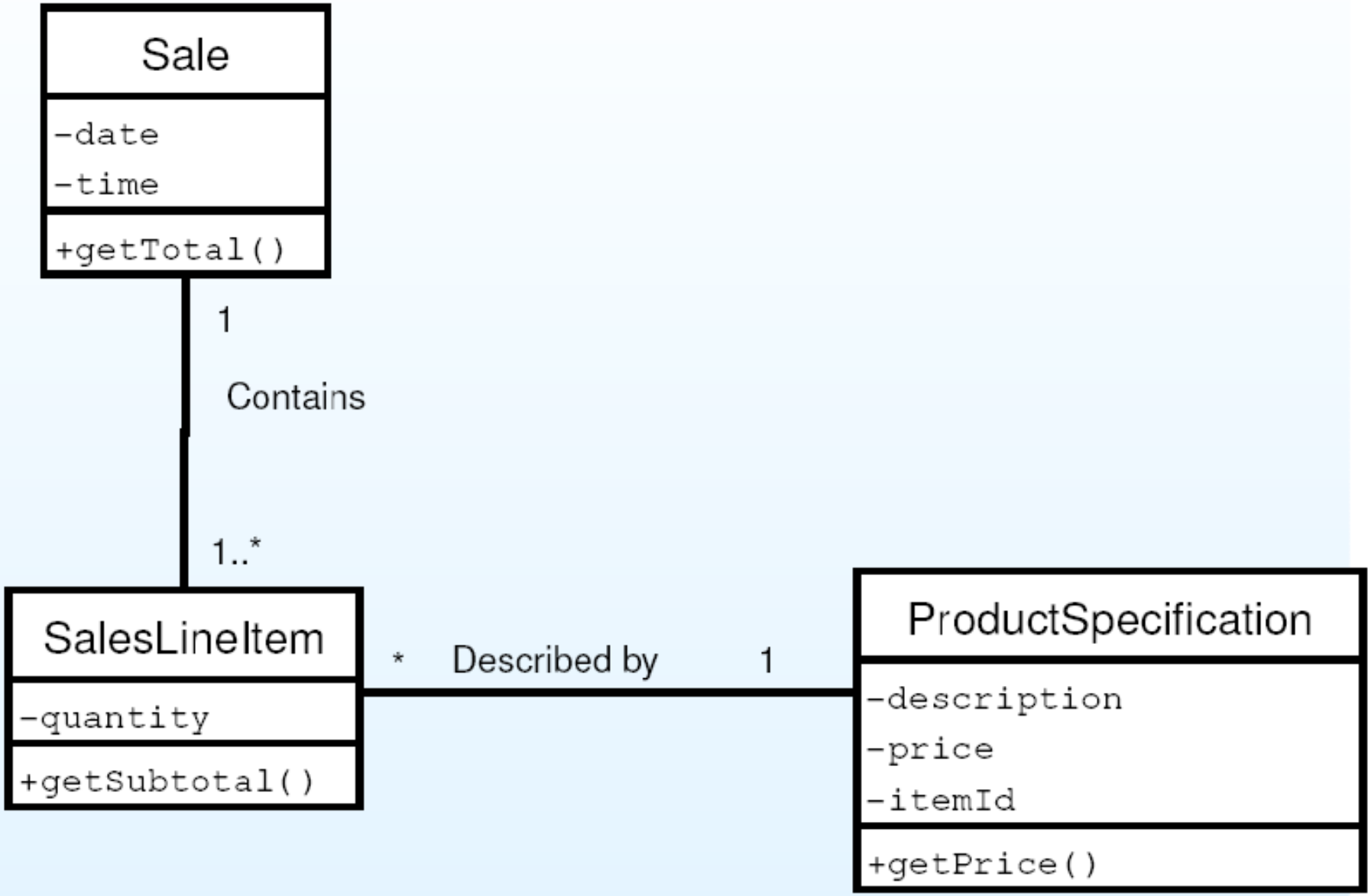

t := getTotal()
:Sale
t := getTotal()
:Sale
1*: st := getSubtotal()
*
:SalesLineItem
t := getTotal()
:Sale
1*: st := getSubtotal()
*
:SalesLineItem
1.1: p:=getPrice()
:ProductSpecification

Soluție posibilă 1

Clasă	Responsabilități
Sale	să cunoască valoarea totală a cumpărăturilor
SalesLineItem	să cunoască subtotalul pentru un produs
ProductSpecification	să cunoască prețul produsului



Soluție posibilă 2

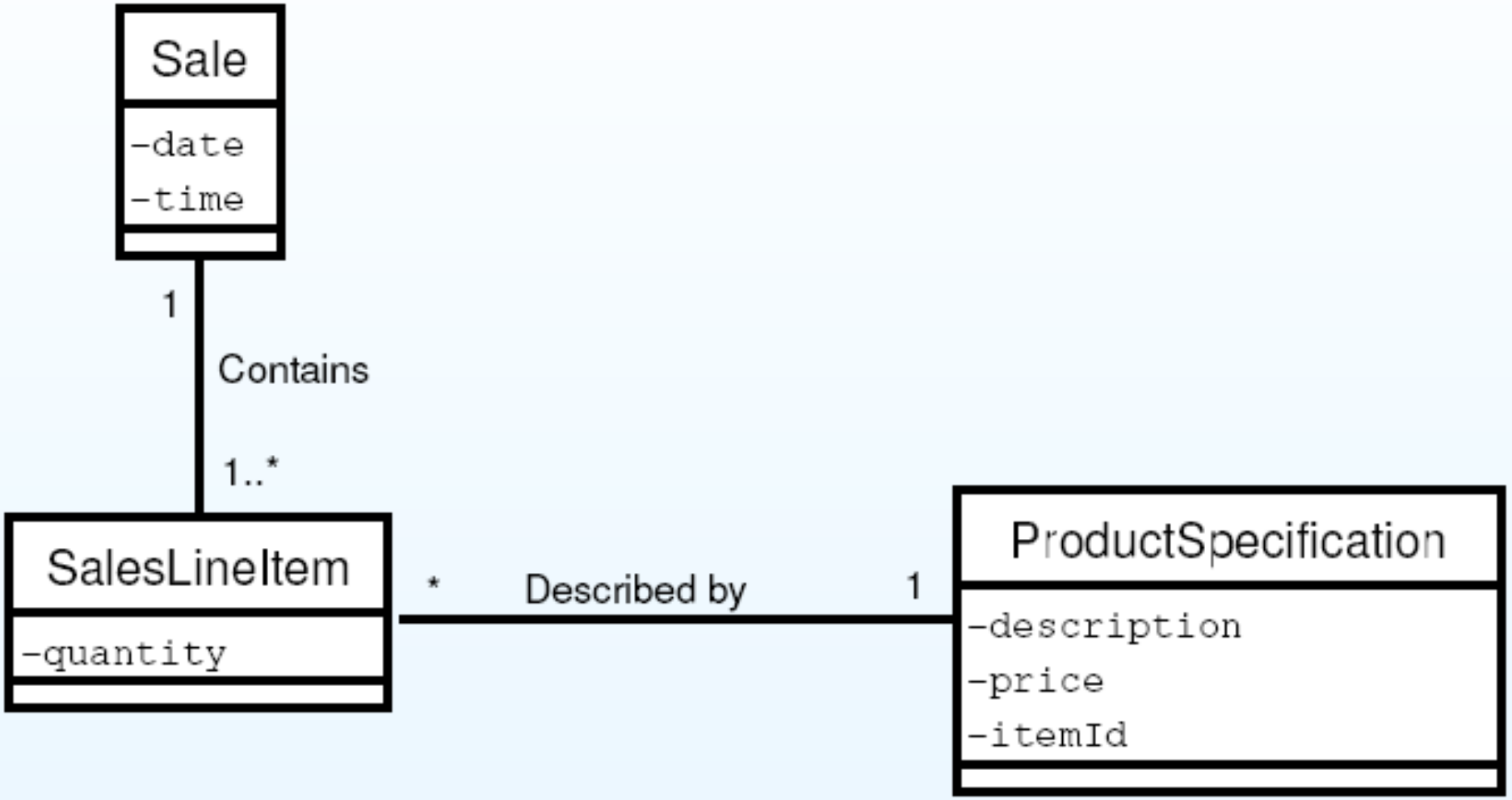


Creator 1

- **Problemă**: cine trebie să fie responsabil cu crearea unei instanțe a unei clase?
- **Soluție**: Asignați clasei B responsabilitatea de a crea instanțe ale clasei A doar dacă cel puțin una dintre următoarele afirmații este adevărată:
  - B *agregă obiecte de tip A*
  - B *conține obiecte de tip A*
  - B *folosește obiecte de tip A*
  - B *are datele de inițializare care trebuie transmise la* instanțierea unui obiect de tip A (B este deci un Expert în ceea ce privește crearea obiectelor de tip A)
- *Factory pattern* este o variantă mai complexă

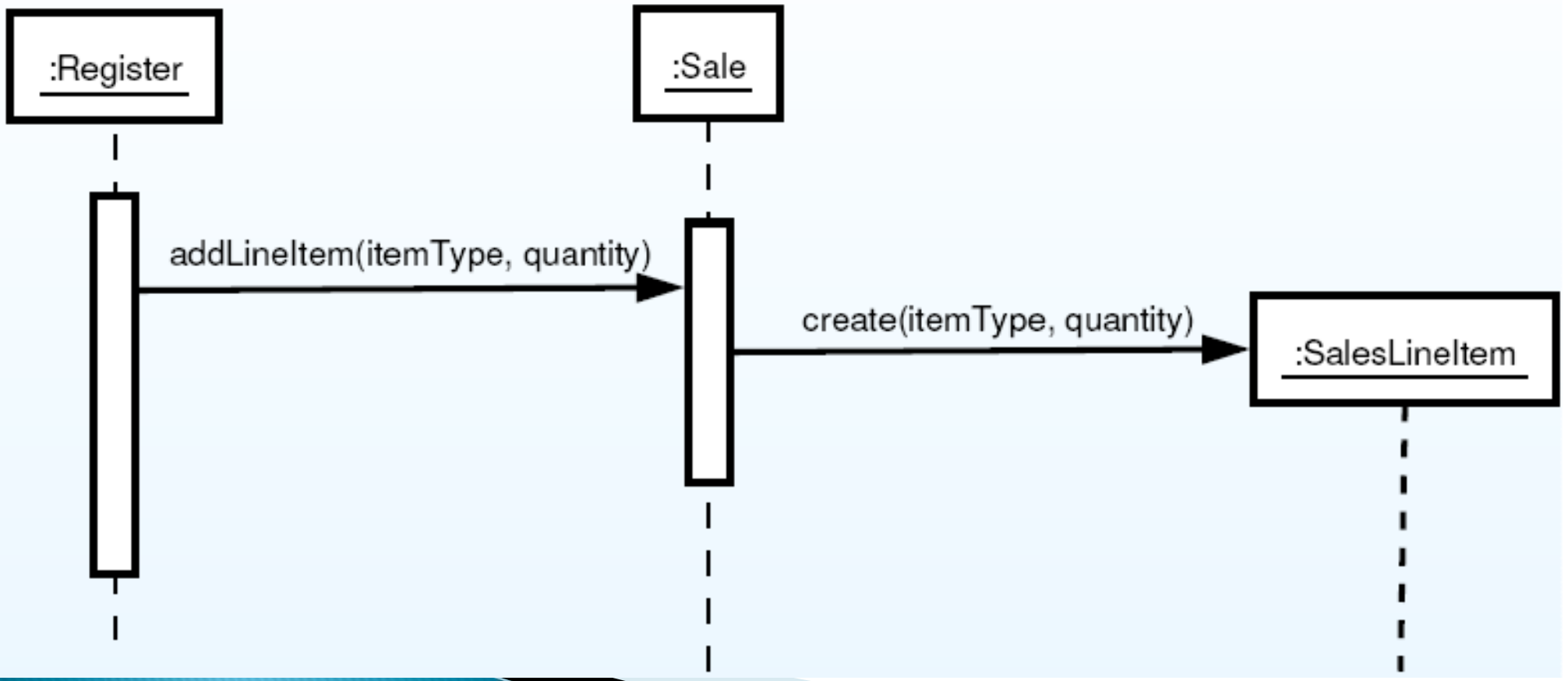
Creator 2

- Cine este responsabil cu crearea unei instanțe a clasei SalesLineItem?



Creator 3

- Deoarece Sale conține (agregă) instanțe de tip SalesLineItem, Sale este un bun candidat pentru a i se atribui responsabilitatea creării acestor instanțe



Low coupling (cuplaj redus)

- Cuplajul este o măsură a gradului de dependență a unei clase de alte clase
- Tipuri de Dependență:
  - este conectată cu
  - are cunoștințe despre
  - se bazează pe
- **O clasă care are cuplaj mic (redus) nu depinde de "multe" alte clase; unde "multe" este dependent de contex**
- O clasă care are cuplaj mare depinde de multe alte clase

Cuplaj 2

- Probleme cauzate de cuplaj:
  - schimbări în clasele relaționate forțează schimbări locale
  - clase **greu de înțeles** în izolare (scoase din context)
  - clase **greu de refolosit** deoarece folosirea lor presupune și prezența claselor de care depind

Cuplaj 3

- Forme comune de cuplaj de la clasa A la clasa B sunt:
  - A are un atribut de tip B
  - O instanță a clasei A apelează un serviciu oferit de un obiect de tip B
  - A are o metodă care referențiază B (parametru, obiect local, obiect returnat)
  - A este subclasă (direct sau indirect) a lui B
  - B este o interfață, iar A implementează această interfață

Legea lui Demeter

- *Don't talk to strangers*

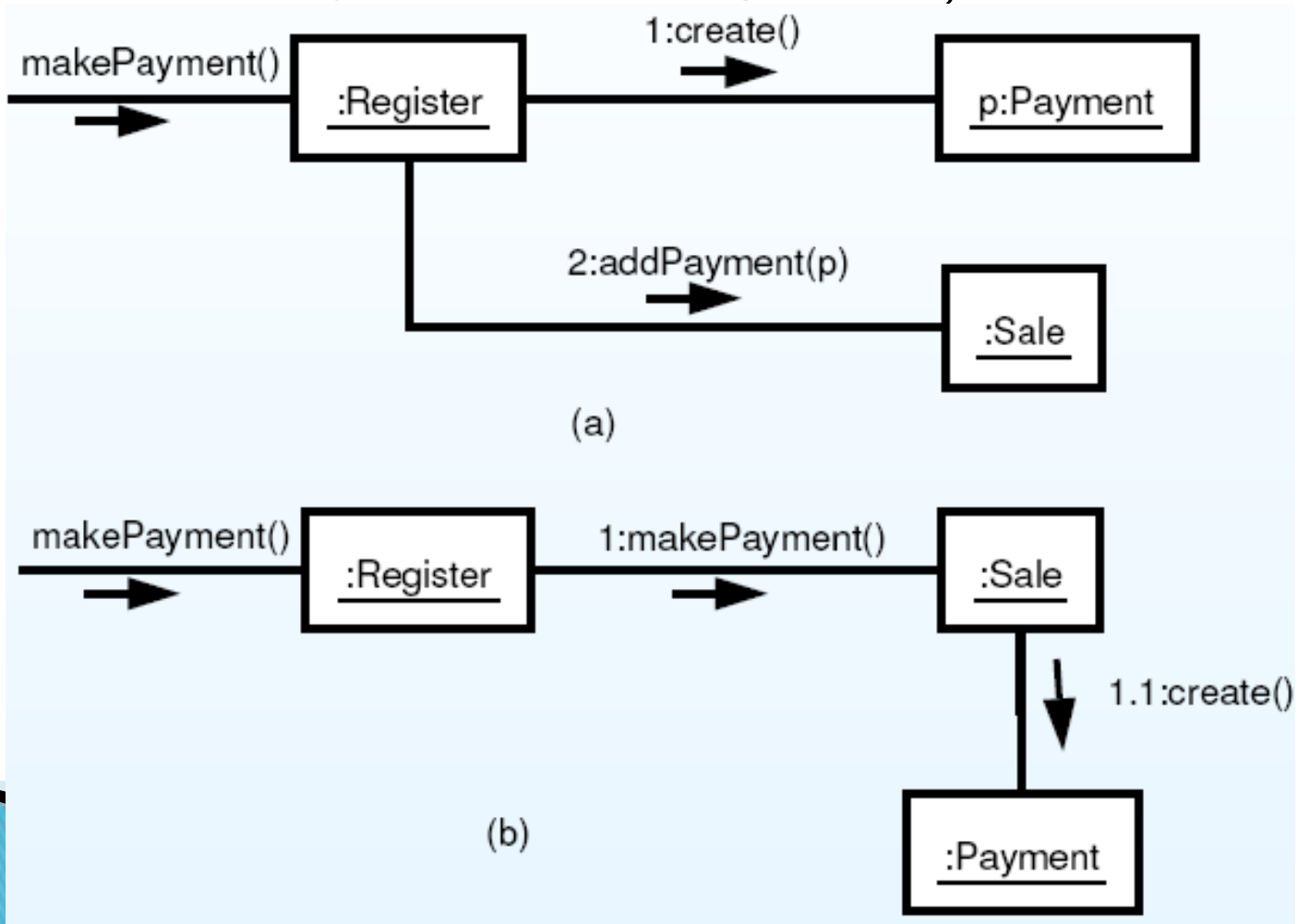
- Orice metodă a unui obiect trebuie să apeleze doar metode aparținând
  - lui însuși
  - oricărui parametru al metodei
  - oricărui obiect pe care l-a creat
  - oricăror obiecte pe care le conține

Vizualizarea Cuplajelor

- Diagrama de clase
- Diagrama de colaborare

Exemplul 1

- Exista legături între toate clasele
- Elimină cuplajul dintre Register și Payment



High Cohesion

- **Coeziunea** este o măsură a cât de puternic sunt focalizate responsabilitățile unei clase
- O clasă ale cărei responsabilități sunt foarte strâns legate și care nu face foarte multe lucruri are o **coeziune mare**
- O clasă care face multe lucruri care nu sunt relaționate sau face prea multe lucruri are o **coeziune mică (slabă)**

Coeziune

- Probleme cauzate de o slabă coeziune:
  - greu de înțeles
  - greu de refolosit
  - greu de menținut
  - delicate; astfel de clase sunt mereu supuse la schimbări

Coeziune și Cuplaj

- Sunt principii vechi în design-ul software
- Promovează un design modular
- Modularitatea este proprietatea unui sistem care a fost descompus într-o mulțime de module coezive și slab cuplate

Controller 1

- **Problemă**: Cine este responsabil cu tratarea unui eveniment generat de un actor?
- Aceste evenimente sunt asociate cu operații ale sistemului
- Un **Controller** este un obiect care nu ține de interfața grafică și care este responsabil cu recepționarea sau gestionarea unui eveniment
- Un Controller definește o metodă corespunzătoare operației sistemului

Controller 2

- **Soluție**: asignează responsabilitatea pentru recepționarea sau gestionarea unui eveniment unei clase care reprezintă una dintre următoarele alegeri:
  - Reprezintă întregul sistem sau subsistem (fațadă controller)
  - Reprezintă un scenariu de utilizare în care apare evenimentul;

Controller 3

- În mod normal, un controller ar trebui să delege altor obiecte munca care trebuie făcută;
- **Controller-ul coordonează sau controlează activitatea, însă nu face prea multe lucruri el însuși**
- O greșeală comună în design-ul unui controller este să i se atribuie prea multe responsabilități (fațade controller)

Concluzii

- Forward & Reverse Engineering

- GRASP
  - Information Expert
  - Creator
  - Low coupling
  - High cohesion
  - Controller

Bibliografie

- Reverse Engineering and Design Discovery: A Taxonomy, Chikofsky, E.J. and Cross, J., January, 1990
- Craig Larman. *Applying UML and Patterns. An Introduction to Object Oriented Analysis and Design*
- Ovidiu Gheorghieș, Curs 6 IP

Links (RE)

- **DJ Java Decompiler 3.10.10.93:** http://www.softpedia.com/progDownload/DJ-Java-Decompiler-Download-13481.html
- **Open Office**: http://ro.wikipedia.org/wiki/OpenOffice.org
- **UML Reverse Engineering for Existing Java, C# , and Visual Basic .NET Code:** **http://www.altova.com/umodel/uml-reverse-engineering.html**
- **Reverse Engineering**: http://en.wikipedia.org/wiki/Reverse_engineering
- **PROTO 3000 3D Engineering Solutions:** http://www.proto3000.com/services.aspx
- HAR2009: http://www.degate.org/HAR2009/
- Degate: http://www.degate.org/screenshots/
- Inteligent: http://www.intelligentrd.com/
- Smartphones RE: http://www.cytraxsolutions.com/2011/01/smartphones-security-and-reverse.html

Links (GRASP)

- WebProjectManager: http://profs.info.uaic.ro/~adrianaa/uml/
- Diagrame de Stare și de Activitate: http://software.ucv.ro/~soimu_anca/itpm/Diagrame%20de%20Stare%20si%20Activitate.doc
- Deployment Diagram: http://en.wikipedia.org/wiki/Deployment_diagram http://www.agilemodeling.com/artifacts/deploymentDiagram.htm
- GRASP: http://en.wikipedia.org/wiki/GRASP_(Object_Oriented_Design)
- http://web.cs.wpi.edu/~gpollice/cs4233-a05/CourseNotes/maps/class4/GRASPpatterns.html
- Introduction to GRASP Patterns: http://faculty.inverhills.edu/dlevitt/CS%202000%20(FP)/GRASP%20Patterns.pdf

Ingineria Programării

Cursul 6 – 27 Martie 2017

adiftene@info.uaic.ro

Cuprins

- Din Cursurile trecute…
- SOLID Principles
- Design Patterns
  - Definitions
  - Elements
  - Example
  - Classification
- JUnit Testing
  - Netbeans (Exemplu 1)
  - Eclipse (Exemplu 2)

Din Cursurile Trecute

- Etapele Dezvoltării Programelor
- Ingineria Cerinţelor
- Diagrame UML
- GRASP

R – GRASP

- Principii, responsabilități
- Information Expert
- Creator
- Low Coupling
- High Cohesion
- Controller

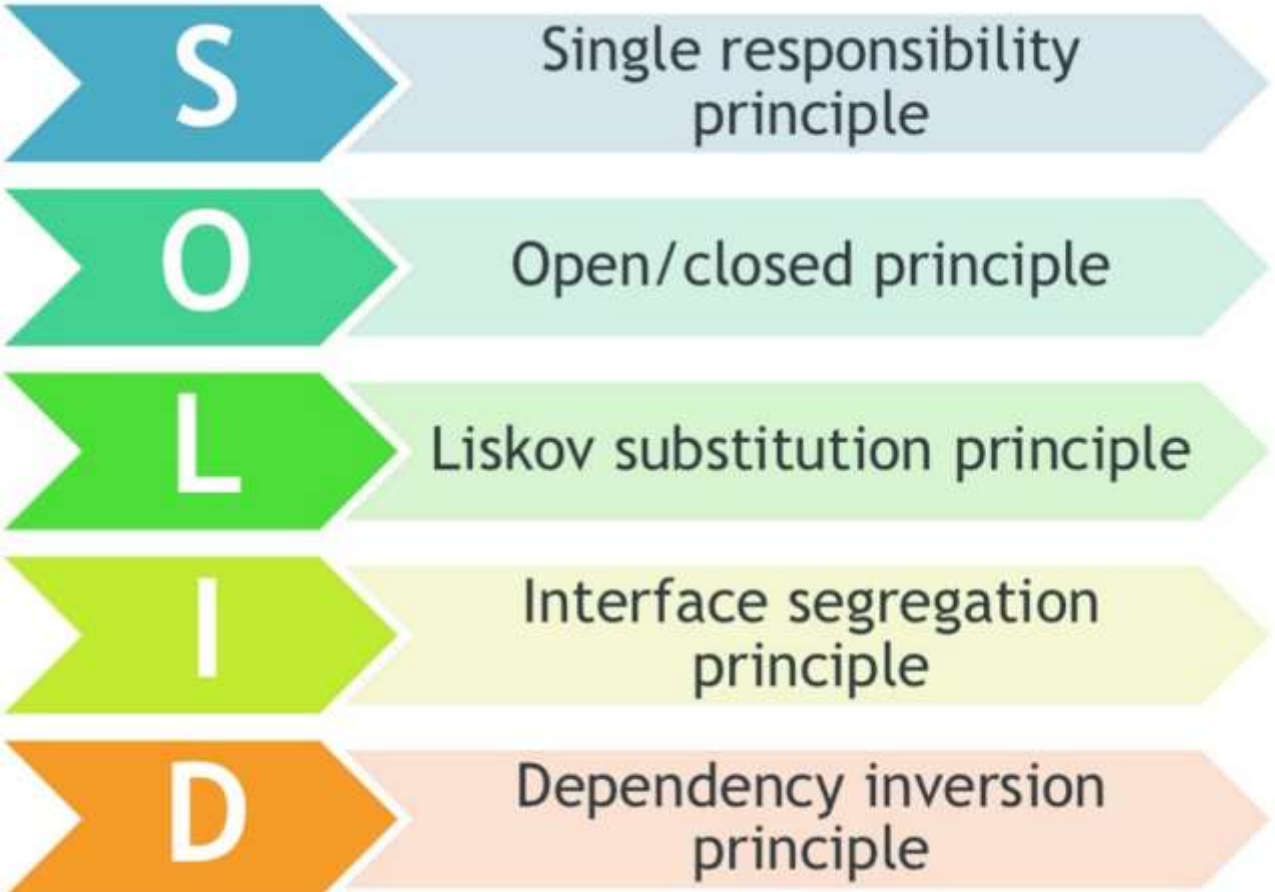
SOLID and Other Principles

- SOLID Principles
  - SRP – Single Responsibility Principle
  - OCP – Open/Closed Principle
  - LSP – Liskov Substitution Principle
  - ISP – Interface Segregation Principle
  - DIP – Dependency Inversion Principle
- DRY – Don't Repeat Yourself
- YAGNI – You Aren't Gonna Need It
- KISS – Keep It Simple, Stupid

S.O.L.I.D.
Principles

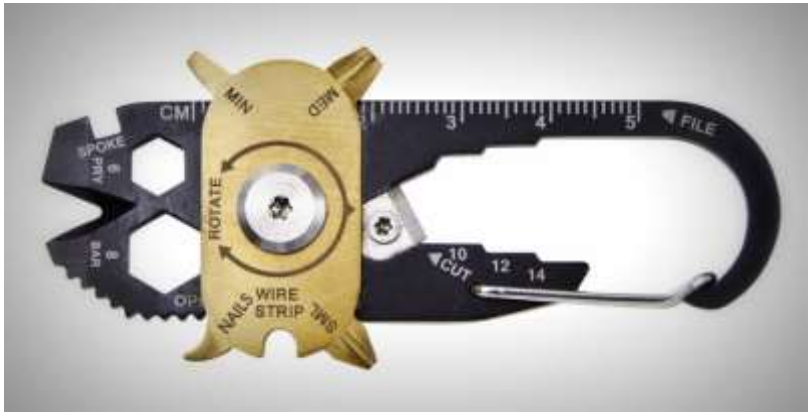
SOLID

- SOLID was introduced by Robert C. Martin in the an article called the “Principles of Object Oriented Design” in the early 2000s



SOLID – Single Responsibility Principle

- Every object should have a single responsibility, and all its services should be narrowly aligned with that responsibility



SOLID – SRP – Definitions

- “The Single Responsibility Principle states that every object should have a single responsibility, and that responsibility should be entirely encapsulated by the class.” – Wikipedia
- “There should never be more than one reason for a class to change.” – Robert Martin
- Low coupling & strong cohesion

SOLID – SRP – Problems & Solutions

- Classic violations
  - Objects that can print/draw themselves
  - Objects that can save/restore themselves
- Classic solution
  - Separate printer & Separate saver
- Solution
  - Multiple small interfaces (ISP)
  - Many small classes
  - Distinct responsibilities
- Result
  - Flexible design
  - Lower coupling & Higher cohesion

SOLID – SRP – Example

- Two responsabilities

```
interface Modem {
 public void dial(String pno);
 public void hangup();

 public void send(char c);
 public char recv();
}
```

- Separated interfaces

```
interface DataChannel {
 public void send(char c);
 public char recv();
}
```

```
interface Connection {
 public void dial(String phn);
 public char hangup();
}
```

SOLID – Open/Closed Principle

- *Open chest surgery is not needed when putting on a coat*
- Bertrand Meyer originated the OCP term in his 1988 book, *Object Oriented Software Construction*

SOLID – OCP – Definitions

- “The Open / Closed Principle states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.” – Wikipedia
- “All systems change during their life cycles. This must be borne in mind when developing systems expected to last longer than the first version.” – Ivar Jacobson
- **Open** to Extension – New behavior can be added in the future
- **Closed** to Modification – Changes to source or binary code are not required

SOLID – OCP – How?

- Change behavior without changing code?!
  - Rely on abstractions, not implementations
  - Do not limit the variety of implementations
- In .NET – Interfaces, Abstract Classes
- In procedural code – Use parameters
- Approaches to achieve OCP
  - Parameters – Pass delegates / callbacks
  - Inheritance / Template Method pattern – Child types override behavior of a base class
  - Composition / Strategy pattern – Client code depends on abstraction, "Plug in" model

SOLID – OCP – Problems & Solutions

- Classic violations
  - Each change requires re-testing (possible bugs)
  - Cascading changes through modules
  - Logic depends on conditional statements
- Classic solution
  - New classes (nothing depends on them yet)
  - New classes (no legacy coupling)
- When to apply OCP?
  - Experience tell you
- OCP add complexity to design (TANSTAAFL)
- No design can be closed against all changes

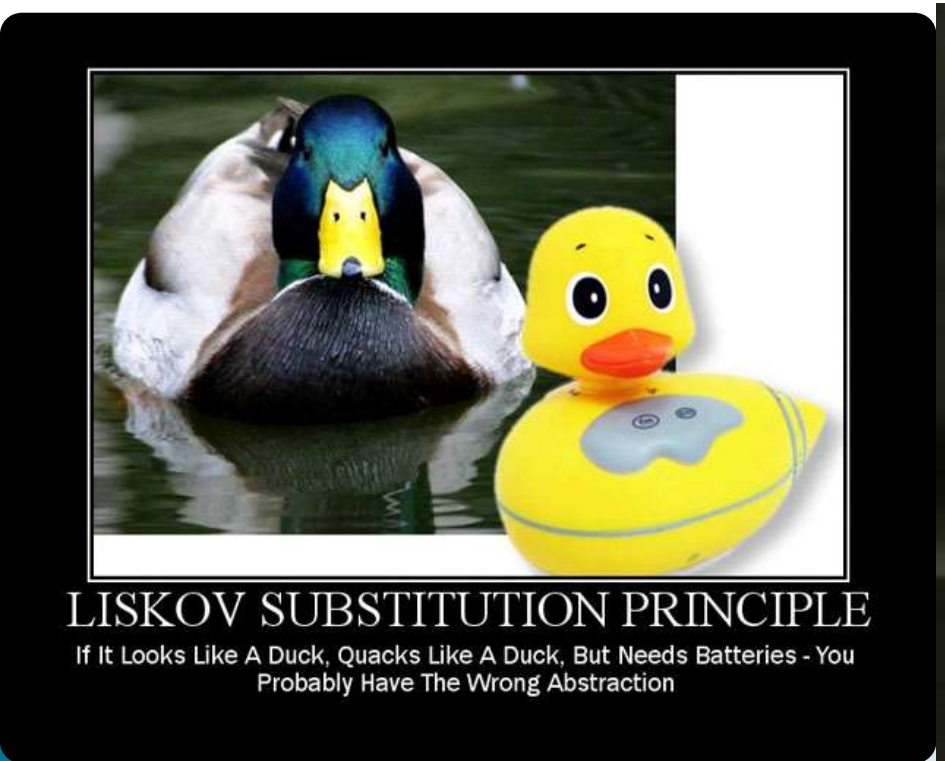
SOLID – OCP – Example

```
// Open-Close Principle - Bad example
class GraphicEditor {
 public void drawShape(Shape s) {
  if (s.m_type==1)
        drawRectangle(s);
  else if (s.m_type==2)
        drawCircle(s);
 }
 public void drawCircle(Circle r)
{....}
 public void drawRectangle(Rectangle r)
{....}
}

class Shape {
 int m_type;
}

class Rectangle extends Shape {
 Rectangle() {super.m_type=1;}
}

class Circle extends Shape {
 Circle() {super.m_type=2;}
}
```

```
// Open-Close Principle - Good
example
class GraphicEditor {
 public void drawShape(Shape s) {
        s.draw();
 }
}

class Shape {
        abstract void draw();
}

class Rectangle extends Shape {
 public void draw() {
        // draw the rectangle
 }
}
```

SOLID – Liskov Substitution

- If it looks like a duck, quacks like a duck, but needs batteries – you probably have the wrong abstraction
- Barbara Liskov described the principle in 1988



SOLID – LSP – Definitions

- "The Liskov Substitution Principle states that Subtypes must be substitutable for their base types.“ – Agile Principles, Patterns, and Practices in C#
- Substitutability – child classes must not
  - Remove base class behavior
  - Violate base class invariants
- Normal OOP inheritance
  - IS-A relationship
- Liskov Substitution inheritance
  - IS-SUBSTITUTABLE-FOR

SOLID – LSP – Problems & Solutions

- The problem
  - Polymorphism break Client code expectations
  - "Fixing" by adding if-then – nightmare (OCP)
- Classic violations
  - Type checking for different methods
  - Not implemented overridden methods
  - Virtual methods in constructor
- Solutions
  - “Tell, Don’t Ask” – Don’t ask for types and Tell the object what to do
  - Refactoring to base class- Common functionality and Introduce third class

SOLID – LSP – Example (1)

```
// Violation of Liskov's Substitution Principle
 class Rectangle{
   int m_width;
   int m_height;

   public void setWidth(int width){
         m_width = width;
   }

   public void setHeight(int h){
         m_height = ht;
   }

   public int getWidth(){
         return m_width;
   }

   public int getHeight(){
         return m_height;
   }

   public int getArea(){
         return m_width * m_height;
   }
 }
```

```
class Square extends Rectangle {
  public void setWidth(int width){
        m_width = width;
        m_height = width;
  }

  public void setHeight(int height){
        m_width = height;
        m_height = height;
  }
 }
```

SOLID – LSP – Example (2)

```
class LspTest
{
private static Rectangle getNewRectangle()
{
        // it can be an object returned by some factory ...
        return new Square();
}

public static void main (String args[])
{
        Rectangle r = LspTest.getNewRectangle();
        r.setWidth(5);
        r.setHeight(10);
// user knows that r it's a rectangle. It assumes that he's able to set the width
 and height as for the base class

        System.out.println(r.getArea());
        // now he's surprised to see that the area is 100 instead of 50.
}
}
```

SOLID – Interface Segregation

- You want me to plug this in. Where?

SOLID – ISP – Definitions

- “The Interface Segregation Principle states that Clients should not be forced to depend on methods they do not use.” – Agile Principles, Patterns, and Practices in C#
- Prefer small, cohesive interfaces – Interface is the interface type + All public members of a class
- Divide "fat" interfaces into smaller ones
  - “fat” interfaces means classes with useless methods, increased coupling, reduced flexibility and maintainability

SOLID – ISP – Problems & Solutions

- Classic violations
  - Unimplemented methods (also in LSP)
  - Use of only small portion of a class
- When to fix?
  - Once there is pain! Do not fix, if is not broken!
  - If the "fat" interface is yours, separate it to smaller ones
  - If the "fat" interface is not yours, use "Adapter" pattern
- Solutions
  - Small interfaces
  - Cohesive interfaces
  - Focused interfaces
  - Let the client define interfaces
  - Package interfaces with their implementation

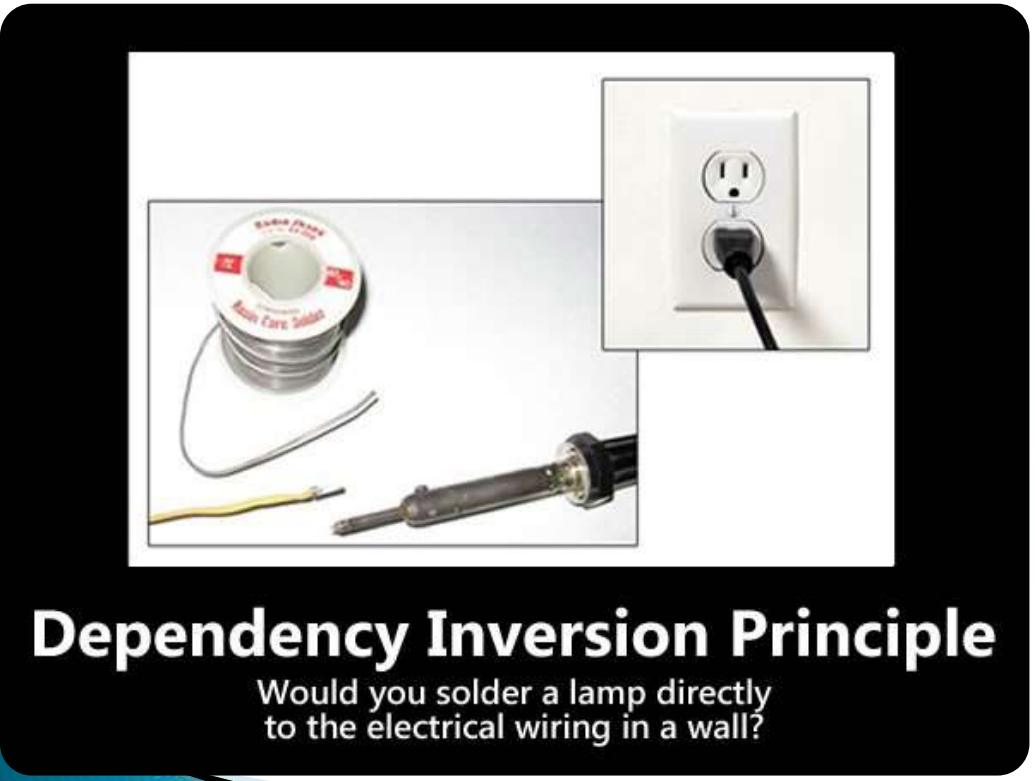
SOLID – ISP – Example

```
//Bad example (polluted interface)


interface Worker {
 void work();
 void eat();
}


ManWorker implements Worker {
        void work() {…};
        void eat() {30 min break;};
}


RobotWorker implements Worker {
        void work() {…};
        void eat() {//Not Appliciable
                for a RobotWorker};
}
```

```
//Solution: split into two interfaces


interface Workable {
        public void work();
 }

 interface Feedable{
        public void eat();
 }
```

SOLID – Dependency Inversion

- Would you solder a lamp directly to the electrical wiring in a wall?



SOLID – DIP – Definitions

- “High-level modules should not depend on low-level modules. Both should depend on abstractions.”

- “Abstractions should not depend on details. Details should depend on abstractions.” – Agile Principles, Patterns, and Practices in C#

SOLID – DIP – Dependency

- Framework
- Third Party Libraries
- Database
- File System
- Email
- Web Services
- System Resources (Clock)
- Configuration

- The new Keyword
- Static methods
- Thread.Sleep
- Random

SOLID – DIP – Problems & Solutions

- How it should be
  - Classes should declare what they need
  - Constructors should require dependencies
  - Dependencies should be abstractions and be shown
- How to do it
  - Dependency Injection
  - The Hollywood principle "Don't call us, we'll call you!"
- Classic violations
  - Using of the new keyword, static methods/properties
- How to fix?
  - Default constructor, main method/starting point
  - Inversion of Control container

SOLID – DIP – Example

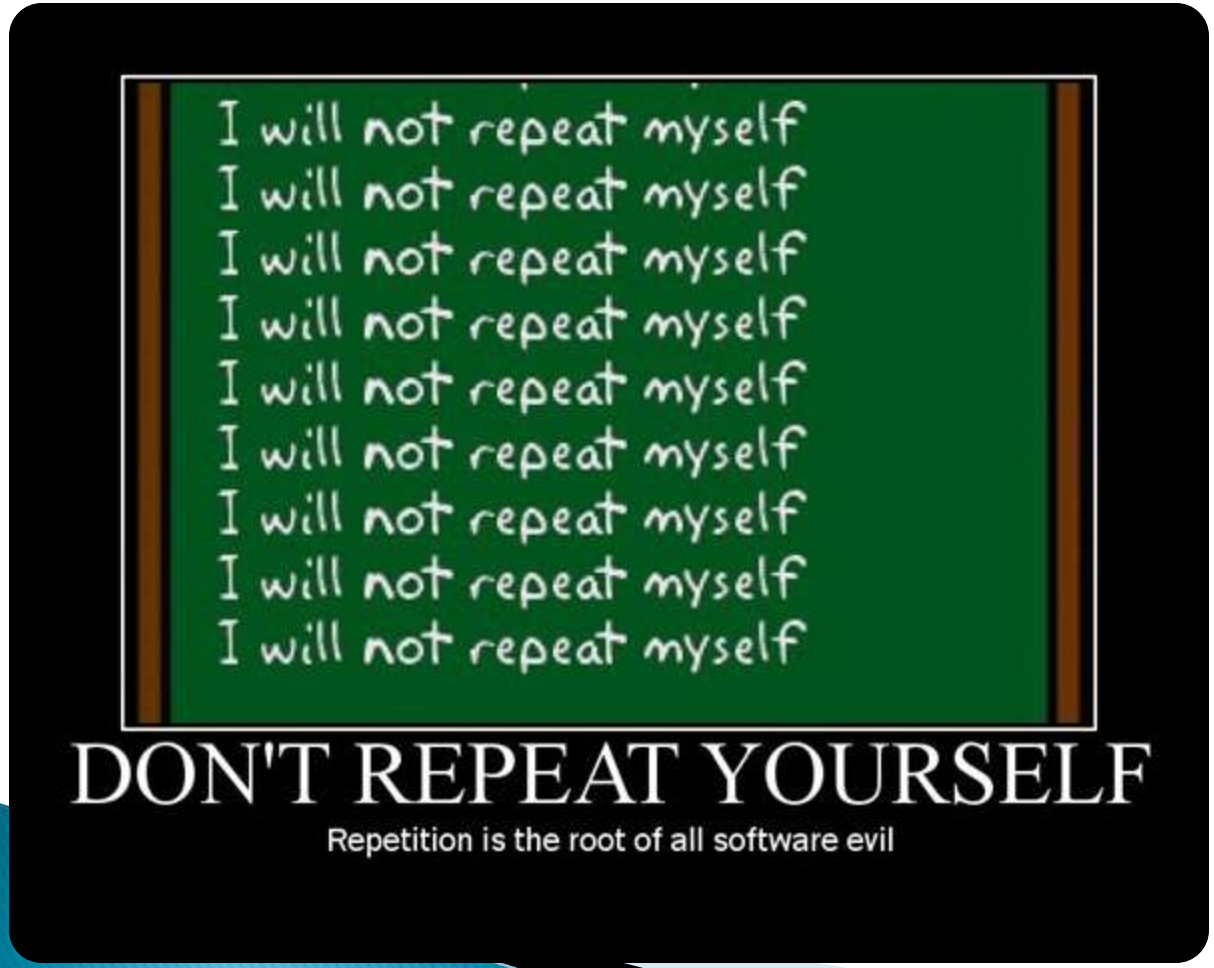
```
//DIP - bad example
public class EmployeeService {
        private EmployeeFinder emFinder //concrete class, not abstract. Can access a SQL DB for instance
        public Employee findEmployee(…) {
                emFinder.findEmployee(…)
        }
}

//DIP - fixed
public class EmployeeService {
        private IEmployeeFinder emFinder //depends on an abstraction, no an implementation
        public Employee findEmployee(…) {
                emFinder.findEmployee(…)
        }
}
//Now its possible to change the finder to be a XmlEmployeeFinder, DBEmployeeFinder, FlatFileEmployeeFinder,
MockEmployeeFinder….
```

Other Principles

- Don't Repeat Yourself (DRY)
- You Ain't Gonna Need It (YAGNI)
- Keep It Simple, Stupid (KISS)

OP – Don't Repeat Yourself

- Repetition is the root of all software evil



OP – DRY – Definitions

- "Every piece of knowledge must have a single, unambiguous representation in the system.“ – The Pragmatic Programmer

- "Repetition in logic calls for abstraction. Repetition in process calls for automation.“ – 97 Things Every Programmer Should Know

- Variations include:
  - Once and Only Once
  - Duplication Is Evil (DIE)

OP – DRY – Problems

- Magic Strings/Values
- Duplicate logic in multiple locations
- Repeated if-then logic
- Conditionals instead of polymorphism
- Repeated Execution Patterns
- Lots of duplicate, probably copy-pasted, code
- Only manual tests
- Static methods everywhere

OP – You Ain't Gonna Need It

- Don’t waste resources on what you might need

OP – YAGNI – Definitions

- "A programmer should not add functionality until deemed necessary.“ – Wikipedia

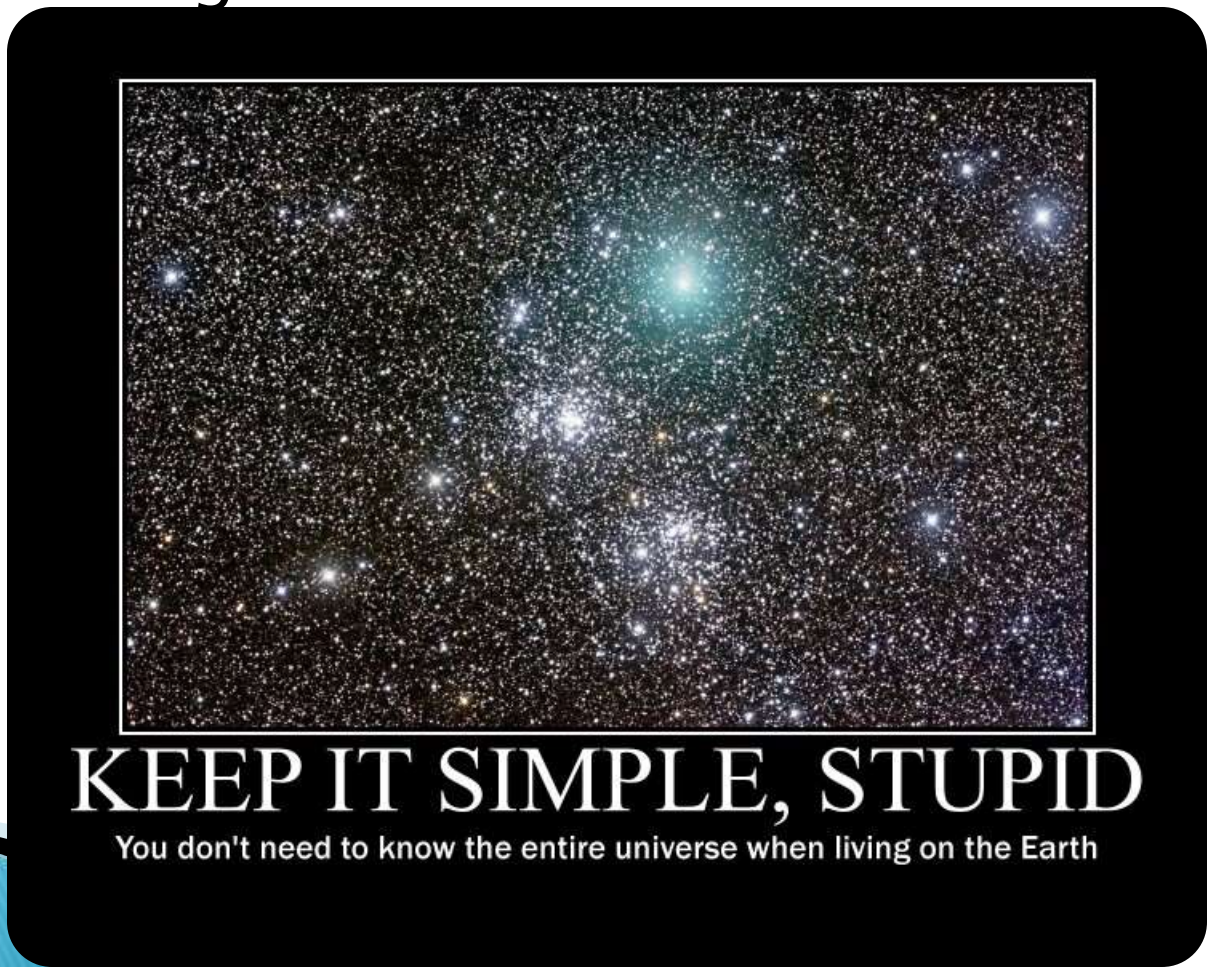
- "Always implement things when you actually need them, never when you just foresee that you need them.“ – Ron Jeffries, XP co-founder

OP – YAGNI – Problems

- Time for adding, testing, improving
- Debugging, documented, supported
- Difficult for requirements
- Larger and complicate software
- May lead to adding even more features
- May be not know to clients

OP – Keep It Simple, Stupid

- You don't need to know the entire universe when living on the Earth



OP – KISS – Definitions

- "Most systems work best if they are kept simple.“ – U.S. Navy
- "Simplicity should be a key goal in design and unnecessary complexity should be avoided.“ – Wikipedia

Design Patterns – Why?

- **If a problem occurs over and over again**, a solution to that problem has been used effectively (solution = pattern)

- When you make a design, you **should know the names of some common solutions**. Learning design patterns is good for people to **communicate each other effectively**

Design Patterns – Definitions

- "Design patterns capture solutions that have developed and evolved over time" (GOF – ***Gang-Of-Four*** (because of the four authors who wrote it), *Design Patterns: Elements of Reusable Object-Oriented Software)*

- **In software engineering (or computer science), a design pattern is a general repeatable solution to a commonly occurring problem in software design**

- The **design patterns** are language-independent strategies for solving common object-oriented design problems

Gang of Four

- Initial was the name given to a leftist political faction composed of four Chinese Communist party officials
- The name of the book (“Design Patterns: Elements of Reusable Object-Oriented Software”) is too long for e-mail, so “book by the gang of four” became a shorthand name for it
- That got shortened to "**GOF book**“. Authors are: *Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides*
- The **design patterns** in their book are *descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context*

Design Patterns – Elements

1. Pattern name
2. Problem
3. Solution
4. Consequences

Design Patterns – Pattern name

- A handle **used to describe a design problem**, its solutions, and consequences in a word or two
- Naming a pattern immediately increases our **design vocabulary**. It lets us design at a higher level of abstraction
- Having a **vocabulary** for patterns lets us talk about them with our colleagues, in our documentation
- Finding good names has been one of the hardest parts of developing our catalog

Design Patterns – Problem

- Describes **when** to apply the pattern. It explains the problem and its **context**
- It might describe specific design problems such as how to represent **algorithms** as objects
- It might describe **class** or **object** structures that are symptomatic of an inflexible design
- Sometimes the problem will include a **list of conditions** that must be met before it makes sense to apply the pattern

Design Patterns – Solution

- Describes the elements that make up the **design, their relationships, responsibilities,** and **collaborations**
- The solution **doesn't describe a particular concrete design or implementation**, because a pattern is like a template that can be applied in many different situations
- Instead, the pattern provides an **abstract description of a design problem** and **how** a general arrangement of elements (classes and objects in our case) **solves it**

Design Patterns – Consequences

- Are the results and trade-offs of applying the pattern
- They are critical for **evaluating design alternatives** and for **understanding the costs and benefits** of applying the pattern
- The consequences for software often concern **space** and **time trade-offs**, they can address **language and implementation issues** as well
- Include its impact on a system's **flexibility, extensibility, or portability**
- Listing these consequences explicitly helps you **understand and evaluate** them

Example of (Micro) pattern

- **Pattern name**: Initialization
- **Problem**: It is important for some code sequence to be executed only once at the beginning of the execution of the program.
- **Solution**: The solution is to use a static variable that holds information on whether or not the code sequence has been executed.
- **Consequences**: The solution requires the language to have a static variable that can be allocated storage at the beginning of the execution, initialized prior to the execution and remain allocated until the program termination.

Describing Design Patterns 1

- **Pattern Name and Classification**
- **Intent** – the answer to question: *What does the design pattern do*?
- **Also Known As**
- **Motivation** – A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem
- **Applicability** – *What are the situations in which the design pattern can be applied*? *How can you recognize these situations*?
- **Related Patterns**

Describing Design Patterns 2

- **Structure** – A graphical representation of the classes in the pattern
- **Participants** – The classes and/or objects participating in the design pattern and their responsibilities
- **Collaborations** – How the participants collaborate to carry out their responsibilities
- **Consequences** – *How does the pattern support its objectives*?
- **Implementation** – *What techniques should you be aware of when implementing the pattern*?
- **Sample Code**
- **Known Uses** – Examples of the pattern found in real systems

Design Patterns – Classification

- **Creational patterns**
- **Structural patterns**
- **Behavioral patterns**
- NOT in GOF: Fundamental, Partitioning, GRASP, GUI, Organizational Coding, Optimization Coding, Robustness Coding, Testing, Transactions, Distributed Architecture, Distributed Computing, Temporal, Database, Concurrency patterns

Creational Patterns

- **Abstract Factory** groups object factories that have a common theme
- **Builder** constructs complex objects by separating construction and representation
- **Factory Method** creates objects without specifying the exact class to create
- **Prototype** creates objects by cloning an existing object
- **Singleton** restricts object creation for a class to only one instance
- Not in GOF book: Lazy initialization, Object pool, Multiton, Resource acquisition (is initialization)

Structural Patterns

- **Adapter** allows classes with incompatible interfaces to work together
- **Bridge** decouples an abstraction from its implementation so that the two can vary independently
- **Composite** composes zero-or-more similar objects so that they can be manipulated as one object.
- **Decorator** dynamically adds/overrides behavior in an existing method of an object
- **Facade** provides a simplified interface to a large body of code
- **Flyweight** reduces the cost of creating and manipulating a large number of similar objects
- **Proxy** provides a placeholder for another object to control access, reduce cost, and reduce complexity

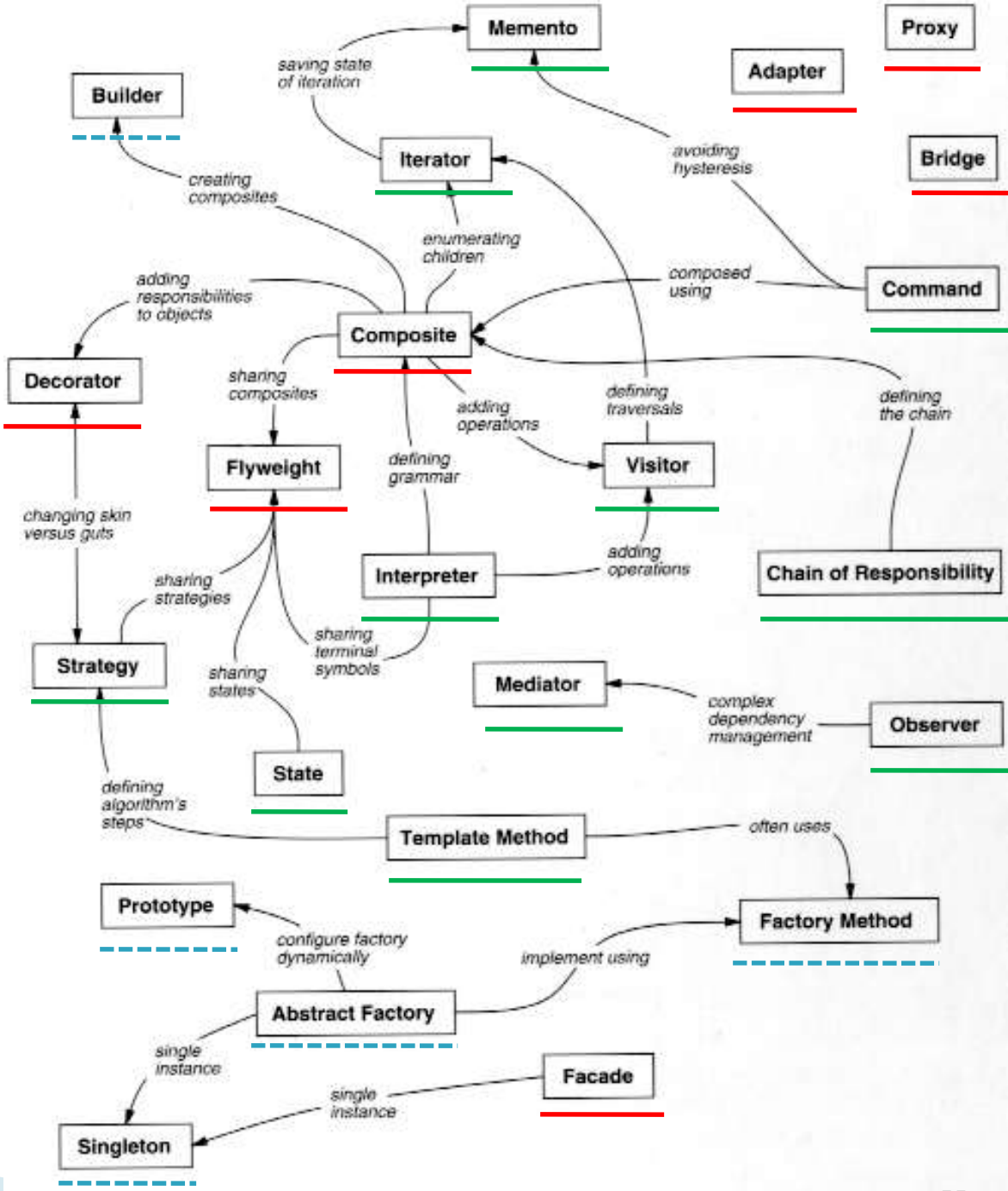
Behavioral patterns 1

- **Chain of responsibility** delegates commands to a chain of processing objects
- **Command** creates objects which encapsulate actions and parameters
- **Interpreter** implements a specialized language
- **Iterator** accesses the elements sequentially
- **Mediator** allows loose coupling between classes by being the only class that has detailed knowledge of their methods
- **Memento** provides the ability to restore an object to its previous state

Behavioral patterns 2

- **Observer** allows to observer objects to see an event
- **State** allows an object to alter its behavior when its internal state changes
- **Strategy** allows one of a family of algorithms to be selected on-the-fly at runtime
- **Template** defines an algorithm as an abstract class, allowing its subclasses to provide concrete behavior
- **Visitor** separates an algorithm from an object structure
- Not in GOF book: Null Object, Specification

- Patterns
  - Creational
  - Structural
  - Behavioral



Design Pattern Relationships

How to Select a Design Pattern?

- With more than 20 design patterns to choose from, it might be hard to find the one that addresses a particular design problem

- Approaches to finding the design pattern that's right for your problem:
  1. *Consider how design patterns solve design problems*
  2. *Scan Intent sections*
  3. *Study relationships between patterns*
  4. *Study patterns of like purpose (comparison)*
  5. *Examine a cause of redesign*
  6. *Consider what should be variable in your design*

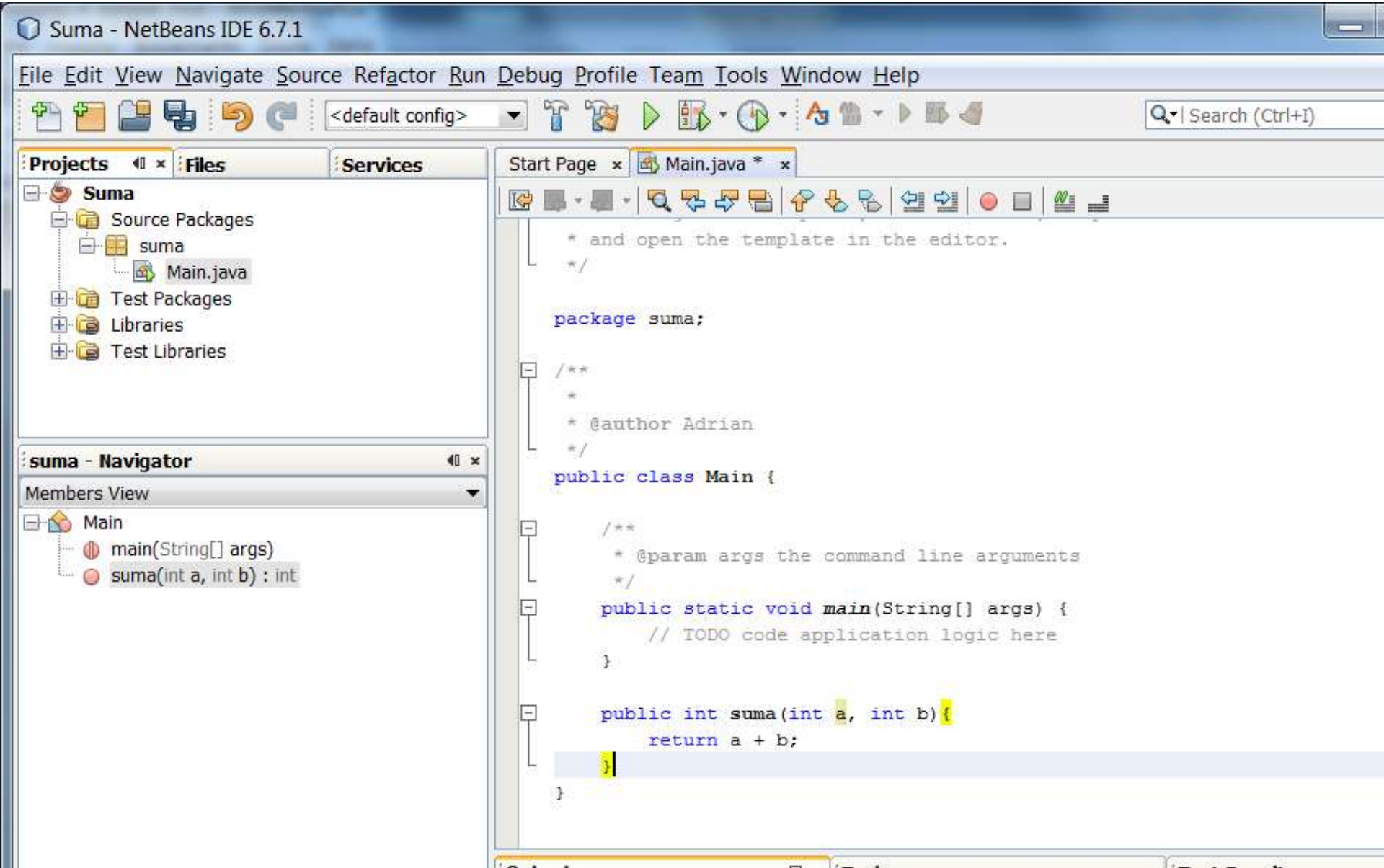
How to Use a Design Pattern?

1. *Read the pattern once through for an overview*
2. *Go back and study the Structure*, Participants, and Collaborations sections
3. *Look at the Sample Code section to see a concrete example*
4. *Choose names for pattern participants that are meaningful in the application context*
5. *Define the classes*
6. *Define application-specific names for operations in the pattern*
7. *Implement the operations to carry out the responsibilities and collaborations in the pattern*

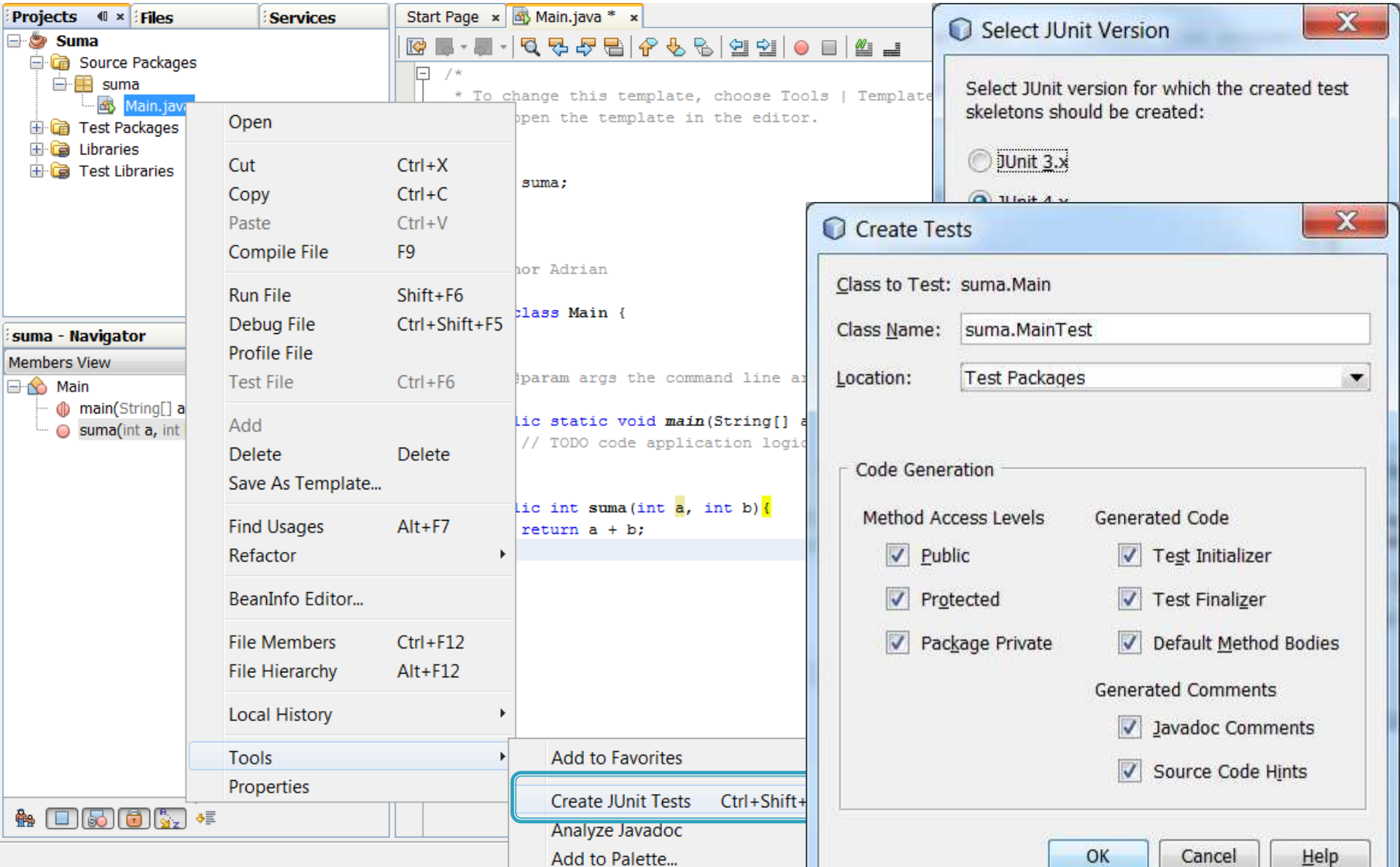
Unit Testing

- Testarea unei funcţii, a unui program, a unui ecran, a unei funcţionalităţi
- Se face de către programatori
- Predefinită
- Rezultatele trebuie documentate
- Se folosesc simulatoare pentru Input şi Output

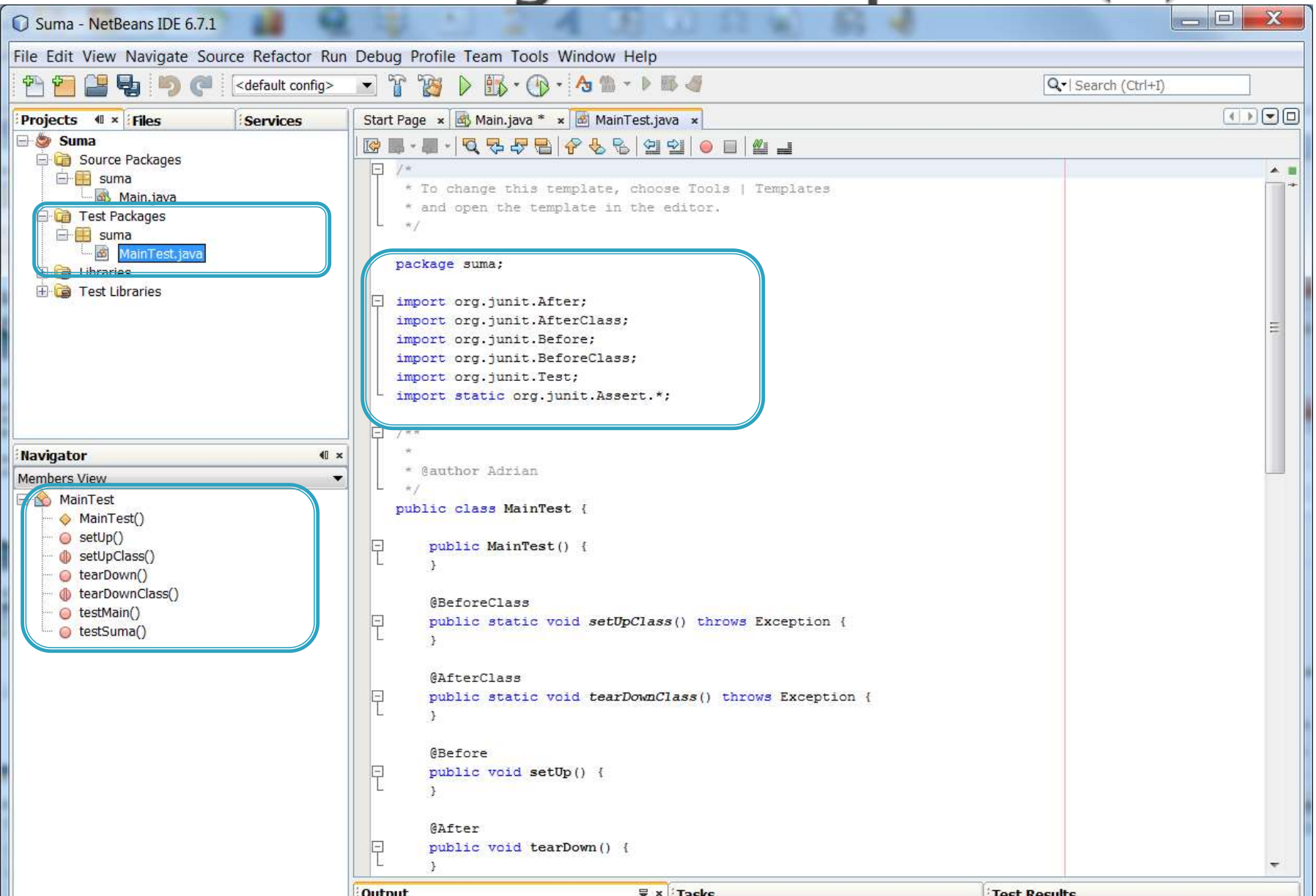
Unit Testing – Exemplu 1 (1)



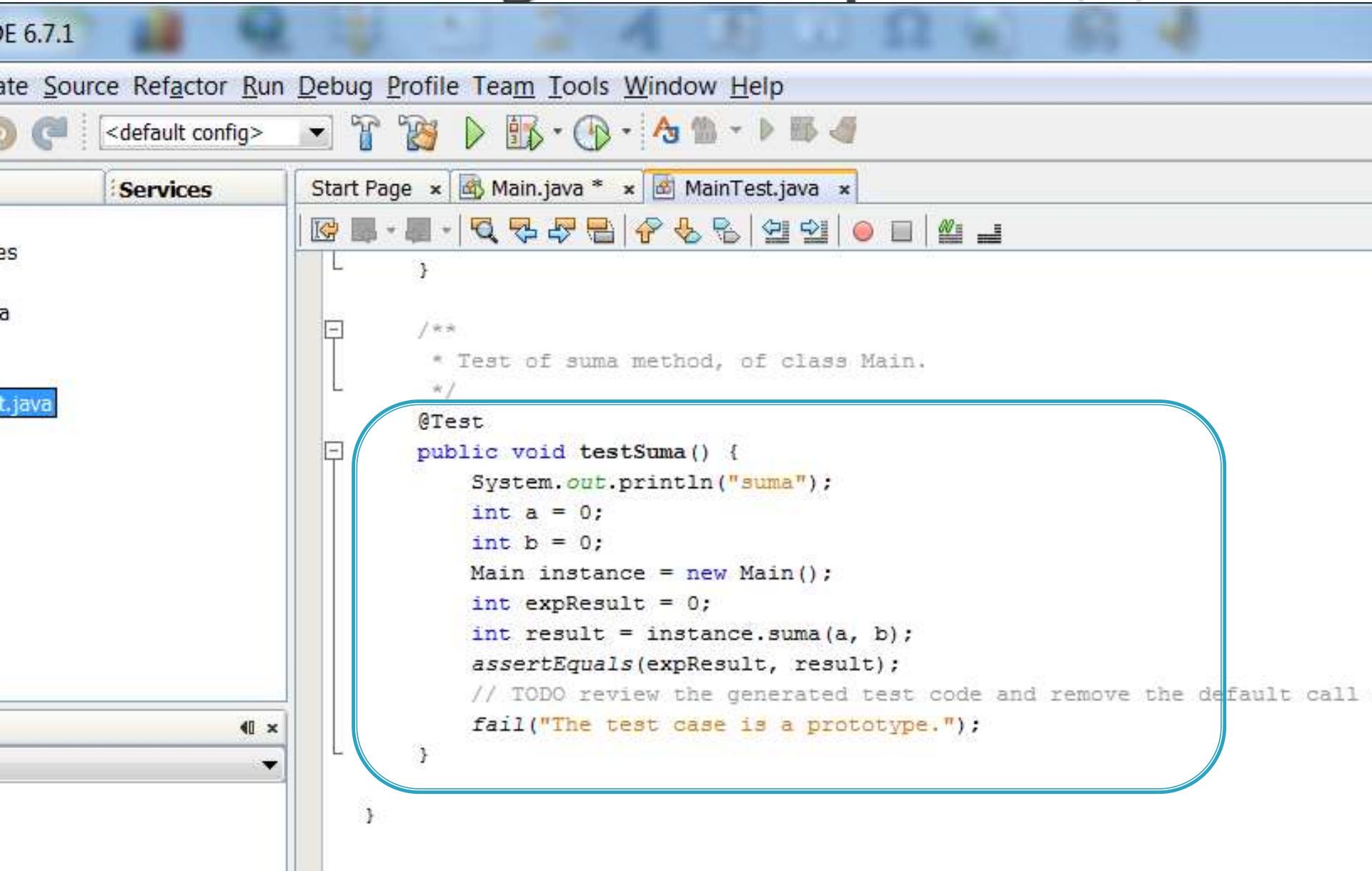
Unit Testing – Exemplu 1 (2)



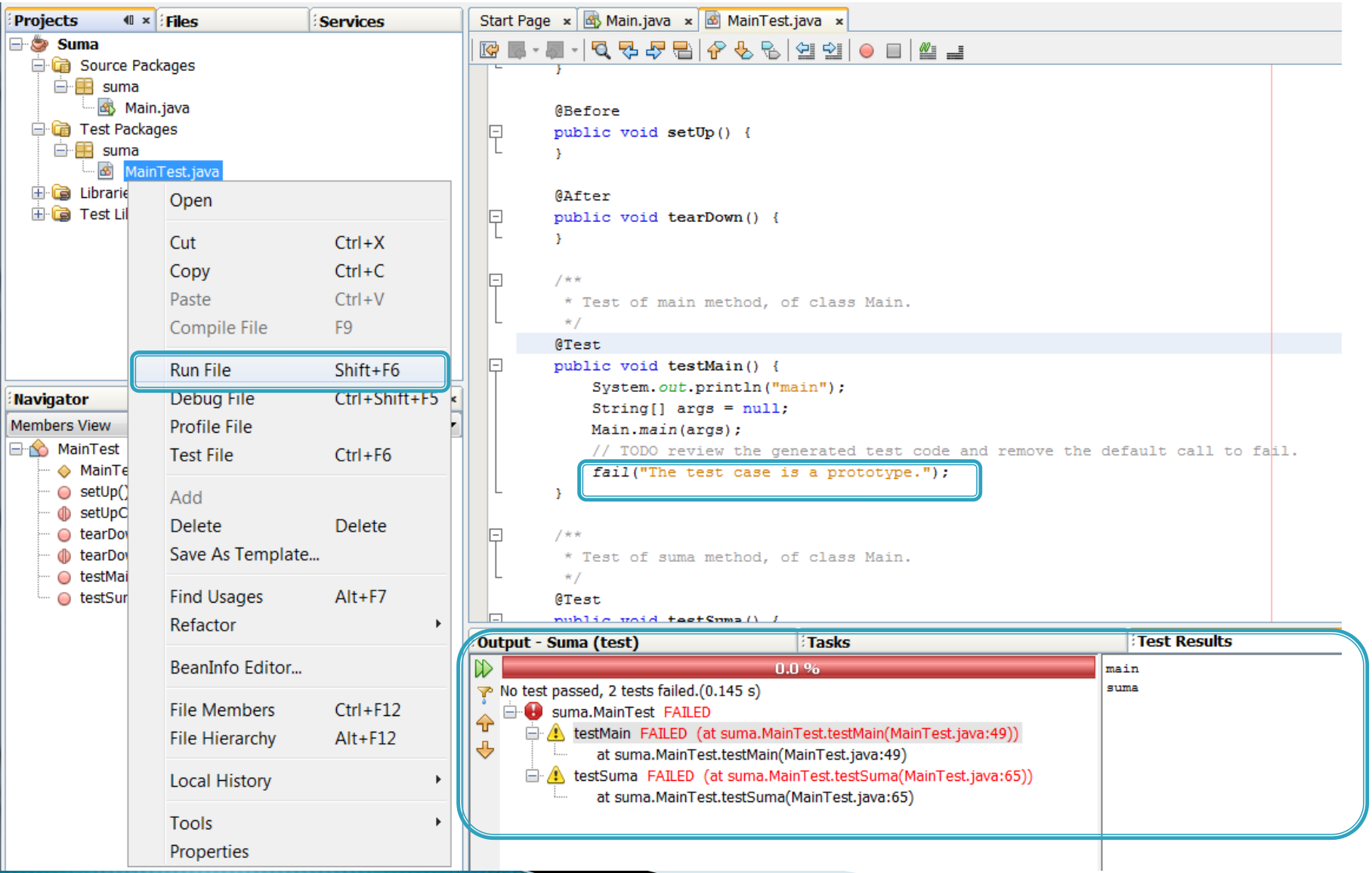
Unit Testing – Exemplu 1 (3)



Unit Testing – Exemplu 1 (4)

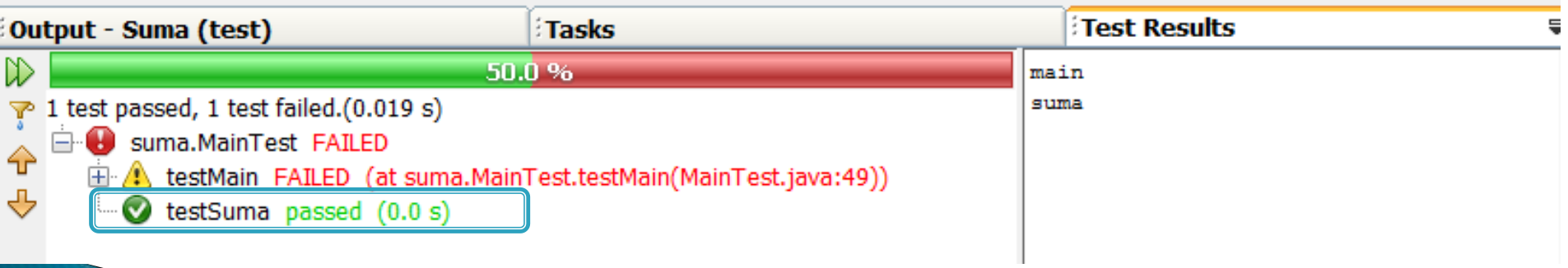


Unit Testing – Exemplu 1 (5)



Unit Testing – Exemplu 1 (6)

```
    @Test
    public void testSuma() {
        System.out.println("suma");
        int a = 0;
        int b = 0;
        Main instance = new Main();
        int expResult = 0;
        int result = instance.suma(a, b);
        assertEquals(expResult, result);
        // TODO review the generated test code and remove the default call to fail.
        //fail("The test case is a prototype.");
    }
}
```

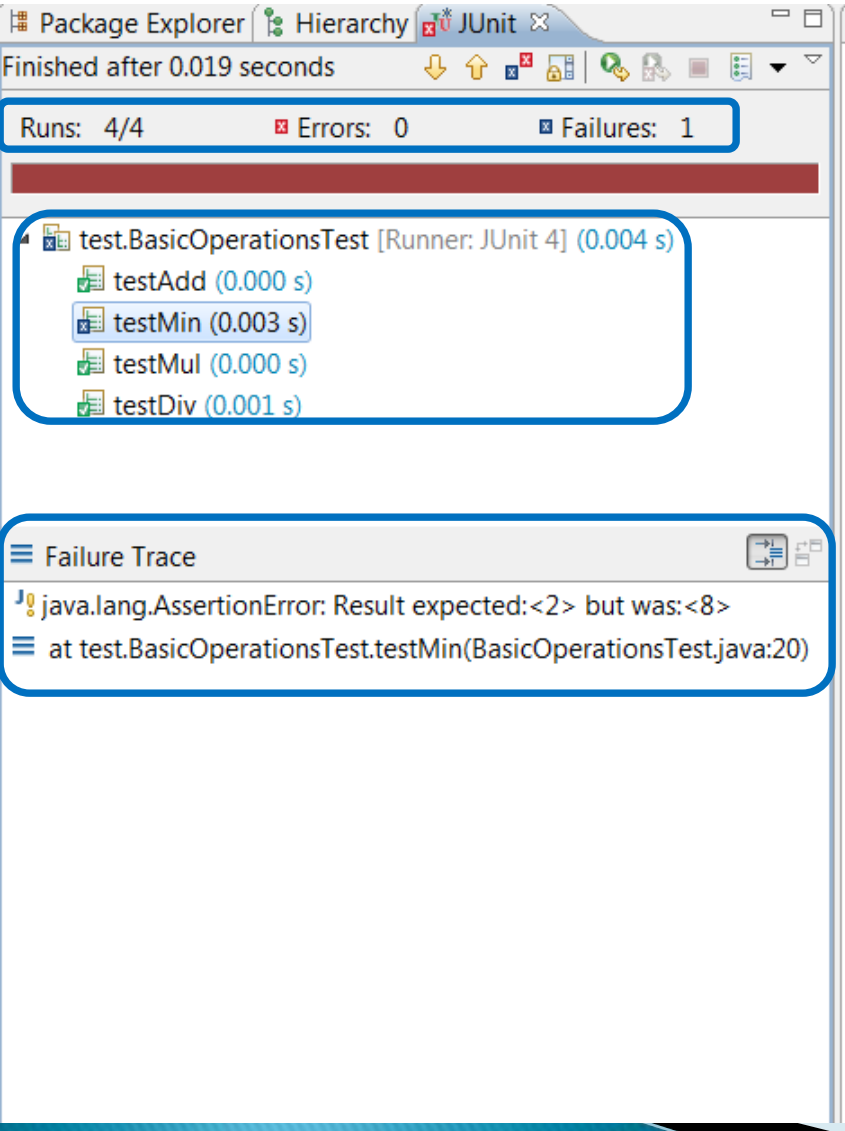


Unit Testing – Example 2 (1)

BasicOperations.java | BasicOperationsTest.java

```
package math;

public class BasicOperations {

    public int add(int x, int y){
        return x + y;
    }

    public int min(int x, int y){
        return x + y;
    }

    public int mul(int x, int y){
        return x * y;
    }

    public int div(int x, int y){
        return x / y;
    }

    /**
     * @param args
     */
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        BasicOperations bc = new BasicOperations();
        System.out.println(bc.add(3,5));
    }

}
```

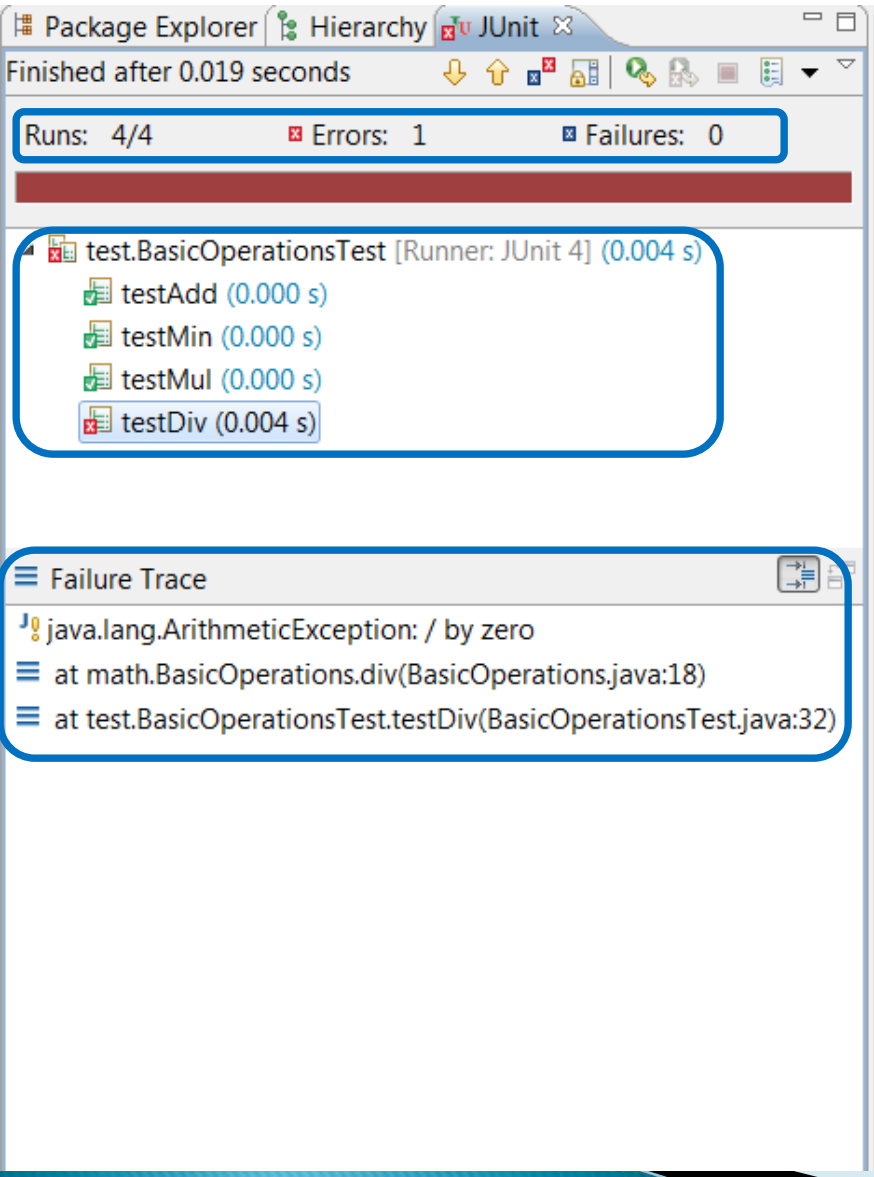
Unit Testing – Example 2 (2)



BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.div(4, 2));
    }
}
```

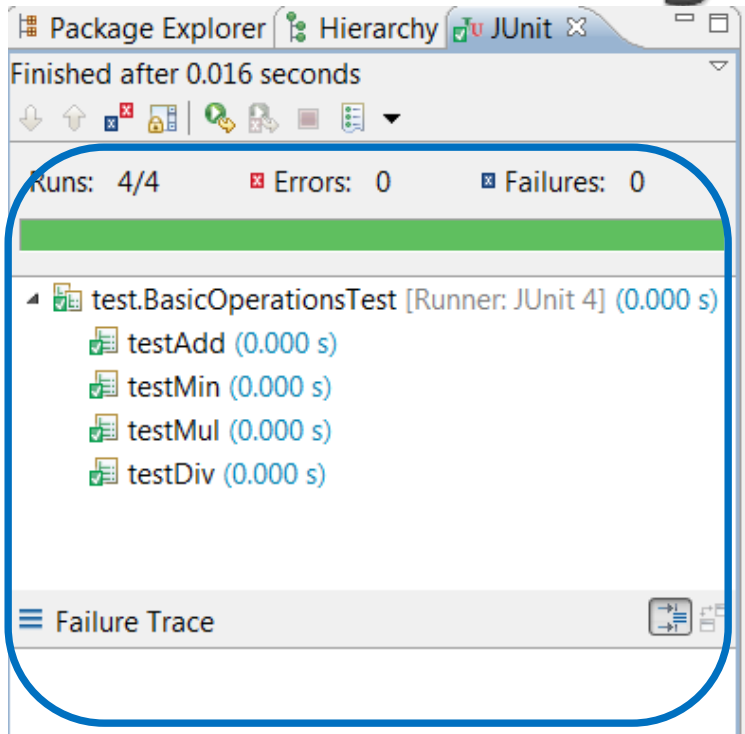
Unit Testing – Example 2 (3)



BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 8, bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 2, bo.min(5, 3));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 0, bo.div(4, 0));
    }
}
```

Unit Testing – Example 2 (4)



BasicOperations.java | BasicOperationsTest.java

```
package test;

import static org.junit.Assert.*;

public class BasicOperationsTest {

    @Test
    public void testAdd() {
        BasicOperations bo = new BasicOperations();
        assertTrue("Result", 8 == bo.add(3, 5));
    }

    @Test
    public void testMin() {
        BasicOperations bo = new BasicOperations();
        assertFalse("Result", ! (3 != bo.min(5, 3)));
    }

    @Test
    public void testMul() {
        BasicOperations bo = new BasicOperations();
        assertEquals("Result", 15, bo.mul(3, 5));
    }

    @Test
    public void testDiv() {
        BasicOperations bo = new BasicOperations();
        if(bo.div(4, 2) == 3)
            fail("Incorrect result!");
    }
}
```

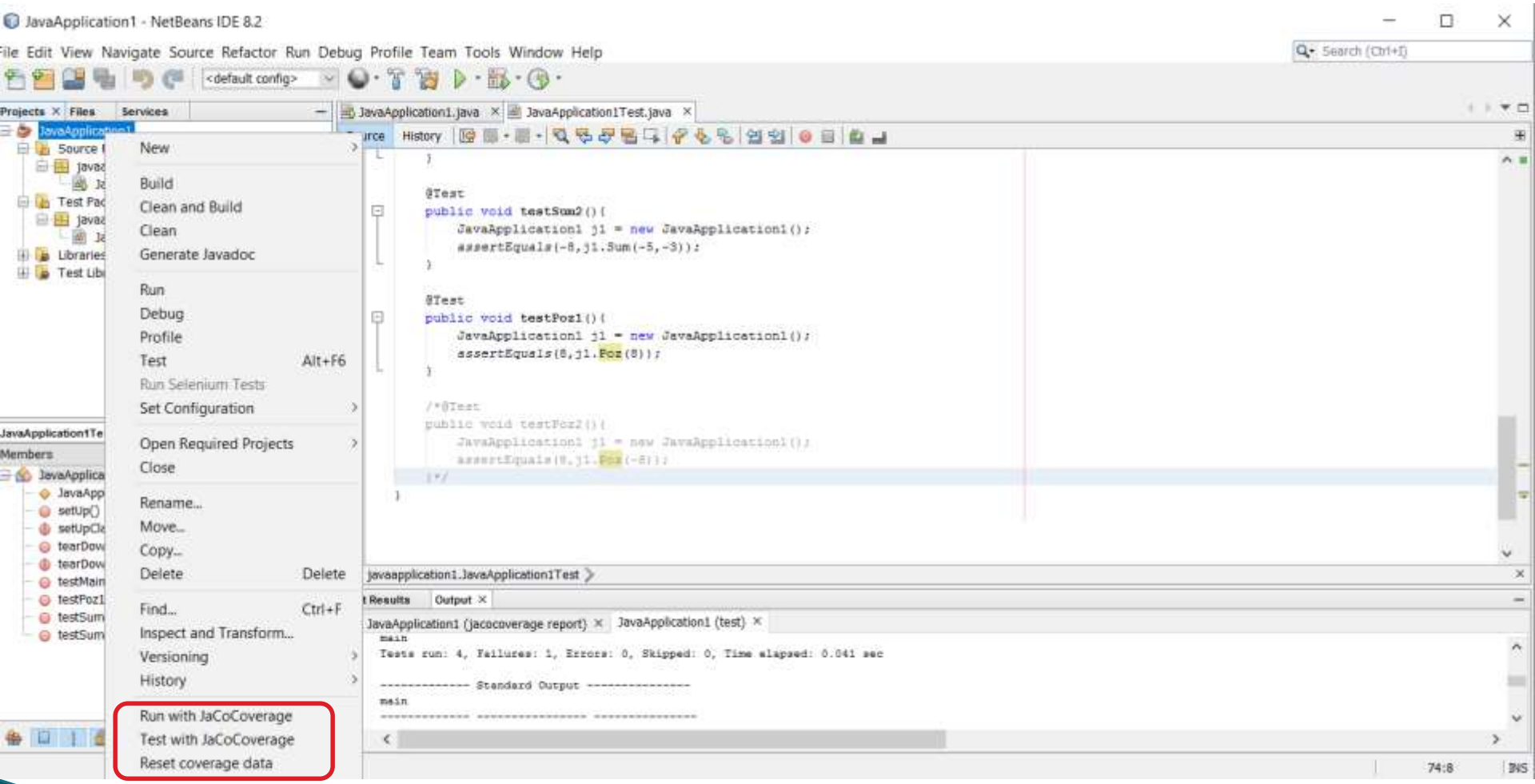
Code Coverage

- NetBeans – TikiOne JaCoCoverage:
- http://plugins.netbeans.org/plugin/48570/tikione-jacocoverage

- Java Code Coverage for Eclipse:
- http://www.eclemma.org/

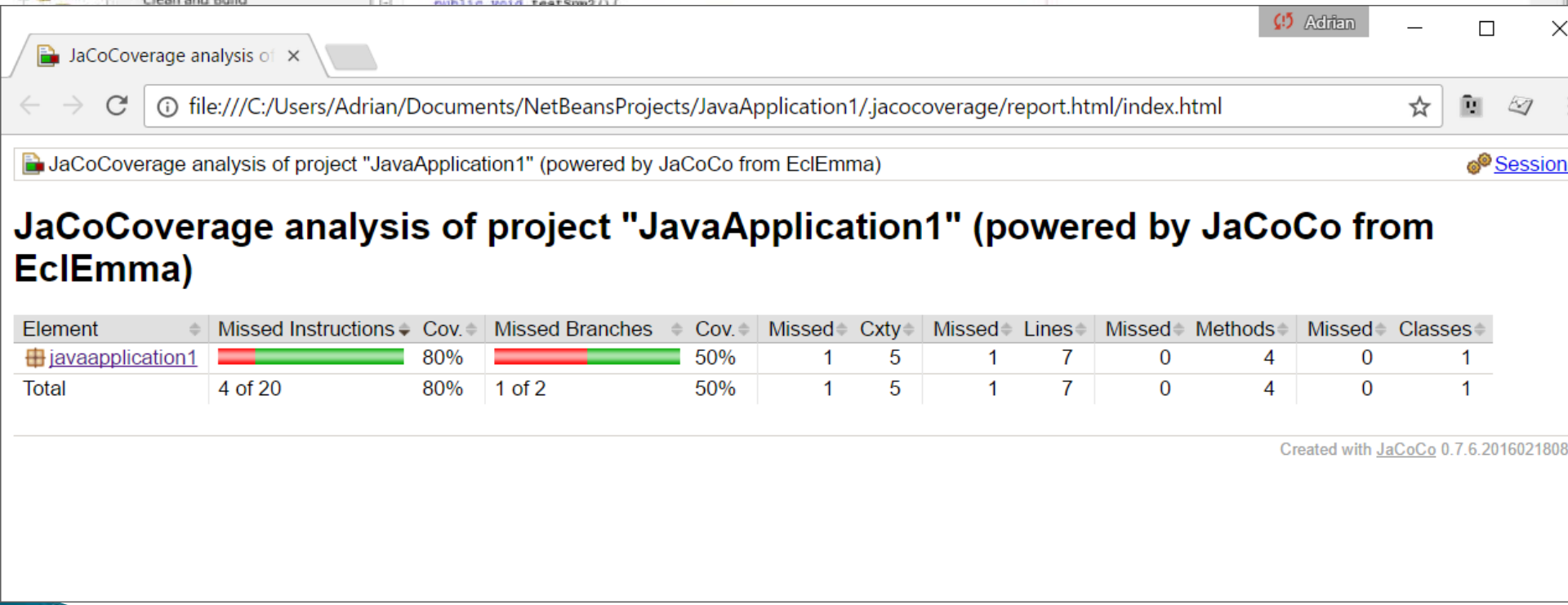
- IntelliJ – Running with coverage:
- https://www.jetbrains.com/help/idea/2016.3/running-with-coverage.html

NetBeans – 1

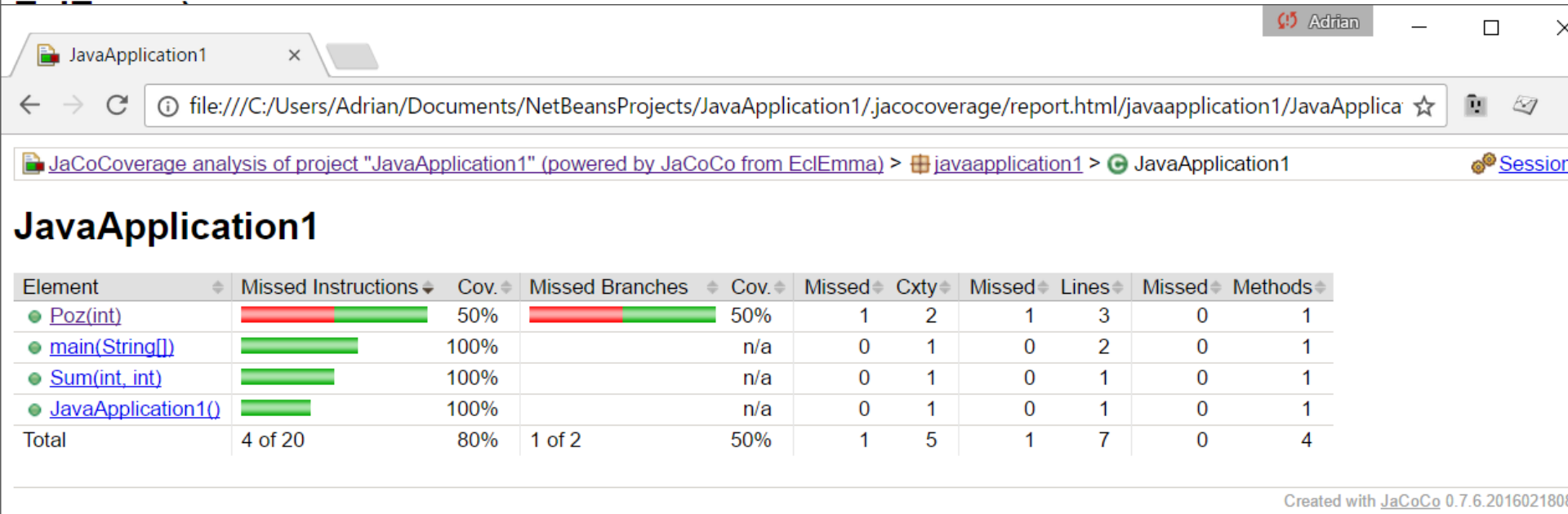

JavaApplication1 - NetBeans IDE 8.2
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Search (Ctrl+I)
<default config>
Projects
Files
Services
JavaApplication1.java
JavaApplication1Test.java
History
New
Build
Clean and Build
Clean
Generate Javadoc
Run
Debug
Profile
Test Alt+F6
Run Selenium Tests
Set Configuration
Open Required Projects
Close
Rename...
Move...
Copy...
Delete Delete
Find... Ctrl+F
Inspect and Transform...
Versioning
History
Run with JaCoCoverage
Test with JaCoCoverage
Reset coverage data
@Test
public void testSum2(){
JavaApplication1 j1 = new JavaApplication1();
assertEquals(-8,j1.Sum(-5,-3));
}
@Test
public void testPoz1(){
JavaApplication1 j1 = new JavaApplication1();
assertEquals(8,j1.Poz(8));
}
javaapplication1.JavaApplication1Test
Output
JavaApplication1 (jacocoverage report)
JavaApplication1 (test)
Tests run: 4, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.041 sec
Standard Output
74:8

NetBeans – 2

JavaApplication1 - NetBeans IDE 8.2
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Search (Ctrl+I)
Projects
Files
Services
JavaApplication1.java
JavaApplication1Test.java
History
New
Build
Clean and Build
@Test


Adrian
JaCoCoCoverage analysis o
file:///C:/Users/Adrian/Documents/NetBeansProjects/JavaApplication1/.jacocoverage/report.html/index.html
JaCoCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma)
Session
JaCoCoCoverage analysis of project "JavaApplication1" (powered by JaCoCo from EclEmma)
Element
Missed Instructions
Cov.
Missed Branches
Cov.
Missed
Cxty
Missed
Lines
Missed
Methods
Missed
Classes
javaapplication1
80%
50%
1
5
1
7
0
4
0
1
Total
4 of 20
80%
1 of 2
50%
1
5
1
7
0
4
0
1
Created with JaCoCo 0.7.6.201602180812

NetBeans – 3



JavaApplication1

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Poz(int)		50%		50%	1	2	1	3	0	1
main(String[])		100%		n/a	0	1	0	2	0	1
Sum(int, int)		100%		n/a	0	1	0	1	0	1
JavaApplication1()		100%		n/a	0	1	0	1	0	1
Total	4 of 20	80%	1 of 2	50%	1	5	1	7	0	4

Created with JaCoCo 0.7.6.201602180

Concluzii

- SOLID
- Design Patterns
  - Definitions, Elements, Example, Classification
- JUnit Testing

Bibliografie

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)
- Ovidiu Gheorghieş, Curs 7 IP
- Adrian Iftene, Curs 9 TAIP: http://thor.info.uaic.ro/~adiftene/Scoala/2011/TAIP/Courses/TAIP09.pdf

Links

- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm
- Gang of Four: http://en.wikipedia.org/wiki/Gang_of_four
- JUnit in Eclipse: http://www.vogella.de/articles/JUnit/article.html
- JUnit in NetBeans: http://netbeans.org/kb/docs/java/junit-intro.html

Ingineria programării

Curs 7 – 3 April

Din Cursurile Trecute

- Design Patterns: Definitions, Elements
- GOF: Creational Patterns, Structural Patterns, Behavioral Patterns

Creational Patterns

- Abstract Factory
- Builder
- Factory Method
- Prototype
- Singleton
- Lazy initialization
- Object pool
- Multiton
- Resource acquisition

Creational Patterns – Introduction

- Abstract the instantiation process
- Help **make a system independent of how its objects are created, composed, and represented**
- There are two recurring themes in these patterns:
  - First, they all encapsulate knowledge about which concrete classes the system uses
  - Second, they hide how instances of these classes are created and put together
- The creational patterns give a lot of flexibility in *what* gets created, *who creates it, how it gets created, and when*

Creational Patterns – Abstract Factory

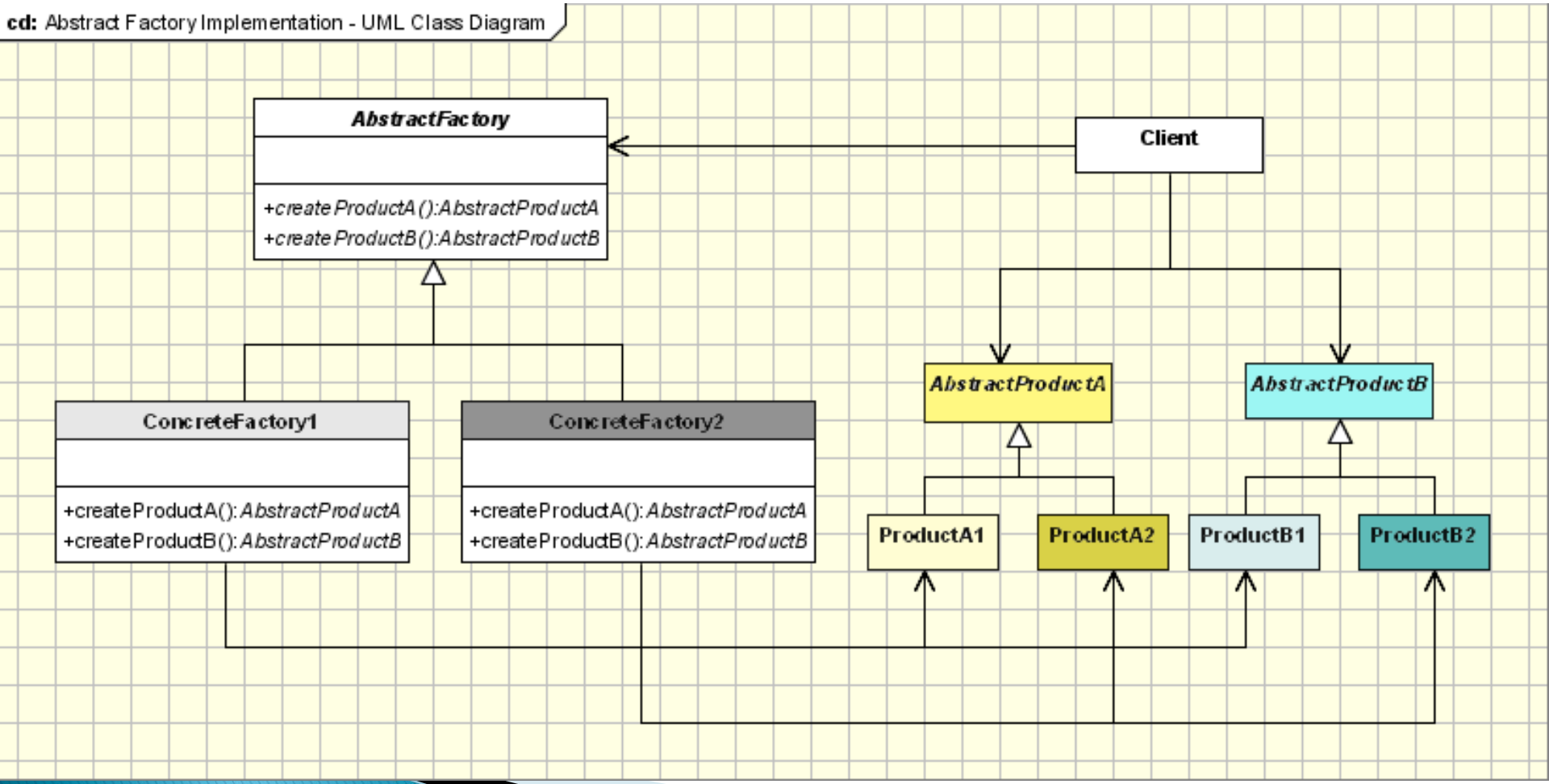
- **Intent** – Provide an interface for creating families of related or dependent objects without specifying their concrete classes
- **Also Known As** – Kit
- **Motivation**
  - Consider a user interface toolkit that supports multiple look-and-feel standards, defines different appearances and behaviors for user interface "widgets" like scroll bars, windows, and buttons
  - To be portable across look-and-feel standards, an application should not hard-code its widgets for a particular look and feel

Abstract Factory 3

- **Applicability** – Use this pattern when:
  - a system should be independent of how its products are created, composed, and represented
  - a system should be configured with one of multiple families of products
  - a family of related product objects is designed to be used together, and you need to enforce this constraint
  - you want to provide a class library of products, and you want to reveal just their interfaces, not their implementations

CP – Abstract Factory

- **Abstract Factory** – maze, computer components

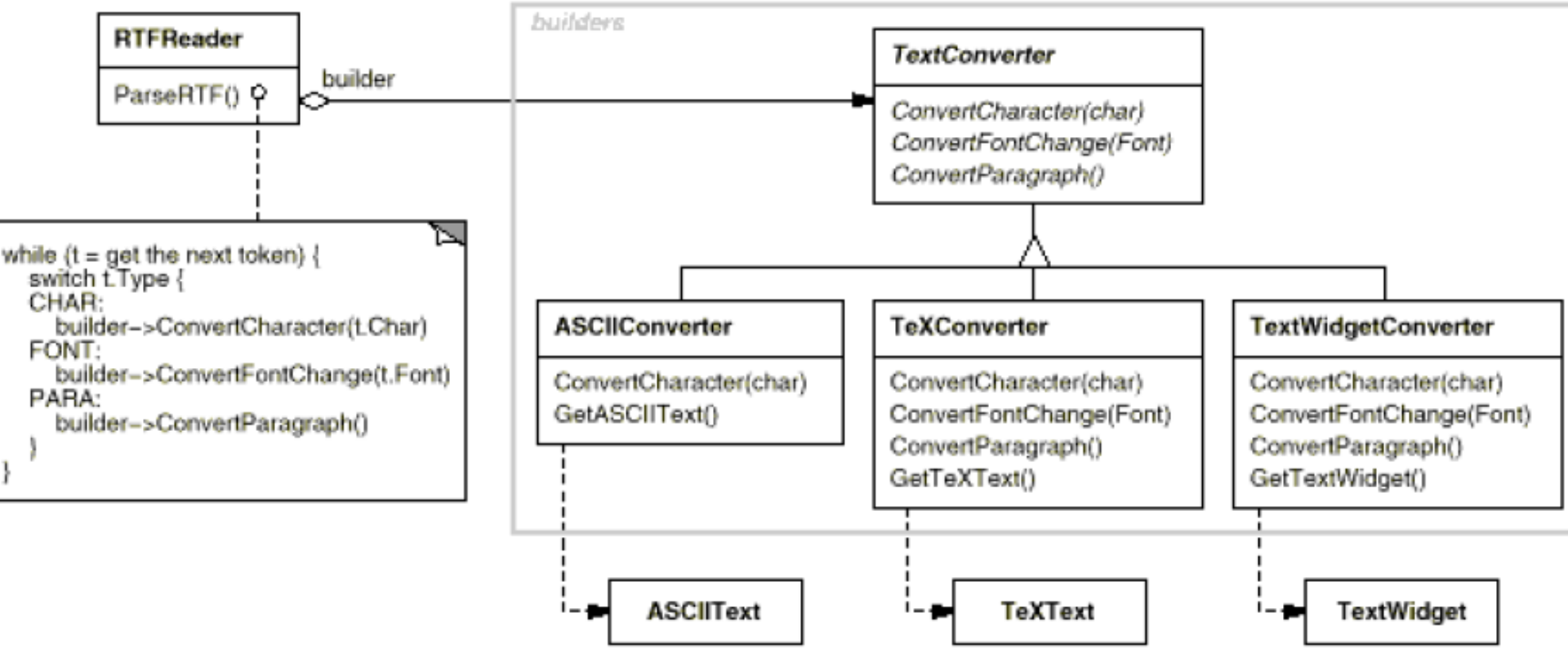


Creational Patterns – Builder

- **Intent – Separate the construction of a complex object from its representation** so that the same construction process can create different representations
- **Motivation**
  - A reader for the RTF (Rich Text Format) document exchange format should be able to convert RTF to many text formats.
  - The reader might convert RTF documents into plain ASCII text or into a text widget that can be edited interactively. The problem, however, is that the number of possible conversions is open-ended. So it should be easy to add a new conversion without modifying the reader.

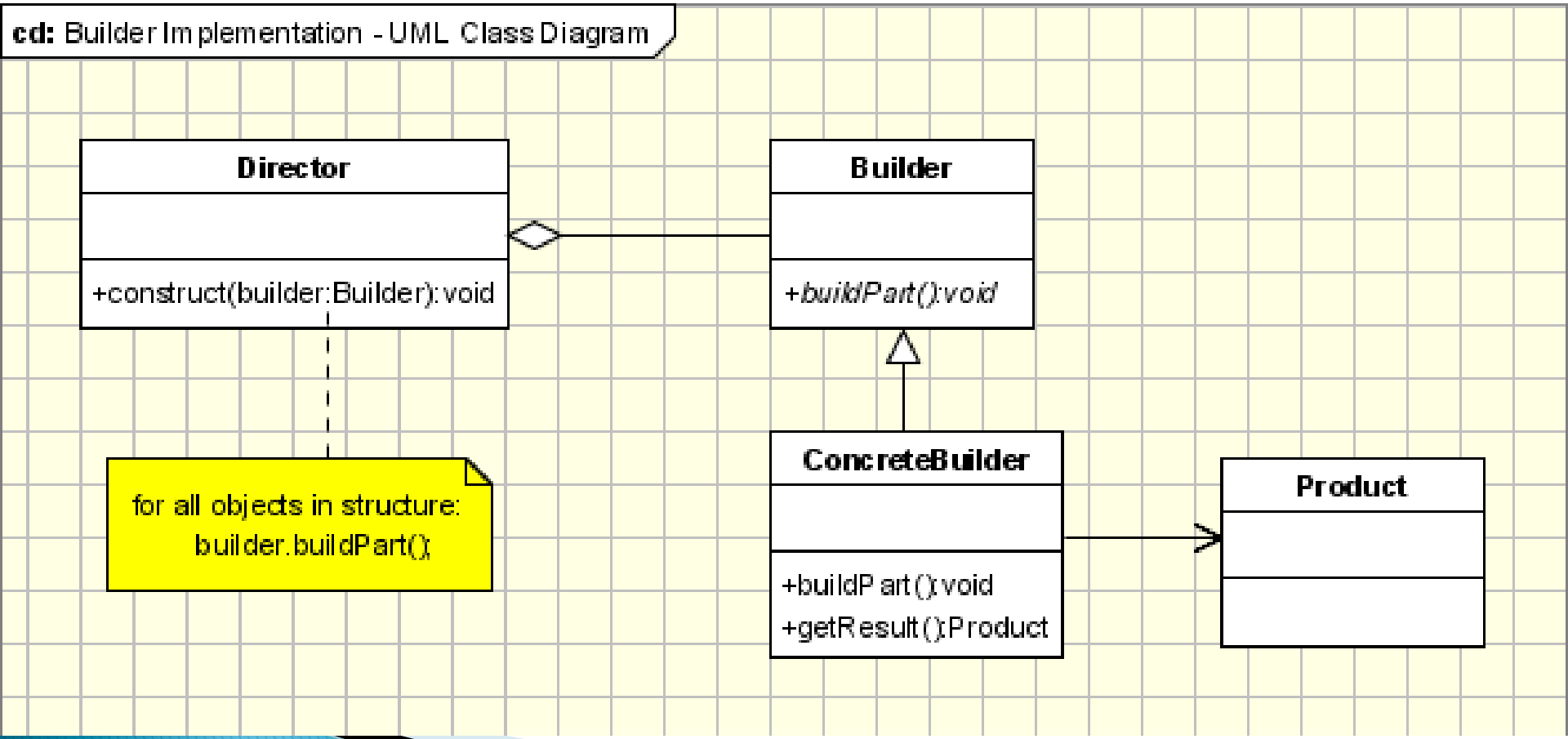
Builder 2

- Each kind of converter class takes the mechanism for creating and assembling
- The converter is separate from the reader



CP – Builder

- **Builder** – RTF Reader, Happy Meal

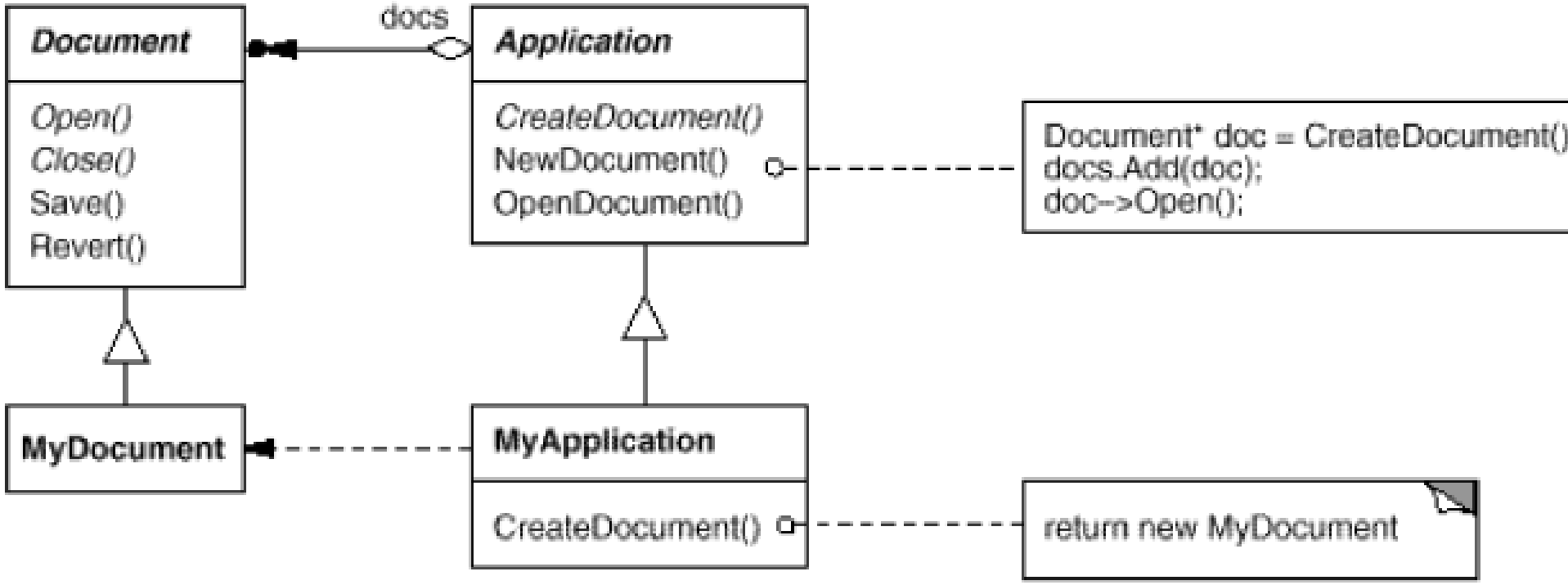


Creational Patterns – Factory Method

- **Intent** – Define an interface for creating an object, but let subclasses decide which class to instantiate
- **Also Known As** – Virtual Constructor
- **Motivation** – To create a drawing application, for example, we define the classes DrawingApplication and DrawingDocument. The Application class is responsible for managing Documents and will create them as required (at Open or New from a menu, for example)

Factory Method 2

- Subclasses redefine an abstract CreateDocument
- It can then instantiate application-specific Documents without knowing their class
- CreateDocument is a **factory method because it's** responsible for "manufacturing" an object



Factory Method 3

- **Applicability** – Use this pattern when
  - a class can't anticipate the class of objects it must create
  - a class wants its subclasses to specify the objects it creates
  - classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate

Factory Method 5

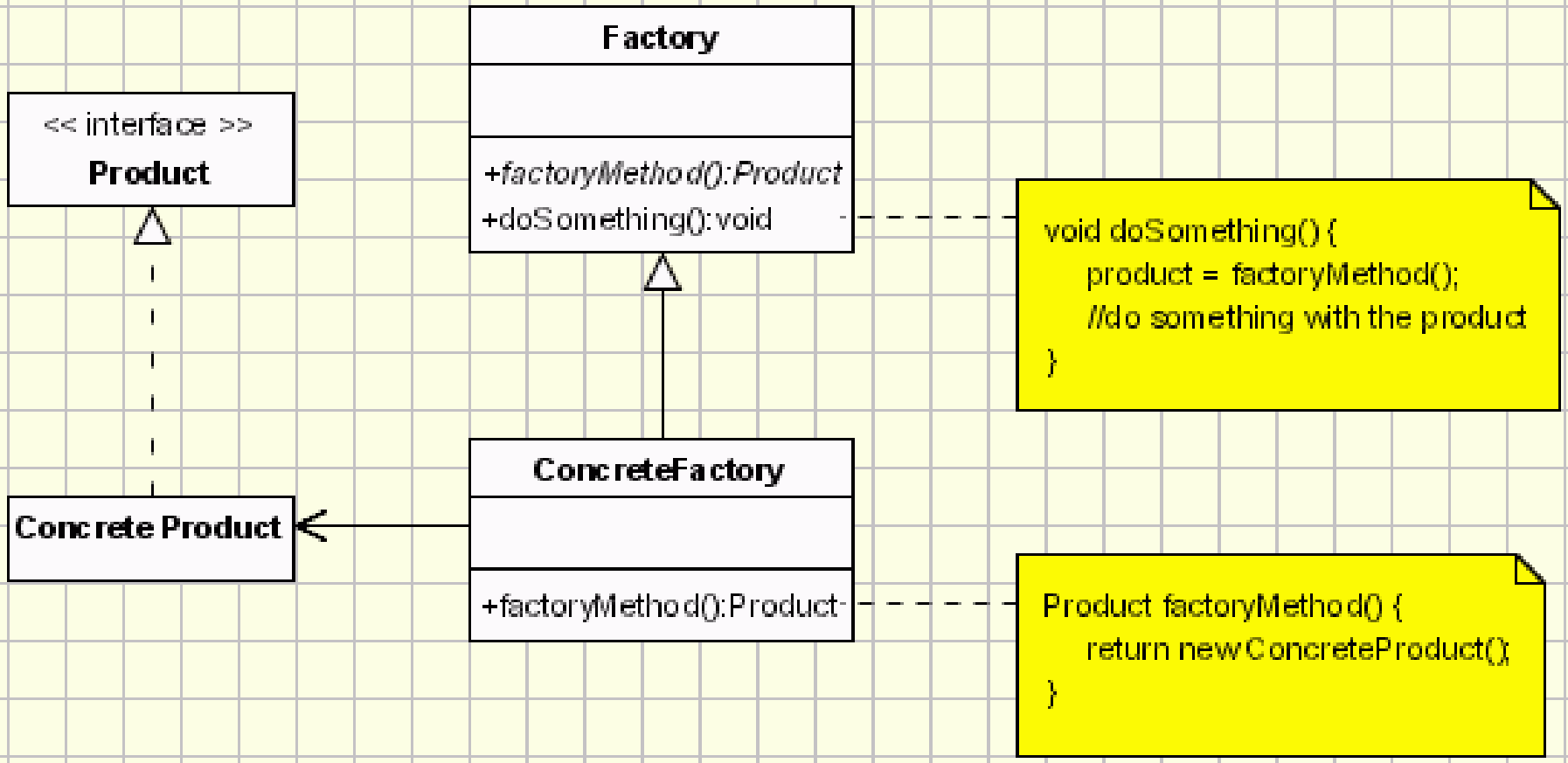
- **Consequences**

1. *Provides interconnections for subclasses*

2. *Connects parallel class hierarchies*

CP – Factory Method

- **Factory Method** – Open/New Project, Hello Mr/Ms

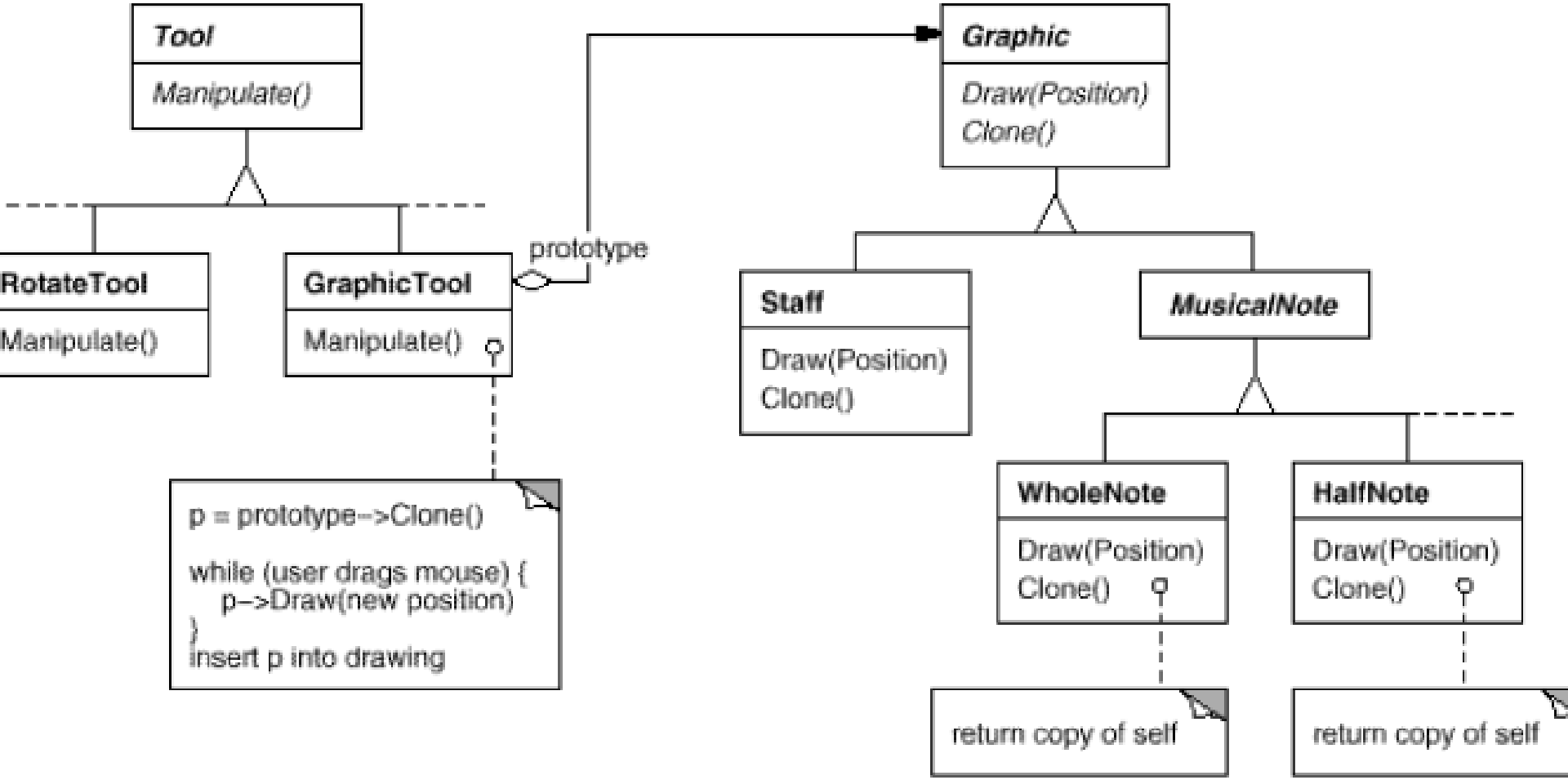


Creational Patterns – Prototype

- **Intent** – Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype
- **Motivation** – You could build an editor for music scores by customizing a general framework for graphical editors and adding new objects that represent notes, rests, and staves
- The editor framework may have a palette of tools for adding these music objects to the score. The palette would also include tools for selecting, moving, and otherwise manipulating music objects

Prototype 2

- We can use the Prototype pattern to reduce the number of classes

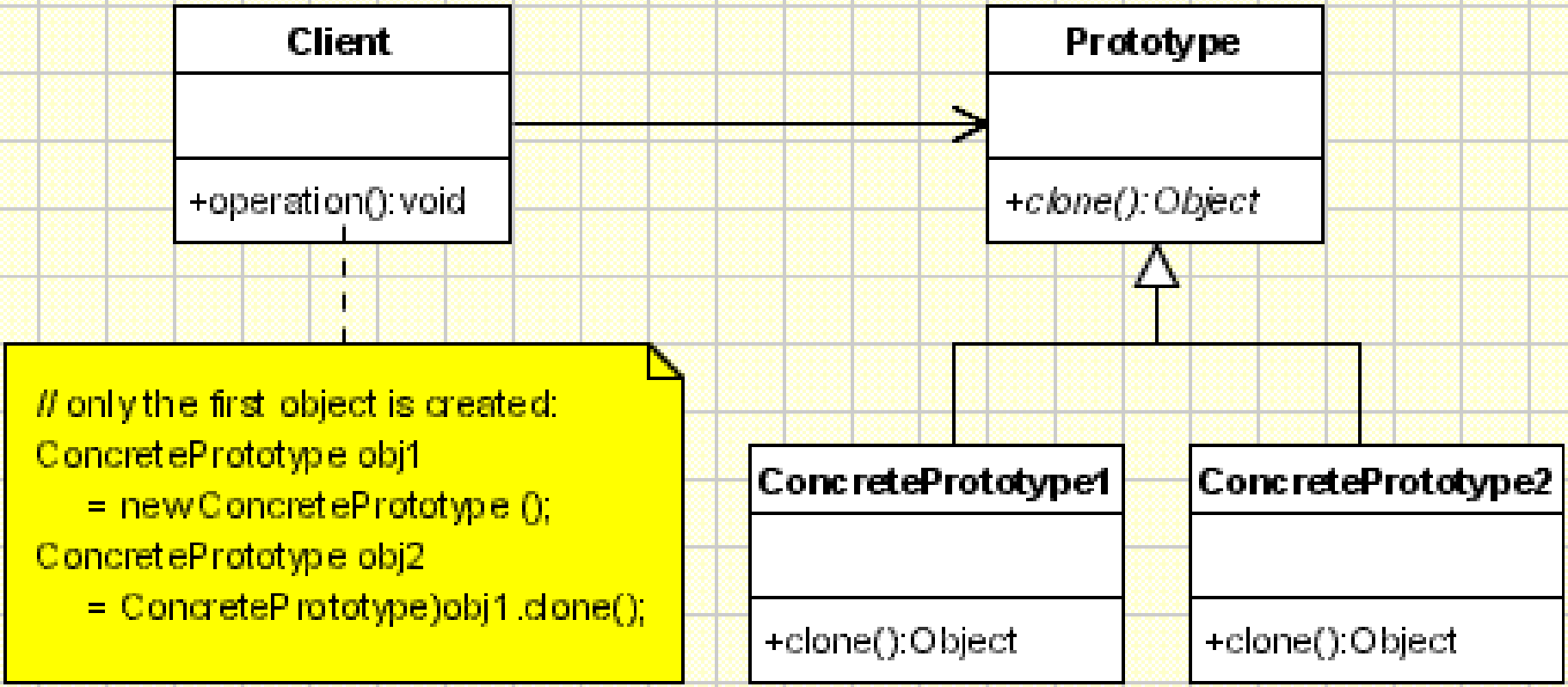


Prototype 3

- **Applicability** – Use the Prototype pattern when a system should be independent of how its products are created, composed, and represented; *and*
  - when the classes to instantiate are specified at run-time, for example, by dynamic loading; *or*
  - to avoid building a class hierarchy of factories that parallels the class hierarchy of products; *or*
  - when instances of a class can have one of only a few different combinations of state

CP – Prototype

- **Prototype** – Music editor, Clonable

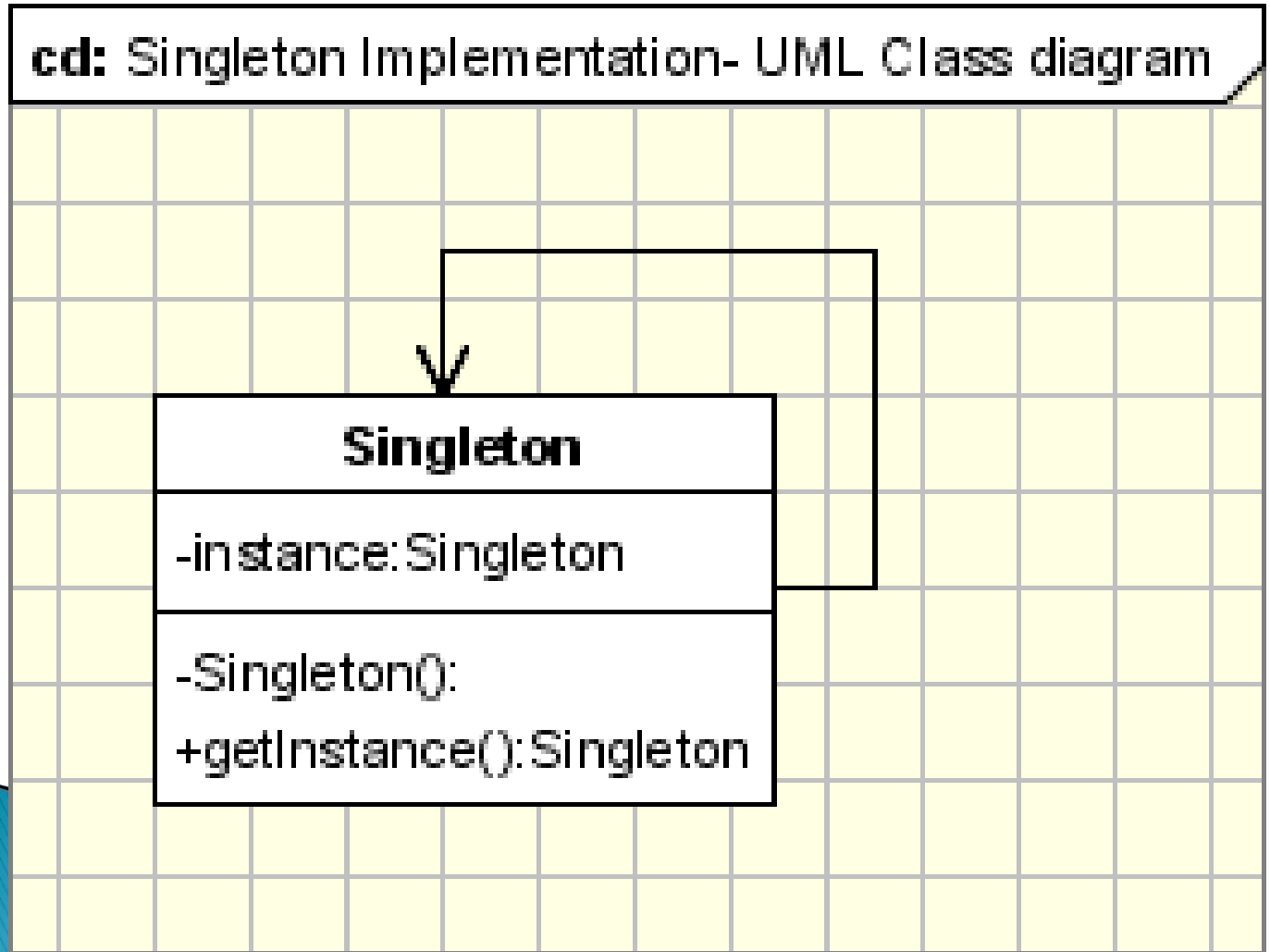


Creational Patterns – Singleton

- **Intent** – Ensure a class only has one instance, and provide a global point of access to it
- **Motivation** – It's important for some classes to have exactly one instance. There should be only one file system and one window manager. An accounting system will be dedicated to serving one company.
- How do we ensure that a class has only one instance and that the instance is easily accessible? A global variable makes an object accessible, but it doesn't keep you from instantiating multiple objects.

CP – Singleton

- **Singleton** – Logger



CP – Lazy initialization

- **Lazy initialization** is the tactic of delaying the creation of an object, the calculation of a value, or some other expensive process until the first time it is needed

- How? With a **flag**
- Each time when the object is called, the **flag** is tested:
  - If it is **ready**, it is **returned**
  - If **not**, it is **initialized**

Lazy initialization – Java 1

```
public class Fruit {
  private static final Map<String,Fruit> types = new
   HashMap<String, Fruit>();
  private final String type;

  //using a private constructor to force use of the factory method
  private Fruit(String type) { this.type = type; }

  public static synchronized Fruit getFruit(String type) {
    if(!types.containsKey(type)) {
      types.put(type, new Fruit(type)); // Lazy initialization
    }
    return types.get(type);
  }
}
```

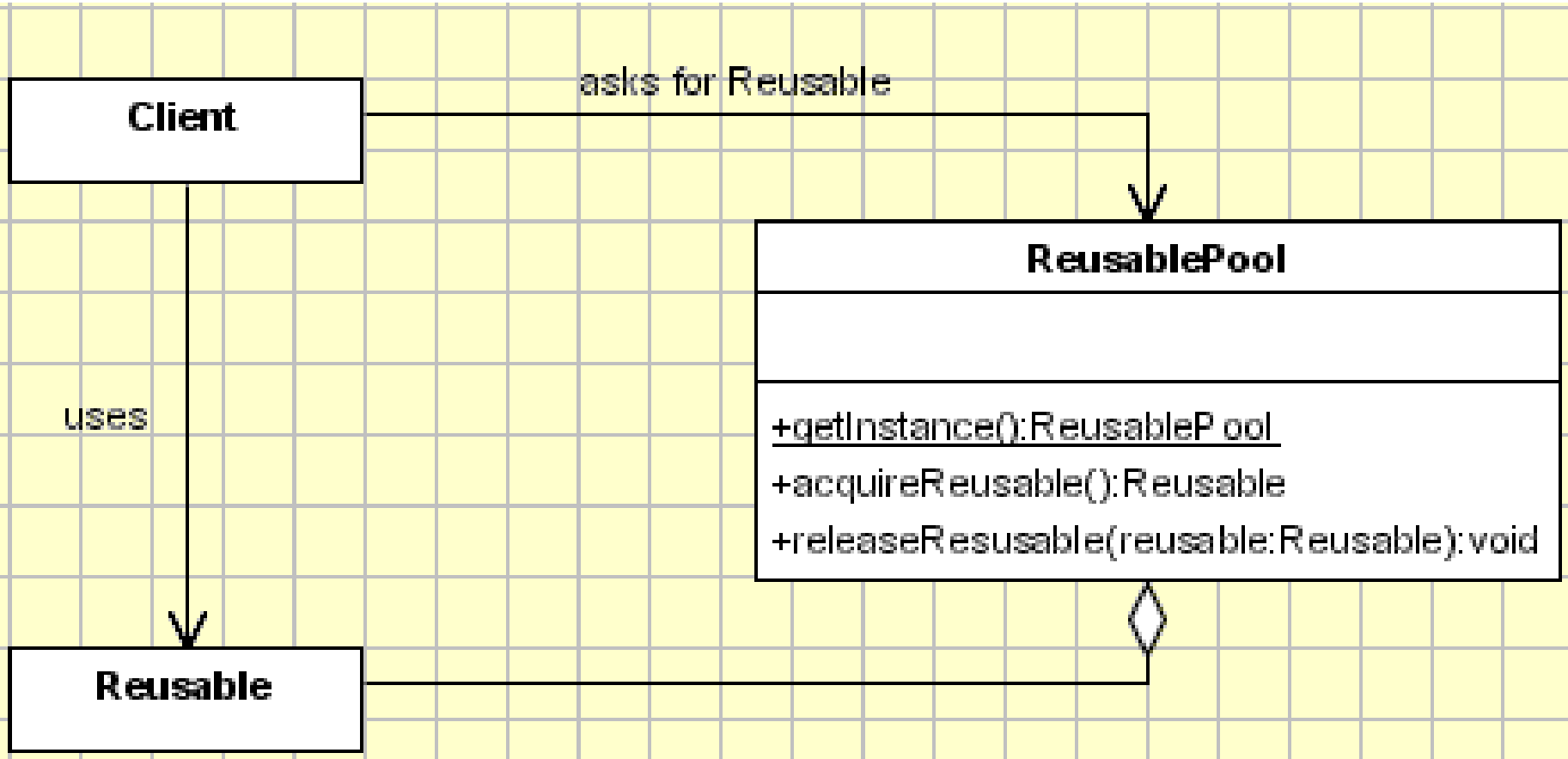
CP – Object Pool

- **Intent**: reuse and share objects that are expensive to create.
- **Motivation**: Performance can be sometimes the key issue during the software development and **the object creation (class instantiation) is a costly step**. The Object Pool pattern offer a mechanism to reuse objects that are expensive to create
- **Why use it?** Basically, we'll use an object pool whenever there are several clients who needs the same stateless resource which is expensive to create

Object Pool – Implementation (1)

- When a client asks for a Reusable object, the pool performs the following actions:
  - **Search** for an available Reusable object and **if it was found** it will be **returned** to the client.
  - **If no Reusable object was found** then it **tries to create a new one.** If this actions **succeeds** the new Reusable object will be **returned** to the client.
  - If the pool **was unable to create a new Reusable**, the pool will **wait** until a reusable object will be released.

Object Pool – Implementation (2)



Object Pool – Applicability

- Lets' take the example of the **database connections**. It's obviously that opening too many connections might affect the performance:
  - **Creating a connection is an expensive** operation
  - When there are **too many connections opened** it **takes longer to create a new one** and the database server will become overloaded

- Here the **object pool comes to manage the connections** and provide a way to reuse and share them. It can also limit the maximum number of objects that can be created

Structural Patterns 1

- **Structural patterns are concerned with how classes and objects are composed to form larger structures**

- **Example**: consider how **multiple inheritance** mixes two or more classes into one. The result is a class that combines the properties of its parent classes. This pattern is particularly useful for making independently developed class libraries work together.

Structural Patterns 2

- Rather than composing interfaces or implementations, structural *object patterns* describe ways to compose objects **to realize new functionality**
- **The added flexibility of object composition** comes from the ability to change the composition at run-time

Structural Patterns

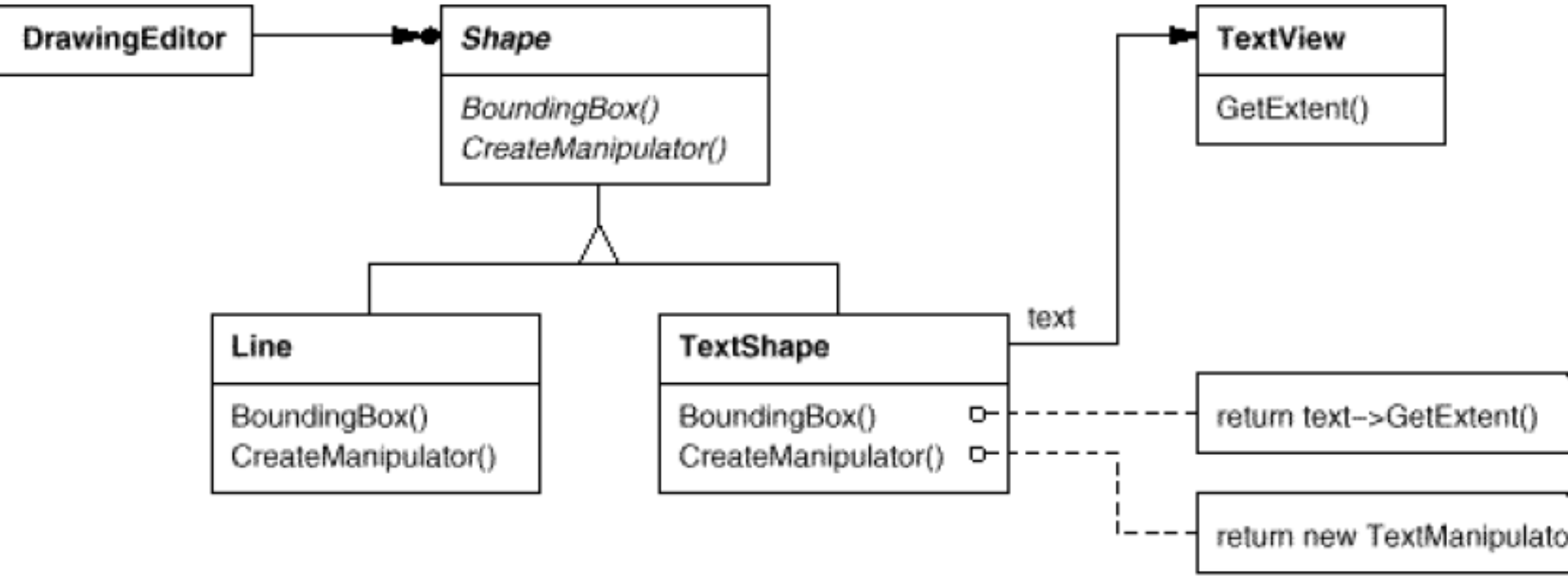
- Adapter
- Bridge
- Composite
- Decorator
- Façade
- Flyweight
- Proxy

Structural Patterns – Adapter

- **Intent**: Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.
- **Also Known As**: Wrapper
- **Motivation**: Consider for example a drawing editor that lets users draw and arrange graphical elements (lines, polygons, text, etc.) into pictures and diagrams. The interface for graphical objects is defined by an abstract class called Shape. The editor defines a subclass of Shape for each kind of graphical object: a LineShape class for lines, a PolygonShape class for polygons, and so forth.

Adapter 1

- A TextShape subclass that can display and edit text is considerably more difficult to implement
- We suppose the existence of a TextView class for displaying and editing text. So we can consider TextShape derived from these classes

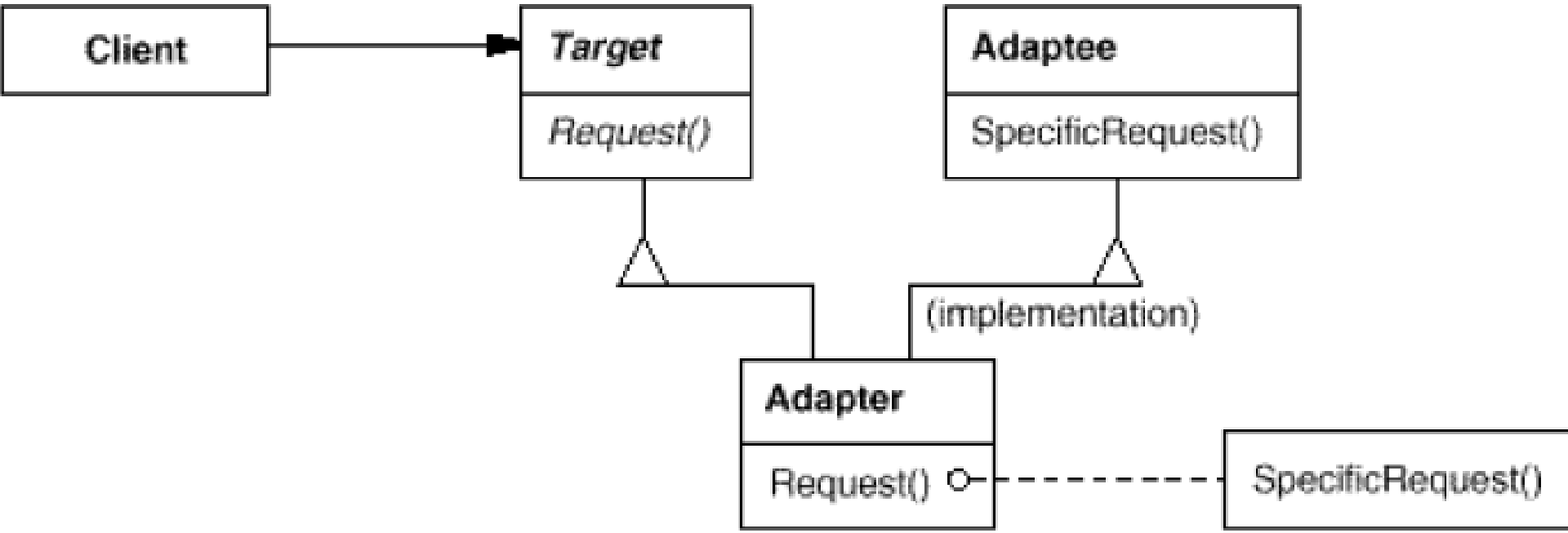


Adapter 2

- **Applicability** – Use the Adapter pattern when
  - you want to use an existing class, and its interface does not match the one you need
  - you want to create a reusable class that cooperates with unrelated or unforeseen classes, that is, classes that don't necessarily have compatible interfaces
  - *(object adapter only) you need to use several existing subclasses, but it's* impractical to adapt their interface by sub classing every one.

Adapter 3

- Structure – The Adapter pattern is used so that two unrelated interfaces can work together. This is something like we convert interface of one class into interface expected by the client.



Adapter – Example

- We all have electric sockets in our houses of different sizes and shapes. I will take an example of a socket of 15 Ampere. This is a bigger socket and the other one which is smaller is of 5 Ampere. A 15 Amp plug cannot fit into a 5 Amp socket => we will use an Adapter

- We have a 5 Amp plug and want a 5 Amp socket so that it can work. We DO NOT have a 5 Amp socket, what we have is a 15 Amp socket in which the 5 Amp plug cannot fit. The problem is how to cater to the client without changing the plug or socket.

Adapter – Java 1

```
/** The socket class has a specs for 15 AMP.*/
public interface Socket {
        public String getOutput();
}

public class Plug {
   private String specification = "5 AMP";
   public String getInput() {
        return specification;
   }
}
```

Adapter – Java 2

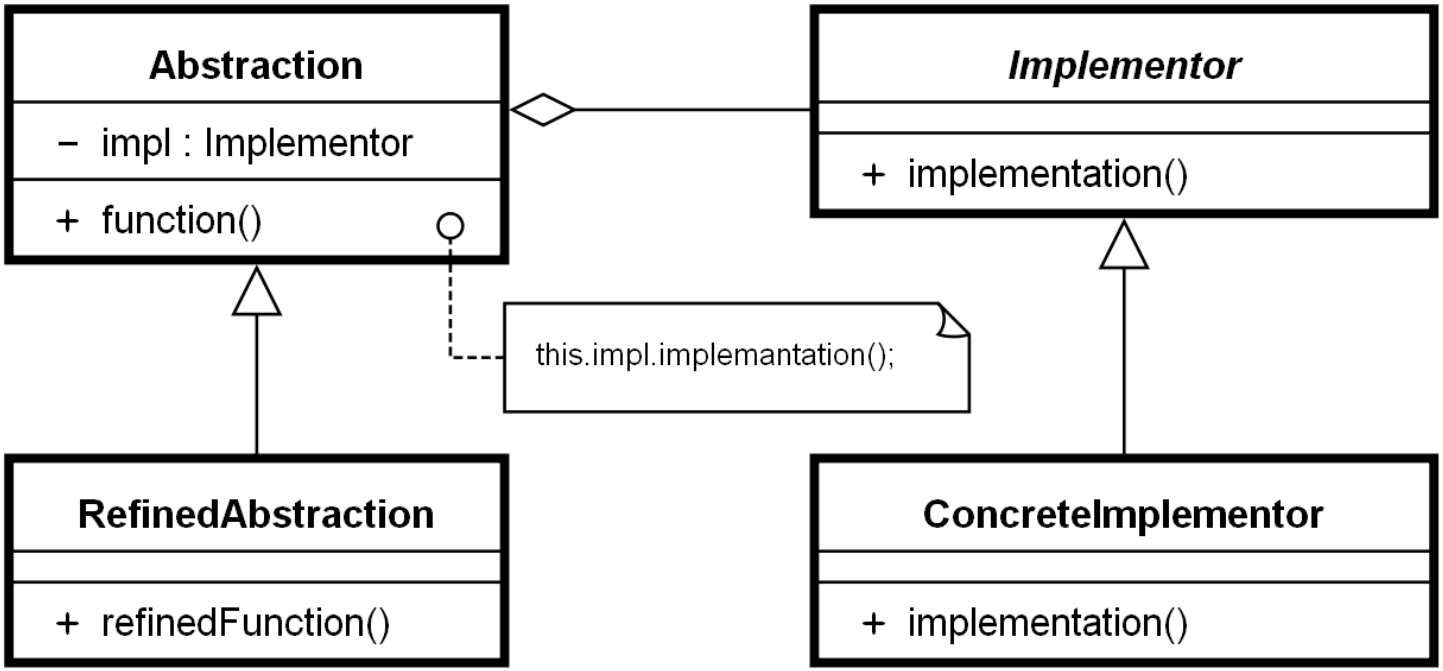
```
public class ConnectorAdapter implements Socket
{
  public String getOutput() {
      Plug plug = new Plug();
      String output = plug.getInput();
      return output;
  }
}
```

Structural Patterns – Bridge

- **Intent** – Decouple an abstraction from its implementation so that the two can vary independently
- **Also Known As** – Handle/Body
- **Motivation** – Consider the abstraction of shapes, each with its own properties. One thing all shapes can do is draw themselves. Drawing graphics to a screen can be dependent on different graphics implementations or operating systems. Shapes have to be able to be drawn on many types of operating systems

Bridge 1

- The bridge helps by allowing the creation of new implementation classes that provide the drawing implementation. Shape class provides methods for getting the size or properties of a shape. Drawing class provides an interface for drawing graphics



Bridge – Java 1

```
interface DrawingAPI {/** "Implementor" */
  public void drawCircle(double x, double y, double
  radius);
}
/** "ConcreteImplementor" 1,2 */
class DrawingAPI1 implements DrawingAPI {
  public void drawCircle(double x, double y, double
  radius) { System.out.printf("API1” + x + y + radius);}
}
class DrawingAPI2 implements DrawingAPI {
  public void drawCircle(double x, double y, double
  radius) { System.out.printf("API2“ + x + y + radius);}
}
```

Bridge – Java 2

```
interface Shape {/** "Abstraction" */
   public void draw();
   public void resizeByPercentage(double pct); }

/** "Refined Abstraction" */
class CircleShape implements Shape {
   private double x, y, radius;
   private DrawingAPI drawingAPI;
   public CircleShape(double x, double y, double radius,
   DrawingAPI drawingAPI) {
         this.x = x; this.y = y; this.radius = radius;
         this.drawingAPI = drawingAPI;
   }
   public void draw() {drawingAPI.drawCircle(x, y, radius); }
   public void resizeByPercentage(double pct) { radius *= pct; }
}
```

Bridge – Java 3

```
/** "Client" */
class BridgePattern {
    public static void main(String[] args) {
        Shape[] shapes = new Shape[2];
        shapes[0] = new CircleShape(1, 2, 3, new DrawingAPI1());
        shapes[1] = new CircleShape(5, 7, 8, new DrawingAPI2());
        for (Shape shape : shapes) {
            shape.resizeByPercentage(2.5);
            shape.draw();
        }
    }
}
```

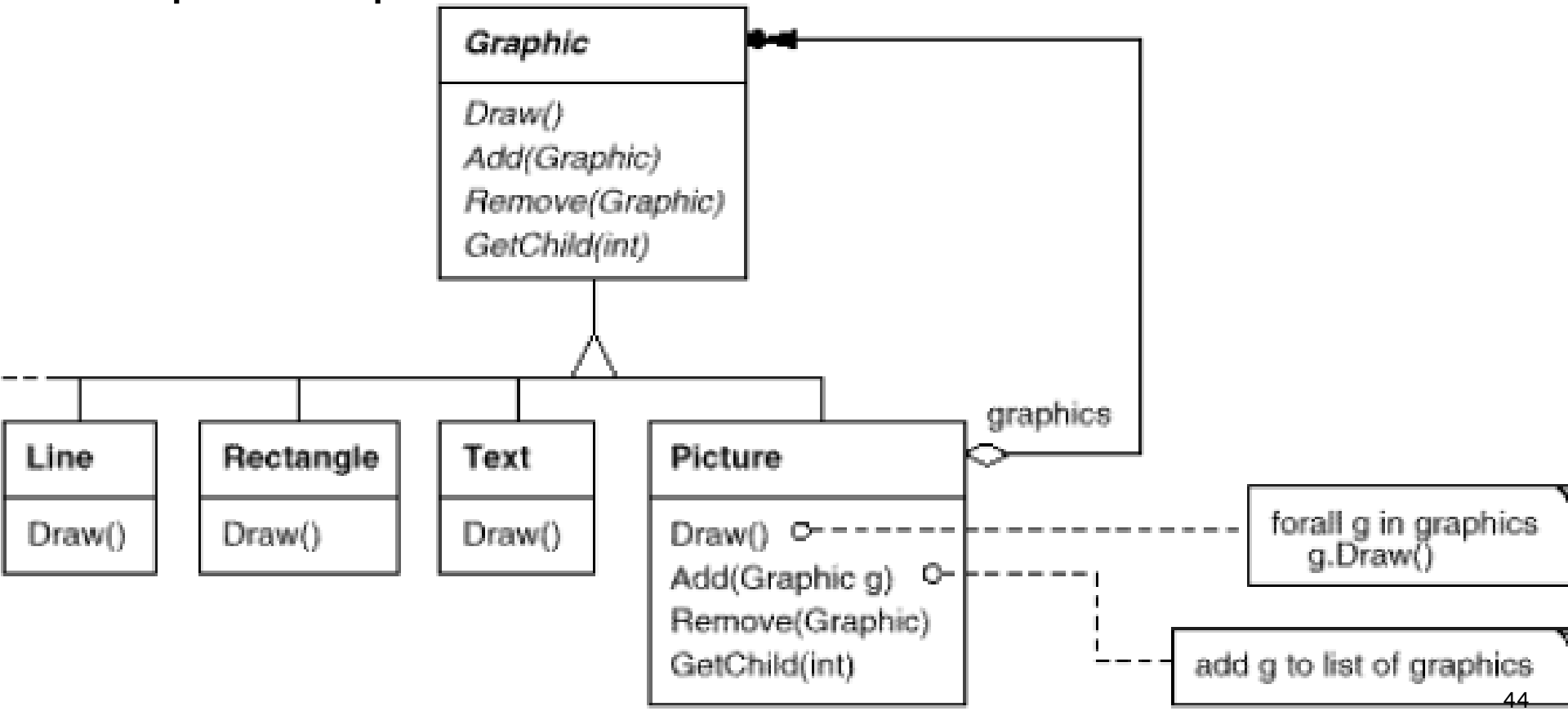
Structural Patterns – Composite

- **Intent** – Compose objects into tree structures to represent part-whole hierarchies. **Composite lets clients treat individual objects and compositions of objects uniformly**

- **Motivation** – Code that uses these classes must treat primitive and container objects differently, even if most of the time the user treats them identically.

Composite 1

- Graphics applications like drawing editors and schematic capture systems let users build complex diagrams out of simple components

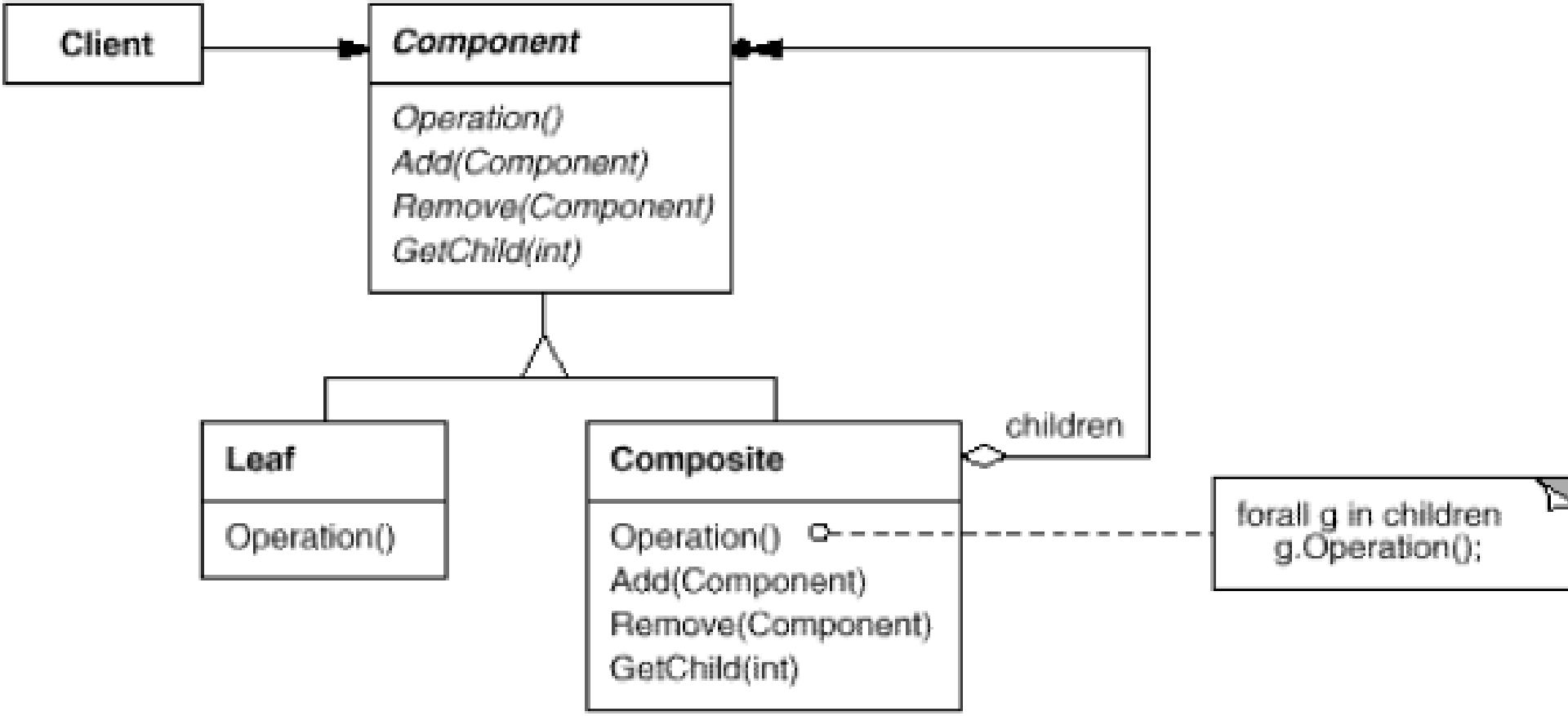


Composite 2

- **Applicability** – Use this pattern when
  - you want to represent part-whole hierarchies of objects
  - you want clients to be able to ignore the difference between compositions of objects and individual objects
  - Clients will treat all objects in the composite structure uniformly

Composite 3

- Structure



Composite – Example

- The most common example in this pattern is of a company's employee hierarchy

- The employees of a company are at various positions. Now, say in a hierarchy, the manager has subordinates; also the Project Leader has subordinates, i.e. employees reporting to him/her. The developer has no subordinates

Composite – Java 1

```
public class Employee {
        private String name; private double salary;
        private Vector subordinates;

        public Vector getSubordinates() {return subordinates;}
        public void setSubordinates(Vector subordinates) {
                this.subordinates = subordinates;}

        public Employee(String name, double sal) {
                setName(name);setSalary(sal);
                subordinates = new Vector();
        }
        public void add(Employee e) {subordinates.addElement(e);}
        public void remove(Employee e) {subordinates.remove(e);}
}
```

Composite – Java 2

```
private void addEmployeesToTree() {
   Employee CFO = new Employee("CFO", 3000);
   Employee headFinance1 = new Employee("HF. North", 2000);
   Employee headFinance2 = new Employee("HF. West", 2200);
   Employee accountant1 = new Employee("Accountant1", 1000);
   Employee accountant2 = new Employee("Accountant2", 900);
   Employee accountant3 = new Employee("Accountant3", 1100);
   Employee accountant4 = new Employee("Accountant4", 1200);

   CFO.add(headFinance1); CFO.add(headFinance2);
   headFinance1.add(accountant1);headFinance1.add(accountant4);
   headFinance2.add(accountant2);headFinance2.add(accountant3);
}
```

CFO = chief financial officer

Composite – Java 3

- Once we have filled the tree up, now we can get the tree for any employee and find out whether that employee has subordinates with the following condition.

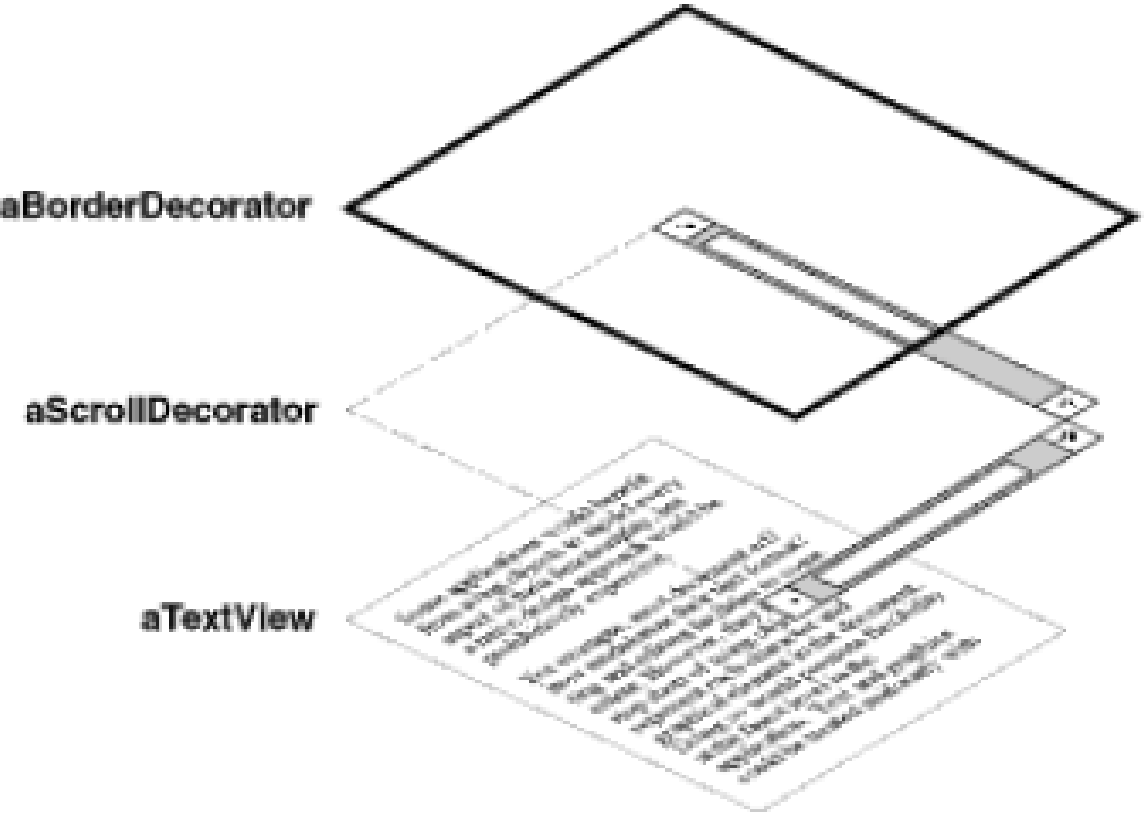
```
Vector subOrdinates = emp.getSubordinates();
if (subOrdinates.size() != 0)
    getTree(subOrdinates);
else
    System.out.println("No Subordinates for the
Employee: "+emp.getName());
```

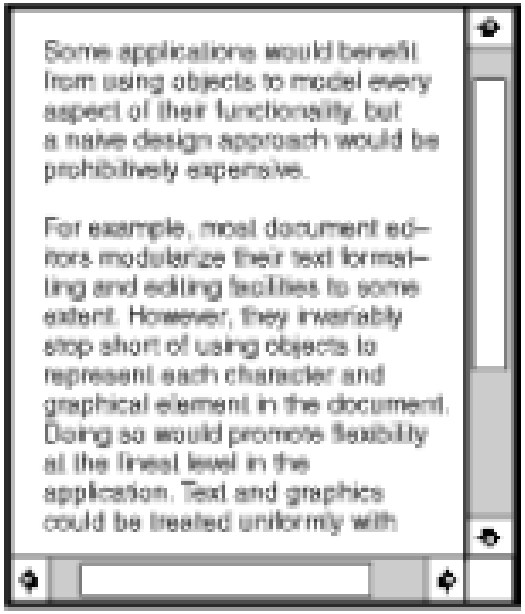
Structural Patterns – Decorator

- **Intent – Attach additional responsibilities to an object dynamically.** Decorators provide a flexible alternative to subclassing for extending functionality
- **Also Known As** – Wrapper (*similar Adapter*)
- **Motivation** – Sometimes we want to add responsibilities to individual objects, not to an entire class. A graphical user interface toolkit, for example, should let you add properties like borders or behaviors like scrolling to any user interface component

Decorator 1

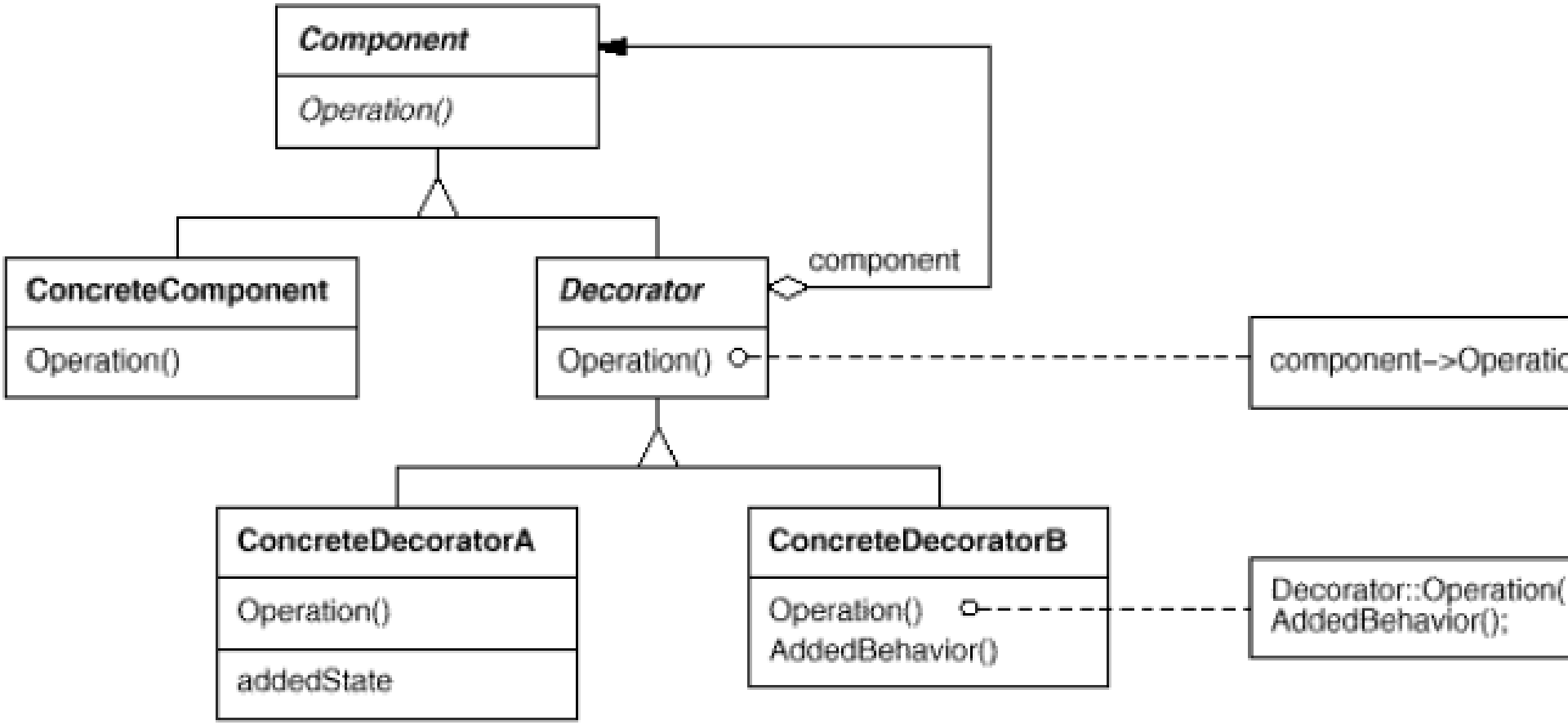
- A more flexible approach is to enclose the component in another object that adds the border. The enclosing object is called a **decorator**





Decorator 2

- Structure



Decorator 3

- **Applicability** – Use Decorator
  - to add responsibilities to individual objects dynamically and transparently
  - for responsibilities that can be withdrawn
  - when extension by subclassing is impractical

Decorator – Example

- Suppose we have some 6 objects and 2 of them need a special behavior, we can do this with the help of a decorator

- Let's take an example of a **Christmas tree**. There is a need to decorate a Christmas tree. Now we have many branches which need to be decorated in different ways

Decorator – Java 1

```
public abstract class Decorator {
  /** The method places each decorative item on
  the tree. */
  public abstract void place(Branch branch);
}
public class ChristmasTree {
  private Branch branch;
  public Branch getBranch() {
      return branch;
  }
}
```

Decorator – Java 2

```
public class BallDecorator extends Decorator {
  public BallDecorator(ChristmasTree tree) {
      Branch branch = tree.getBranch();
      place(branch);
  }

  public void place(Branch branch) {
      branch.put("ball");
  }
}
```

Decorator – Java 3

- Similarly, we can make StarDecorator and RufflesDecorator

StarDecorator decorator = new StarDecorator(new ChristmasTree());

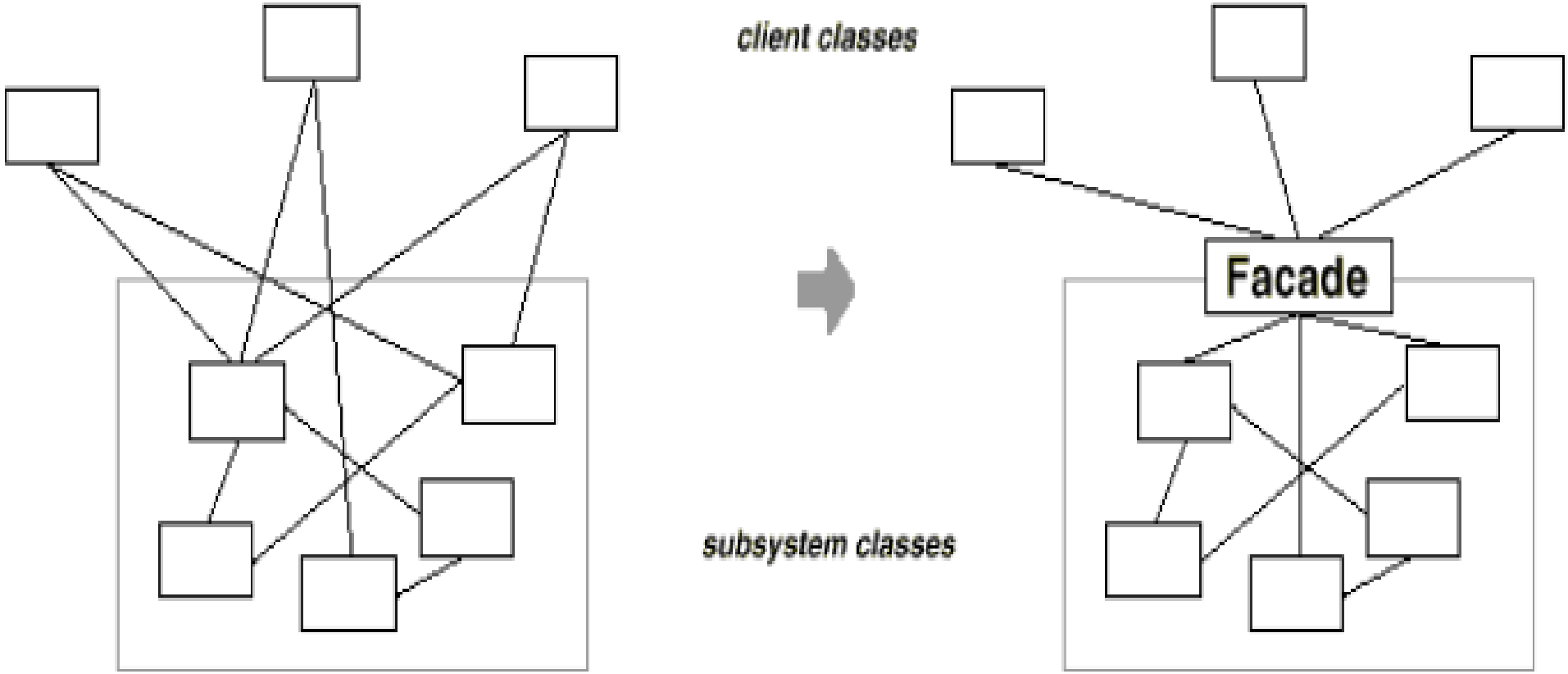
- This way the decorator will be instantiated and a branch of the Christmas tree will be decorated.

Structural Patterns – Façade

- **Intent – Provide a unified interface to a set of interfaces in a subsystem**
- **Motivation** – Consider for example a programming environment that gives applications access to its compiler subsystem. This subsystem contains classes such as *Scanner, Parser, ProgramNode, BytecodeStream,* and *ProgramNodeBuilder* that implement the compiler. Some specialized applications might need to access these classes directly. But most clients of a compiler want to compile some code

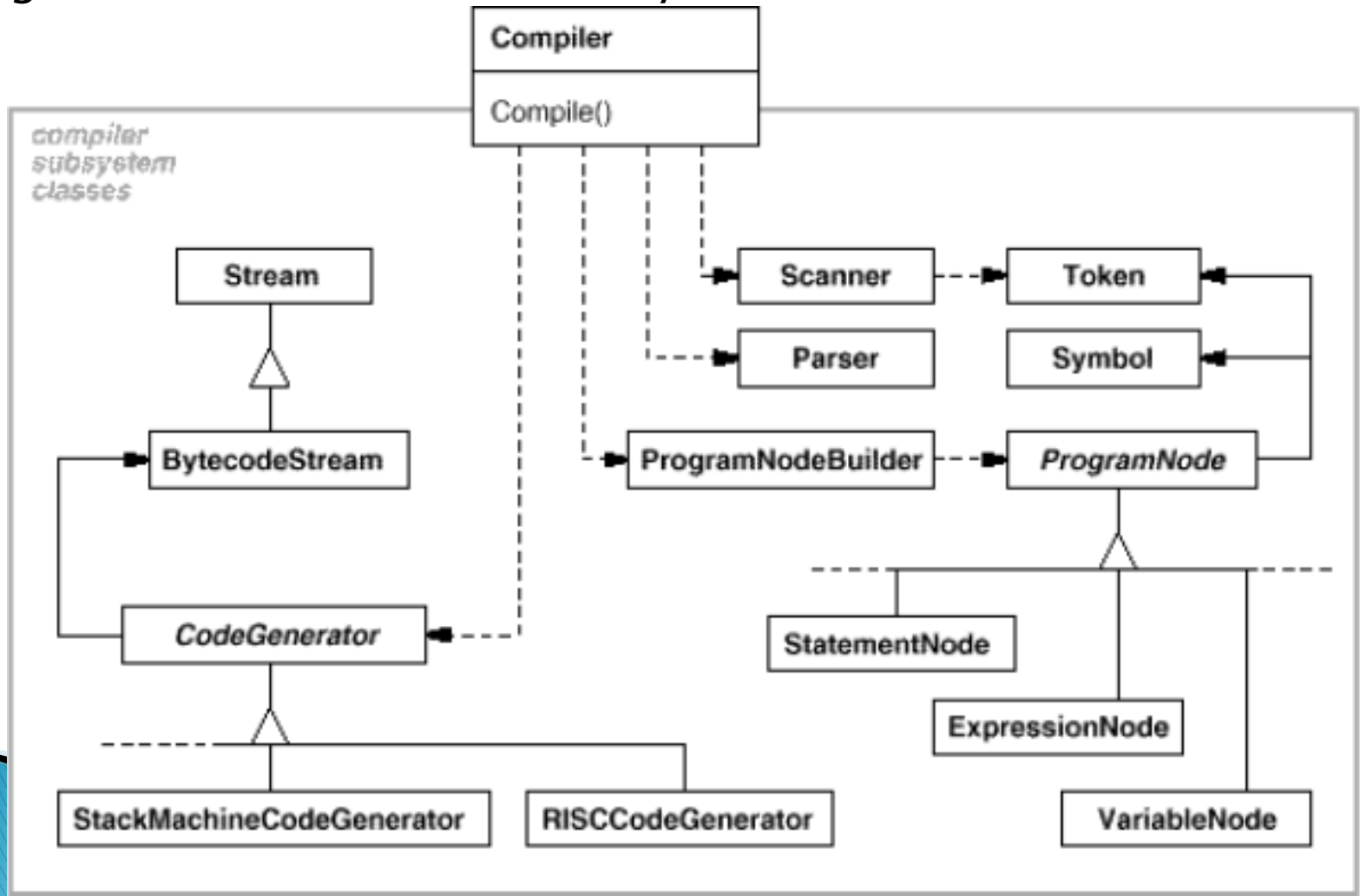
Façade 1

- A common design goal **is to minimize the communication and dependencies between** subsystems



Façade 2

- The compiler facade makes life easier for most programmers without hiding the lower-level functionality from the few that need it



Façade 3

- **Applicability** – Use the Facade pattern when
  - you want to provide a simple interface to a complex subsystem
  - there are many dependencies between clients and the implementation classes of an abstraction
  - you want to layer your subsystems

Façade – Example 1

- Facade as the name suggests means the face of the building. The people walking past the road can only see this glass face of the building. The face hides all the complexities of the building and displays a friendly face.
- Facade hides the complexities of the system and provides an interface to the client from where the client can access the system. In Java, the interface JDBC can be called a façade
- Other examples?

Façade – Example 2

- Let's consider a **store**. This store has a store keeper. In the storage, there are a lot of things stored e.g. **packing material, raw material and finished goods**.
- You, as client want access to different goods. You do not know where the different materials are stored. You just have access to store keeper who knows his store well. Here, the store keeper acts as the facade, as he hides the complexities of the system Store.

Façade – Java 1

```
public interface Store {
  public Goods getGoods();
}

public class FinishedGoodsStore implements Store
{
  public Goods getGoods() {
      FinishedGoods finishedGoods = new FinishedGoods();
      return finishedGoods;
  }
}
```

Façade – Java 2

```
public class StoreKeeper {
  public RawMaterialGoods getRawMaterialGoods() {
      RawMaterialStore store = new RawMaterialStore();
      RawMaterialGoods rawMaterialGoods =
  (RawMaterialGoods)store.getGoods();
      return rawMaterialGoods;
  }
  …
}
```

Façade – Java 3

```
public class Client {
  public static void main(String[] args) {
      StoreKeeper keeper = new StoreKeeper();
      RawMaterialGoods rawMaterialGoods =
      keeper.getRawMaterialGoods();
  }
}
```

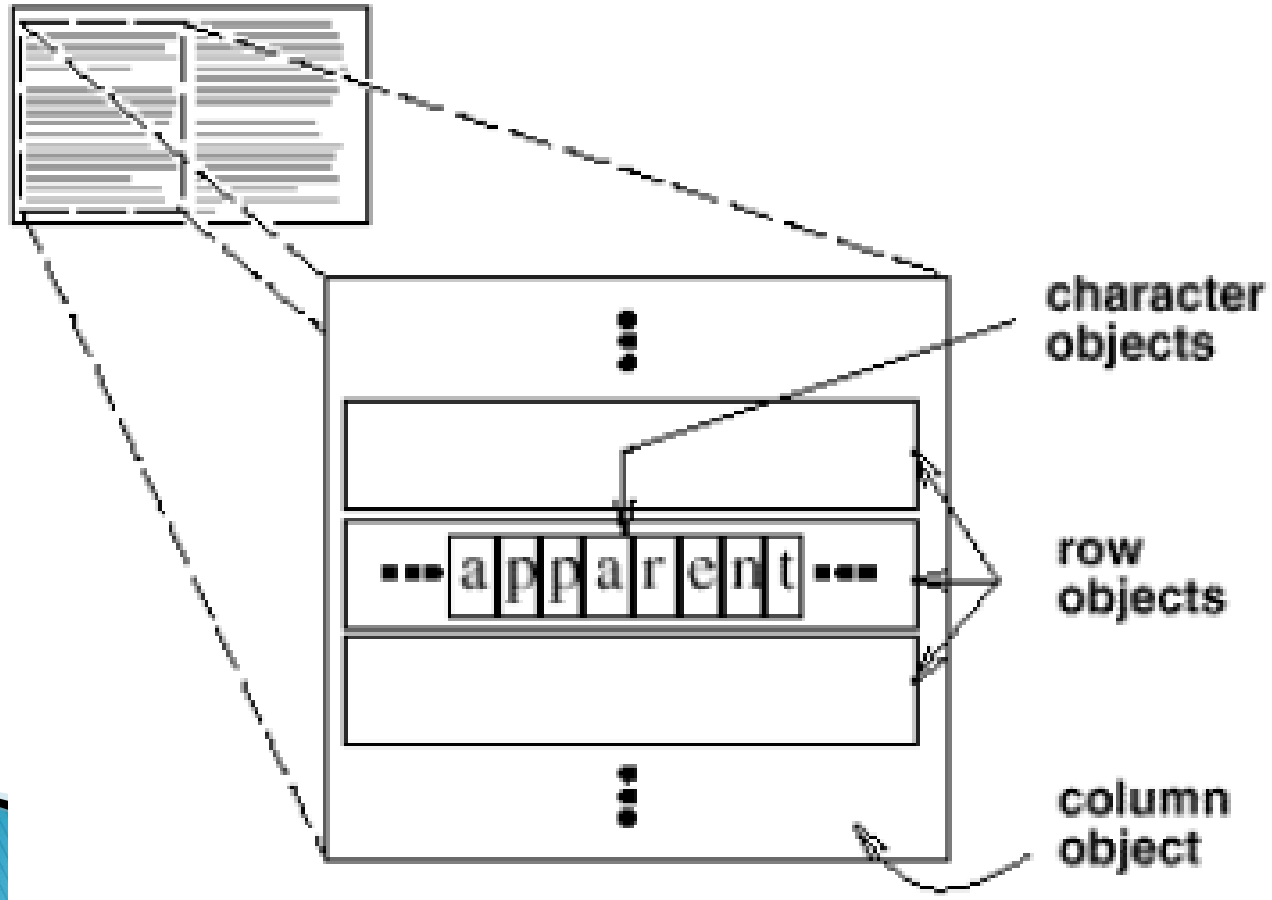
Structural Patterns – Flyweight

- **Intent – Use sharing to support large numbers of fine-grained objects efficiently**

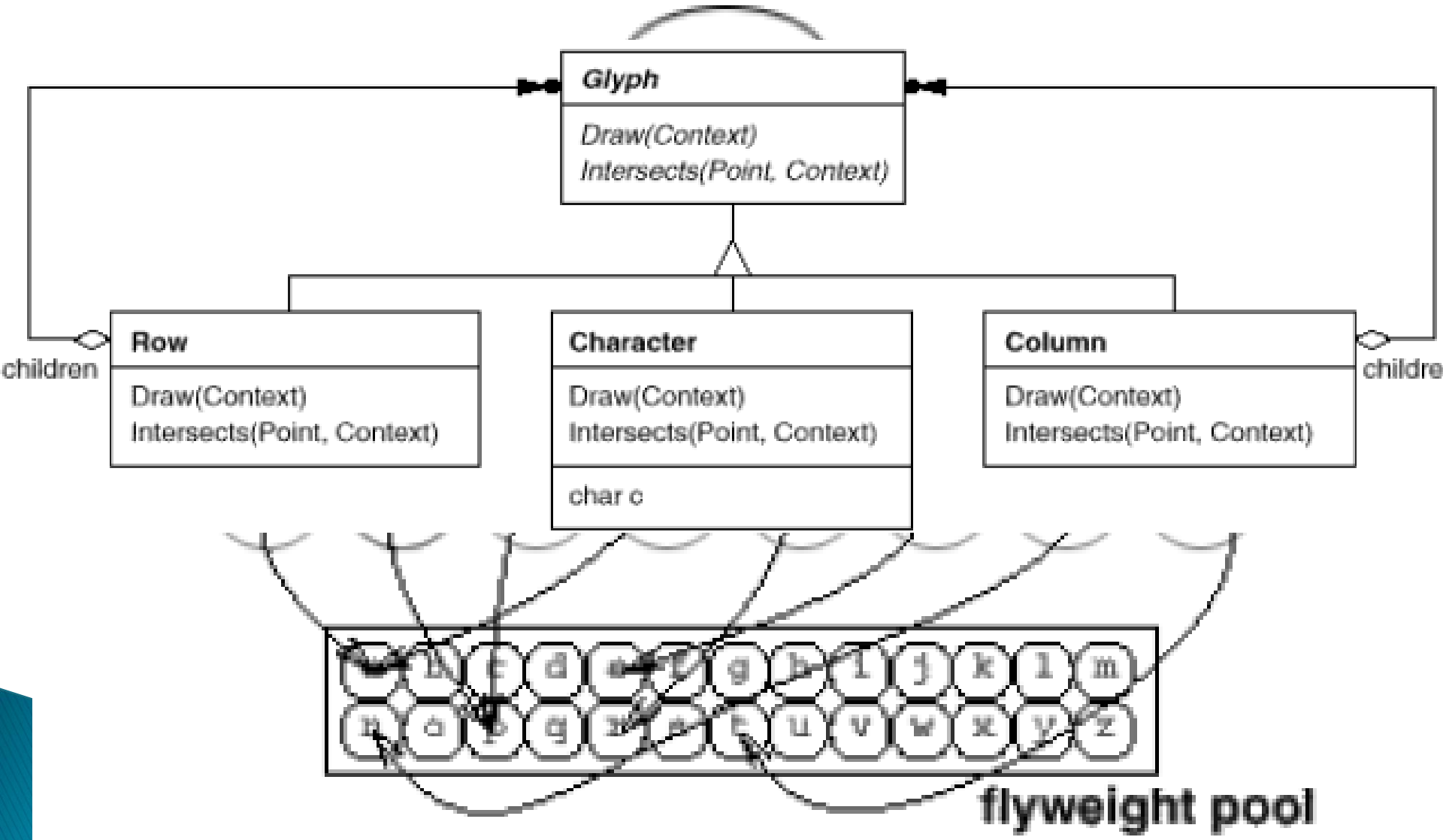
- **Motivation –** Some applications could benefit from using objects throughout their design, but a naive implementation would be prohibitively expensive.
- For example, most document editor implementations have text formatting and editing facilities that are modularized to some extent.

Flyweight 1

- The following diagram shows how a document editor can use objects to represent characters



Flyweight 2


Glyph
Draw(Context)
Intersects(Point, Context)
Row
Draw(Context)
Intersects(Point, Context)
Character
Draw(Context)
Intersects(Point, Context)
char c
Column
Draw(Context)
Intersects(Point, Context)
children
childre
flyweight pool

Flyweight – Example

- **A Flyweight is an object that minimizes memory use by sharing as much data as possible with other similar objects**

- A classic example usage of the flyweight pattern are the data structures for graphical representation of characters in a word processor. It would be nice to have, for each character in a document, a glyph object containing its *font outline, font metrics*, and other formatting data, but it would amount to hundreds or thousands of bytes for each character. Instead, are used the flyweights called **FontData**

Flyweight – Java 1

```
public enum FontEffect {
   BOLD, ITALIC, SUPERSCRIPT, SUBSCRIPT, STRIKETHROUGH
}
public final class FontData {
   private static final WeakHashMap<FontData,
   WeakReference<FontData>> FLY_WEIGHT_DATA = new
   WeakHashMap<FontData, WeakReference<FontData>>();
   private final int pointSize;
   private final String fontFace;
   private final Color color;
   private final Set<FontEffect> effects;

   private FontData(int pointSize, String fontFace, Color color,
   EnumSet<FontEffect> effects) {
         this.pointSize = pointSize;
         this.fontFace = fontFace;
         this.color = color;
         this.effects = Collections.unmodifiableSet(effects);
   }
```

Flyweight – Java 2

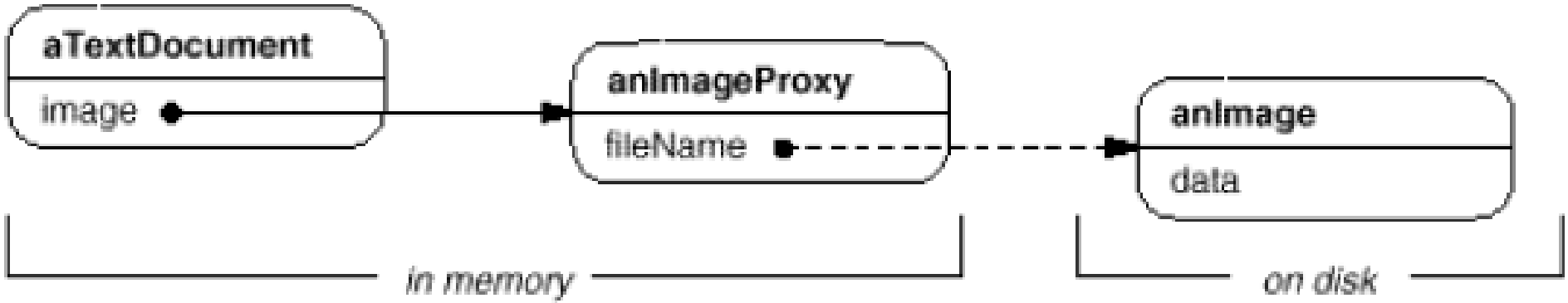
```
public static FontData create(int pointSize, String
  fontFace, Color color, FontEffect... effects) {
  EnumSet<FontEffect> effectsSet =
  EnumSet.noneOf(FontEffect.class);
  for (FontEffect fontEffect : effects) {
  effectsSet.add(fontEffect); }
  FontData data = new FontData(pointSize, fontFace,
  color, effectsSet);
  if (!FLY_WEIGHT_DATA.containsKey(data)) {
       FLY_WEIGHT_DATA.put(data, new
       WeakReference<FontData> (data));
  }
  return FLY_WEIGHT_DATA.get(data).get();
}
```

Structural Patterns – Proxy

- **Intent – Provide a surrogate or placeholder for another object to control access to it.**
- **Also Known As** – Surrogate
- **Motivation** – Consider a document editor that can embed graphical objects in a document. Some graphical objects, like large raster images, can be expensive to create. But opening a document should be fast, so we should avoid creating all the expensive objects at once when the document is opened. This isn't necessary anyway, because not all of these objects will be visible in the document at the same time

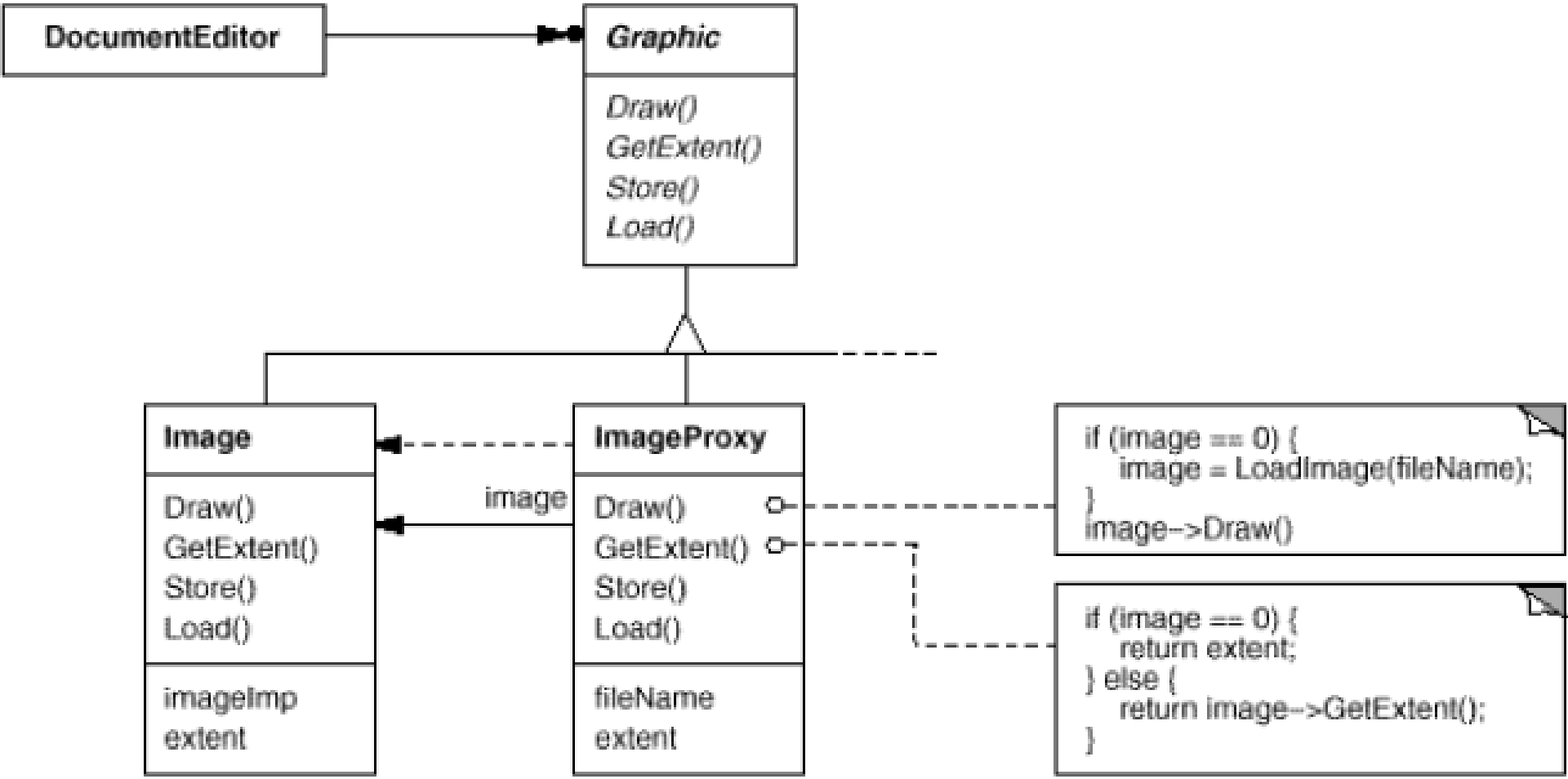
Proxy 1

- The solution is to use another object, an image **proxy, that acts as a stand-in** for the real image. The proxy acts just like the image and takes care of instantiating it when it's required.



Proxy 2

- The following class diagram illustrates this example in more detail



Proxy – Example

- Let’ say we need to withdraw money to make some purchase. The way we will do it is, go to an ATM and get the money, or purchase straight with a cheque.
- In old days when ATMs and cheques were not available, what used to be the way??? Well, get your passbook, go to bank, get withdrawal form there, stand in a queue and withdraw money. Then go to the shop where you want to make the purchase.
- In this way, we can say that ATM or cheque in modern times act as proxies to the Bank.

Proxy – Java 1

```
public class Bank {
   private int numberInQueue;

   public double getMoneyForPurchase(double amountNeeded) {
         You you = new You("Prashant");
         Account account = new Account();
         String accountNumber = you.getAccountNumber();
         boolean gotPassbook = you.getPassbook();
         int number = getNumberInQueue();

         while (number != 0) {number--; }

         boolean isBalanceSufficient =
   account.checkBalance(accountNumber, amountNeeded);
         if(isBalanceSufficient)
                  return amountNeeded;
         else
                  return 0;
   }

   private int getNumberInQueue() {
         return numberInQueue;}
}
```

Proxy – Java 2

```
public class ATMProxy {
  public double getMoneyForPurchase(double amountNeeded){
    You you = new You("Prashant");
    Account account = new Account();
    boolean isBalanceAvailable = false;
    if(you.getCard()) {
        isBalanceAvailable =
        account.checkBalance(you.getAccountNumber(),
        amountNeeded);
   }

   if(isBalanceAvailable)
        return amountNeeded;
   else
        return 0;
   }
}
```

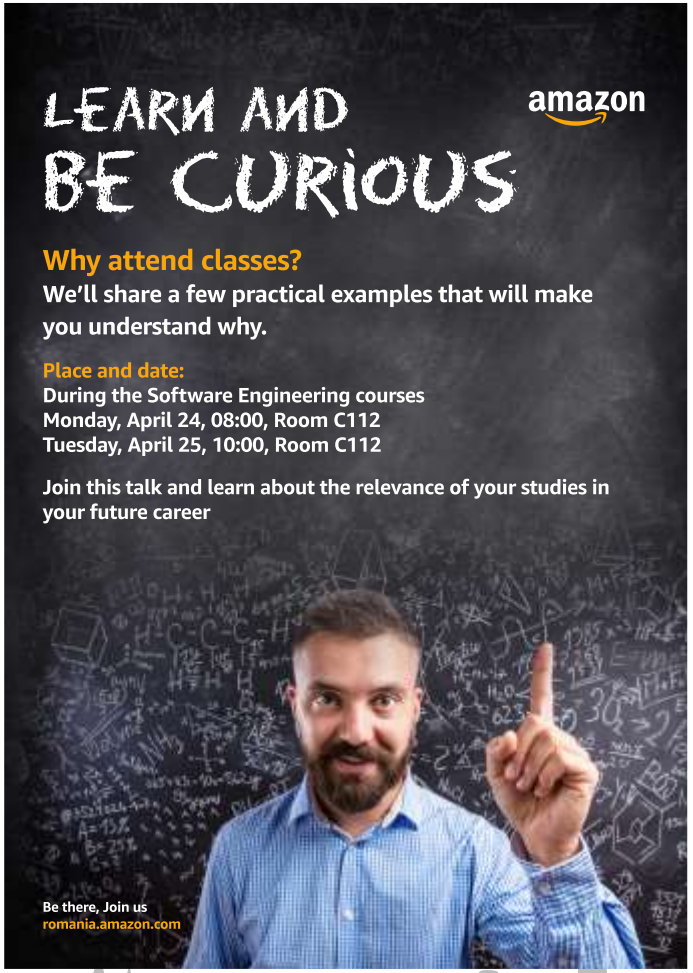
Bibliografie

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)

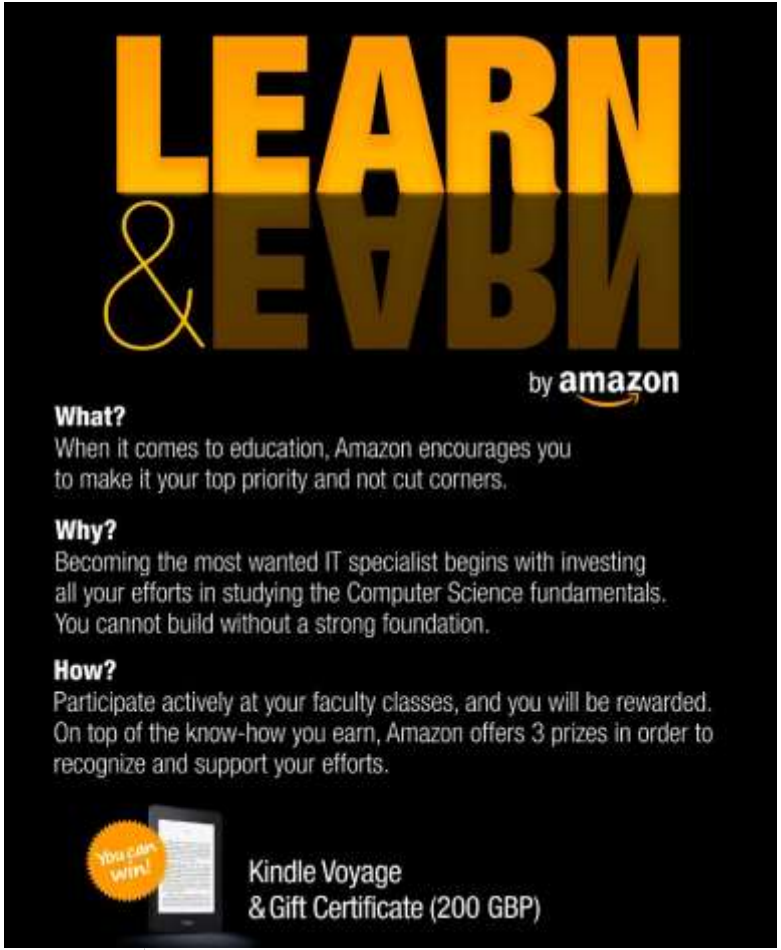
Links

- Design Patterns: http://www.oodesign.com/
- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm
- Lazy initialization: http://en.wikipedia.org/wiki/Lazy_initialization
- Use Lazy Initialization to Conserve Resources: http://www.devx.com/tips/Tip/18007

- **What** is **Learn&Be Curious?**
- **Why** we are here
- **What** we'll talk about



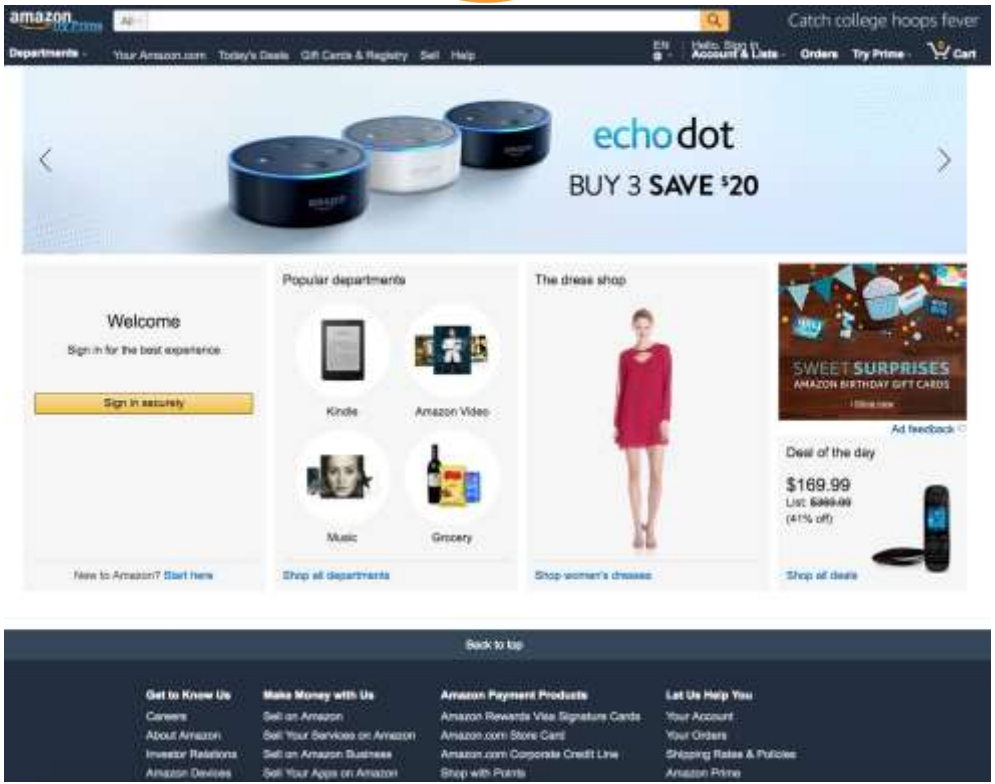
- **What** is **Learn&Earn**
- **Why** award prizes?



Small or world changing,
at the heart of any software is engineering.

What is amazon?

Online retail



What is amazon?

Online Retail

Robotics

What is amazon?

- Online Retail
- Robotics
- **Prime Air Delivery**

What is amazon?

- Online Retail
- Robotics
- Prime Air Delivery
- **AWS**

What is amazon?

- Online Retail
- Robotics
- Prime Air Delivery
- AWS
- **Kindle**

What is amazon?

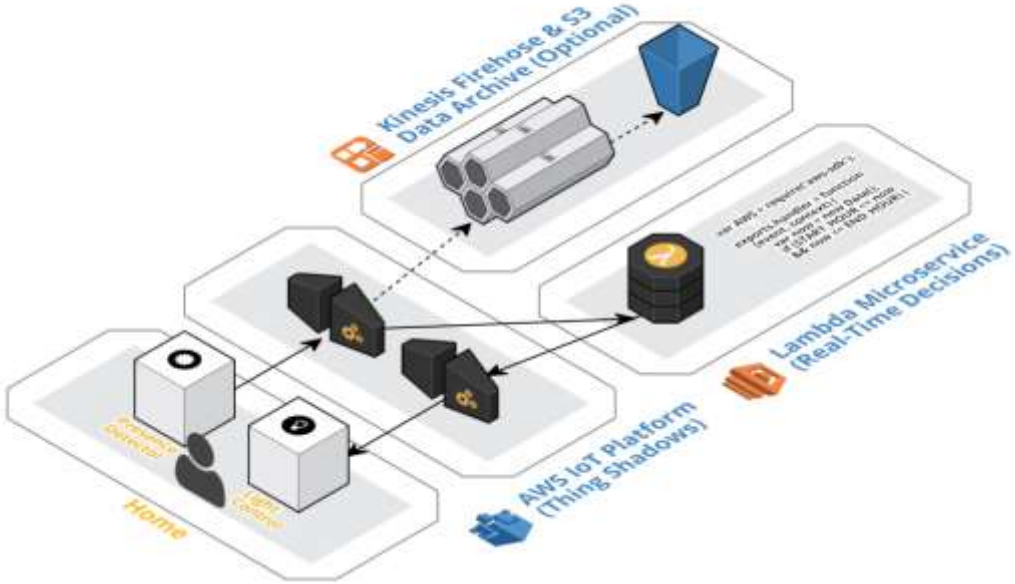
- Online Retail
- Robotics
- Prime Air Delivery
- AWS
- Kindle
- **Amazon Go**

What is amazon?

- Online Retail
- Robotics
- Prime Air Delivery
- AWS
- Kindle
- Amazon Go
- **Echo**

What is amazon?

- Online Retail
- Robotics
- Prime Air Delivery
- AWS
- Kindle
- Amazon Go
- Echo
- **Internet of Things**



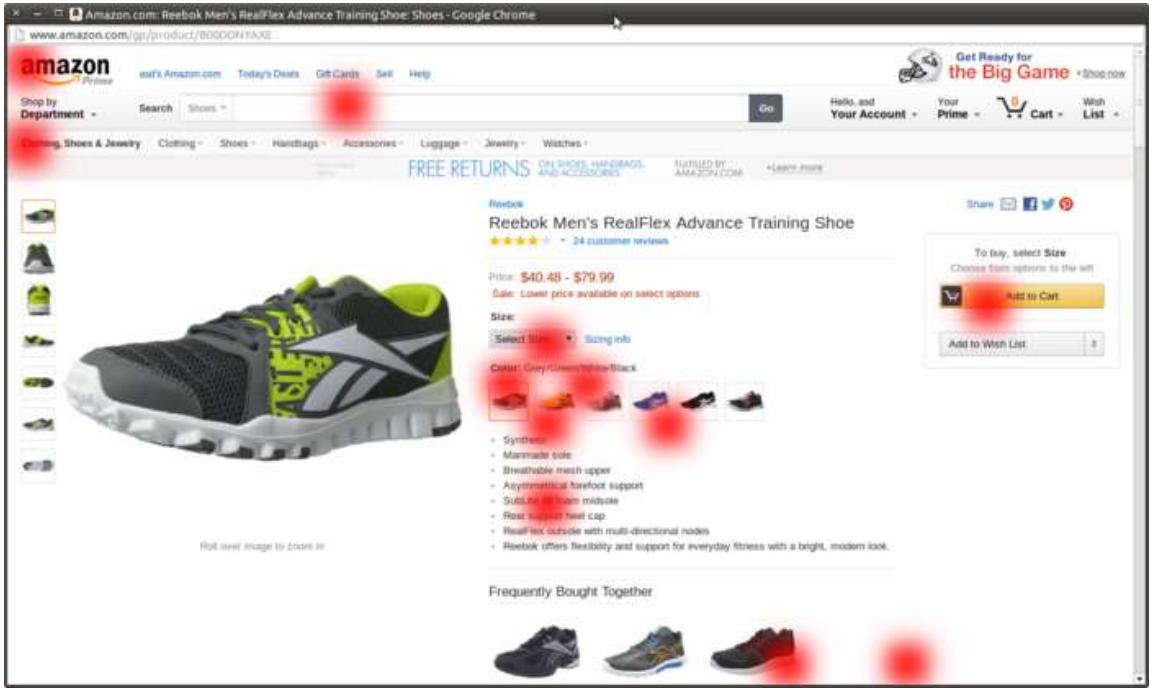
“Put the customer first. **Invent**. And be patient.”

Optimizing Amazon.com

- **How can we understand what the user's journey is on a page?**
- **In-page analytics.** ***Everyone does it!***

Optimizing Amazon.com

- **Measure, measure, measure!**
- **In the wild, not in the lab.**
- **Improve on the basis of real data.**



Analytics hooks

- A web page is composed of web elements.
- A user interacts with web elements.
- Interaction triggers more than one unit of work to be run.

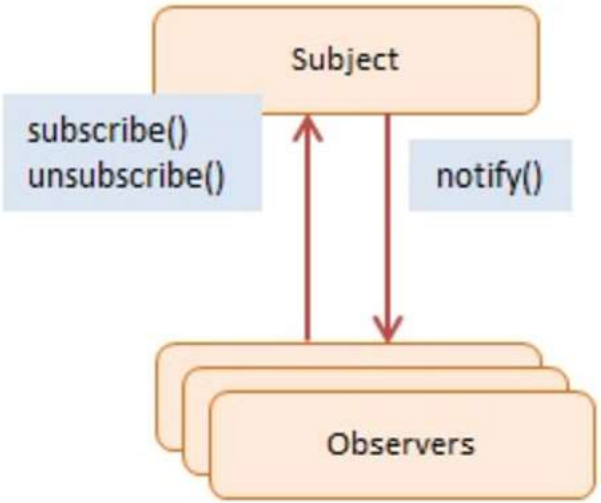
Analytics hooks

```
function buildDetailPage(product) {
   WebPage detailPage = new WebPage("detailPage", product);
   var addToCartButton = new Button("Add to cart");
   WebPage.add(addToCartButton);
   addToCartButton.onClick = new function(clickEvent) {
      doAnalytics(clickEvent);
      doAddToCart(product);
   };
   // other elements added to the page
}
```

Can we decouple the analytics from the actual add to cart invocation?

Observer

- **Problem:** A large monolithic design does not scale well as new graphing or monitoring requirements are levied.
- **Definition**: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Web apps** ⇔ writing many **event handlers**.
- **Event handlers** = functions that will be notified when a certain event fires.



http://www.dofactory.com/javascript/observer-design-pattern

Observer

```
function Button() {
    this.handlers = [];  // observers
}
Button.prototype = {
    subscribe: function(fn) {
        this.handlers.push(fn);
    },
    unsubscribe: function(fn) {
        this.handlers = this.handlers.filter(
            function(item) {
                return (item !== fn) ? item : undefined
            }
        );
    },
    fire: function(event) {
        var scope = thisObj || window;
        this.handlers.forEach(function(item) {
            item.call(scope, event);
        });
    }
}
```

Observer

```
// new Detail page code, decoupled from Analytics code
function buildDetailPage(product) {
   WebPage detailPage = new WebPage("detailPage", product);
   var addToCartButton = new Button("Add to cart");
   WebPage.add(addToCartButton);
   function addToCart() {
      // call add to cart service
   }
   addToCartButton.subscribe(addToCart);
   // other elements added to the page
}
```

Observer

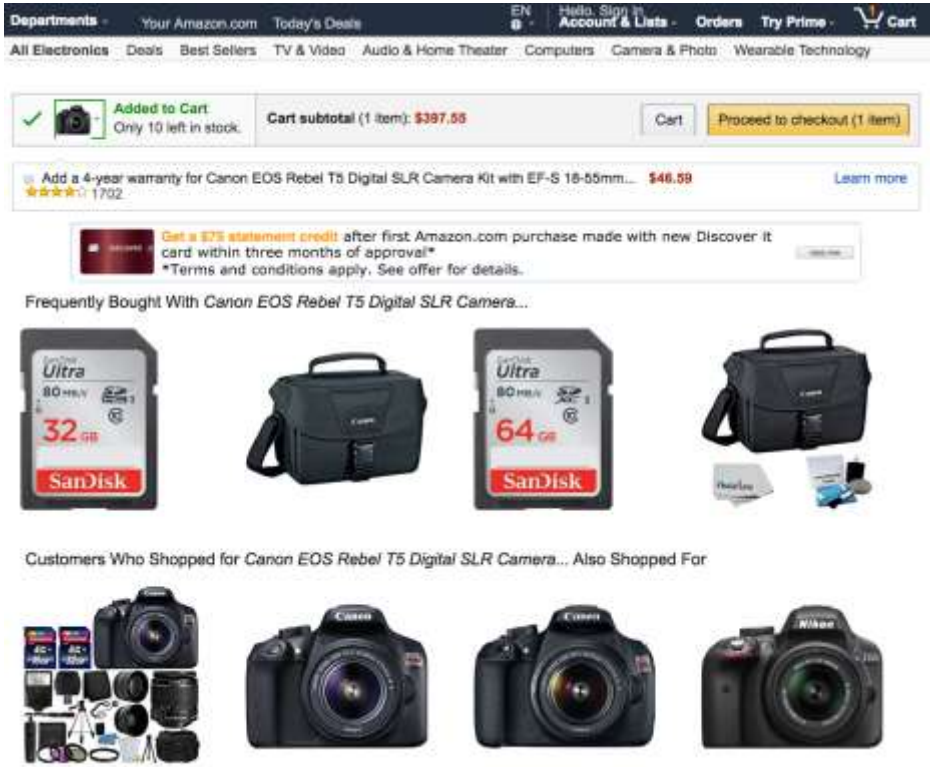
```
function analyticsModule(webPage) {
    function doAnalytics(elementEvent) {
        // call analytics service and register event
    }
    webPage.getButtons().forEach(function(button) {
        button.subscribe(doAnalytics);
    });
    // analytics module code goes here
}
```

Observer - benefits

- Decoupling of subject and observers: the subject doesn’t need to know about how many or which observers will be interested in changes.
- Open-closed software: we only need to write new observer code, not modify existing one.

Shopping engagement

- How are these recommendations served?
- What if we want new types of recommendations to be displayed?



Shopping engagement

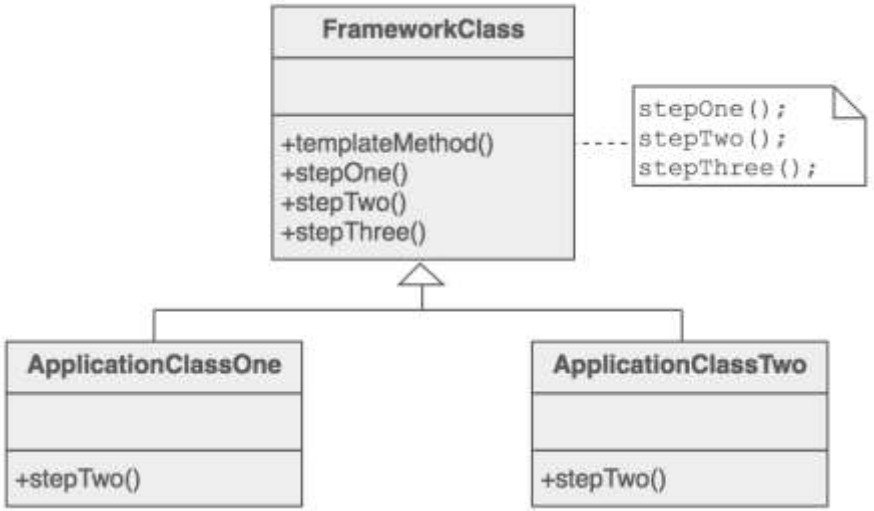
```
public class TopSellersWidget extends WidgetBase {
  static final String TOP_SELLERS_RANK = "top-sellers";
  @Override
  protected Set<Products> getRecommendedProducts() {
    if(KindleService.isKindleInCart(this.addToCartRequest.getCartProducts())) {
        return Collections.emptySet();
    }

    ProductRakingService service = ProductRankingServiceFactory.getService();
    GetRankedProductsRequest request = service.newGetRankedProductsRequest(
              this.addToCartRequest.getMarketplaceID(),
              TOP_SELLERS_RANK,
              this.getCategories(this.addToCartRequest.getCartProducts()));

    GetRankedProductsResult result = request.callAsync();
    while (!result.isReady() && !currentThread().isInterrupted()) {
       wait(100);
    }
    if (result.isReady()) {
        Map<Category, Set<Product>> topRankedProducts = result.getTopRankedProducts();
        return topRankedProducts.entrySet().stream()
                    .flatMap(mapEntry -> mapEntry.getValue().stream())
                    .collect(Collectors.toSet());
    }
    return Collections.emptySet();
  }
}
```

Shopping engagement

```
public class PurchaseSimilaritiesWidget extends WidgetBase {
  @Override
  protected Set<Products> getRecommendedProducts() throws Exception {
    SimilaritiesService simsService = SimilaritiesServiceFactory.getService();
    GetSimilaritiesRequest simsRequest = simsService.newGetSimilaritiesRequest(
                  this.addToCartRequest.getMarketplaceID(),
                  this.addToCartRequest.getCartProducts());
    simsRequest.setMaxResults(10);
    GetSimilaritiesResult simsResult = simsRequest.callAsync();
    while (!simsResult.isReady() && !currentThread().isInterrupted()) {
        wait(50);
    }
    if (simsResult.isReady()) {
        Map<Product, Map<Product, Float>> similarProducts = simsResult.getSimilarProducts();
        return topRankedProducts.entrySet().stream()
                  .flatMap(mapEntry -> mapEntry.getValue().stream())
                  .sorted(Map.Entry.<Product, Float>comparingByValue().reversed())
                  .limit(10)
                  .map(Map.Entry::getKey)
                  .collect(Collectors.toSet());
    }
    return Collections.emptySet();
  }
}
```

Template method

- **Problem**: two different components have significant similarities. A change common to both components implies duplicate effort.
- **Definition**: Define the skeleton of an algorithm in an operation, deferring some steps to client subclasses.
- **Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.**



https://sourcemaking.com/design_patterns/template_method

Template method

```
public enum RecommendedProductRetrievalTemplate {
    INSTANCE;
    public final Set<Product> retrieve(Widget widget) {
        if (widget.shouldStop()) {
            return Collections.emptySet();
        }
        Request serviceRequest = widget.buildServiceRequest();
        Result result = serviceRequest.callAsync();
        waitForServiceResult(result);
        return widget.processServiceResult(result);
    }
    private void waitForServiceResult(final Result result) {
        // do complicated synchronization on results ready-ness
        ...
    }
}
public interface Widget {
    boolean shouldStop();
    Request buildServiceRequest();
    Set<Products> processServiceResult(Result result);
}
```

Template method

```
public class TopSellersWidget extends WidgetBase {
  static final String TOP_SELLERS_RANK = "top-sellers";
  @Override
  boolean shouldStop() {
    return KindleService.isKindleInCart(this.addToCartRequest.getCartProducts());
  }
  @Override
  Request buildServiceRequest() {
    ProductRakingService service = ProductRankingServiceFactory.getService();
    GetRankedProductsRequest request = service.newGetRankedProductsRequest(
              this.request.getMarketplaceID(),
              TOP_SELLERS_RANK,
              this.getCategories(this.addToCartRequest.getCartProducts()));
    return request;
  }
  @Override
  Set<Products> processServiceResult(Result result) {
    Map<Category, Set<Product>> topRankedProducts = result.getTopRankedProducts();
    return topRankedProducts.entrySet().stream()
                      .flatMap(mapEntry -> mapEntry.getValue().stream())
                      .collect(Collectors.toSet());
  }
}
```

Template method

```
public class PurchaseSimilaritiesWidget extends WidgetBase {
  @Override
  boolean shouldStop() { return false; }
  @Override
  Request buildServiceRequest() {
    SimilaritiesService simsService = SimilaritiesServiceFactory.getService();
    GetSimilaritiesRequest simsRequest = simsService.newGetSimilaritiesRequest(
                this.addToCartRequest.getMarketplaceID(),
                this.addToCartRequest.getCartProducts());
    simsRequest.setMaxResults(10);
    return simsRequest;
  }
  @Override
  Set<Products> processServiceResult(Result result) {
    Map<Product, Map<Product, Float>> similarProducts = simsResult.getSimilarProducts();
    return topRankedProducts.entrySet().stream()
              .flatMap(mapEntry -> mapEntry.getValue().stream())
              .sorted(Map.Entry.<Product, Float>comparingByValue().reversed())
              .limit(10)
              .map(Map.Entry::getKey)
              .collect(Collectors.toSet());
  }
}
```

Template method - benefits

- All widgets know what they have to define in order to return recommendations.
- They don’t reinvent the wheel.
- They don’t duplicate async calls and results synchronization code.

Design patterns in low level programming

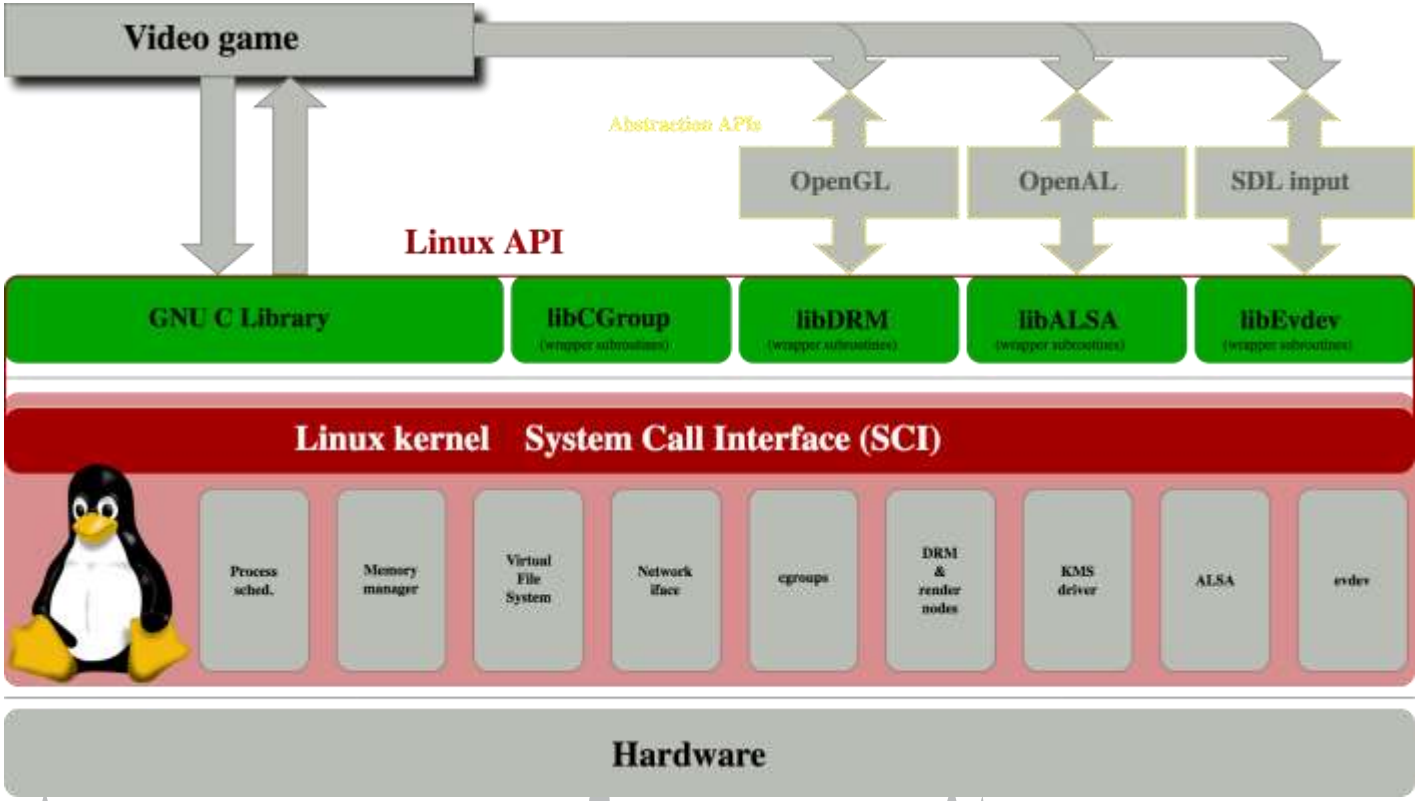
Design patterns in low level programming

- Do you trust Amazon to delegate credit card processing?
- We work hard to protect customer data

Linux Security Modules

- How access control works in kernel?
- Different security modules:
  - SELinux (NSA)
  - AppArmor (Canonical)
  - Smack (Intel)
  - TOMOYO (NTT Data Corp)
  - Yama (Canonical)

Linux Security Modules



https://upload.wikimedia.org/wikipedia/commons/4/43/Linux_API.svg

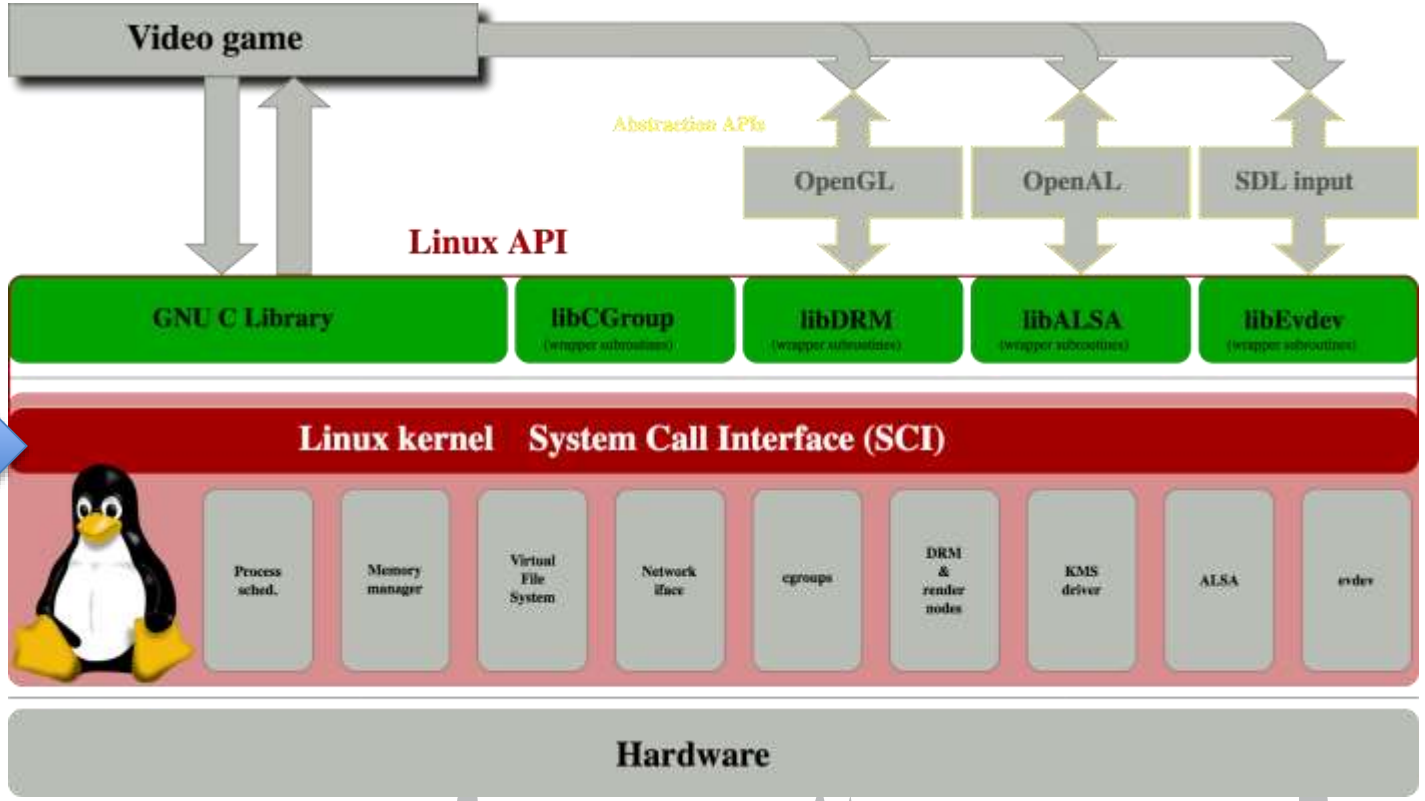
Linux Security Modules

How can one restrict syscall access?

Linux Security Modules

- In early 2000 there was no standardized kernel security framework
- NSA proposes SELinux as a monolithic solution

Linux Security Modules



2001 SELinux

https://upload.wikimedia.org/wikipedia/commons/4/43/Linux_API.svg

Linux Security Modules

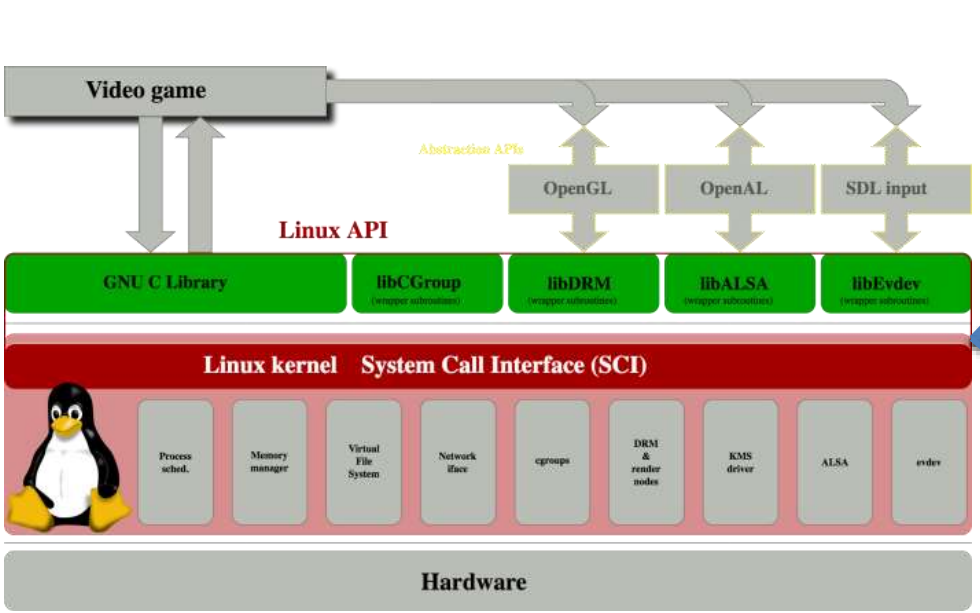
High-level pseudocode:

```
open_syscall():
        …
        selinux_access_checks()
        ...
        open_call()
```

Linux Security Modules

- In 2003 there was still the same problem
- LSM is introduced as a generic security framework
- SELinux is approved in the mainline

Linux Security Modules



Hooks

- SELinux
- AppArmor
- Smack
- Tomoyo
- Yama

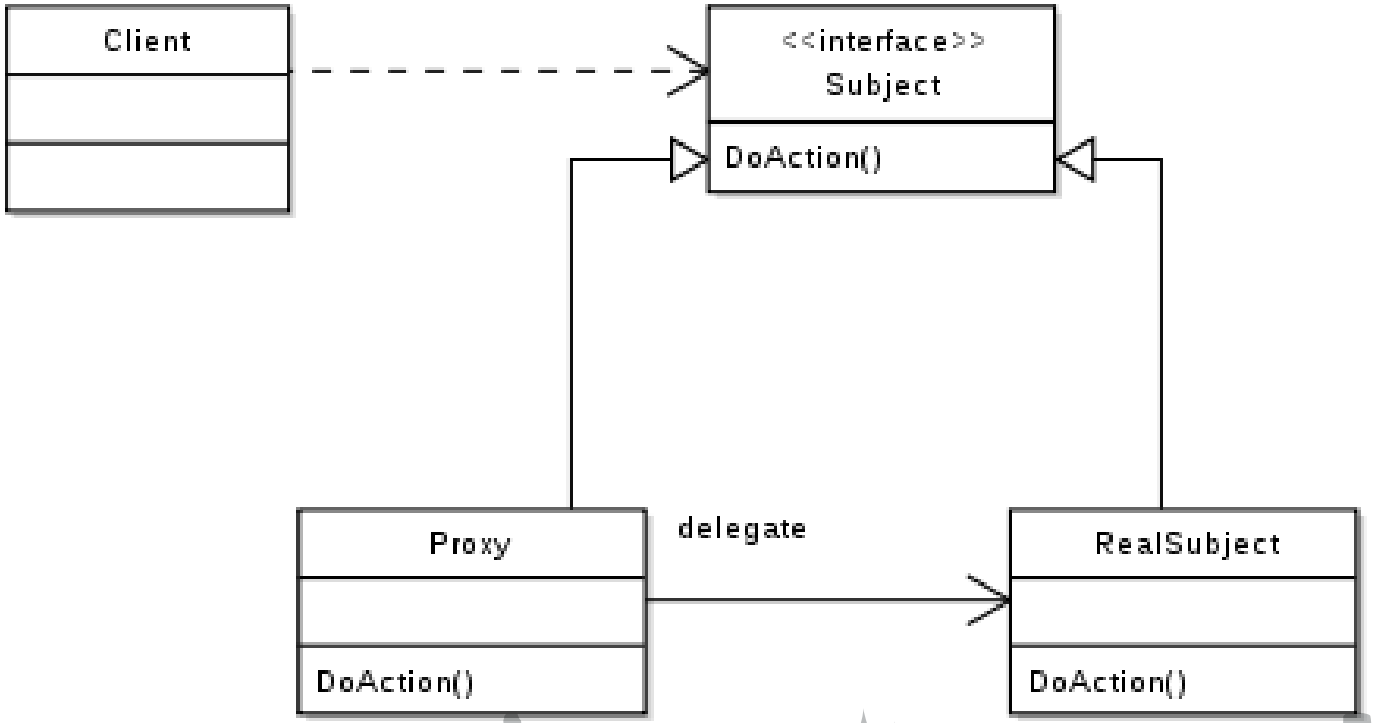
https://upload.wikimedia.org/wikipedia/commons/4/43/Linux_API.svg

Linux Security Modules

High-level pseudocode:

```
register_selinux_hooks()
open_syscall():
    …
    process_generic_hooks()
    ...
    open_call()
```

Linux Security Modules – Proxy Pattern



https://upload.wikimedia.org/wikipedia/commons/thumb/7/75/Proxy_pattern_diagram.svg/800px-Proxy_pattern_diagram.svg.png

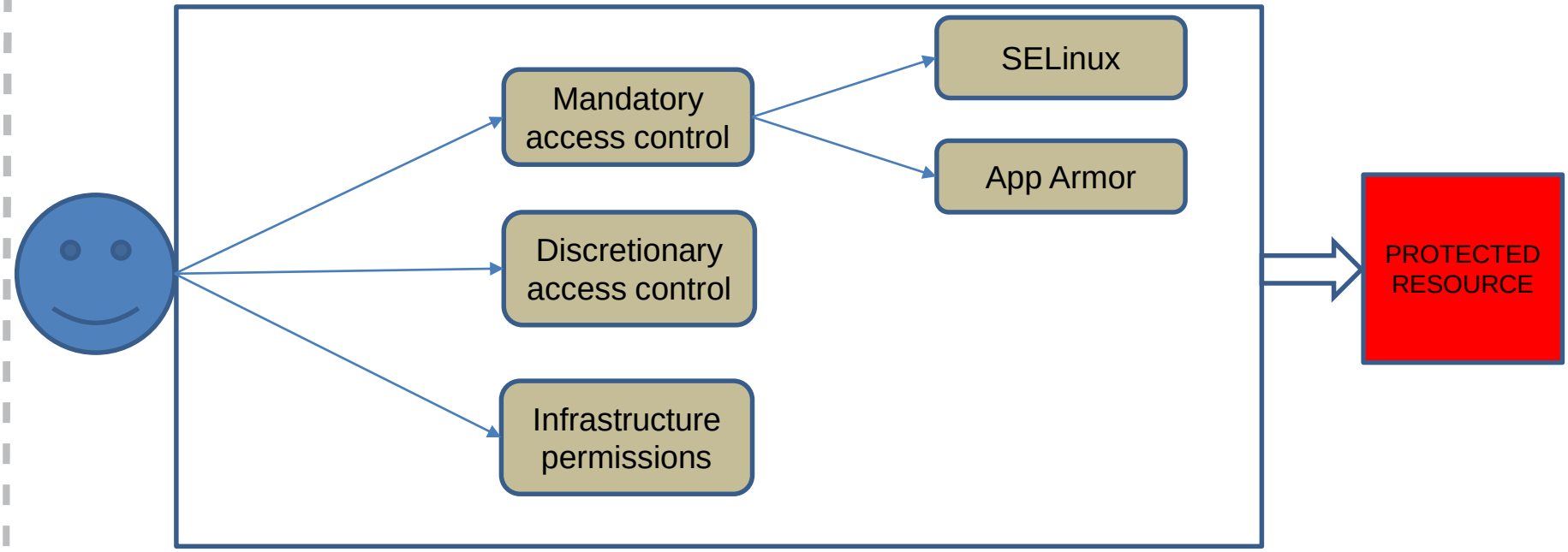
Linux Security Modules – Proxy Pattern

- Rejecting initial SELinux proposal was a chance to build generic frameworks
- Proxy pattern adds an indirection layer
- Reduces complexity from critical components
- Adds a plus on availability since critical code is not modified often

Managing permissions

- Default Discretionary Access Control(DAC) in Linux has: read, write and execute
- Sometimes it is not enough, you want additional granularity (heap dump, stack trace)
- Existing frameworks might be too complex with a steep learning curve
- There are multiple kinds of permissions for each resource
- Sometimes you must combine different technologies to achieve a goal

Managing permissions



Managing permissions

High-level pseudocode

```
init_mac_system()
init_dac_system()
init_infrastructure_permissions()
…
check_mac_access(user, resource, action)
check_dack_access(user, resource, action)
check_infrastructure_permissions(user,
resource, action)
```

Managing permissions - Facade

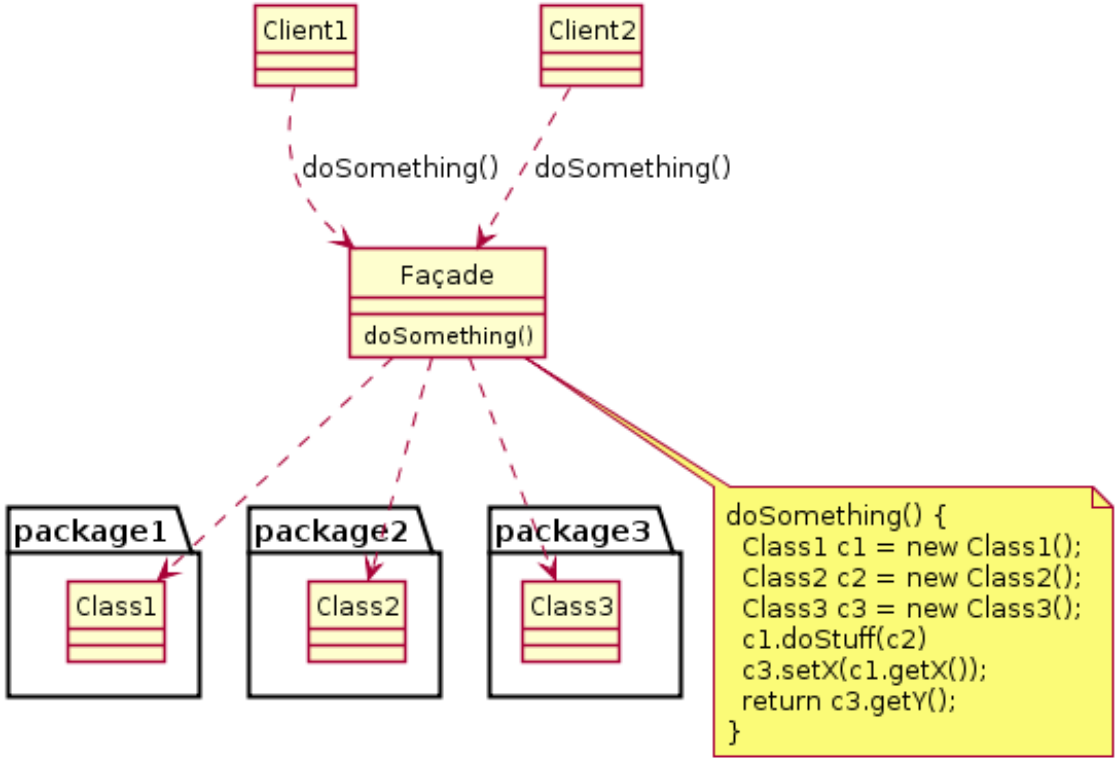
How can we make it simpler?

Managing permissions - Facade

High-level pseudocode:

```
init_permission_system()
…
check_permissions(user, resource, action)
```

Managing permissions - Facade

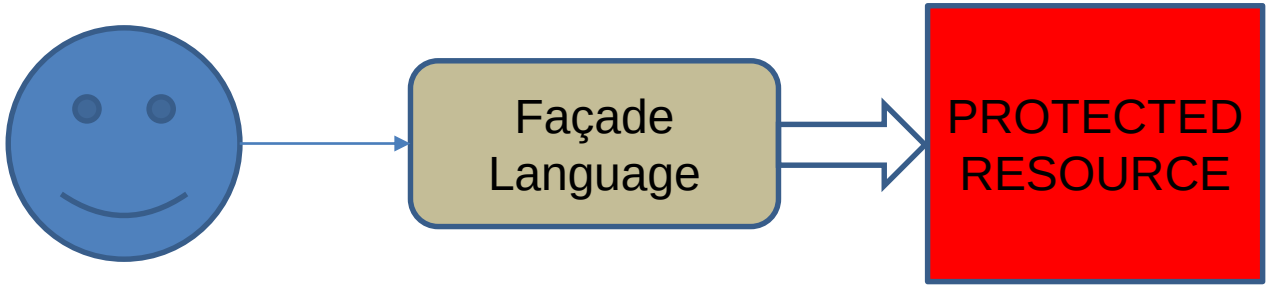


https://en.wikipedia.org/wiki/Facade_pattern#/media/File:Example_of_Facade_design_pattern_in_UML.png

Managing permissions - Facade

- Provide a unified interface to a set of interfaces in a subsystem
- Transform a complicated subsystem by using a simpler interface

Managing permissions - Facade



Security work is never ending story.

Follow design patterns to make it easier to understand and maintain.

A project's lifecycle

A project's lifecycle

http://www.ign.com/articles/2015/10/05/this-life-size-lego-batmobile-is-unbelievable

A project's lifecycle

- Handover
- Requirements
- Design
- Develop & Test
- Maintenance

The Vision

The Result

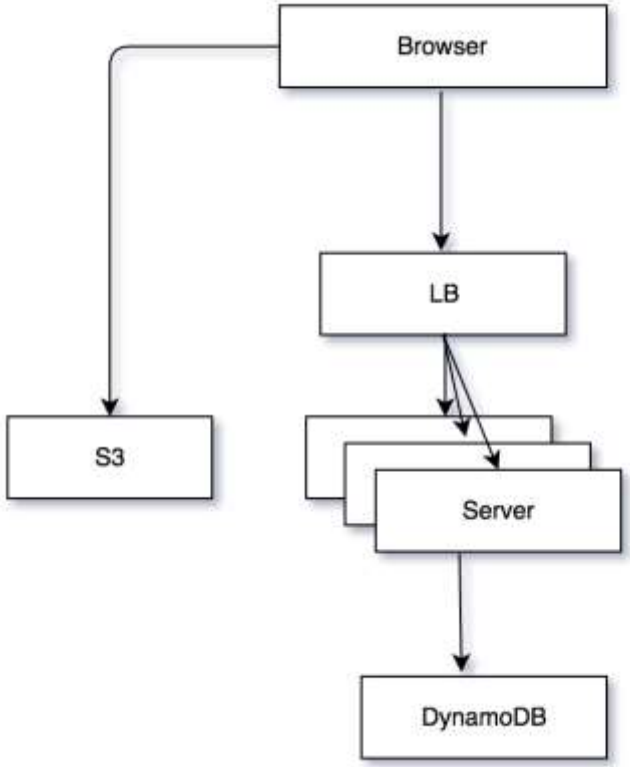
© barryoreilly.com

Handover

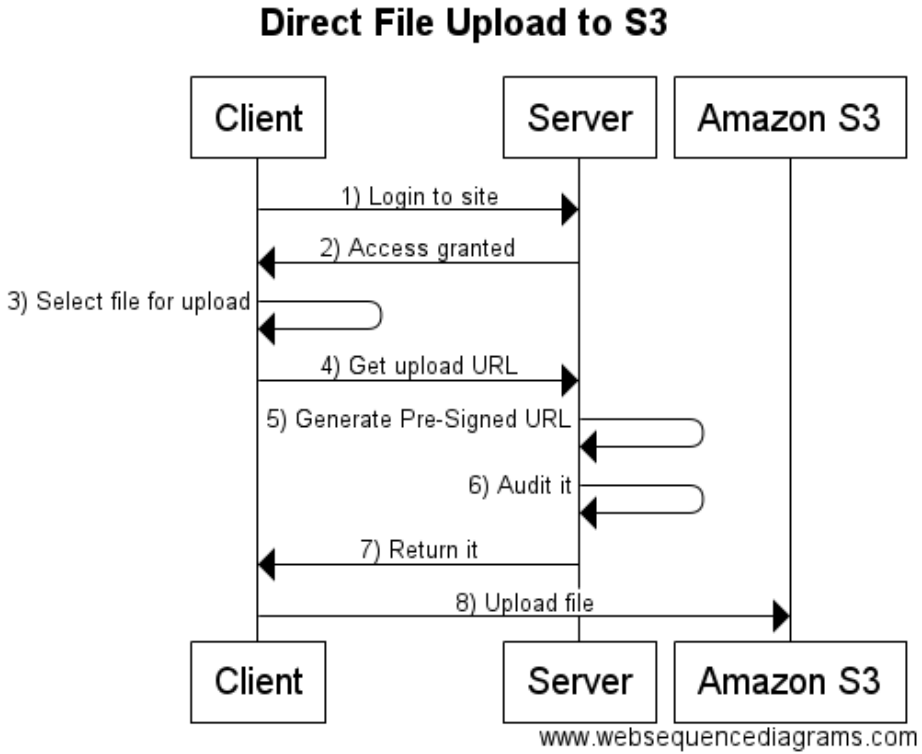
- **The project**
  - How big is the project
  - How many users does the system have
- **The communication**
  - Reverse engineering
  - You are given a presentation
  - Q/A on conference call
  - Emails

Handover

- Documentation
  - Architecture diagram
  - Database diagram
  - Sequence diagram
  - Deployment document



Handover


Direct File Upload to S3
Client
Server
Amazon S3
1) Login to site
2) Access granted
3) Select file for upload
4) Get upload URL
5) Generate Pre-Signed URL
6) Audit it
7) Return it
8) Upload file
Client
Server
Amazon S3
www.websequencediagrams.com

Requirements

- A list of business requirements
  - Forward engineering
  - Small requirements or lots of new functionalities
- More requirements from developers
  - Amazon’s working backwards philosophy
  - Developer in the driver seat
  - Start with the customer
  - User research
  - Discover new customers
- Innovation
- Review the list of requirements with peers and stakeholders

Design

- **Steps for a successful design**
  - Create an architecture overview document
  - Create a prototype
  - Create final design document
  - Validate the design with engineering leaders in the company

Design

- **Things to consider**
- System integration
  - The other systems it needs to communicate with
  - Message bus? Caching layer? Search? Big data?
  - The scale at which it will operate
  - Create a new module / service or build on existing stack?
    - More services / less services
    - SOA at Amazon
- Technologies
  - Languages and frameworks to use
  - Should it use an existing library (Open source, AWS)?

Develop, develop, develop

- Handover
- Requirements
- Design
- **Develop & Test**

- Is this all?

Maintenance

- Define environments
  - Pre-production and production stages
  - Host infrastructure
- Monitoring
  - Performance metrics
  - Health metrics
- Alarms
- Optimizations

Nagios®

AWS CloudWatch

Thank you!

Please help us improve by filling out the survey for the presentation.

Ingineria Programării

Cursul 8 – 24 Aprilie

Cuprins

- Din Cursurile trecute…
  - Design Patterns (Creational Patterns, Structural Patterns)
- Behavioral Patterns
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Template Method
  - Visitor

Din Cursurile Trecute

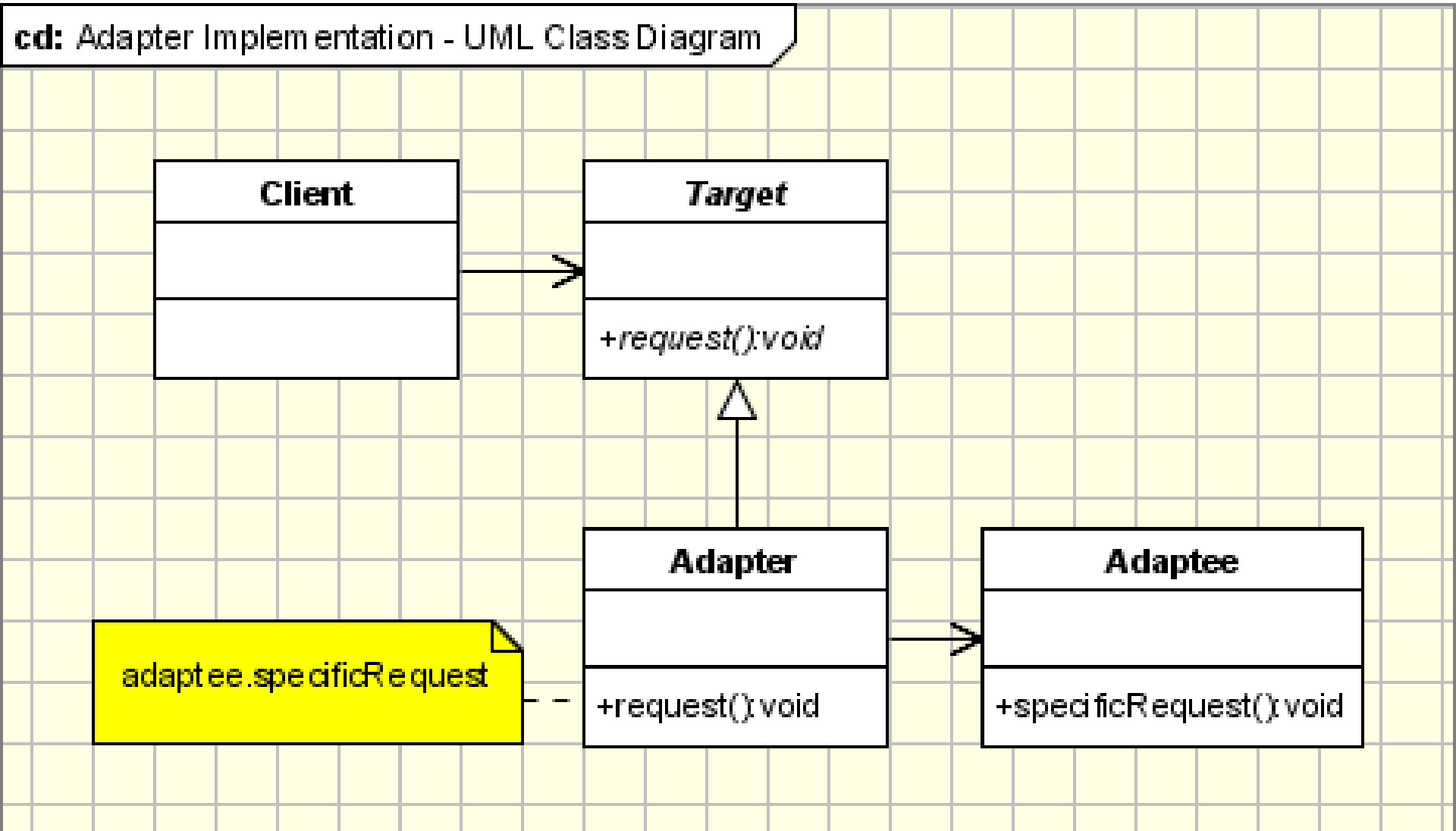
- GOF: Creational Patterns, Structural Patterns, Behavioral Patterns
- Creational Patterns
- Structural Patterns

Din cursurile trecute – CP

- **Abstract Factory** – computer components
- **Builder** – children meal
- **Factory Method** – Hello <Mr/Ms>
- **Prototype** – Cell division
- **Singleton** – server log files

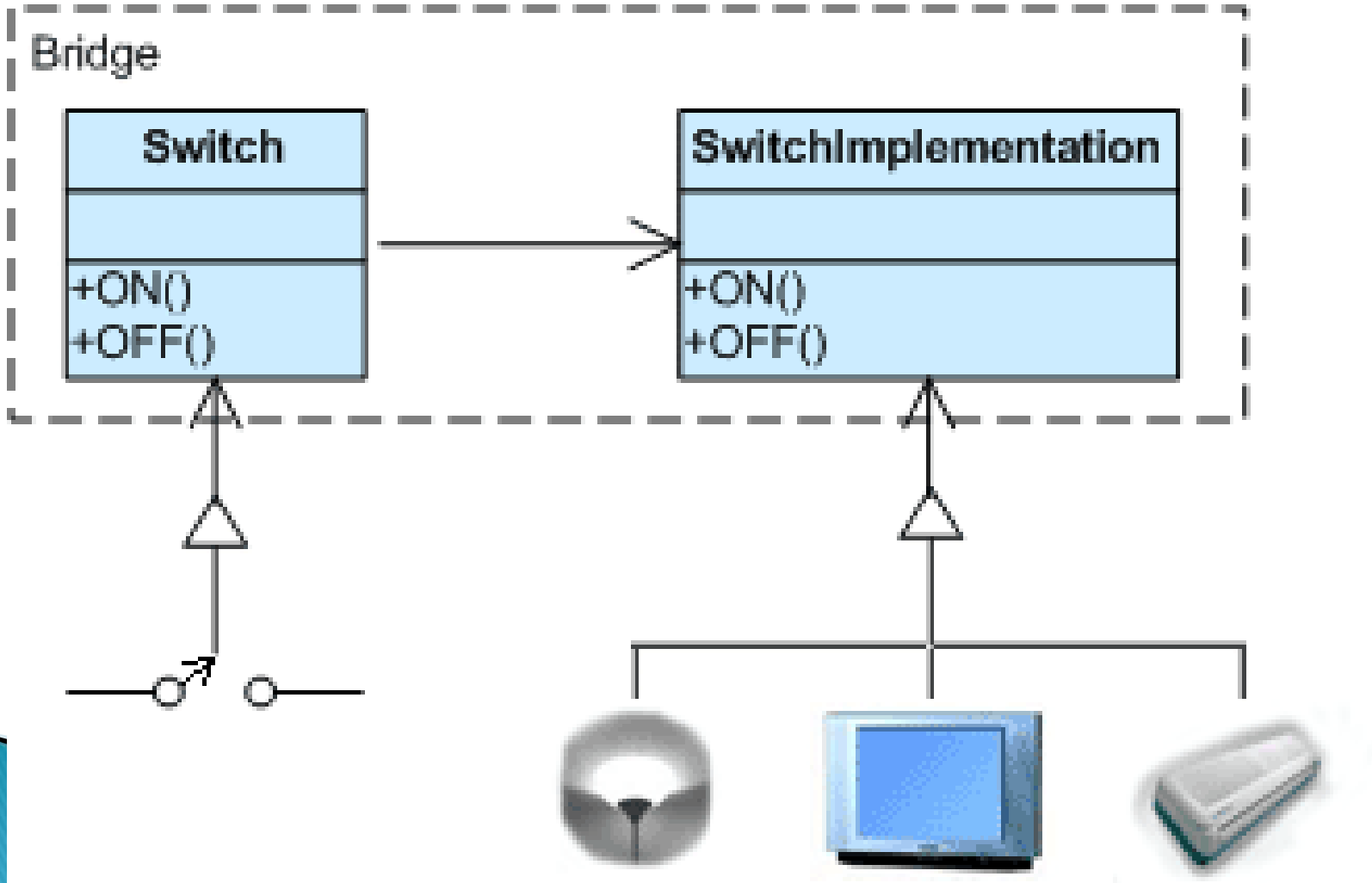
SP – Adapter

- **Adapter** – socket–plug



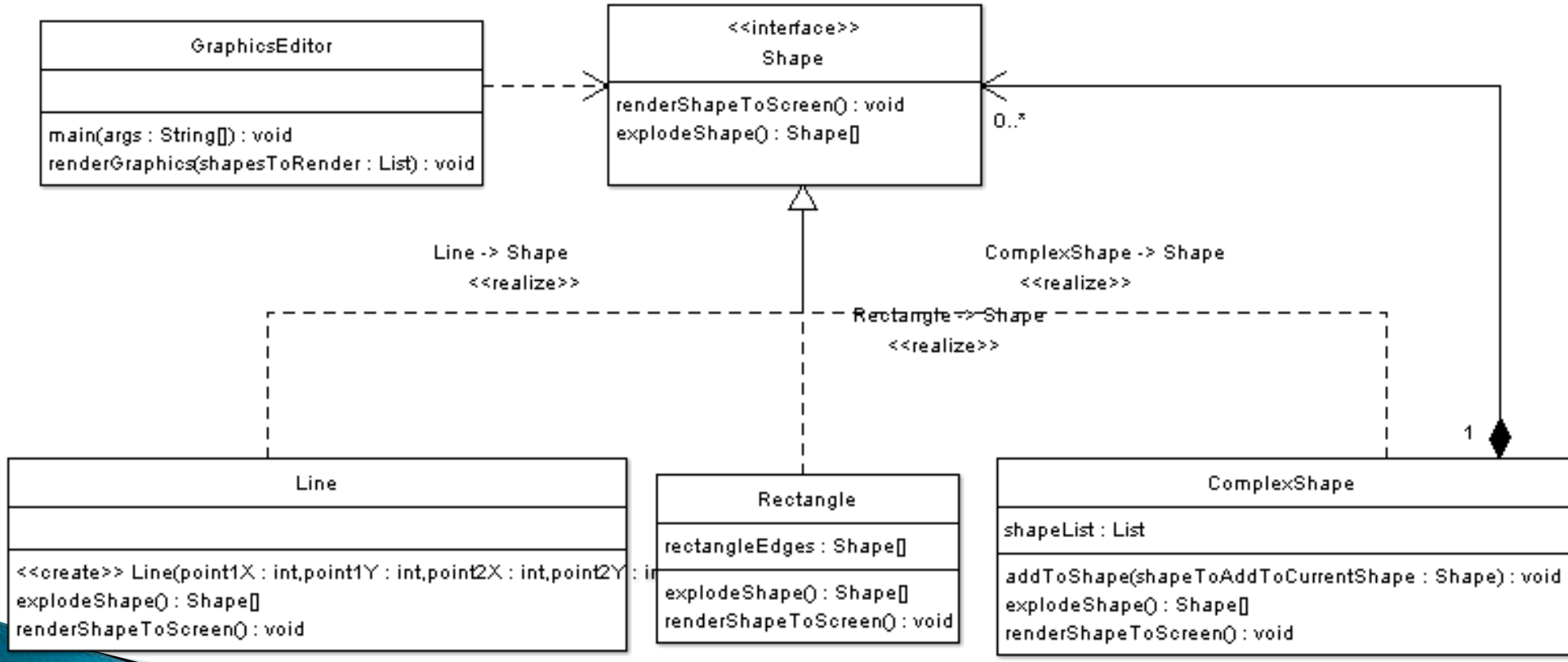
SP – Bridge

- **Bridge** – drawing API



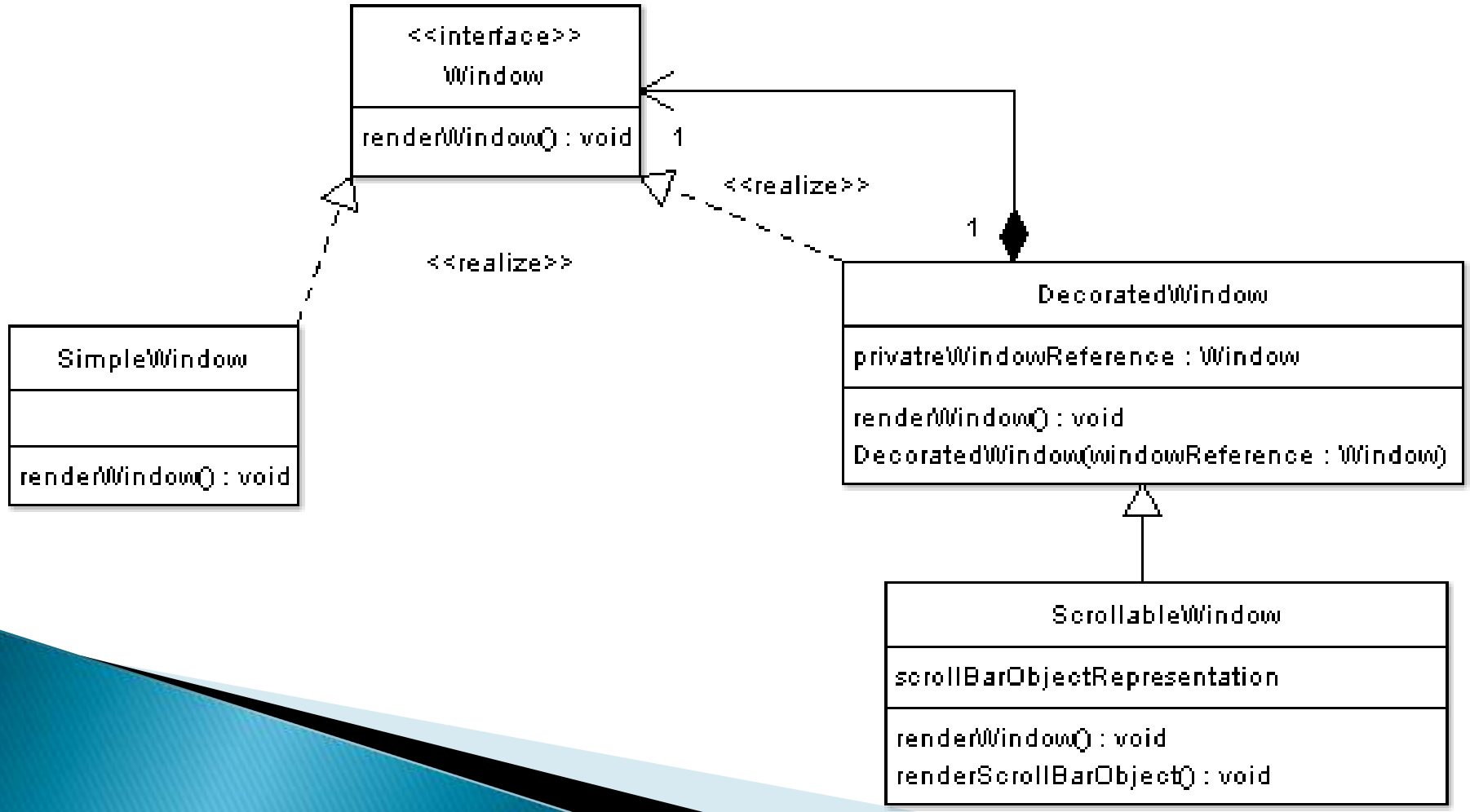
SP – Composite

- **Composite** – employee hierarchy



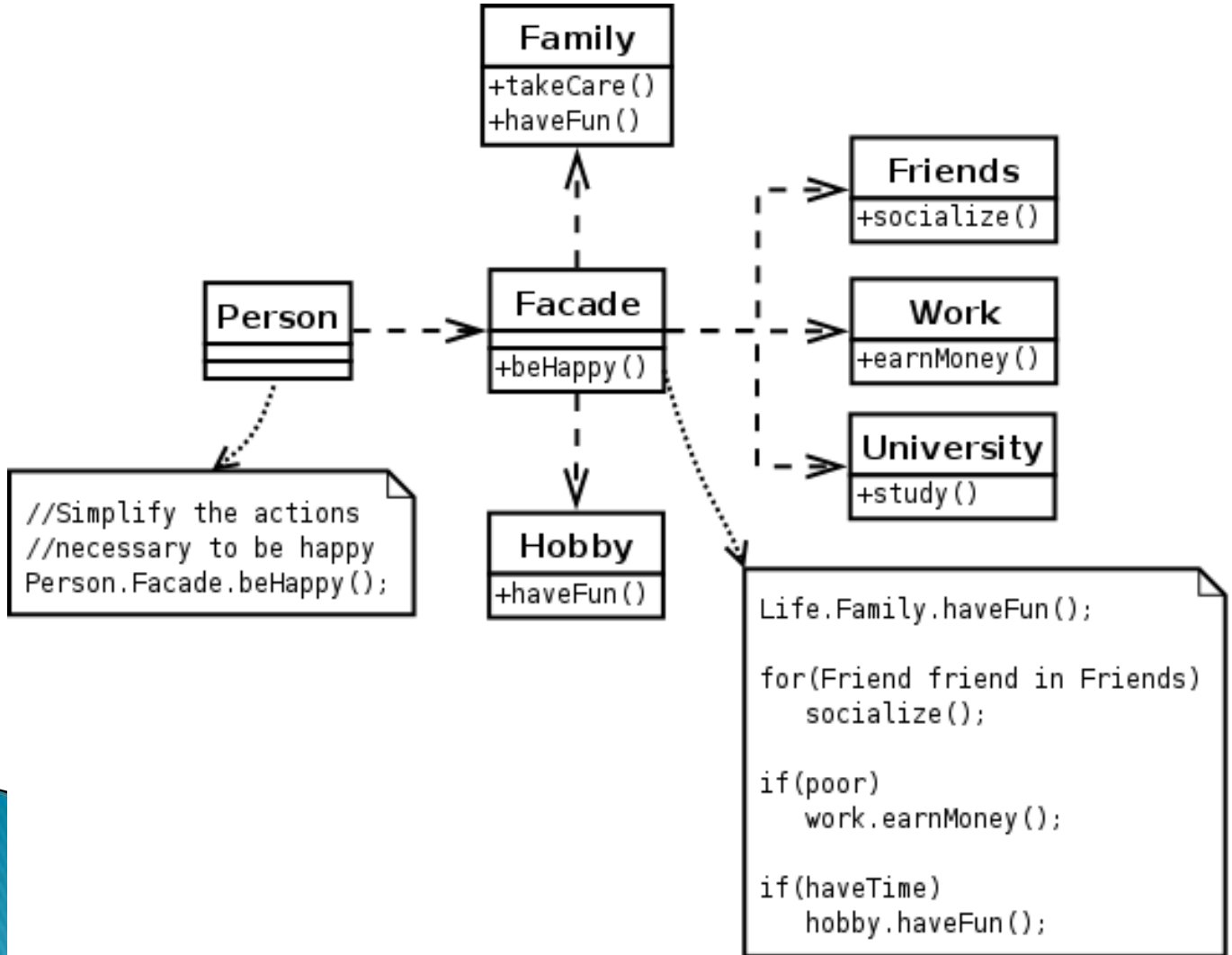
SP – Decorator

- **Decorator** – Christmas tree



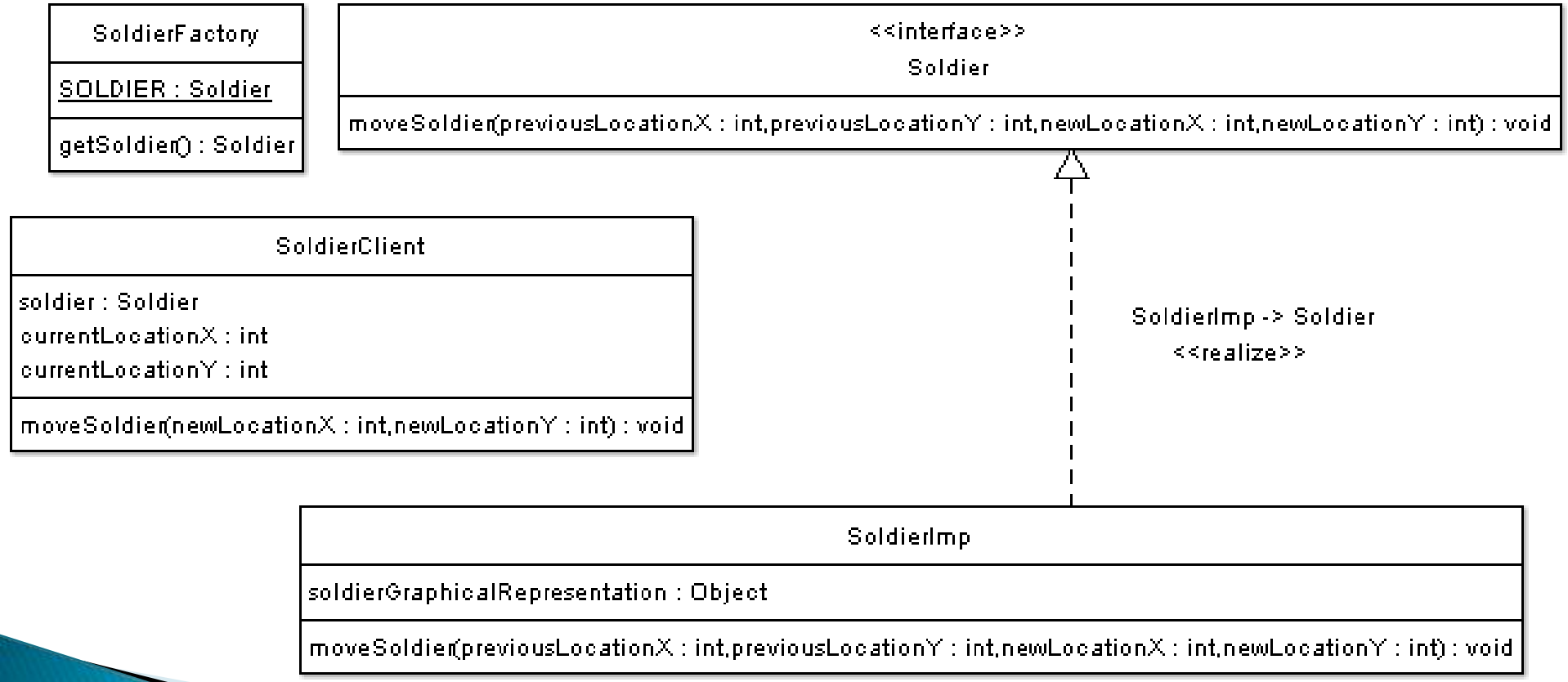
SP – Façade

- **Façade** – store keeper



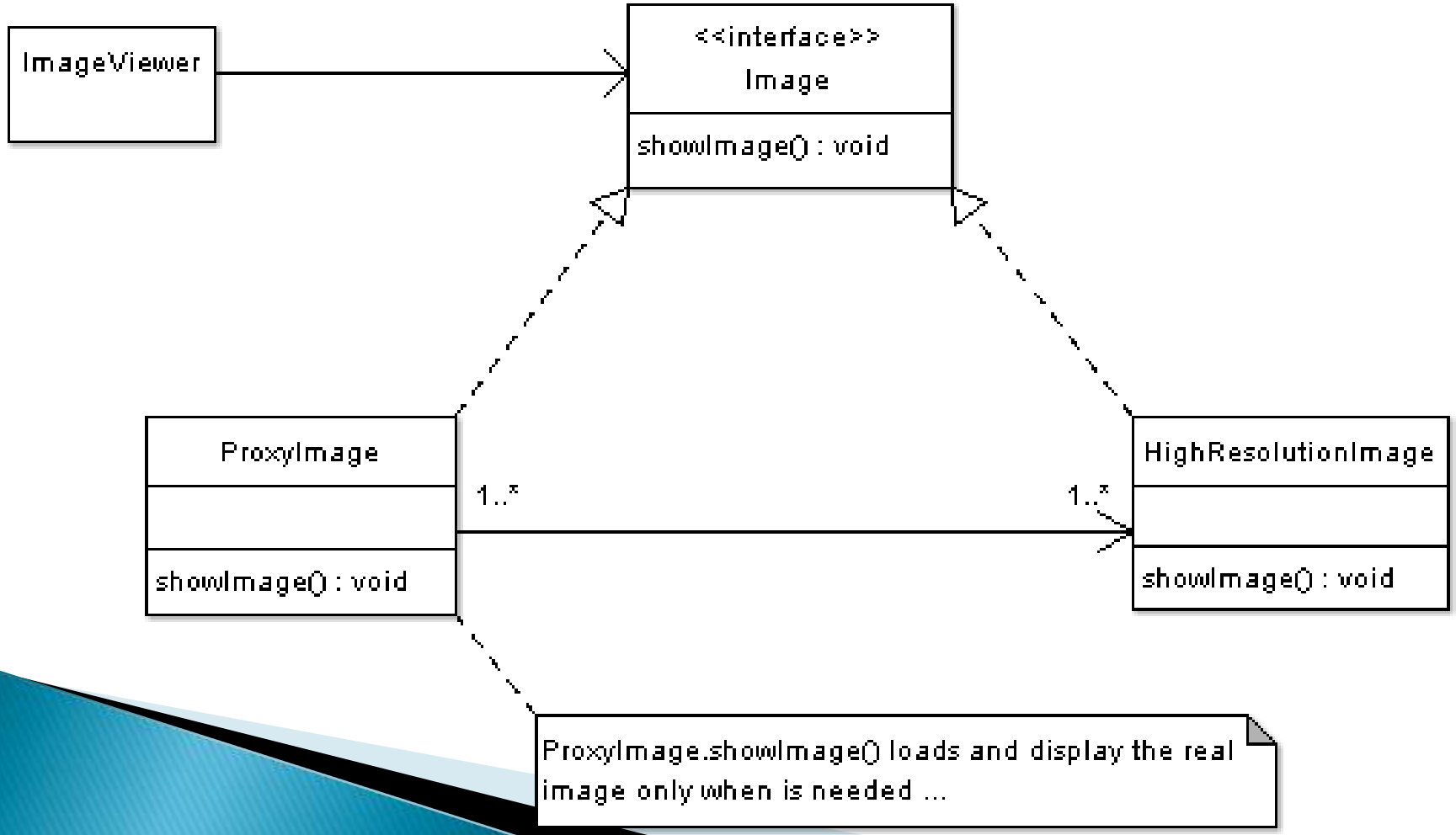
SP – Flyweight

- **Flyweight** – FontData



SP – Proxy

- **Proxy** – ATM access



Behavioral Patterns 1

- Behavioral patterns are concerned with **algorithms** and the **assignment of responsibilities between objects**
- These patterns **characterize complex control flow** that's difficult to follow at run-time
- They shift your focus away from flow of control to let you **concentrate just on the way objects are interconnected**

Behavioral Patterns 2

- **Encapsulating variation** is a theme of many behavioral patterns
- When an **aspect of a program changes frequently**, these patterns define an object that encapsulates that aspect
- Then other **parts of the program can collaborate with the object** whenever they depend on that aspect

Behavioral Patterns 3

- These patterns describe **aspects of a program that are likely to change**
- Most patterns have two kinds of objects:
  - the **new object**(s) that encapsulate the aspect,
  - and the **existing object**(s) that use the new ones
- Usually the **functionality of new objects would be an integral part of the existing objects** were it not for the pattern

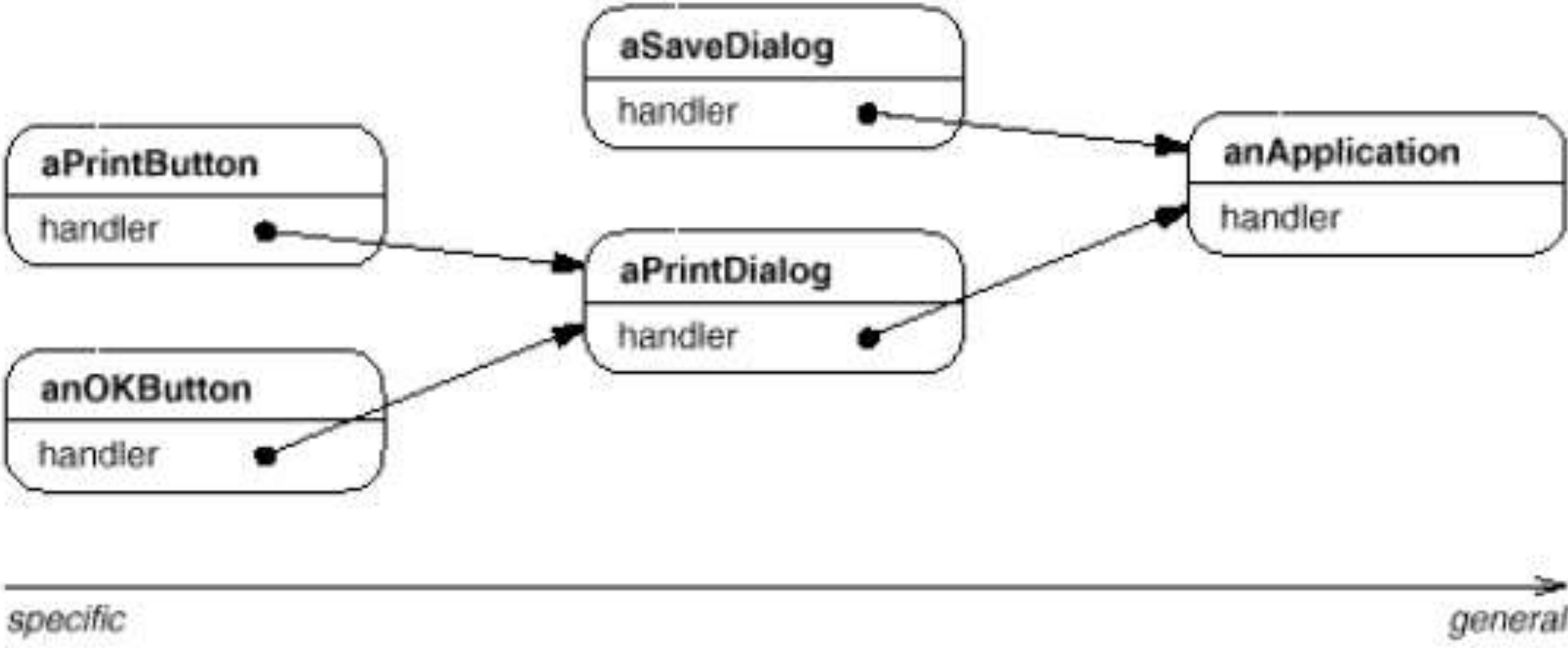
Chain of Responsibility

- **Intent** – Chain the receiving objects and **pass the request along the chain until an object handles it**

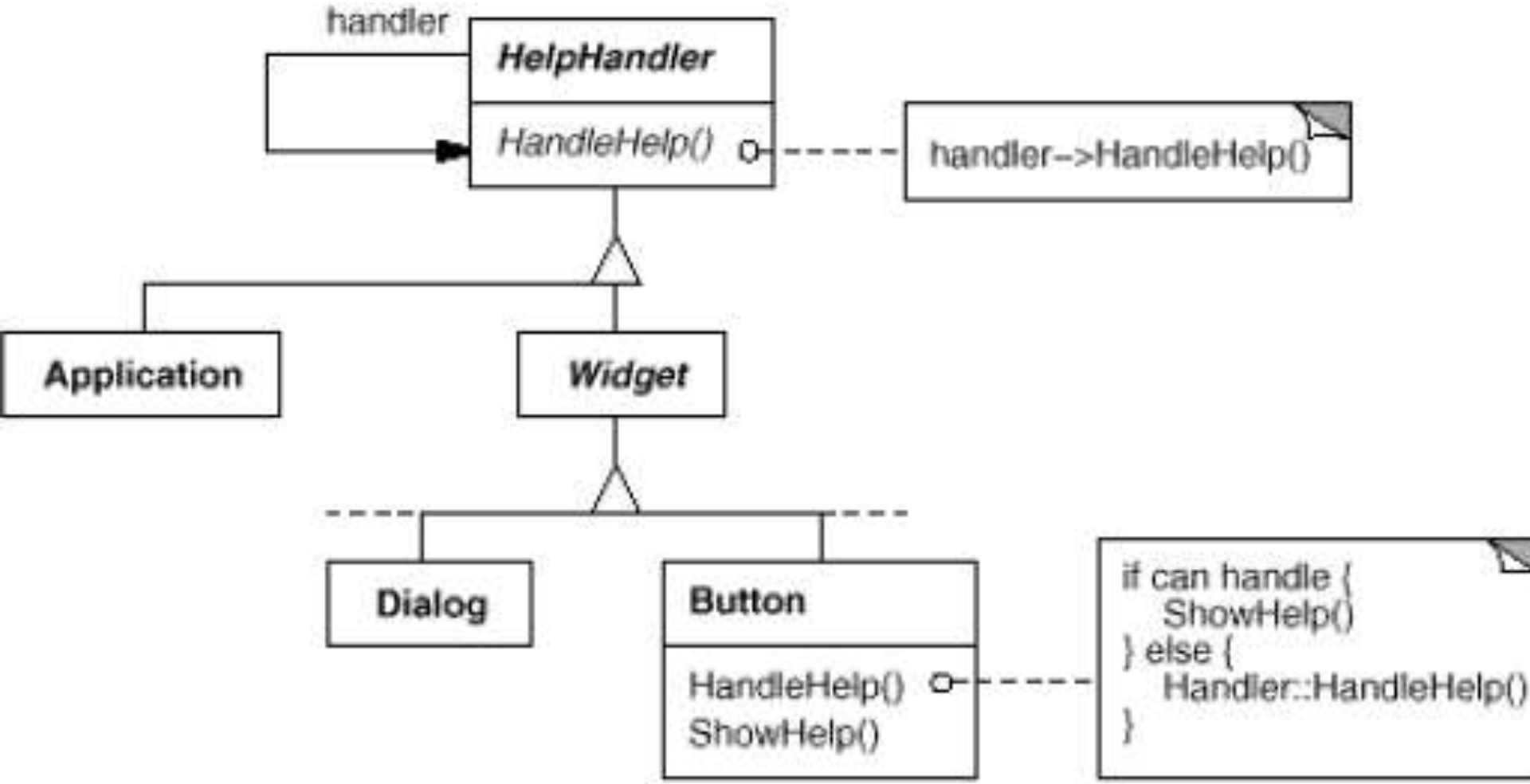
- **Motivation** – Consider a context-sensitive help facility for a graphical user interface. The help that's provided depends on the part of the interface that's selected and its context. If no specific help information exists for that part of the interface, then the help system should display a more general help message about the immediate context

Chain of Responsibility – Idea

- It's natural to organize help information **from the most specific to the most general**



Chain of Responsibility – Structure



Chain of Responsibility

- **Applicability** – Use this pattern when
  - more than one object may handle a request, and the handler isn't known *a priori*
  - you want to issue a request to one of several objects without specifying the receiver explicitly
  - the set of objects that can handle a request should be specified dynamically

Chain of Responsibility – Example

- Suppose, we have a multi level filter and gravel of different sizes and shapes. We need to filter this gravel of different sizes to approx size categories
- We will put the gravel on the multi-level filtration unit, with the filter of maximum size at the top and then the sizes descending. The gravel with the maximum sizes will stay on the first one and rest will pass, again this cycle will repeat until, the finest of the gravel is filtered and is collected in the sill below the filters
- Each of the filters will have the sizes of gravel which cannot pass through it. And hence, we will have approx similar sizes of gravels grouped

Chain of Responsibility – Java 1

```
public class Matter {
   private int size;
   private int quantity;

   public int getSize() {return size;}

   public void setSize(int size) {this.size = size;}

   public int getQuantity() {return quantity;}

   public void setQuantity(int quantity) {
        this.quantity = quantity;
   }
}
```

Chain of Responsibility – Java 2

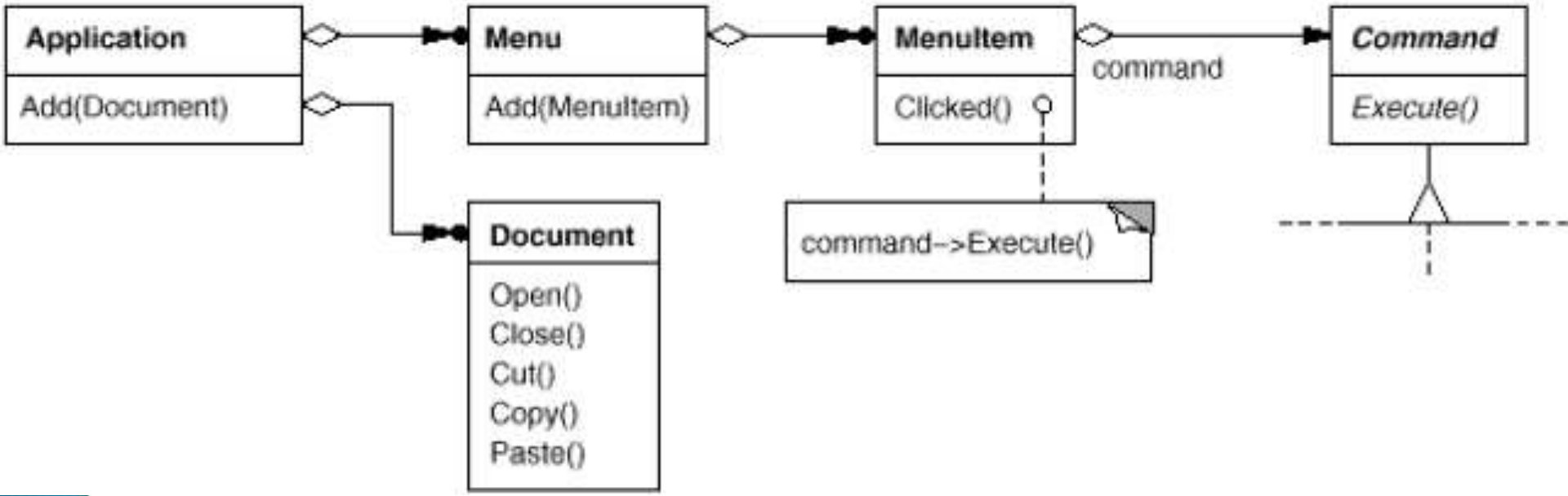
```
public class Sill {
   public void collect(Matter gravel) {}
}

public class Filter1 extends Sill {
   private int size;

   public Filter1(int size) {this.size = size;}

   public void collect(Matter gravel) {
   for(int i = 0; i < gravel.getQuantity(); i++) {
         if(gravel.getSize() < size) {
                  super.collect(gravel);}
         else {
   //collect here. that means, only matter with less size will
   pass
         }
   }}}
```

Command

- **Intent – Encapsulate a request as an object**, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations
- **Also Known As** – Action, Transaction
- **Motivation** – Sometimes **it's necessary to issue requests to objects without knowing anything about the operation being requested or the receiver of the request**
- For example, user interface toolkits include objects like buttons and menus that carry out a request in response to user input**. But the toolkit can't implement the request explicitly in the button or menu**, because only applications that use the toolkit know what should be done on which object

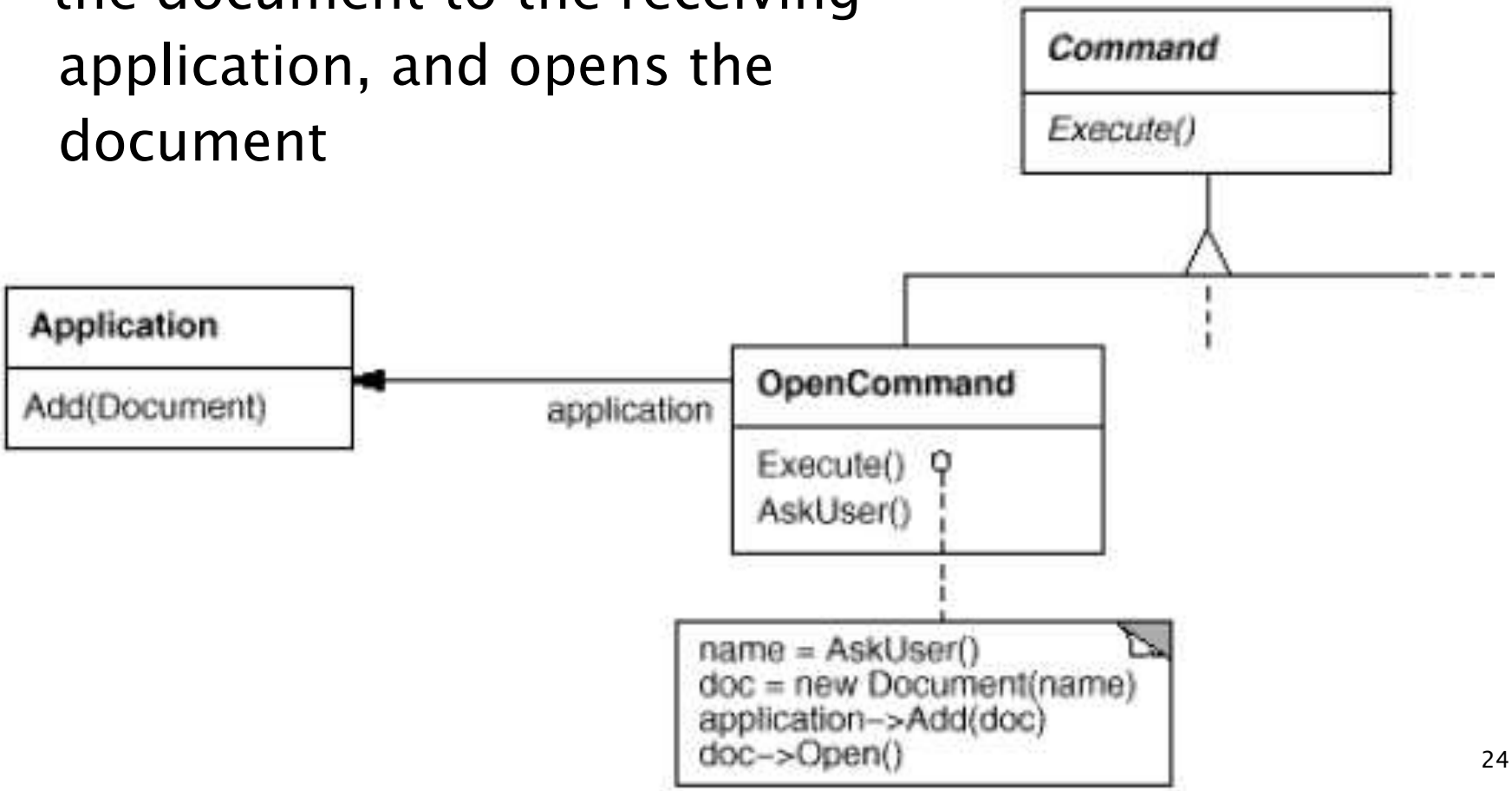
Command 2

- The key to this pattern is an abstract Command class, which declares an interface for executing operations



Command – Structure

- OpenCommand prompts the user for a document name, creates a corresponding Document object, adds the document to the receiving application, and opens the document



Command – Example

- A classic example of this pattern is a restaurant:
  - A **customer** goes to restaurant and orders the food according to his/her choice
  - The **waiter/ waitress** takes the order (command, in this case) and hands it to the *cook* in the kitchen
  - The **cook** can make several types of food and so, he/she prepares the ordered item and hands it over to the *waiter/waitress* who in turn serves to the *customer*

Command – Java 1

```
public class Order {
  private String command;
  public Order(String command) {
      this.command = command;
  } }
public class Waiter {
  public Food takeOrder(Customer cust, Order
  order) {
      Cook cook = new Cook();
      Food food = cook.prepareOrder(order, this);
      return food;
  } }
```

Command – Java 2

```
public class Cook {
  public Food prepareOrder(Order order, Waiter
  waiter) {
      Food food = getCookedFood(order);
      return food;
  }
  public Food getCookedFood(Order order) {
      Food food = new Food(order);
      return food;
  }
}
```

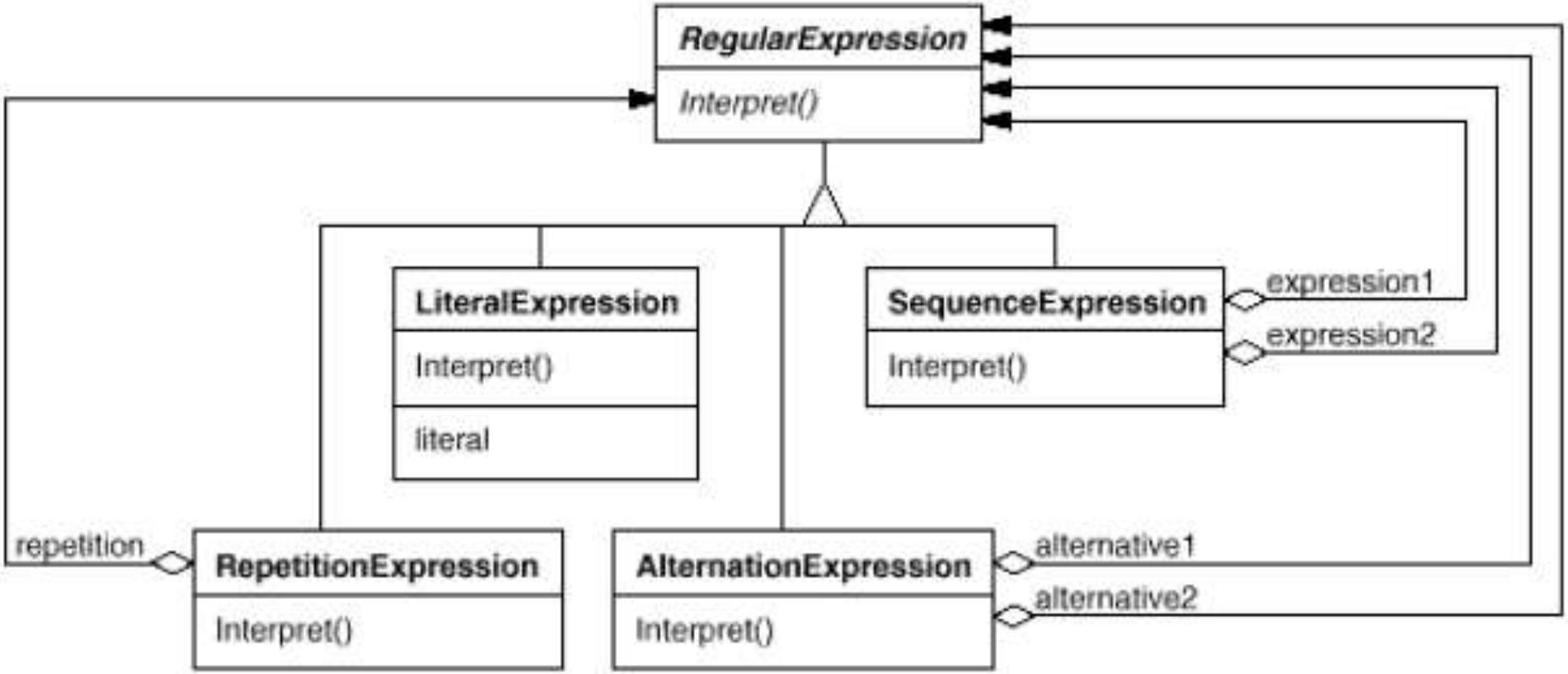
Interpreter

- **Intent – Given a language, define a representation for its grammar along with an interpreter**
- **Motivation** – If a particular kind of problem occurs **often enough**, then it might be worthwhile to **express instances of the problem as sentences in a simple language**. Then you can build an **interpreter** that solves the problem by interpreting these sentences.
- For example, *searching for strings that match a pattern is a common problem*. Regular expressions are a standard language for specifying patterns of strings

Interpreter – Grammar

- Suppose the following grammar defines the regular expressions:
  - expression ::= literal | alternation | sequence | repetition | '(' expression ')'
  - alternation ::= expression '|' expression
  - sequence ::= expression '&' expression
  - repetition ::= expression '*'
  - literal ::= 'a' | 'b' | 'c' | ... { 'a' | 'b' | 'c' | ... }*

Interpreter – Regular expressions



Interpreter – Example

- The “musical notes” is an “Interpreted Language”. The musicians read the notes, interpret them according to “Sa, Re, Ga, Ma…” or “Do, Re, Mi… ” etc. and play the instruments, what we get in output is musical sound waves. Think of a program which can take the Sa, Re, Ga, Ma etc. and produce the sounds for the frequencies.
- For Sa, the frequency is 256 Hz, similarly, for Re, it is 288Hz and for Ga, it is 320 Hz etc…
- We can have it at one of the two places, one is a constants file, “token=value” and the other one being in a properties file

Interpreter – Java 1

- *MusicalNotes.properties*
  Sa=256
  Re=288
  Ga=320

```
public class NotesInterpreter {
   private Note note;
   public void getNoteFromKeys(Note note) {
         Frequency freq = getFrequency(note);
         sendNote(freq);
   }

   private Frequency getFrequency(Note note) {
   // Get the frequency from properties file using ResourceBundle
   // and return it.
         return freq;
   }

   private void sendNote(Frequency freq) {
         NotesProducer producer = new NotesProducer();
         producer.playSound(freq);
   }
}
```

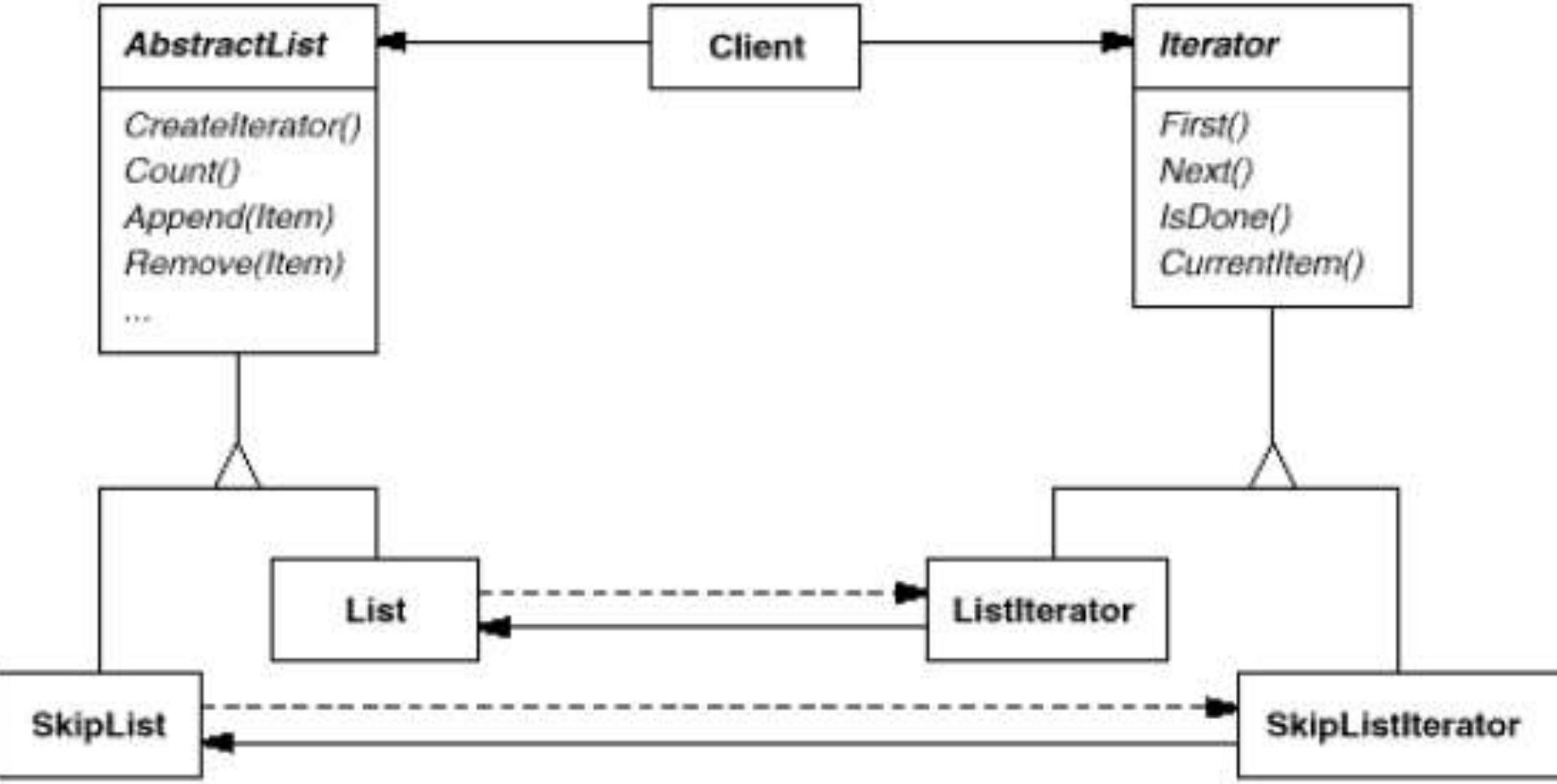
Interpreter – Java 2

```
public class NotesProducer {

  private Frequency freq;

  public NotesProducer() {
      this.freq = freq;
  }

  public void playSound(Frequency freq) {
  }
}
```

Iterator

- **Intent** – Provide a way to access the elements of an aggregate object sequentially
- **Also Known As** – Cursor
- **Motivation** – *An aggregate object such as a list should give you a way to access its elements without exposing its internal structure*. Moreover, you might want to traverse the list in different ways, depending on what you want to accomplish

Iterator – Structure



Iterator – Applicability

- to access an aggregate object's contents without exposing its internal representation
- to support multiple traversals of aggregate objects
- to provide a uniform interface for traversing different aggregate structures (that is, to support polymorphic iteration)

Iterator – Example

- For example, remote control of TV. Any remote control we use, either at home/hotel or at a friend's place, we just pick up the TV remote control and start pressing Up and Down or Forward and Back keys to iterate through the channels

Iterator – Java 1

```
public interface Iterator {
   public Channel nextChannel(int currentChannel);
   public Channel prevChannel(int currentChannel);
}

public ChannelSurfer implements Iterator {
   public Channel nextChannel (int currentChannel) {
         Channel channel = new Channel(currentChannel+1);
         return channel;
   }
   public Channel prevChannel (int currentChannel) {
         Channel channel = new Channel(currentChannel-1);
         return channel;
   }
}
```

Iterator – Java 2

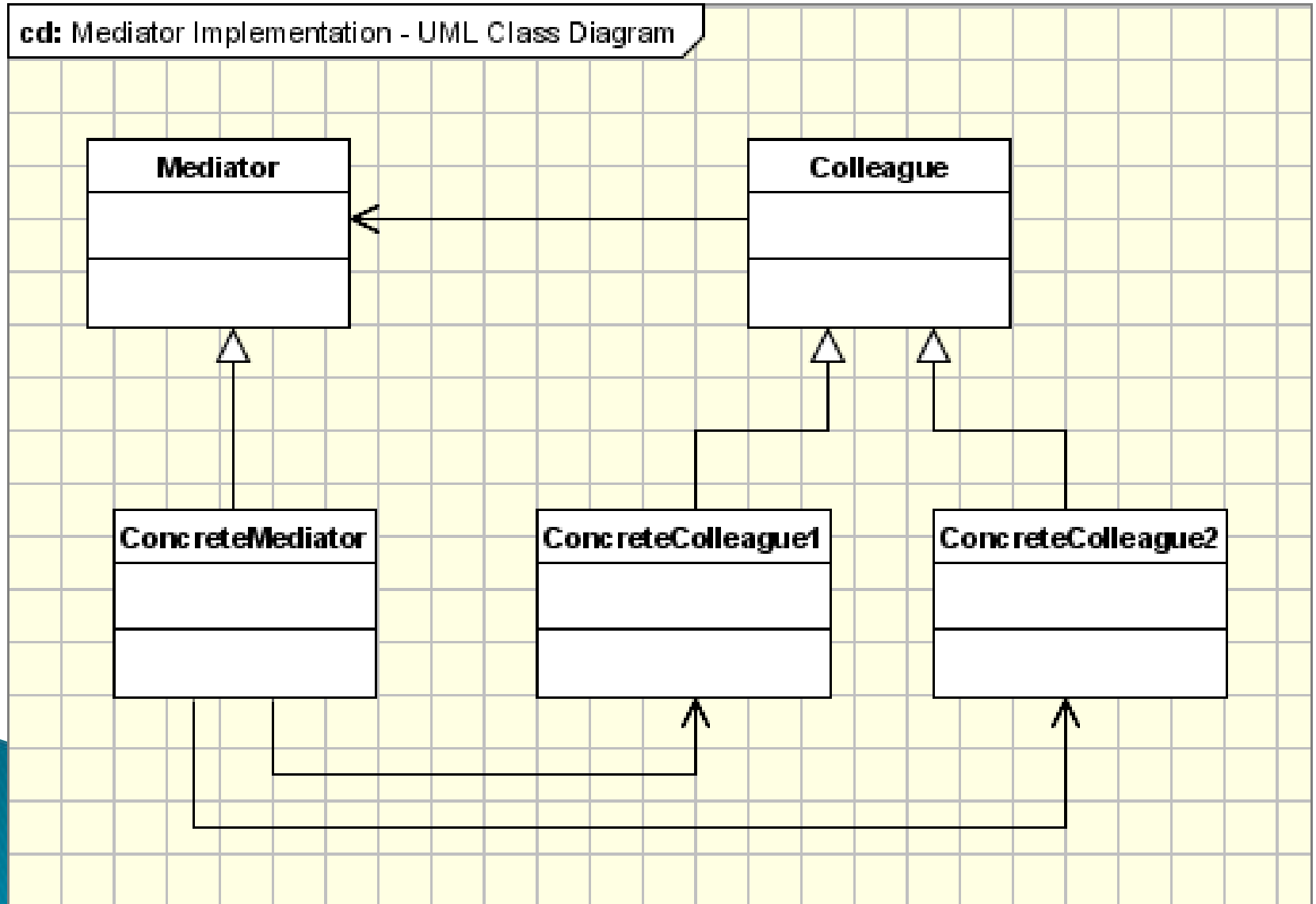
```
public class RemoteControl {
  private ChannelSurfer surfer;
  private Settings settings;

  public RemoteControl() {
      surfer = new ChannelSurfer();
      settings = new Settings();
  }
  public getProgram(ChannelSurfer surfer) {
      return new Program(surfer.nextChannel());
  }
}
```

Mediator

- **Intent** – Define **an object that encapsulates how a set of objects interact**

- **Motivation – Object-oriented design encourages the distribution of behavior among objects.** Such distribution can result in an object structure with many connections between objects; in the worst case, every object ends up knowing about every other

Mediator – Structure



Mediator – Applicability

- According to (Gamma et al), the Mediator pattern should be used when:
  - a set of objects communicate in well-defined but complex ways. The resulting interdependencies are unstructured and difficult to understand.
  - reusing an object is difficult because it refers to and communicates with many other objects.
  - a behavior that's distributed between several classes should be customizable without a lot of subclassing.

Mediator – Examples

- A very common example can be airplanes interacting with the control tower and not among themselves

- Another popular example is Stock exchange

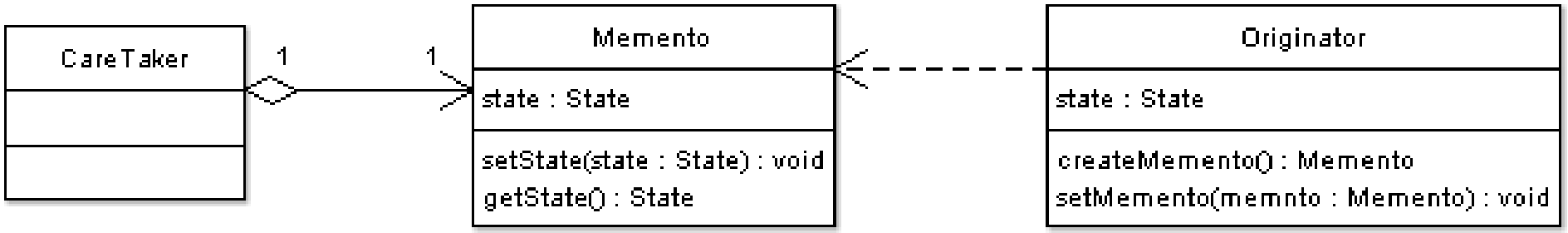
- The chat application is another example of the mediator pattern

- Other examples?

Memento

- **Intent** – Without violating encapsulation, capture and externalize an object's internal state so that the **object can be restored to this state later**
- **Also Known As** – Token
- **Motivation** – Sometimes it's necessary to record the internal state of an object. This is required when implementing checkpoints and undo mechanisms that let users back out of tentative operations or recover from errors. **You must save state information somewhere so that you can restore objects to their previous states**

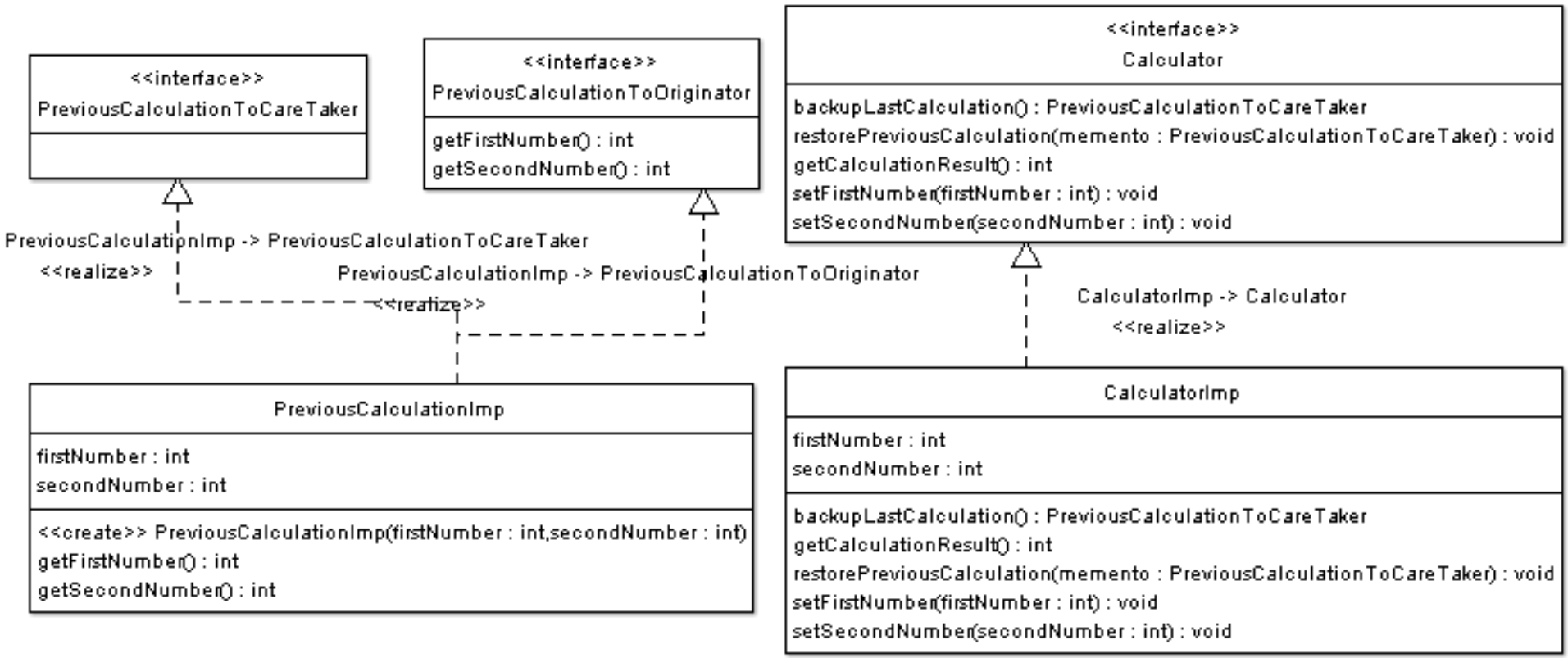
Memento – Structure



- **Memento**
  - Stores internal state of the Originator object
  - Allows the originator to restore previous state
- **Originator**
  - Creates a memento object capturing it's internal state
  - Use the memento object to restore its previous state.
- **Caretaker**
  - Responsible for keeping the memento.
  - The memento is opaque to the caretaker, and the caretaker must not operate on it.

Memento – Example

- **Simple Calculator with Undo Operation**



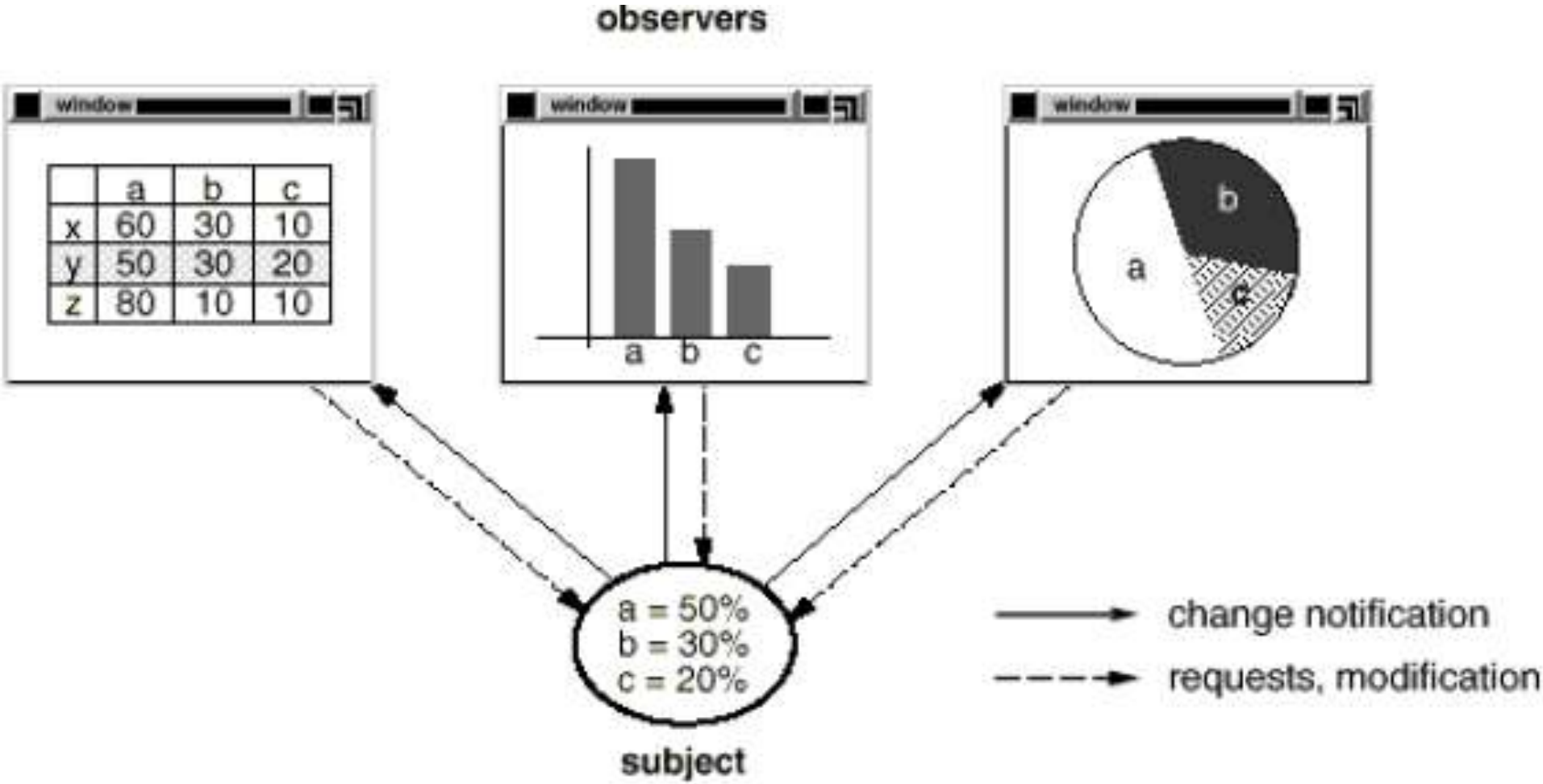
Memento – Database Transactions

- Transactions are operations on the database that occur in an atomic, consistent, durable, and isolated fashion
- If all operations succeed, the transaction would commit and would be final
- And if any operation fails, then the transaction would fail and all operations would rollback and leave the database as if nothing has happened
- This mechanism of rolling back uses the memento design pattern

Observer

- **Intent** – Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically
- **Also Known As** – Dependents, Publish-Subscribe
- **Motivation** – A common side-effect of partitioning a system into a collection of cooperating classes is the need to maintain consistency between related objects

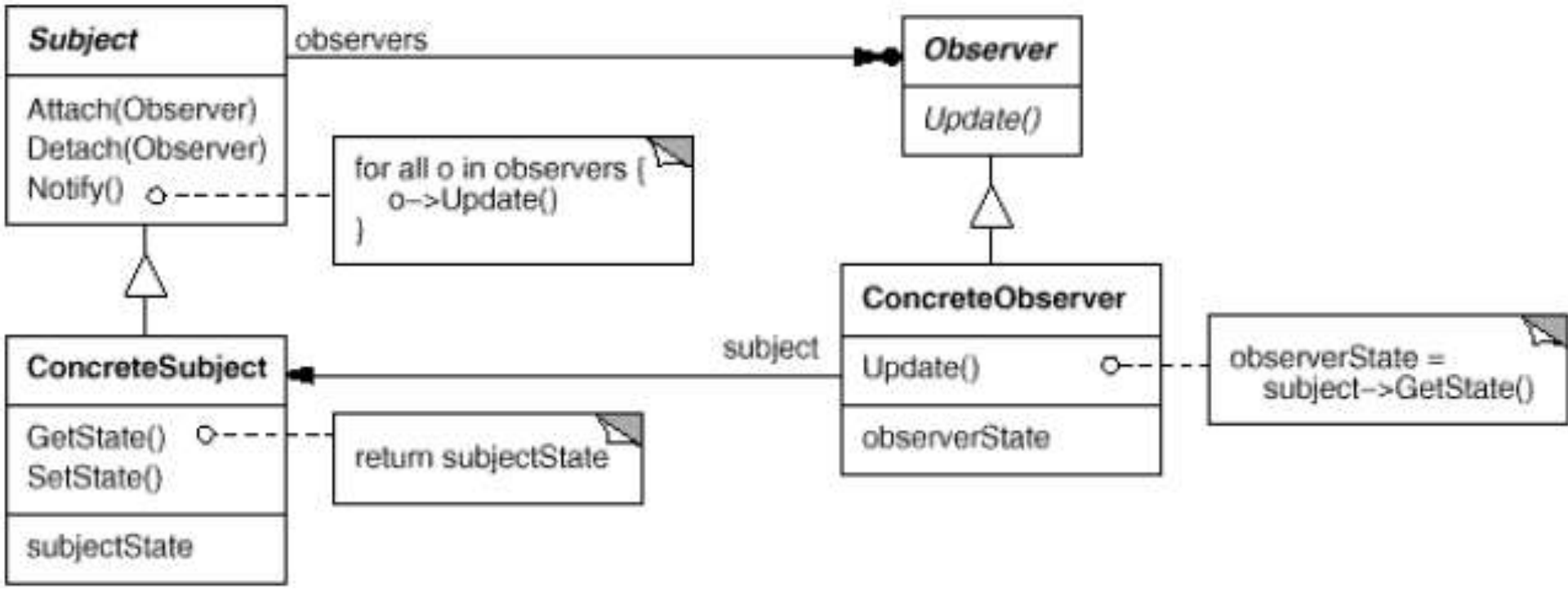
Observer – Example



Observer – Applicability

- When an abstraction has two aspects, one dependent on the other
- When a change to one object requires changing others, and you don't know how many objects need to be changed
- When an object should be able to notify other objects without making assumptions about who these objects are

Observer – Structure



Observer – Example

- Below is an example that takes keyboard input and treats each input line as an event. The example is built upon the library classes **java.util.Observer** and **java.util.Observable**
- When a string is supplied from System.in, the method notifyObservers is then called, in order to notify all observers of the event's occurrence, in the form of an invocation of their 'update' methods – in our example, ResponseHandler.update(...).
- *The Java Swing library makes extensive use of the observer pattern for event management*

Observer – Java 1

```
public class EventSource extends Observable
  implements Runnable {
  public void run() {
   try {
    final InputStreamReader isr = new
  InputStreamReader( System.in );
   final BufferedReader br = new BufferedReader( isr );
    while( true ) {
       final String response = br.readLine();
       setChanged();
       notifyObservers( response ); }
  }
  catch (IOException e) { e.printStackTrace(); } }
}
```

Observer – Java 2

```
public class ResponseHandler implements Observer {
  private String resp;

  public void update (Observable obj, Object arg) {
    if (arg instanceof String) {
       resp = (String) arg;
       System.out.println("\nReceived Response: "+ resp );
    }
  }
}
```

Observer – Java 3

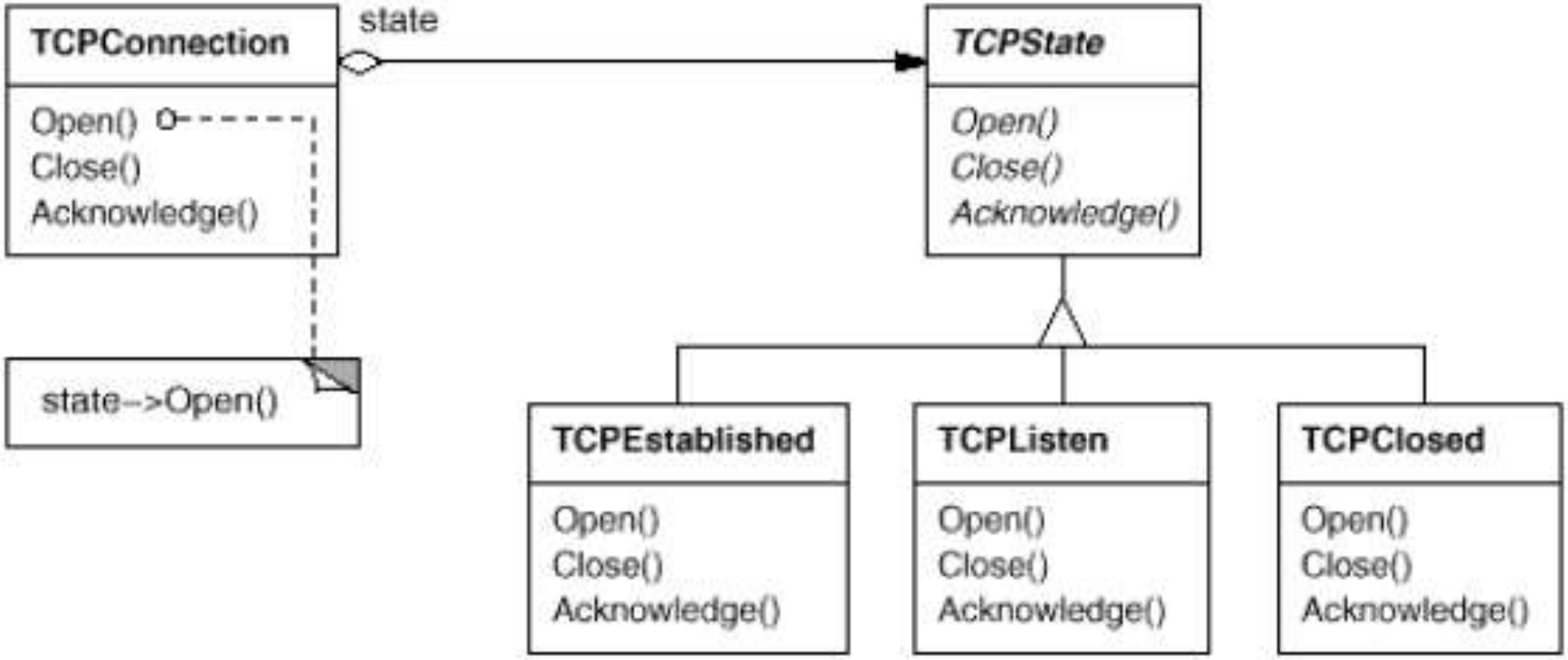
```
public class MyApp {
  public static void main(String args[]) {
    System.out.println("Enter Text >");
    // create an event source - reads from stdin
      final EventSource evSrc = new EventSource();
    // create an observer
      final ResponseHandler respHandler = new
      ResponseHandler();
    // subscribe the observer to the event source
      evSrc.addObserver( respHandler );
    // starts the event thread
      Thread thread = new Thread(evSrc);
      thread.start();
    }
}
```

State

- **Intent** – Allow an object to alter its behavior when its internal state changes
- **Also Known As** – Objects for States
- **Motivation** – Consider a class TCPConnection that represents a network connection. A TCPConnection object can be in one of several different states: Established, Listening, Closed. When a TCPConnection object receives requests from other objects, it responds differently depending on its current state

State – Idea

- The key idea in this pattern is to introduce an abstract class called TCPState to represent the states of the network connection.



State – Applicability

- Use the State pattern in either of the following cases:
  - An object's behavior depends on its state
  - Operations have large, multipart conditional statements that depend on the object's state

Strategy

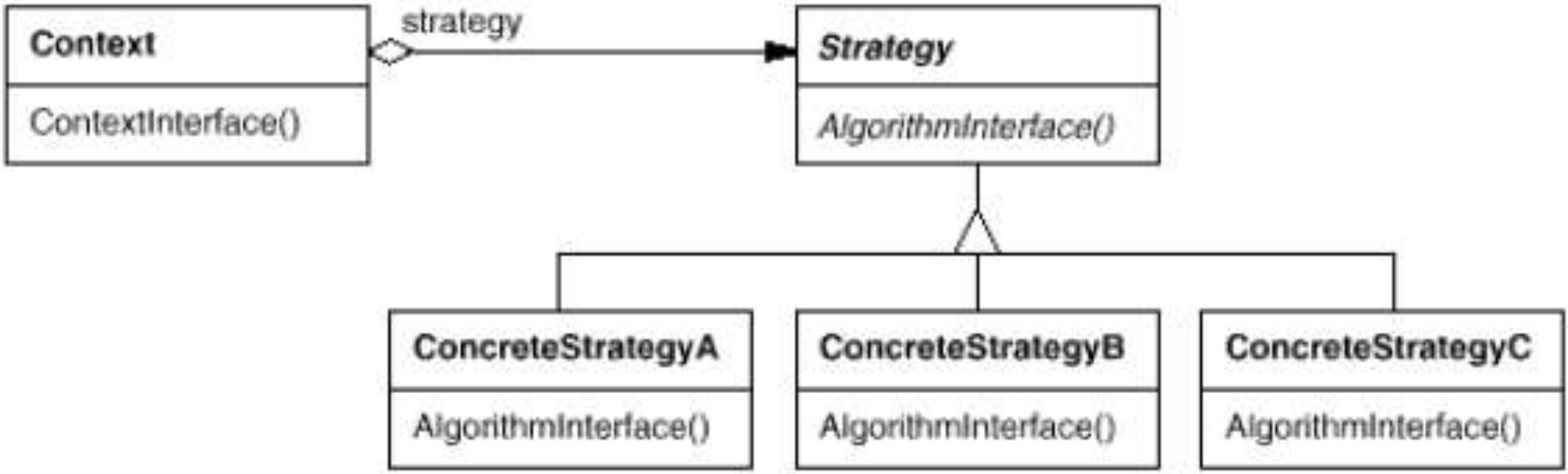
- **Intent** – Define a family of algorithms, encapsulate each one, and make them inter changeable

- **Also Known As** – Policy

- **Motivation** – Many algorithms exist for breaking a stream of text into lines. Hard-wiring all such algorithms into the classes that require them isn't desirable for several reasons

Strategy – Structure

- With Strategy pattern, we can define classes that encapsulate different line breaking algorithms



Strategy – Example

- In the strategy pattern algorithms can be selected at runtime.
- A standard calculator that implements basic operations: +, –, *

Strategy – Java 1

```
interface Strategy {
   int execute(int a, int b);
}
class ConcreteStrategyAdd implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyA's execute()");
         return (a + b);
}}
class ConcreteStrategySub implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyB's execute()");
         return (a - b);
}}
class ConcreteStrategyMul implements Strategy {
   public int execute(int a, int b) {
         System.out.println("Called ConcreteStrategyC's execute()");
         return a * b;
} }
```

Strategy – Java 2

```
class Context {
   Strategy strategy;
   public Context(Strategy strategy) {
        this.strategy = strategy;
   }
   public int execute(int a, int b) {
        return this.strategy.execute(a, b);
   }
}
```

Strategy – Java 3

```
class StrategyExample {

   public static void main(String[] args) {
    Context context;
    context = new Context(new ConcreteStrategyAdd());
    int resultA = context.execute(3,4);
    context = new Context(new ConcreteStrategySub());
    int resultB = context.execute(3,4);
    context = new Context(new ConcreteStrategyMul());
    int resultC = context.execute(3,4);
   }
}
```

Template Method

- **Intent – Define the skeleton of an algorithm** in an operation, deferring some steps to subclasses
- **Motivation** – Consider an application framework that provides Application and Document classes.
- The Application class is responsible for opening existing documents stored in an external format, such as a file. A Document object represents the information in a document once it's read from the file

Template Method – Example

- The template pattern is often referred to as the **Hollywood Principle**: "*Don't call us, we'll call you*." Using this principle, the template method in a parent class controls the overall process by calling subclass methods as required

- This is shown in several games in which players play against the others, but only one is playing at a given time

Template Method – Java 1

```
abstract class Game {
   protected int playersCount;
   abstract void initializeGame();
   abstract void makePlay(int player);
   abstract boolean endOfGame();
   abstract void printWinner();

   final void playOneGame(int playersCount) {
        this.playersCount = playersCount;
        initializeGame(); int j = 0;
        while (!endOfGame()) {
            makePlay(j); j = (j + 1) % playersCount; }
         printWinner();
   }
}
```

Template Method – Java 2

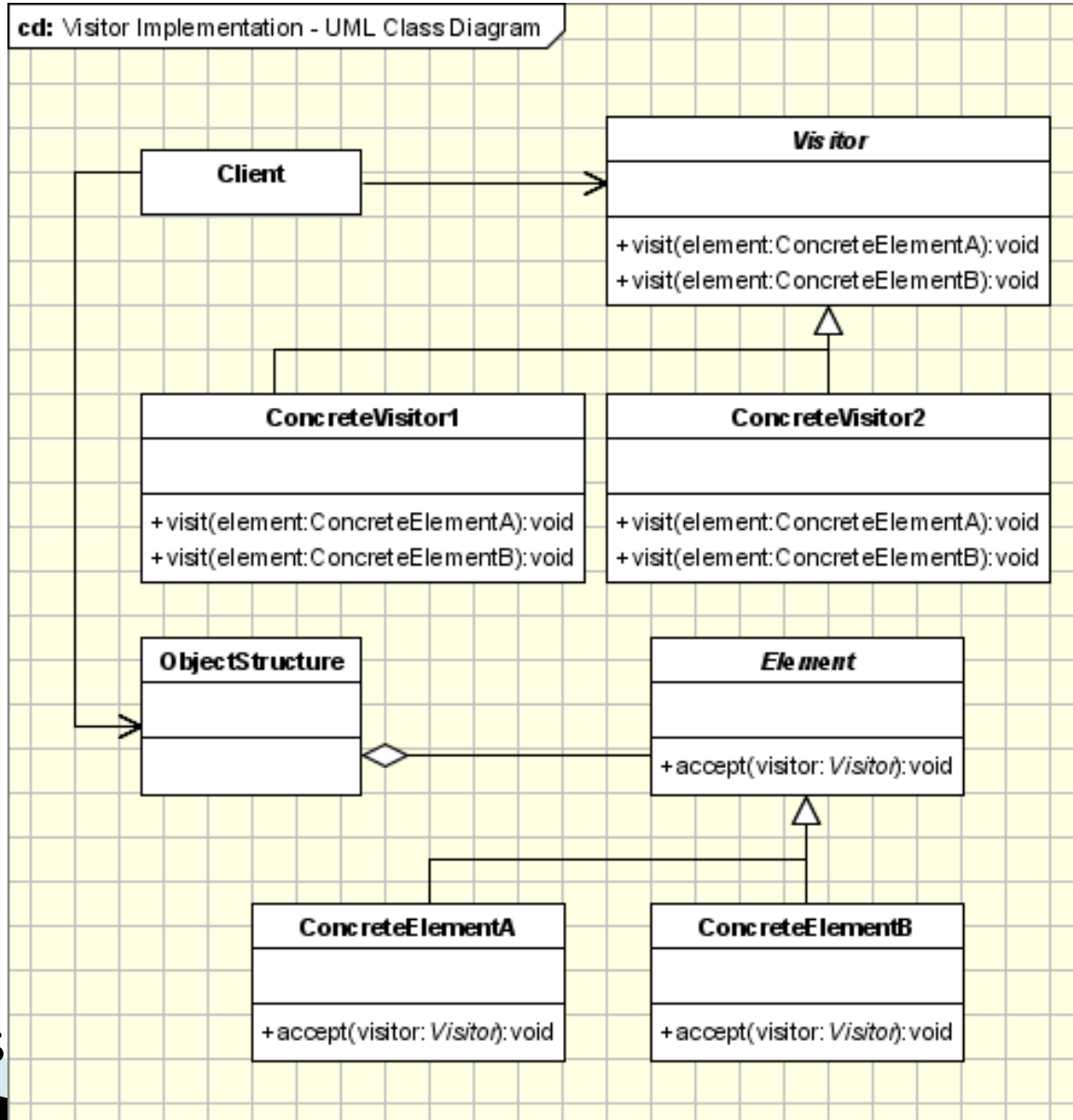
```
class Monopoly extends Game {
   // Implementation of necessary concrete methods

   void initializeGame() { // ... }
   void makePlay(int player) { // ... }
   boolean endOfGame() { // ... }
   void printWinner() { // ... }

   // Specific declarations for the Monopoly game.
}

class Chess extends Game {
...}
```

Visitor

- **Intent** – Represent an operation to be performed on the elements of an object structure. **Visitor lets you define a new operation without changing the classes** of the elements on which it operates.
- **Motivation** – Collections are data types widely used in object oriented programming. Often collections contain objects of different types and in those cases some operations have to be performed on all the collection elements without knowing the type

Visitor – Structure Example

- We want to create a reporting module in our application to make statistics about a group of customers
- The statistics should be very detailed so all the data related to the customer must be parsed
- All the entities involved in this hierarchy must accept a visitor so the CustomerGroup, Customer, Order and Item are visitable objects



Visitor- Applicability

- The visitor pattern is used when:
  - Similar operations have to be performed on objects of different types grouped in a structure
  - There are many distinct and unrelated operations needed to be performed
  - The object structure is not likely to be changed but is very probable to have new operations which have to be added

Visitor Pattern using Reflection

- Reflection can be used to overcome the main drawback of the visitor pattern
- When the standard implementation of visitor pattern is used the method to invoke is determined at runtime
- **Reflection is the mechanism used to determine the method to be called at compile-time**

Alte tipuri de DP 1

- **Concurrency Patterns** – deal with multi-threaded programming paradigm
  - **Single Threaded Execution** – Prevent concurrent calls to the method from resulting in concurrent executions of the method
  - **Scheduler** – Control the order in which threads are scheduled to execute single threaded code using an object that explicitly sequences waiting threads
  - **Producer-Consumer** – Coordinate the asynchronous production and consumption of information or objects

Alte tipuri de DP 2

- **Testing Patterns 1**
  - **Black Box Testing** – Ensure that software satisfies requirements
  - **White Box Testing** – Design a suite of test cases to exhaustively test software by testing it in all meaningful situations
  - **Unit Testing** – Test individual classes
  - **Integration Testing** – Test individually developed classes together for the first time
  - **System Testing** – Test a program as a whole entity

Alte tipuri de DP 3

- **Testing Patterns 2**
  - **Regression Testing** – Keep track of the outcomes of testing software with a suite of tests over time
  - **Acceptance Testing** – Is done to ensure that delivered software meets the needs of the customer or organization that the software was developed for
  - **Clean Room Testing** – People designing software should not discuss specifications or their implementation with people designing tests for the software

Alte tipuri de DP 4

- **Distributed Architecture Patterns**
  - **Mobile Agent** – An object needs to access very large volume of remote data => move the object to the data
  - **Demilitarized Zone** – You don't want hackers to be able to gain access to servers
  - **Object Replication** – You need to improve the throughput or availability of a distributed computation

Alte clase de DP

- **Transaction patterns** – Ensure that a transaction will never have any unexpected or inconsistent outcome. Design and implement transactions correctly and with a minimum of effort
- **Distributed computing patterns**
- **Temporal patterns** – distributed applications to function correctly, the clocks on the computers they run on must be synchronized. You may need to access pervious or future states of an object. The values of an object's attributes may change over time
- **Database patterns**

Bibliografie

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)

Links

- Structural Patterns: http://www.oodesign.com/structural-patterns/
- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm

Microservices from zero to hero

DAN NASTASA
FLORIN OLARIU

April 30, 2025

centric
connect.engage.succeed.

AGENDA

- About Centric
- “Italian” Architecture and Microservices ☺
- Onion and Spring
- What Is a Microservice?
- Building Monolithic Applications
- Marching Toward Monolithic Hell
- Microservices – Tackling the Complexity
- The Benefits of Microservices
- The Drawbacks of Microservices
- Why Microservices?
- Demo
- About Centric Internship
- Summary
- Bibliography

ABOUT CENTRIC

ABOUT CENTRIC

- It is a Dutch company

ABOUT CENTRIC

- It is a Dutch company
- Portfolio

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions
  - IT Outsourcing

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions
  - IT Outsourcing
  - Business process outsourcing

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions
  - IT Outsourcing
  - Business process outsourcing
  - Staffing services

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions
  - IT Outsourcing
  - Business process outsourcing
  - Staffing services
- Geographic Area

ABOUT CENTRIC

- It is a Dutch company
- Portfolio
  - Software solutions
  - IT Outsourcing
  - Business process outsourcing
  - Staffing services
- Geographic Area

centric

“ITALIAN” ARCHITECTURE AND MICROSERVICES ☺

“ITALIAN” ARCHITECTURE AND MICROSERVICES ☺

“ITALIAN” ARCHITECTURE AND MICROSERVICES ☺

“ITALIAN” ARCHITECTURE AND MICROSERVICES ☺

ONION AND SPRING

ONION AND SPRING

- **Domain Model layer**, where our entities and classes closely related to them e.g. value objects reside

ONION AND SPRING

- **Domain Model layer**, where our entities and classes closely related to them e.g. value objects reside
- **Domain Services layer**, where domain-defined processes reside

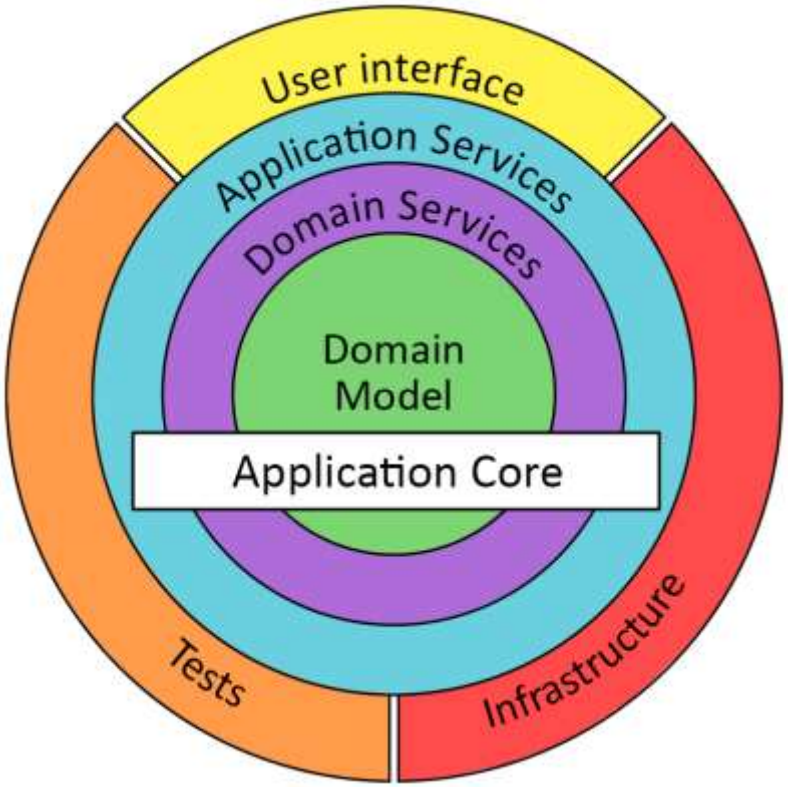
ONION AND SPRING

- **Domain Model layer**, where our entities and classes closely related to them e.g. value objects reside
- **Domain Services layer**, where domain-defined processes reside
- **Application Services layer**, where application-specific logic i.e. our use cases reside

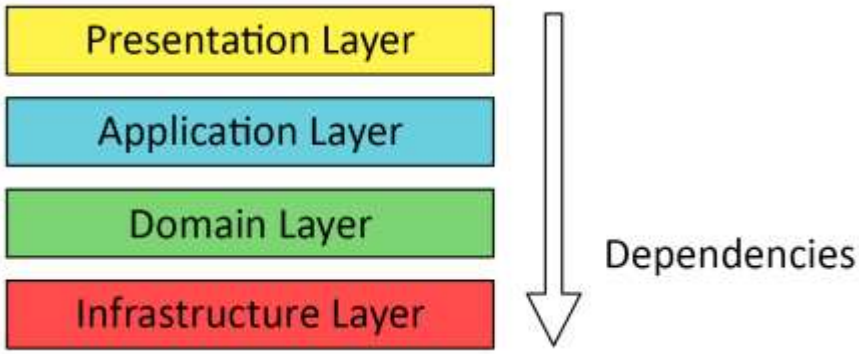
ONION AND SPRING

- **Domain Model layer**, where our entities and classes closely related to them e.g. value objects reside
- **Domain Services layer**, where domain-defined processes reside
- **Application Services layer**, where application-specific logic i.e. our use cases reside
- **Outer layer**, which keeps peripheral concerns like UI, databases or tests

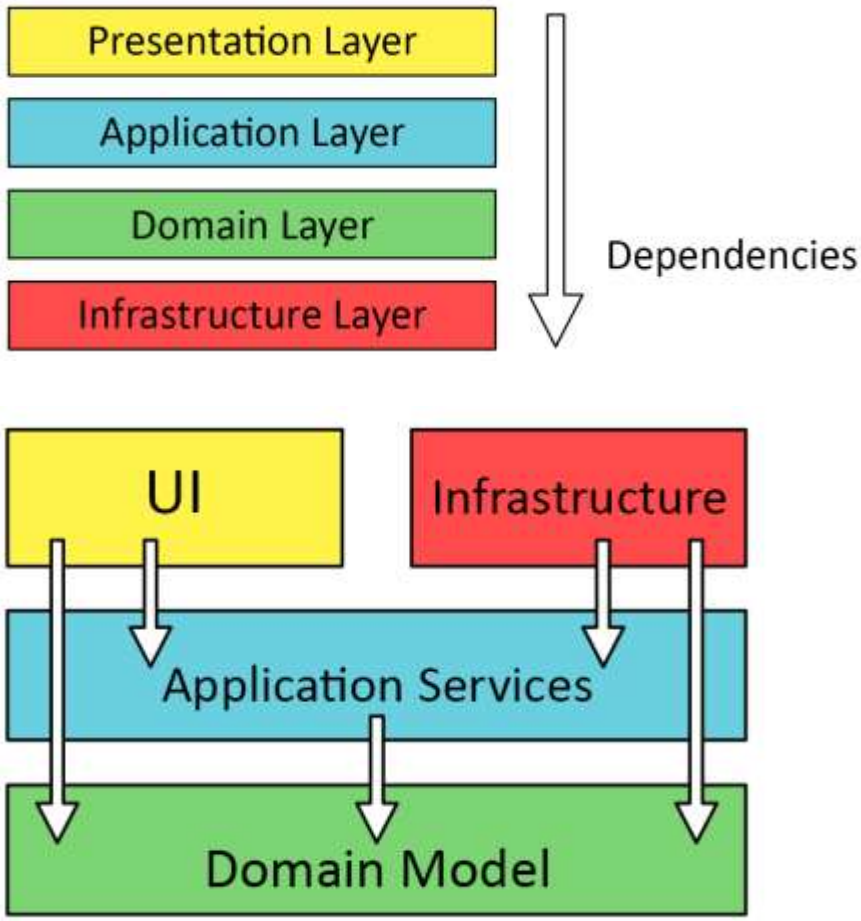
ONION AND SPRING



ONION AND SPRING

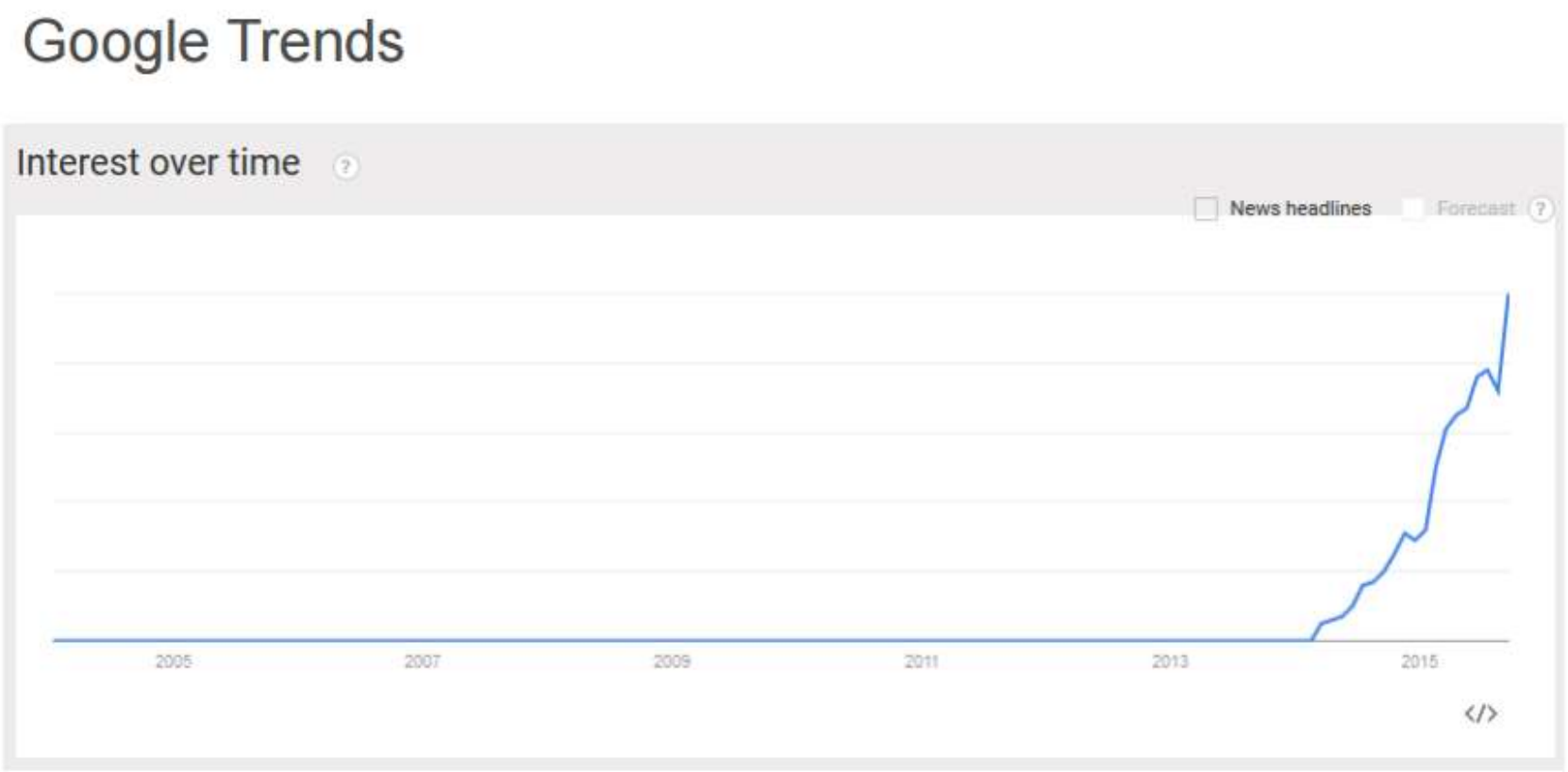


ONION AND SPRING



WHAT IS A MICROSERVICE?

WHAT IS A MICROSERVICE?



centric

WHAT IS A MICROSERVICE?

- “Microservices are a thing these days.”

  Phil Calçado, former Director of Engineering, SoundCloud

WHAT IS A MICROSERVICE?

- “Microservices are small, autonomous services that work together.”

  Sam Newman, Thoughtworks

WHAT IS A MICROSERVICE?

- “Loosely coupled service-oriented architecture with bounded contexts.”

  Adrian Cockcroft, Battery Ventures

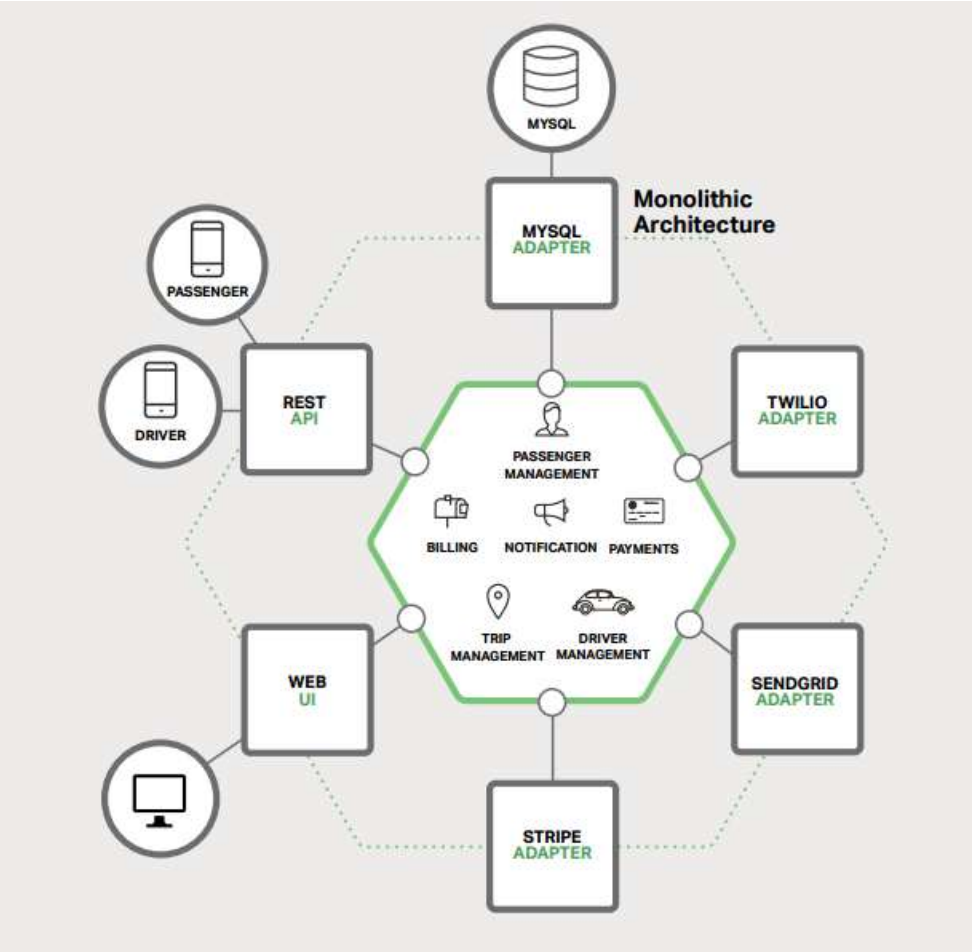
WHAT IS A MICROSERVICE?

- “A microservice is an independently deployable component of bounded scope that supports interoperability through message-based communication.”

  “Microservice Architecture-Aligning Principles, Practices, and Culture”

BUILDING MONOLITHIC APPLICATIONS

BUILDING MONOLITHIC APPLICATIONS



centric

BUILDING MONOLITHIC APPLICATIONS

- These applications are simple to test and debug.

BUILDING MONOLITHIC APPLICATIONS

- These applications are simple to test and debug.
- These applications are also simple to deploy.

BUILDING MONOLITHIC APPLICATIONS

- These applications are simple to test and debug.
- These applications are also simple to deploy.
- These applications are scalable.

MARCHING TOWARD MONOLITHIC HELL

MARCHING TOWARD MONOLITHIC HELL

- In time the application become too complex.

MARCHING TOWARD MONOLITHIC HELL

- In time the application become too complex.
- Being too large is very difficult for any developer to fully understand.

MARCHING TOWARD MONOLITHIC HELL

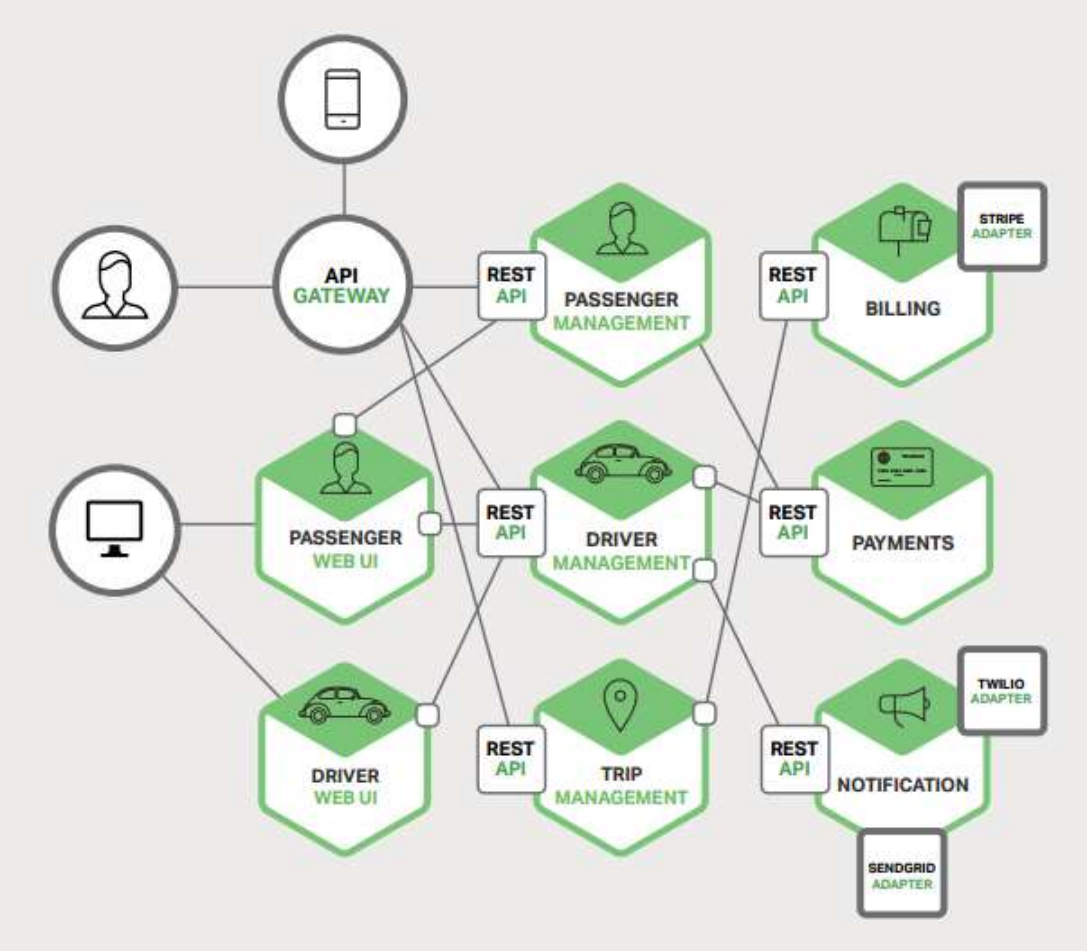
- In time the application become too complex.
- Being too large is very difficult for any developer to fully understand.
- A large application is an obstacle to continuous deployment.

MARCHING TOWARD MONOLITHIC HELL

- In time the application become too complex.
- Being too large is very difficult for any developer to fully understand.
- A large application is an obstacle to continuous deployment.
- Another problem with monolithic applications is reliability.

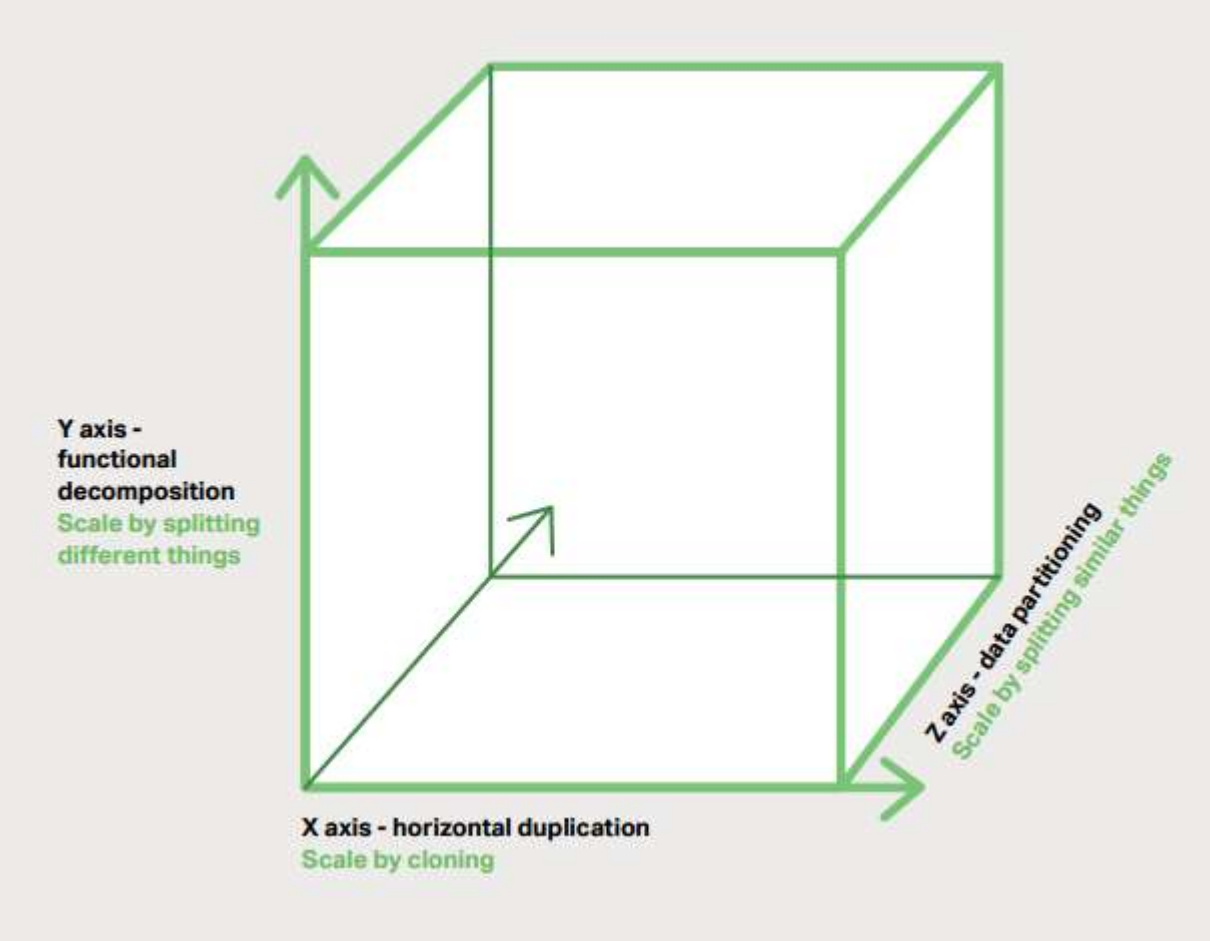
MICROSERVICES – TACKLING THE COMPLEXITY

MICROSERVICES – TACKLING THE COMPLEXITY



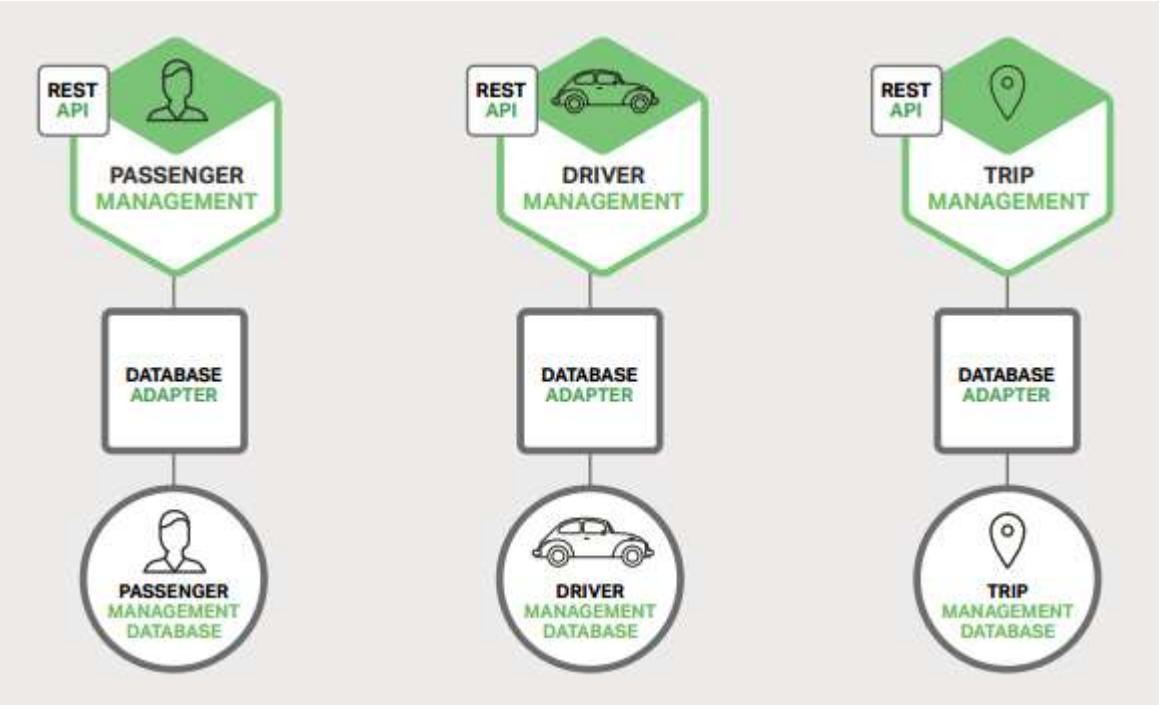
centric

MICROSERVICES – TACKLING THE COMPLEXITY



centric

MICROSERVICES – TACKLING THE COMPLEXITY



centric

THE BENEFITS OF MICROSERVICES

THE BENEFITS OF MICROSERVICES

- It tackles the problem of complexity.

THE BENEFITS OF MICROSERVICES

- It tackles the problem of complexity.
- Enforces modularity.

THE BENEFITS OF MICROSERVICES

- It tackles the problem of complexity.
- Enforces modularity.
- Enables each service to be developed independently by a team that is focused on that service.

THE BENEFITS OF MICROSERVICES

- It tackles the problem of complexity.
- Enforces modularity.
- Enables each service to be developed independently by a team that is focused on that service.
- Enables each microservice to be deployed independently.

THE BENEFITS OF MICROSERVICES

- It tackles the problem of complexity.
- Enforces modularity.
- Enables each service to be developed independently by a team that is focused on that service.
- Enables each microservice to be deployed independently.
- Enables each service to be scaled independently.

THE DRAWBACKS OF MICROSERVICES

THE DRAWBACKS OF MICROSERVICES

- The name itself.

THE DRAWBACKS OF MICROSERVICES

- The name itself.
- The complexity that arises from the fact that a microservices application is a distributed system.

THE DRAWBACKS OF MICROSERVICES

- The name itself.
- The complexity that arises from the fact that a microservices application is a distributed system.
- The partitioned database architecture.

THE DRAWBACKS OF MICROSERVICES

- The name itself.
- The complexity that arises from the fact that a microservices application is a distributed system.
- The partitioned database architecture.
- Testing is also much more complex.

THE DRAWBACKS OF MICROSERVICES

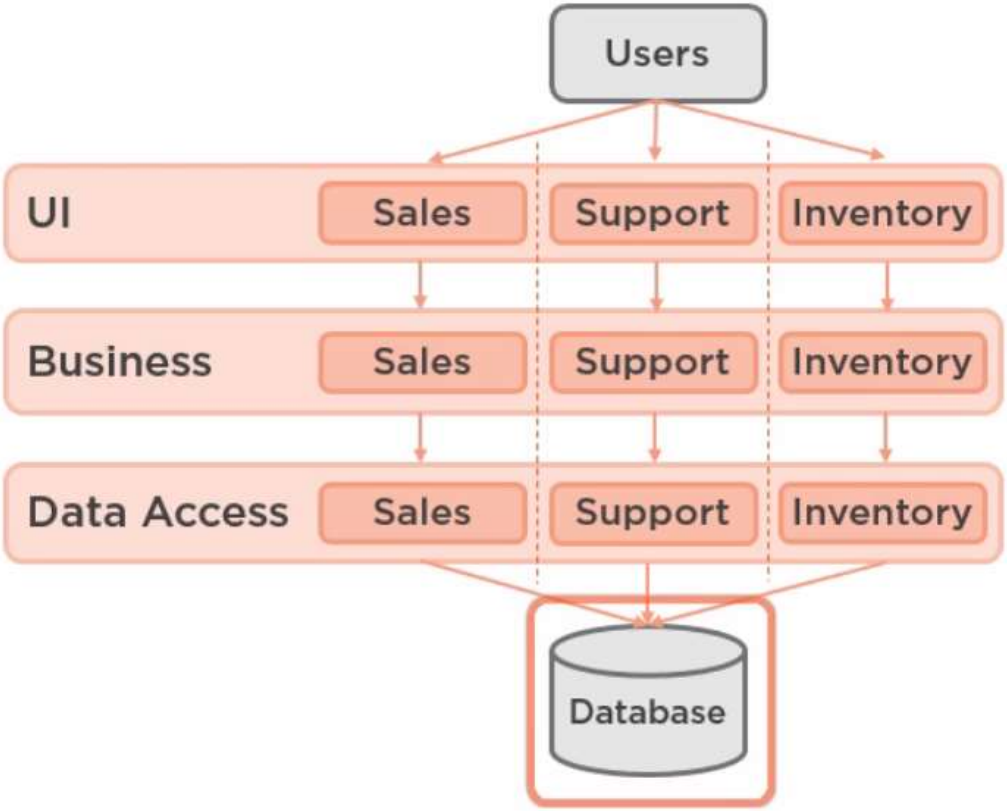
- The name itself.
- The complexity that arises from the fact that a microservices application is a distributed system.
- The partitioned database architecture.
- Testing is also much more complex.
- Deploying a microservices-based application is also much more complex.

WHY MICROSERVICES?

WHY MICROSERVICES?

- **Gilt**: “From Monolith Ruby App to Distributed Scala Micro-Services” (NYC Tech Talks) [Link]
- **Nike**: “Nike’s Journey to Microservices” (AWS Re:Invent 2014) [Link]
- **SoundCloud**: ”Building Products at SoundCloud - Part III: Microservices in Scala and Finagle” [Link]
- **Capital One**: “Lack Of Legacy Lets Capital One Build Nimble Infrastructure” [Link]
- **Hailo**: “A Journey into Microservices” [Link]
- **Autoscout24**: “Why Autoscout24 changes its technology” [Link]
- **Zalando**: “From Monolith to Microservices” [Link]

WHY MICROSERVICES?



WHY MICROSERVICES?

Sales

Sales opportunity

Contact

Sales person

Product

Sales territory

Support

Support ticket

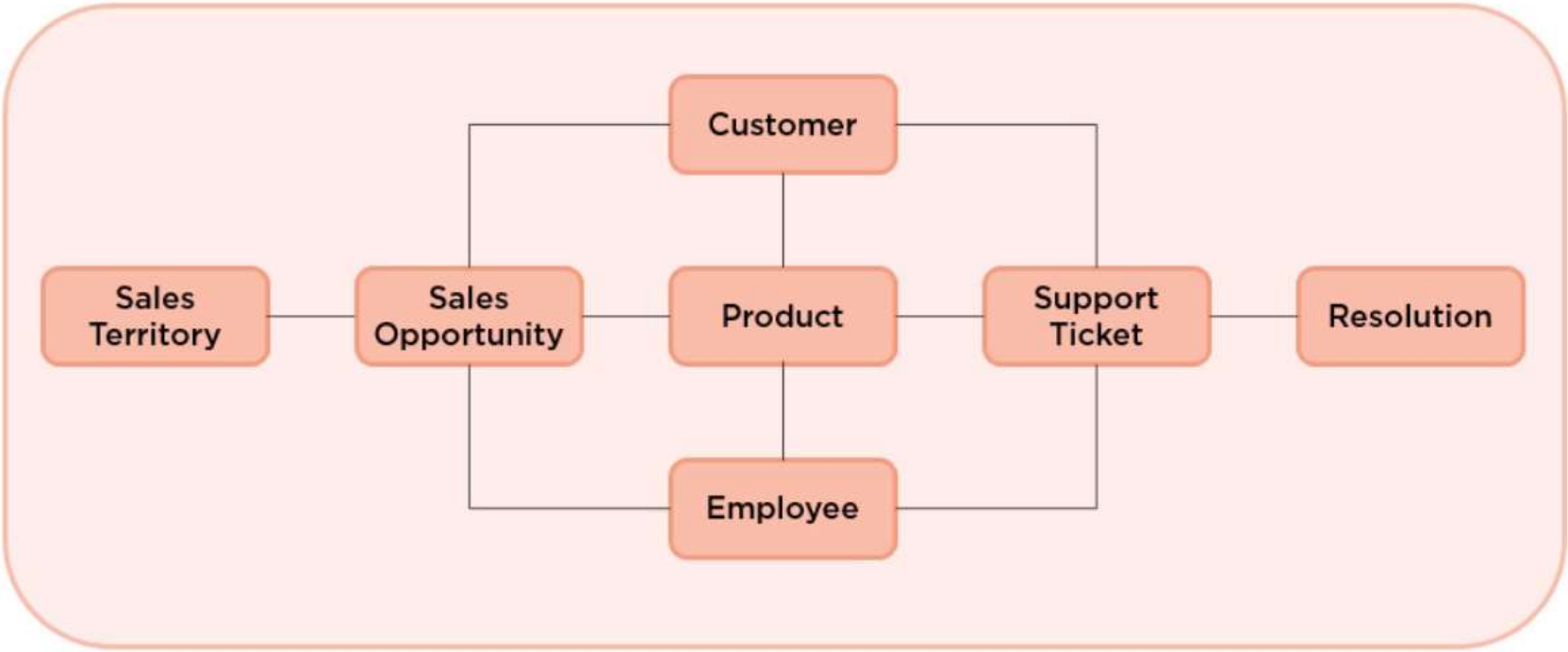
Customer

Support person

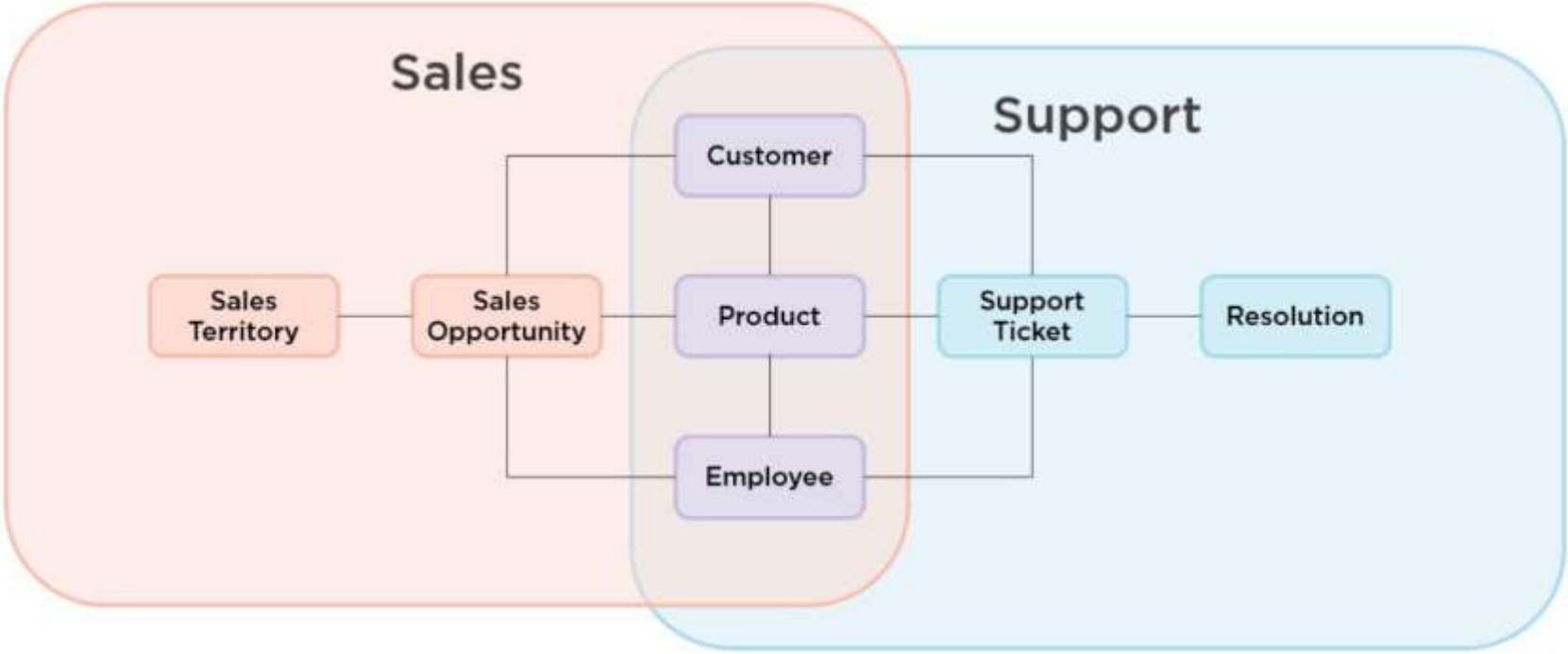
Product

Resolution

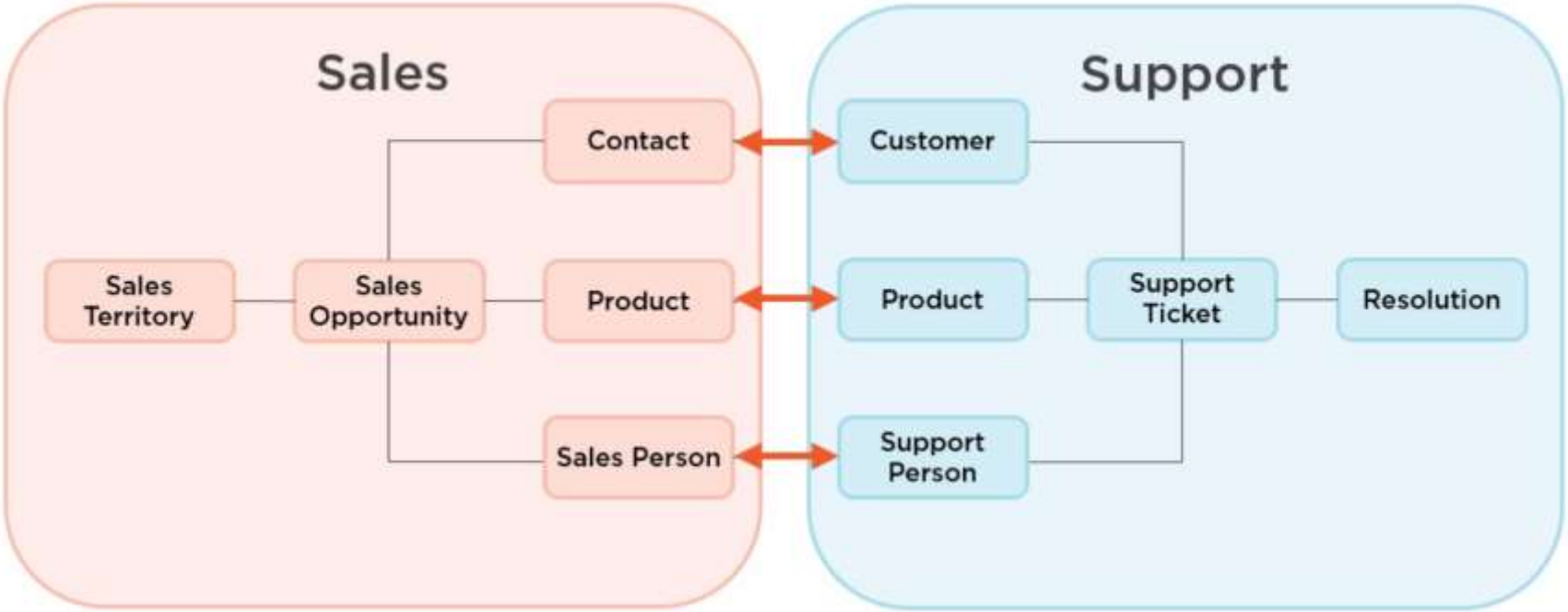
WHY MICROSERVICES?



WHY MICROSERVICES?



WHY MICROSERVICES?



centric

WHY MICROSERVICES?

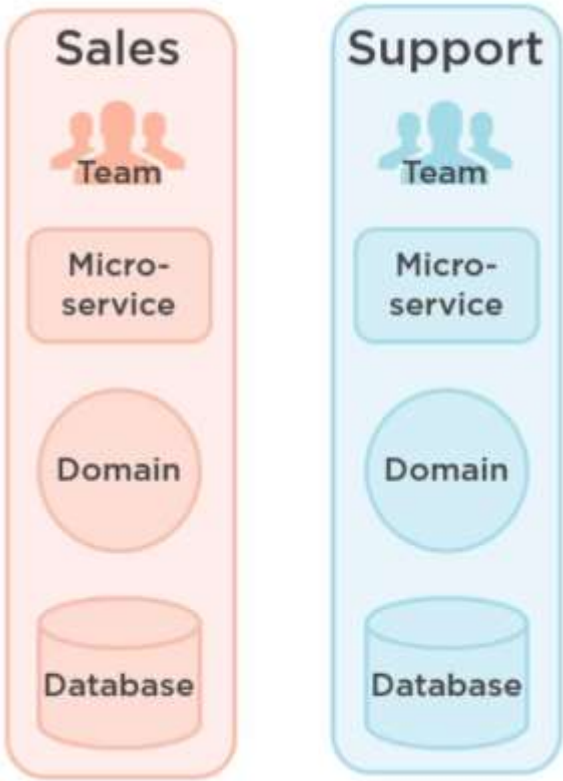
Bounded context

Cohesion/coupling

Single domain of knowledge

Consistent data model

Independence



centric

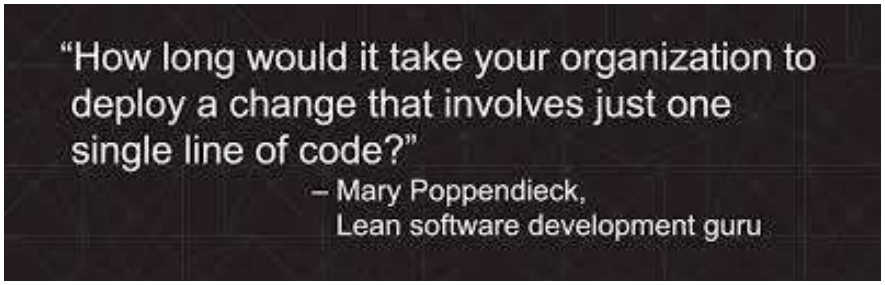
DEMO

ABOUT CENTRIC INTERNSHIP

centric

ONE MORE THING …

ONE MORE THING …



SUMMARY

SUMMARY

- Building complex applications is not an easy task.

SUMMARY

- Building complex applications is not an easy task.
- The Monolithic Architecture pattern only makes sense for simple, lightweight applications.

SUMMARY

- Building complex applications is not an easy task.
- The Monolithic Architecture pattern only makes sense for simple, lightweight applications.
- The Microservices Architecture pattern is the better choice for complex, evolving applications, despite the drawbacks and implementation challenges.

SUMMARY

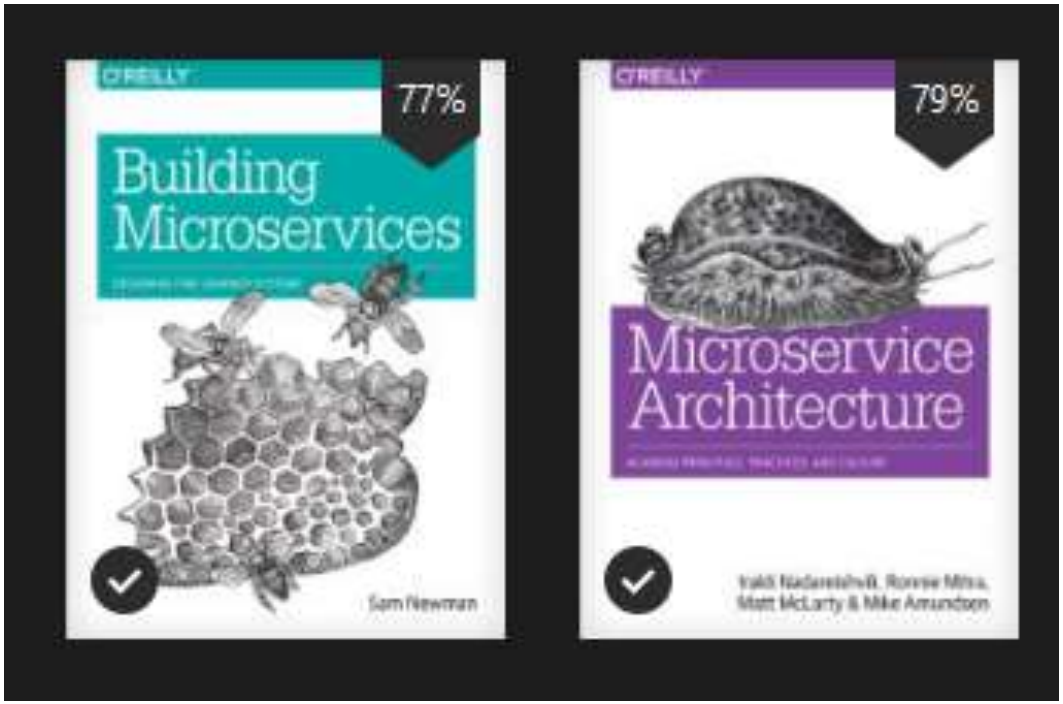
- Building complex applications is not an easy task.
- The Monolithic Architecture pattern only makes sense for simple, lightweight applications.
- The Microservices Architecture pattern is the better choice for complex, evolving applications, despite the drawbacks and implementation challenges.
- Probably the best way to define boundaries for microservices is by using Bounded Context from DDD (Domain Driven Design)

BIBLIOGRAPHY

BIBLIOGRAPHY

- Alagarasan, Vijay. “Seven Microservices Anti-patterns”, August 24, 2015.
- Cockcroft, Adrian. “State of the Art in Microservices”, December 4, 2014.
- Fowler, Martin. “Microservice Prerequisites”, August 28, 2014.
- Fowler, Martin. “Microservice Tradeoffs”, July 1, 2015.
- Humble, Jez. “Four Principles of Low-Risk Software Release”, February 16, 2012.
- Humble, Jez, Chris Read, and Dan North. “The Deployment Production Line”. In Proceedings of the conference on AGILE 2006, 113– 118.
- IEEE Computer Society. Kniberg, Henrik, and Anders Ivarsson. “Scaling Agile at Spotify”, October 2012.
- http://microservices.io/patterns/microservices.html

BIBLIOGRAPHY



centric

BIBLIOGRAPHY

centric

QUESTIONS?

centric
connect.engage.succeed.

THANK YOU!

dan.nastasa@centric.eu
florin.olariu@centric.eu

DAN NASTASA
FLORIN OLARIU

April 30, 2025

centric
connect.engage.succeed.

Ingineria Programării

Cursul 11 – 9 Mai

Cuprins

- Din Cursurile trecute…
  - Design Patterns
    - Creational Patterns
    - Structural Patterns
    - Behavioral Patterns
- Alte tipuri de Design Patterns
- Quality Assurance
  - Software Testing
  - Testing Methodologies
  - Testing process
  - Manual Testing vs Automatic Testing

Din Cursurile Trecute

- GOF = ?
- Creational Patterns
- Structural Patterns
- Behavioral Patterns

Din cursurile trecute – CP

- Abstract Factory
- Builder
- Factory Method
- Prototype = ?
- Singleton

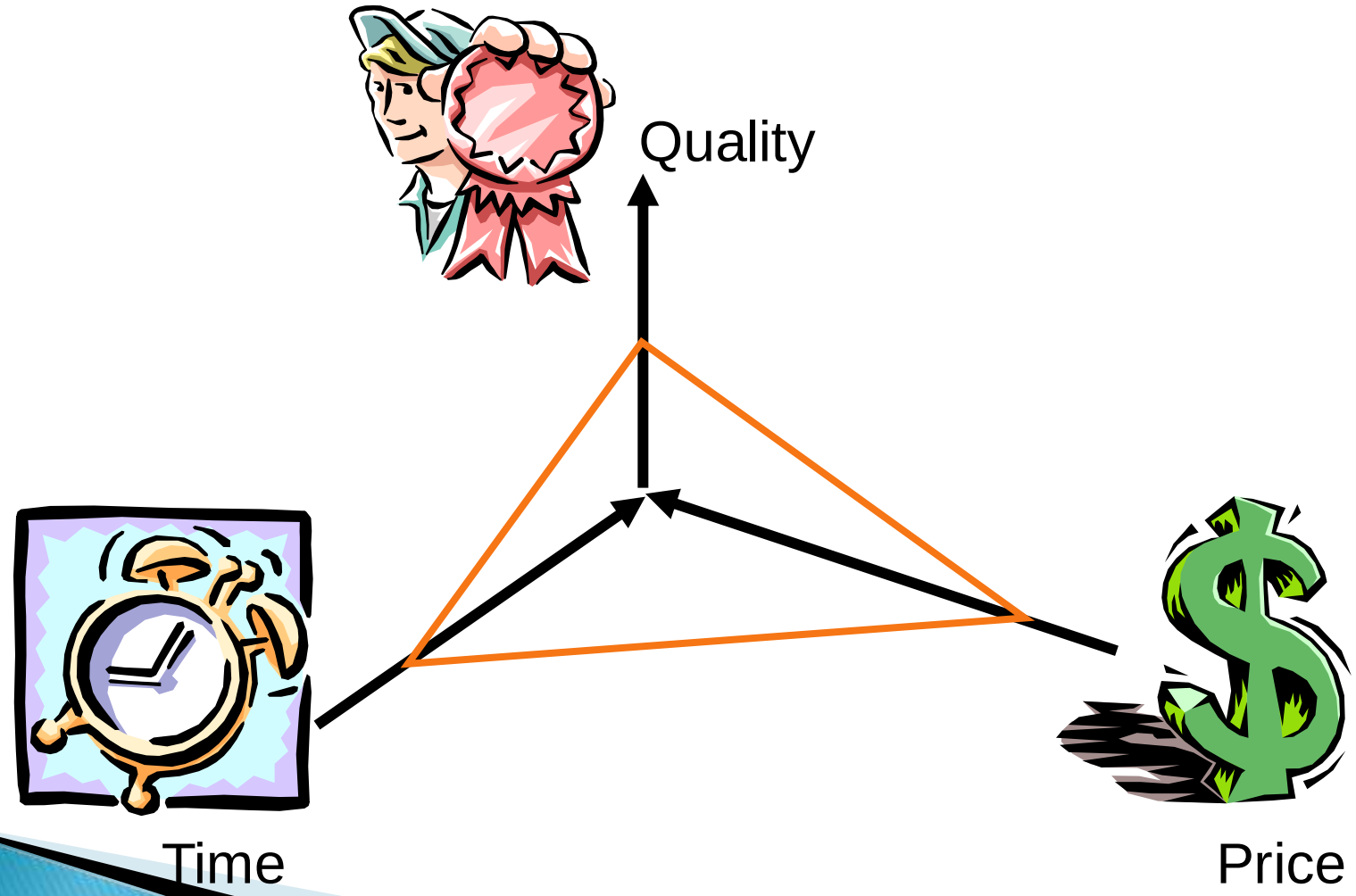
Din cursurile trecute – SP

- Adapter
- Bridge
- Composite
- Decorator
- Façade
- Flyweight = ?
- Proxy

Quality Assurance

- Refers to **planned and systematic production processes** that provide confidence in a product's suitability for its intended purpose.
- A set of activities intended to ensure that **products satisfy customer requirements**
- *QA cannot absolutely guarantee the production of quality products but makes this more likely*
- Two key principles characterize QA:
  - "fit for purpose" – the product should be suitable for the intended purpose, and
  - "right first time" – mistakes should be eliminated

Quality Assurance Dilemma



Quality Assurance – Definition

"The process of exercising or evaluating a system by manual or automated means to verify that it satisfies specified <u>requirements</u> or to identify differences between expected and actual results."

(IEEE Standard Glossary, 1983)

Software Quality Assurance

- (SQA) consists of a means of **monitoring the software engineering processes and methods used to ensure quality**
- May include ensuring conformance to one or more standards, such as ISO 9000 or CMMI
- SQA encompasses the entire software development process, which includes processes such as *software design, coding, source code control, code reviews, change management, configuration management, and release management*

ISO 9000

- ISO 9000 is a family of standards for quality management systems
- Some of the requirements in ISO 9001 (from ISO 9000 family) include
  - a set of procedures;
  - monitoring processes;
  - keeping adequate records;
  - checking output for defects;
  - regularly reviewing individual processes;
  - facilitating continual improvement

Software Testing – Introduction

- An empirical investigation conducted to provide information about the *quality of the product or service under test, with respect to the context in which it is intended to operate*.
- Allow the business to *appreciate and understand* the risks at implementation of the software
- Test techniques include the process of executing a program or application with **the intent of finding software bugs**
- The process of **validating and verifying** that a software program/application/product meets the business and technical requirements that guided its design and development

Software Testing – When?

- Can be implemented at any time in the development process
- However the most test effort is employed after the requirements have been defined and coding process has been completed
- In XP…

Software Testing

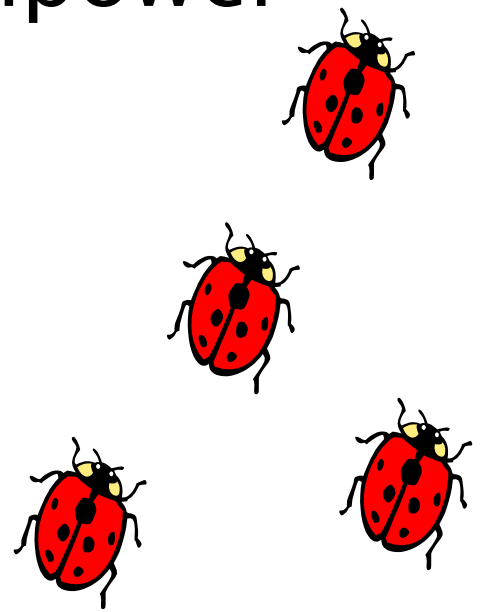
- Software Testing is NOT a phase
- It is integrated in all phases of software development
- Each development step has an attached testing documentation

What is the purpose of testing?

- We need not only to find bugs but also to prevent them (which is better)
- To know when to stop because effectiveness and economics of the process is essential.
- To know that not all system requires the same level of quality (mission critical against IT).
- Testing is not only for the SOFTWARE it is for all DELIVERABLES

Why are there bugs in software?

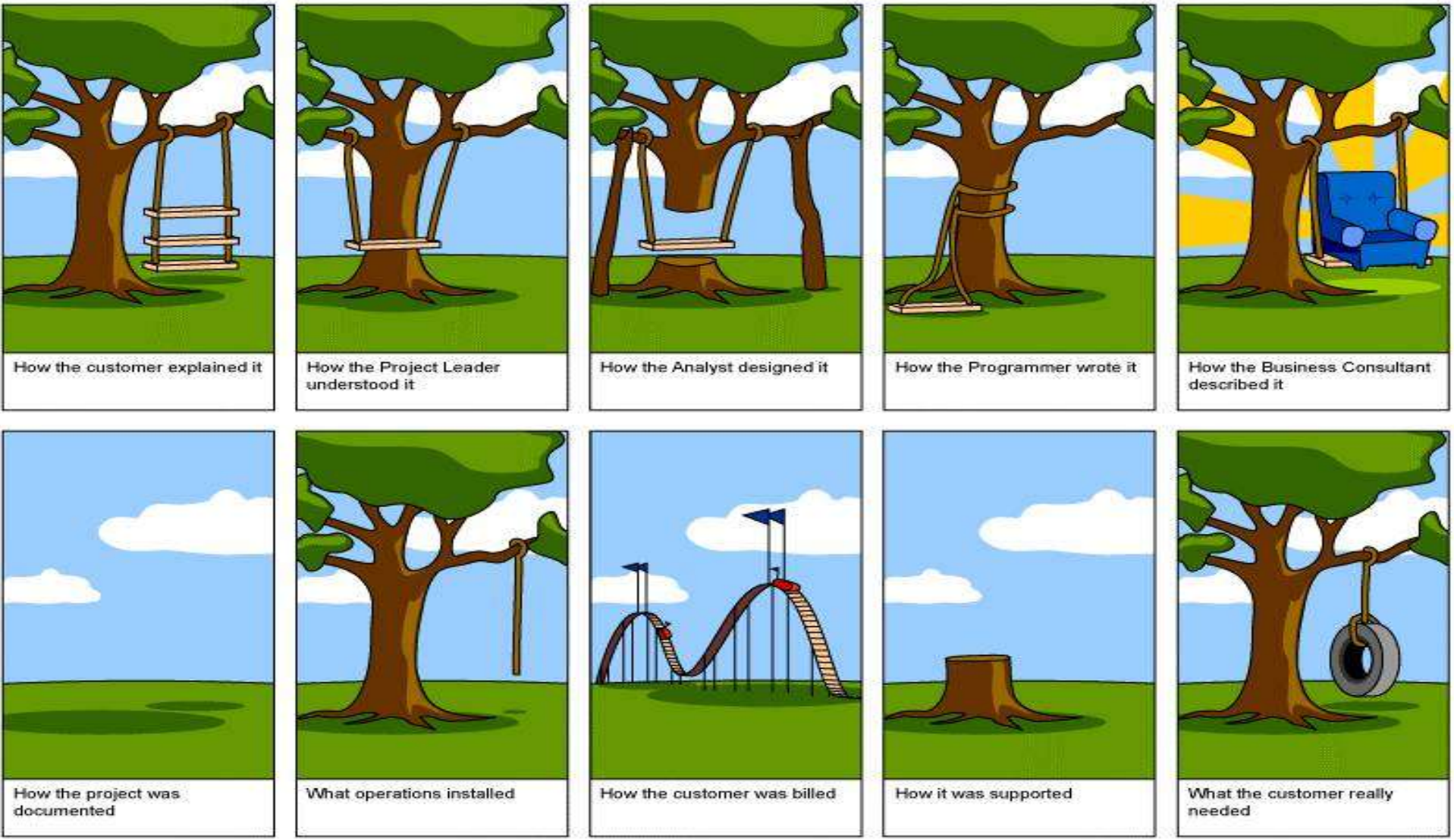
- Miscommunication
- Misunderstanding
- Low professional manpower
- Time pressures



Miscommunication

"Didn't you get my e-mail?"

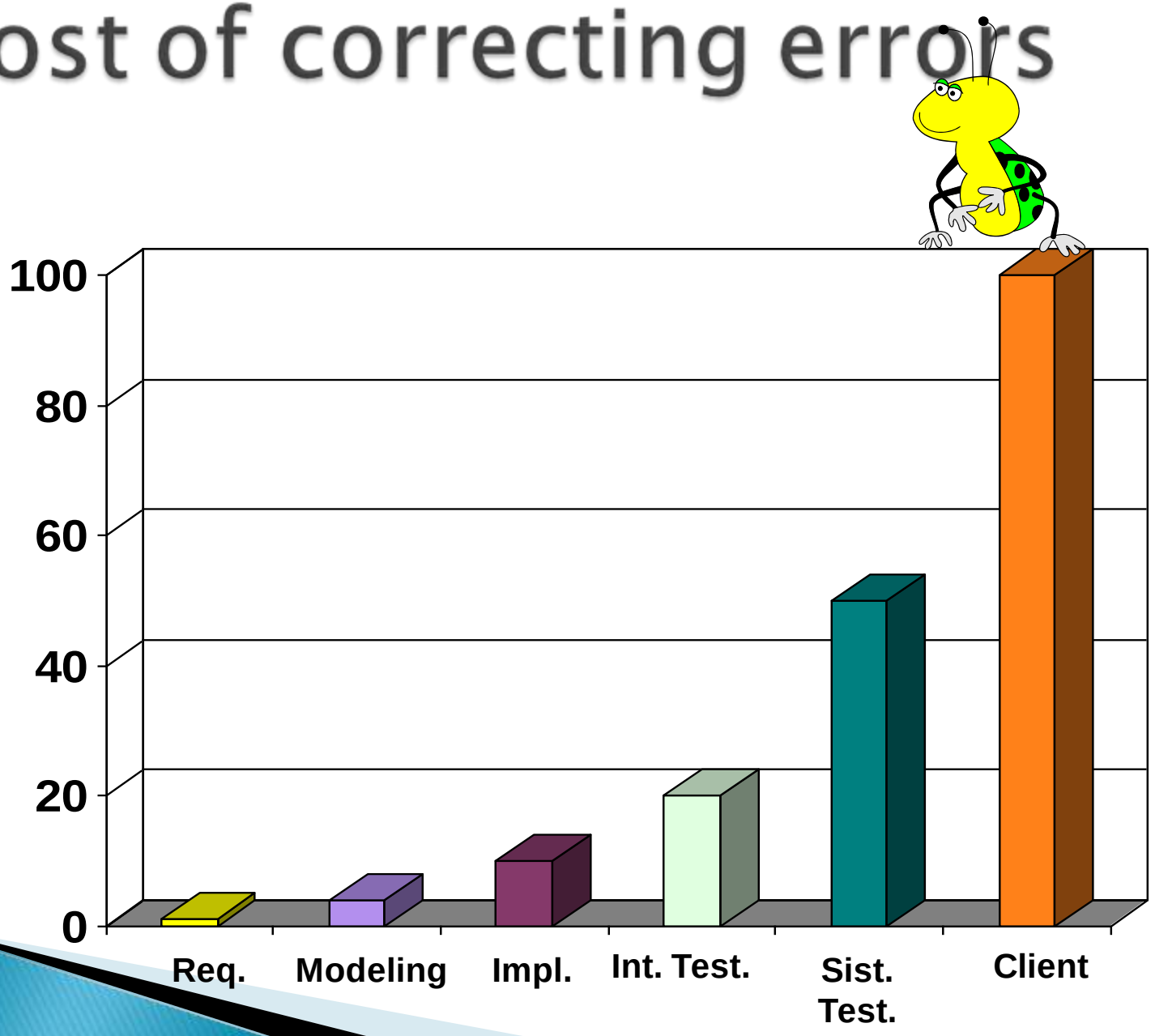
Misunderstandings



What genenerates most errors?

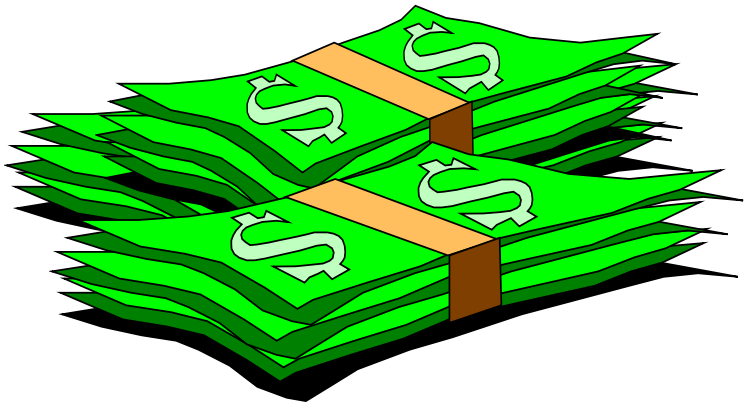
- Incorrect or incomplete requirements 50%
- Ambiguous or incomplete modeling 30%
- Programming errors 20%

Cost of correcting errors

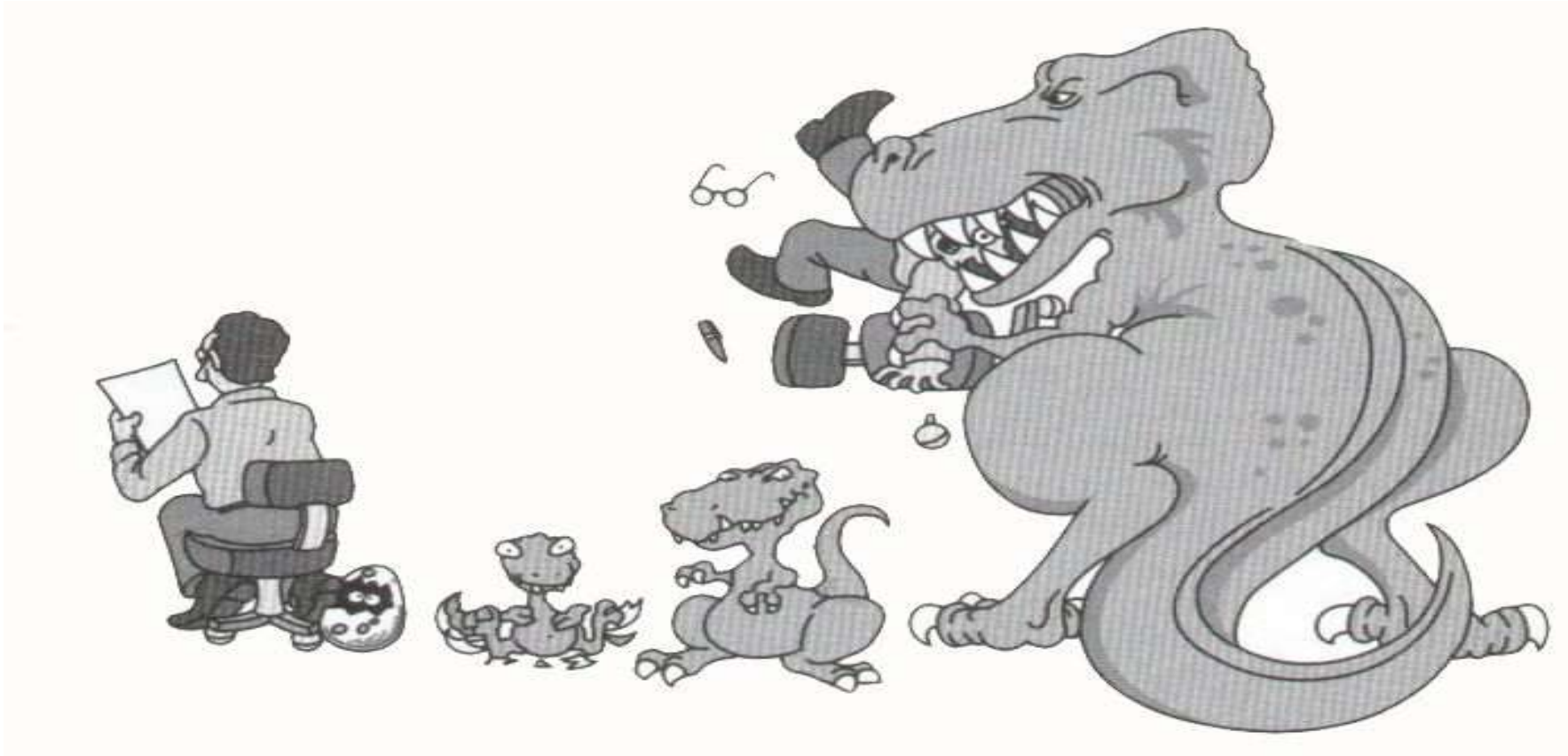


Note

Late error detection⇒ greater repair cost



Error must be repaired as soon as possible



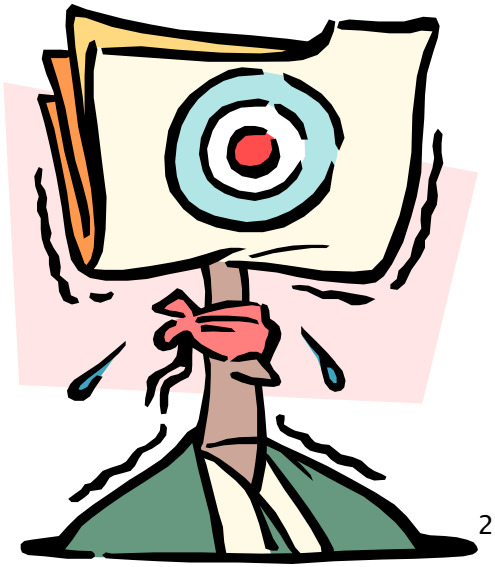
REQ. MODELING IMPLEM. CLIENT TESTING

Professional Testing

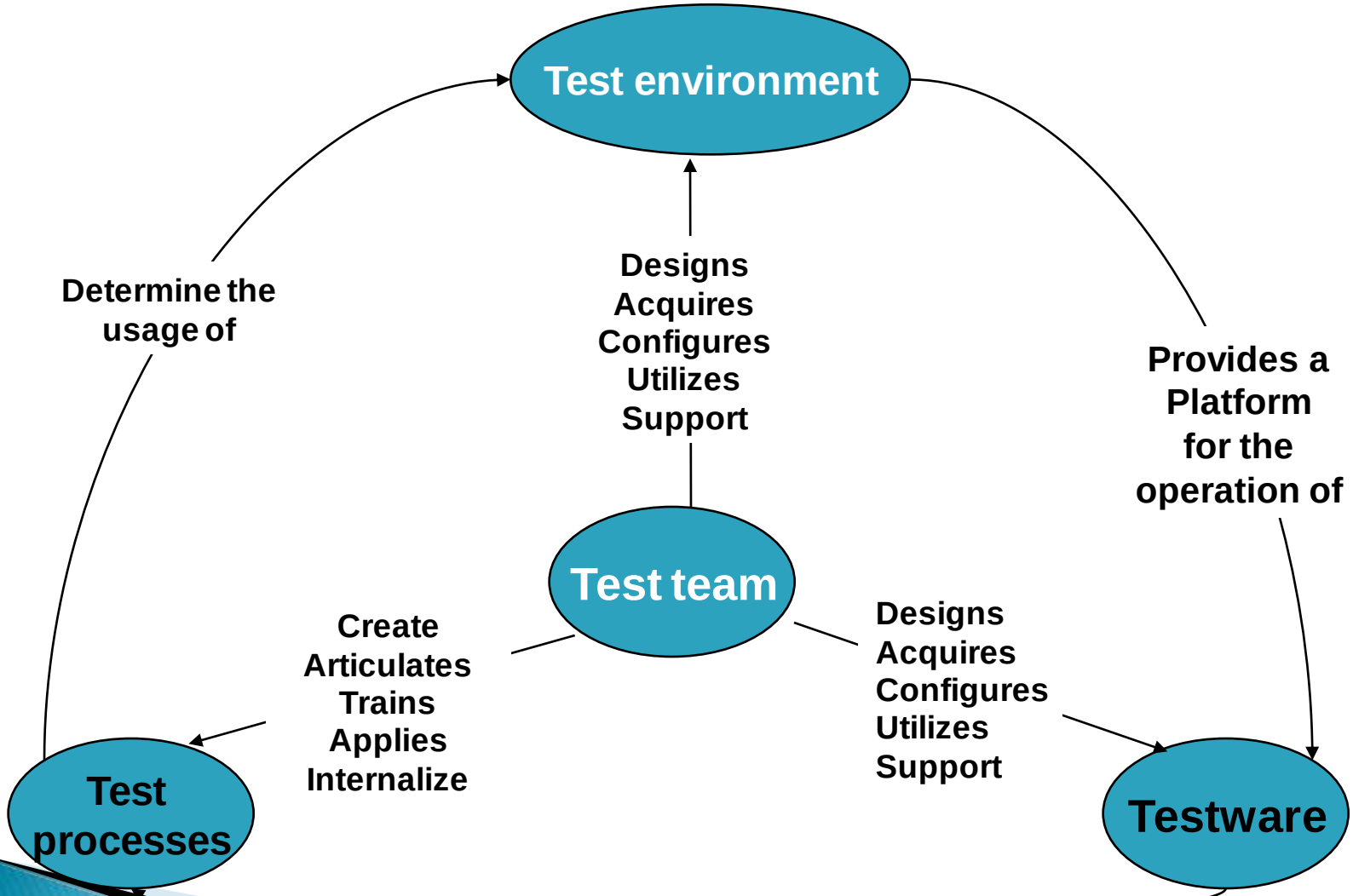
Professional testing means finding the least amount of test cases which will check the most amount of system features.

When does testing stop?

- Never
- When the number of errors found in a test cycle is lower than a set amount
- When no more critical faults are found
- When we run out of time



Schema of a Testing System



Testing Methodology

- Differences between testing and debugging
- Layers of testing
- Testing methods
- Testing content
- Manual vs Automatic Testing

Testing vs. Debugging

Testing

- Check compliance to requirements
- Normally carried out by an external and neutral party
- Is planned and controlled

Debugging

- Check validity of program sections
- Run by the developer
- Is a random process

Layers of testing

- Unit testing or debugging
- Module/Sub-System
- Integration
- System
- Acceptance

Unit Testing

- Testing a function, program screen, feature
- Run by programmers
- Predefined
- Results must be documented
- Input and Output simulators are used

Integration testing

- Testing of several modules at the same time
- Testing coexistence
- Run by programmers or testers
- Pre-planned testing
- Results must be documented

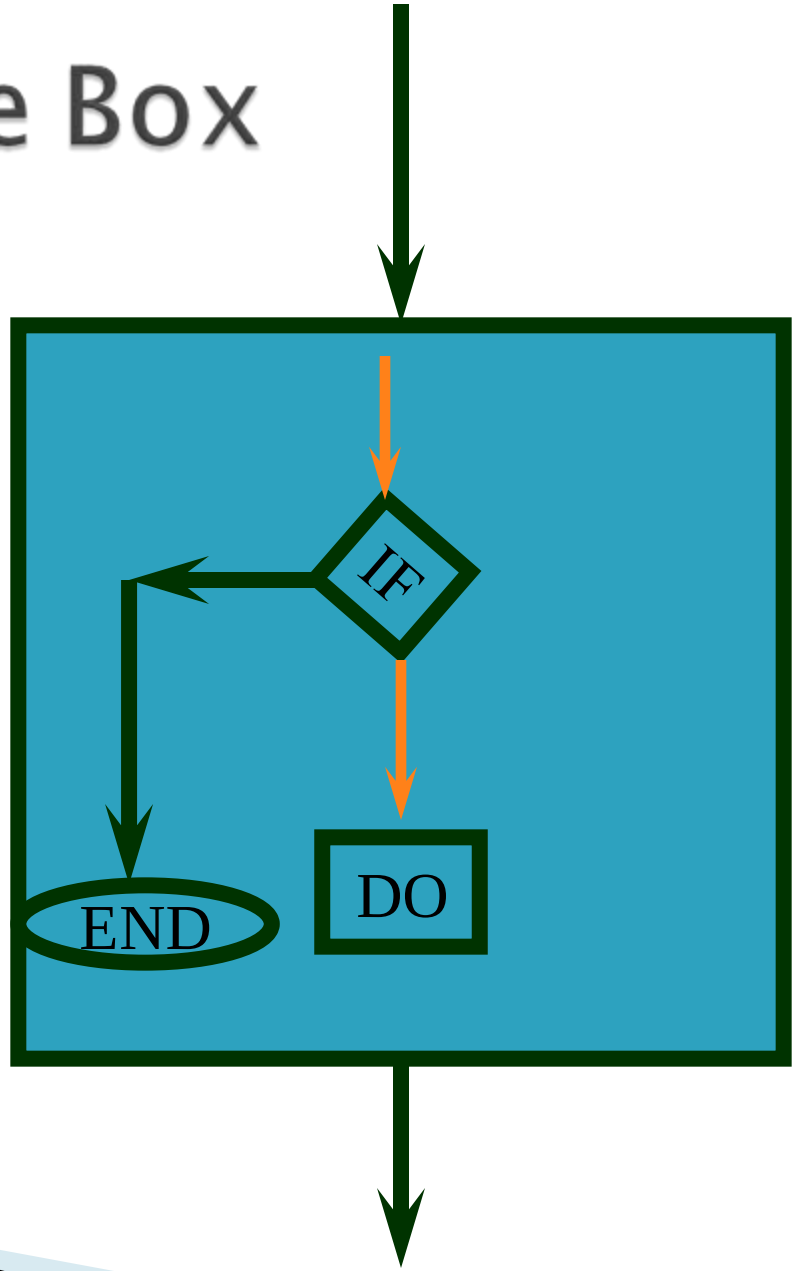
System Testing

- System testing of software or hardware is testing conducted **on a complete, integrated system** to evaluate the system's compliance with its specified requirements.
- System testing falls within the scope of **black box testing**
- System testing is a more limiting type of testing; it seeks to detect defects both within the "inter-assemblages" and also within the system as a whole.

Testing methods

- White Box
- Black Box
- Gray Box
- Graphical user Interface Testing
- Acceptance Testing
- Regression Testing

White Box

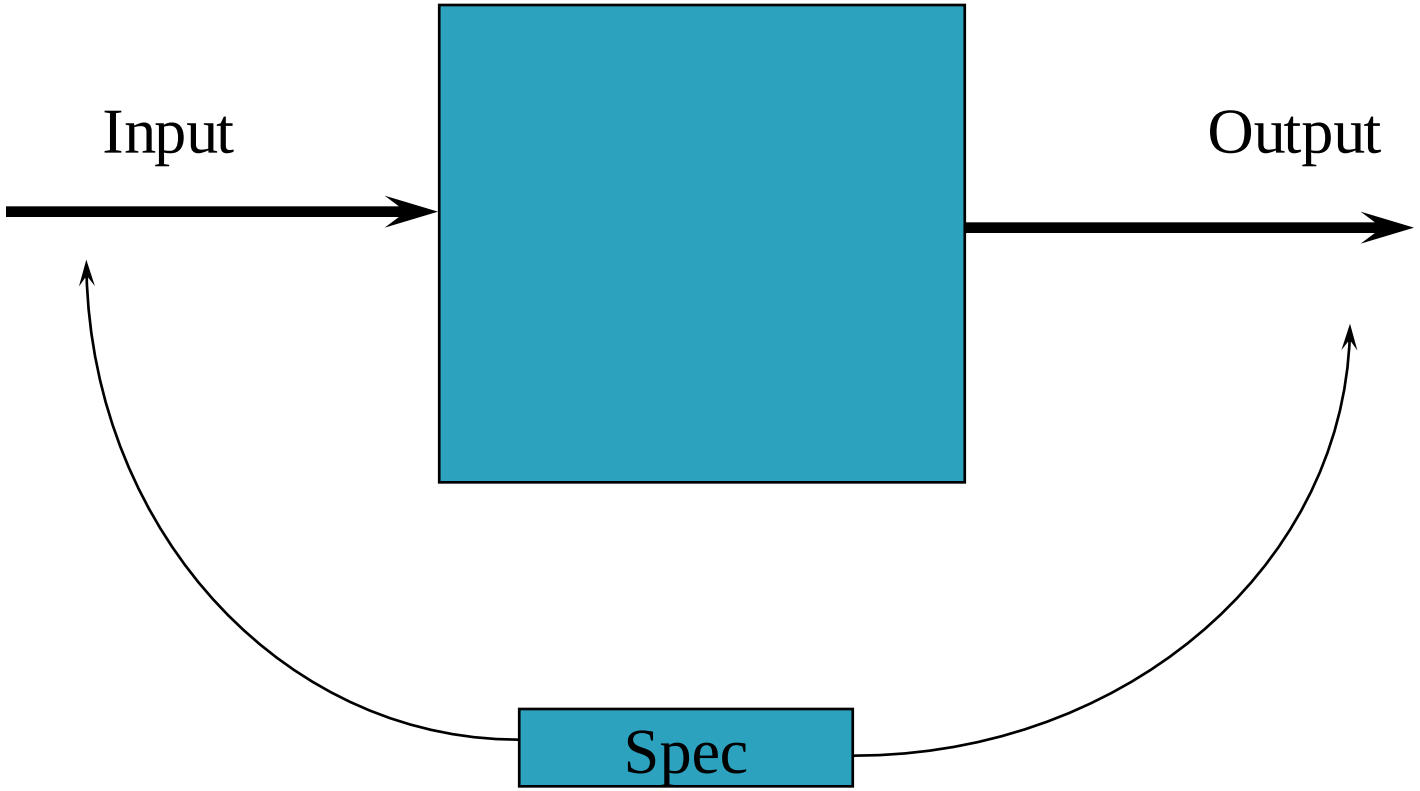


White Box (2)

- The tester has access to the internal data structures and algorithms

- Types of white box testing
  - api testing – Testing of the application using Public and Private APIs
  - code coverage – creating tests to satisfy some criteria of code coverage
  - fault injection methods
  - mutation testing methods
  - static testing – White box testing includes all static testing

Black Box



- "like a walk in a dark labyrinth without a flashlight,"

Black Box (2)

- Specification-based testing

- Black box testing methods include: equivalence partitioning, boundary value analysis, all-pairs testing, fuzz testing, model-based testing, traceability matrix, exploratory testing and specification-based testing.

Grey Box

- This involves having access to internal data structures and algorithms for purposes of designing the test cases, but testing at the user, or black-box level
- Manipulating input data and formatting output do not qualify as "grey-box," because the input and output are clearly outside of the "black-box" that we are calling "the software under test"

GUI Testing

- In computer science, GUI software testing is the process of testing a product that uses a graphical user interface, to ensure it meets its written specifications.
- The variety of errors found in GUI applications:
  - Data validation, Incorrect field defaults, Mandatory fields, not mandatory, Wrong fields retrieved by queries, Incorrect search criteria
  - Field order, Multiple database rows returned, single row expected
  - Currency of data on screens, Correct window modality?
  - Control state alignment with state of data in window?

Acceptance Testing

- **A black-box testing performed on a system** prior to its delivery
- In software development, acceptance testing by the system provider is often distinguished from acceptance testing by the customer (the user or client) prior to accepting transfer of ownership.
- In such environments, acceptance testing performed by the customer is known as **user acceptance testing** (UAT).
- This is also known as **end-user testing**, site (acceptance) testing, or field (acceptance) testing.

Regression Testing

- Regression testing is **any type of software testing** which seeks to uncover software regressions.
- Such regressions occur whenever software functionality that was previously working correctly, stops working as intended.
- Typically regressions **occur as an unintended consequence of program changes**.
- Common methods of regression testing include **re-running previously run tests and checking whether previously fixed faults have re-emerged**

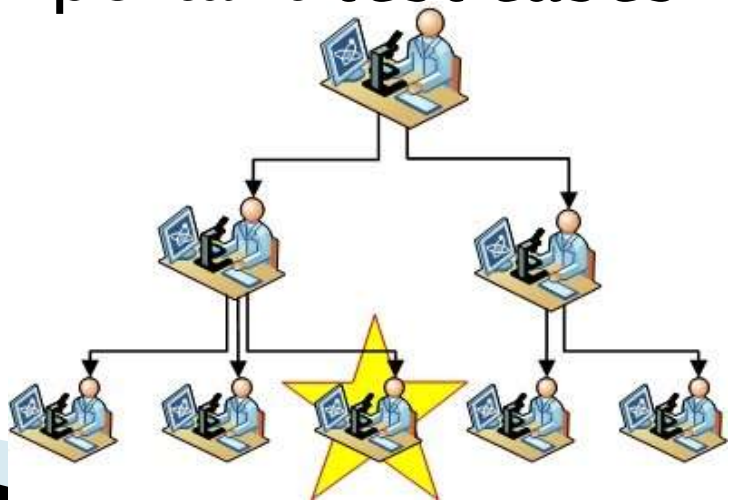
Automatic vs Manual Testing

- *Problems are found quickly*
- *Cheap to repeat*
- *The process of writing code is more flexible*
- *Less manual testing*
- *Software development becomes predictable and can be planned*

- *Solves interface issues: correctness of text, messages, page layout, element order, visibility etc.*
- *Writing test scenarios can be difficult and implies technical knowledge of the entire system*

Manual Testing

- *Manual testing is the process of manually testing software for defects*
- It requires a **tester** to play the role of an end user, and use most of all features of the application to ensure correct behavior
- To ensure completeness of testing, the tester often follows a written **test plan** that leads them through a set of important **test cases**



Definitions

- **Test Strategy** is developed by the "Project manager" which contains what type of technique to follow and which module to test
- **Test Plan** is developed by the Test Lead, which contains "what to test", "how to test", "when to test", "who to test"
- **Test Scenario** is a name given to test case. It is dealt with by the Test Engineer
- **Test Case** specifies a testable condition to validate functionality. The test cases are deal by Test Engineer

Test Plan

- **A systematic approach to testing a system**
- Contains a detailed understanding of what the eventual workflow will be
- Documents the strategy that will be used to verify and ensure that a product or system meets its design specifications and other requirements
- Is usually prepared by or with significant input from Test Engineers

Test Plan Components

- May include one or more of the following:
  - *Design Verification or Compliance test*
  - *Manufacturing or Production test*
  - *Acceptance or Commissioning test*
  - *Service and Repair test*
  - *Regression test*

Test Plan Structure (IEEE 829-1998)

- Test plan identifier
- Introduction
- Test items
- Features to be tested
- Features not to be tested
- Approach
- Item pass/fail criteria
- Suspension criteria
- Test deliverables
- Testing tasks
- Environmental needs
- Responsibilities
- Staffing and training needs
- Schedule
- Risks and contingencies
- Approvals

Test Plan – Life Cycle

Test Case

- **A set of conditions or variables** under which a tester will determine whether an application or software system meets specifications
- **A sequence of steps** to test the correct behavior/functionalities, features of an application
- In order to fully test that all the requirements of an application are met, there must be *at least one test case for each requirement* (*two recommended*)

Test Case Format

- Test case ID
- Test case Description
- Expected Output
- Actual Output
- Pass/Fail
- Remarks
- Test step or order of execution number
- Related requirement(s)
- Depth
- Test category
- Author
- Check boxes for whether the test is automatable and has been automated.

Large Scale Engineering Projects

- Need a systematic approach:
  1. Choose a high level test plan
  2. Write detailed test cases
  3. Assign the test cases to testers, who manually follow the steps and record the results.
  4. Author a test report, detailing the findings of the testers.

- The report is used by managers to determine whether the software can be released

Test Automation

- **A process of writing a computer program to do testing** that would otherwise need to be done manually
- **The use of software to control the execution of tests**, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions
- Commonly, test automation involves automating a manual process already in place that uses a formalized testing process

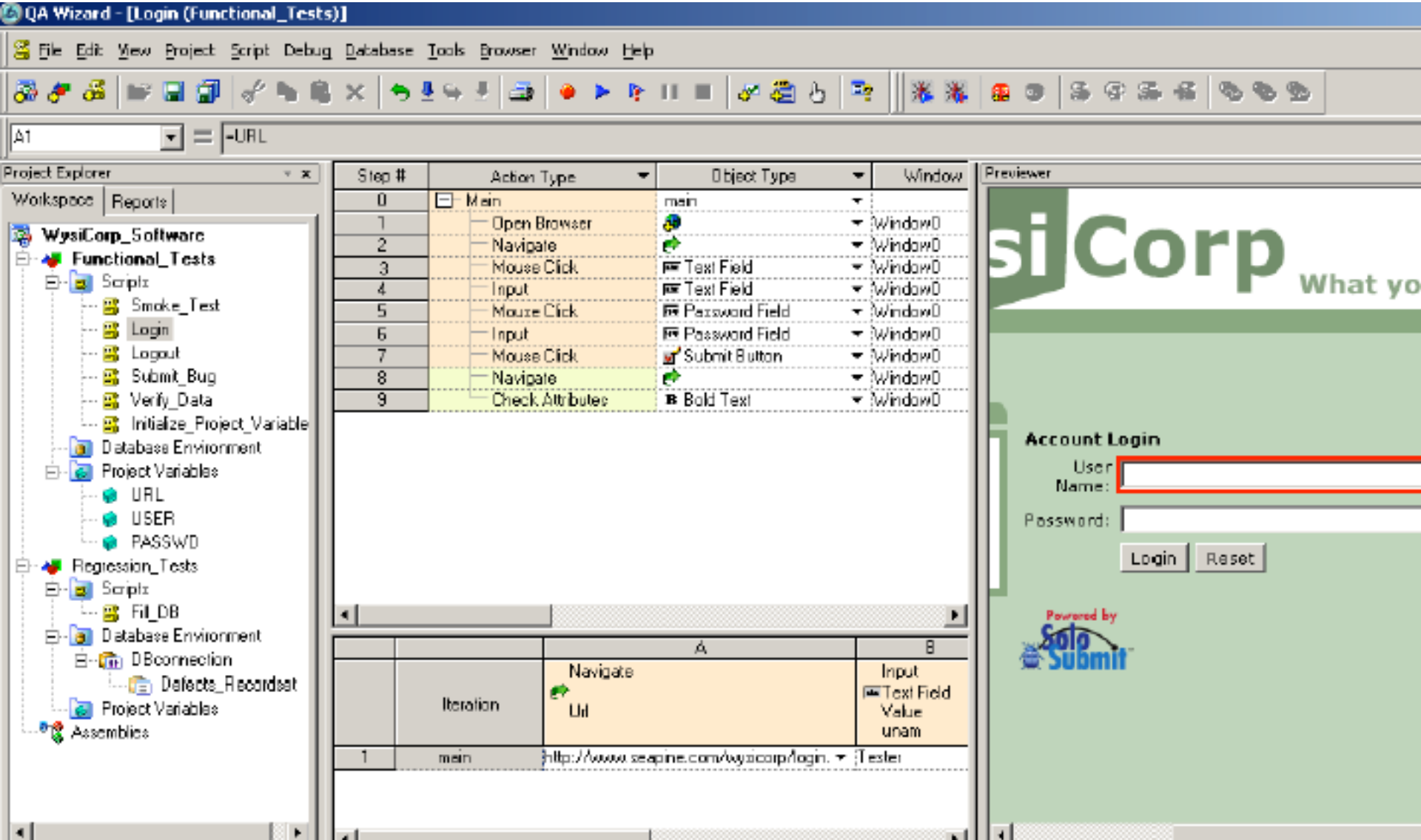
Test Automation – Approaches

- **Graphical user interface testing**. A testing framework generates user interface events such as keystrokes and mouse clicks, and **observes the changes** that result in the user interface, to validate that the observable behavior of the program is correct
- **Code-driven testing**. The public (usually) interface to classes, modules, or libraries are tested with a variety of input arguments to validate that the results that are returned are correct

TA – What to test

- Testing tools can help automate tasks such as *product installation, test data creation, GUI interaction, problem detection, defect logging*, etc.
- Important points when thinking about TA:
  - Platform and OS independence
  - Data driven capability (Input Data, Output Data, Meta Data)
  - Customizable Reporting (DB Access, crystal reports)
  - Email Notifications
  - Easy debugging and logging
  - Version control friendly
  - Extensible & Customizable
  - Support distributed execution environment
  - Distributed application support

Test Automation – Example



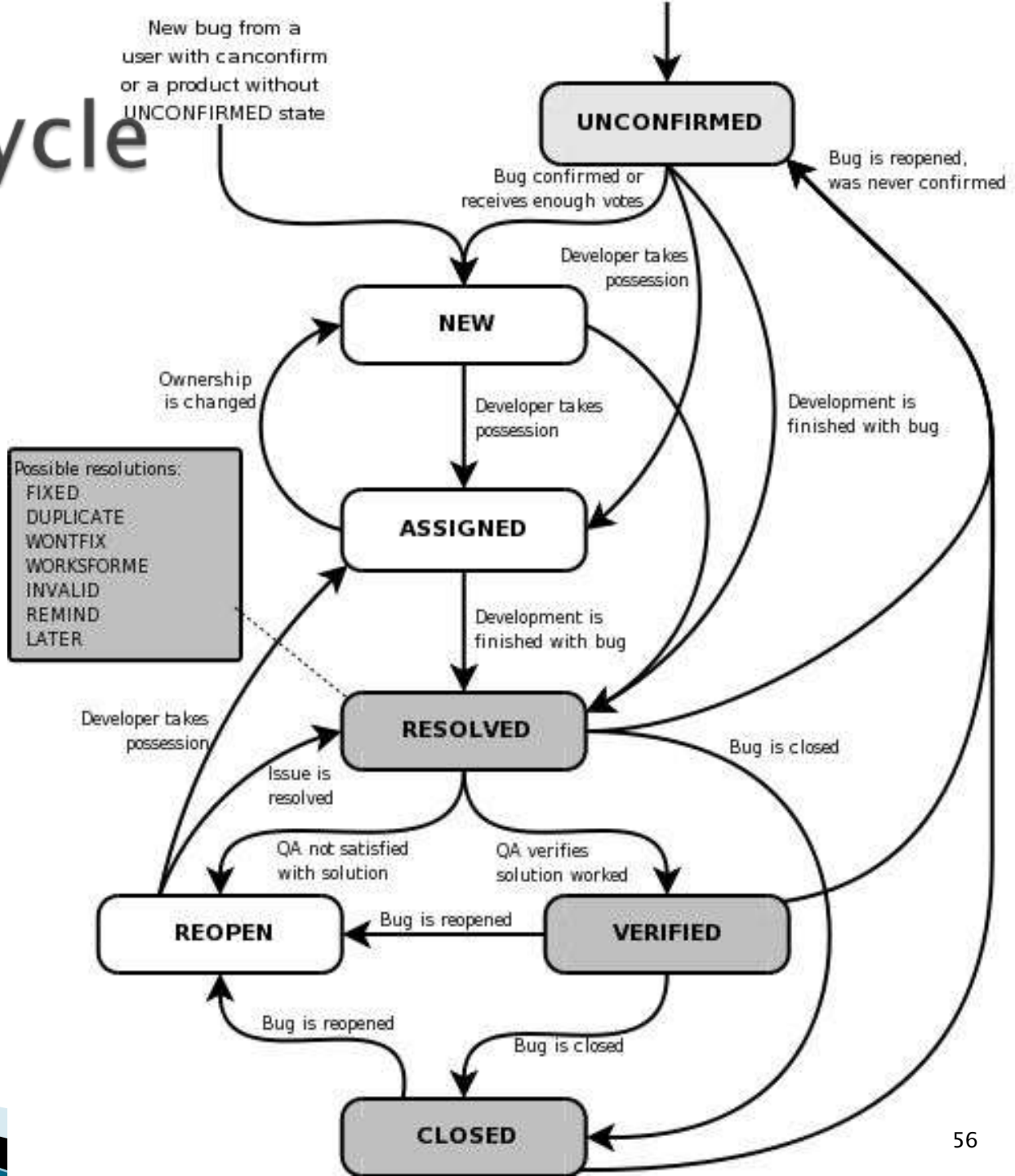
Software Bug

- **A software bug is an error, flaw, mistake, failure, or fault in a computer program**
- Most bugs arise from mistakes and errors made by people (in program or in its design), and a few are caused by compilers
- Reports detailing bugs in a program are commonly known as **bug reports**, **fault reports, problem reports, trouble reports, change requests**, and so forth.

Bugs Effects

- Bugs can have only a subtle effect on the program's functionality
- More serious bugs may cause the program to crash or freeze leading to a denial of service. Others qualify as security bugs
- Bugs in the code controlling the Therac-25 radiation therapy machine were directly responsible for some patient deaths in the 1980s
- In 2002, a study commissioned by the US DCNIST concluded that software bugs, or errors cost the US economy an estimated $59 billion annually (0.6% of gross domestic product)

Bug Life Cycle



Bug Prevention

- Programming style
- Programming techniques
- Development methodologies
- Programming language support
- Code analysis
- Instrumentation

Coding Style – Motivation

- Coding conventions are important because:
  - 80% of the work on software is maintenance
  - Usually, a product is not maintained by the persons who created it
  - Code conventions improve code readability and allow a software developer to quickly understand new programs

Coding Style – General Reuquirements

- Use comments: what do functions do, what do variables represent, explain steps of algorithms, etc.
- Use suggestive names for variables and functions
- Develop modular code
- Use dual pairs: set/get, start/stop, add/remove, save/load

Correctness

- Correctness of an algorithm is asserted when it is said that **the algorithm is correct with respect to a specification**
- **Functional correctness** refers to the input-output behavior of the algorithm (i.e., for each input it produces the correct output)
- A distinction is made between **total correctness**, which additionally requires that the algorithm terminates, and **partial correctness**, which simply requires that *if* an answer is returned it will be correct.

Non Functional Software Testing

- Verifies that the software functions properly even when it receives invalid or unexpected inputs
- Example: software fault injection (fuzzy form)
- Methods:
  - **Performance testing** or **Load Testing** checks to see if the software can handle large quantities of data or users (software scalability).
  - **Usability testing** checks if the user interface is easy to use and understand.
  - **Security testing** is essential for software which processes confidential data and to prevent system intrusion by hackers.
  - **Internationalization and localization** is needed to test these aspects of software, for which a pseudo localization method can be used.

Software Performance Testing

- **Types**
  - **load testing** – can be the expected concurrent number of users on the application (database is monitored)
  - **stress testing** – is used to break the application (2 x users, extreme load) (application's robustness)
  - **endurance testing** – if the application can sustain the continuous expected load (for memory leaks)
  - **spike testing** – spiking the number of users and understanding the behavior of the application whether it will go down or will it be able to handle dramatic changes in load

Performance Testing Analysis

Usability testing

- A technique used to evaluate a product by testing it on users
- Usability testing focuses on measuring a human-made product's capacity to meet its intended purpose.
- **Examples** of products that commonly benefit from usability testing are web sites or web applications, computer interfaces, documents, or devices
- **Goals**
  - *Performance* – How much time, steps?
  - *Accuracy* – How many mistakes/fatal did people make?
  - *Recall* – How much does the person remember afterwards or after periods of non-use?
  - *Emotional response* – How does the person feel about the tasks completed? Is the person confident, stressed? Would the user recommend this system to a friend?

Usability testing steps

Security testing

- The Process to determine that an Information System protects data and maintains functionality as intended.
- The six basic security concepts that need to be covered by security testing are:
  - **Confidentiality**,
  - **Integrity** – information which it receives has not been altered in transit or by other than the originator of the information
  - **Authentication** – validity of a transmission, message, or originator,
  - **Authorization** – determining that a requester is allowed to receive a service or perform an operation,
  - **Availability** – Assuring information and communications services will be ready for use when expected,
  - **Non-repudiation** – prevent the later denial that an action happened, or a communication that took place

Security logo

Internationalization and localization

- Means of adapting computer software to different languages and regional differences
- **Internationalization** is the process of designing a software application so that it can be adapted to various languages and regions without engineering changes.
- **Localization** is the process of adapting software for a specific region or language by adding locale-specific components and translating text.

Measuring software testing

- Usually, quality is constrained to such topics as **correctness, completeness, security**
- Can also include capability, reliability, efficiency, portability, maintainability, compatibility, and usability
- There are a number of common software measures, often called "metrics", which are used to measure the state of the software or the adequacy of the testing.

Testing artifacts

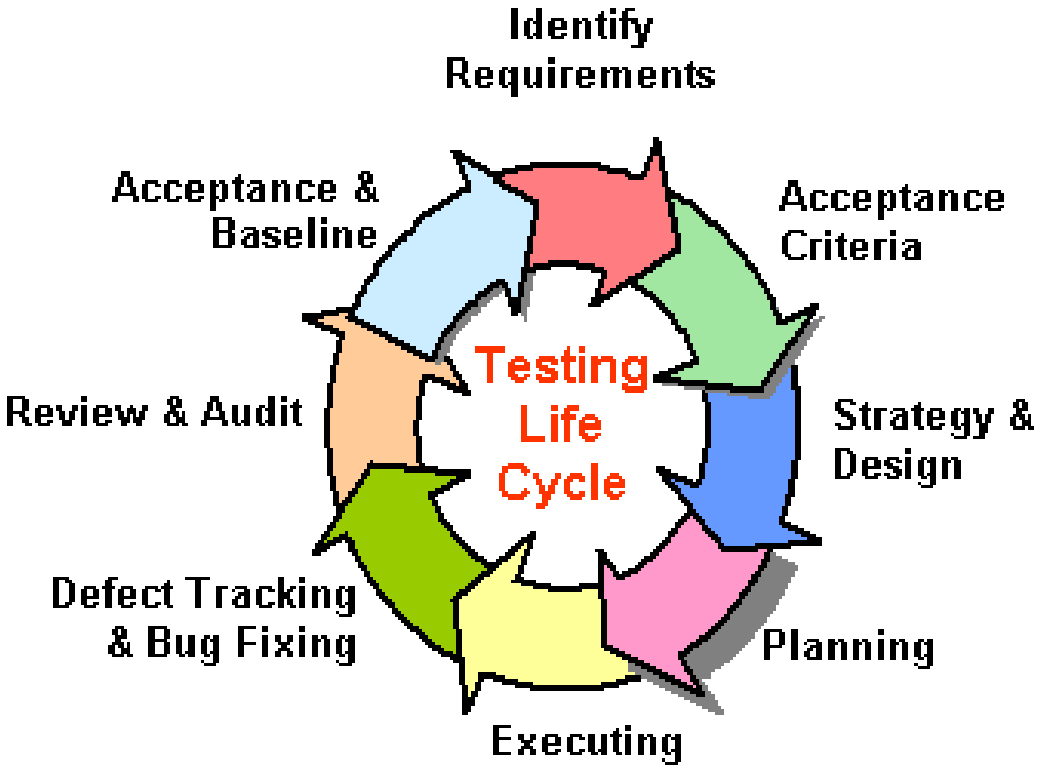
- **Test case** – consists of a unique identifier, requirement references from a design specification, preconditions, events, a series of steps (also known as actions) to follow, input, output, expected result, and actual result
- **Test script** – the combination of a test case, test procedure, and test data
- **Test data** – multiple sets of values or data are used to test the functionality of a particular feature

Testing artifacts (2)

- **Test suite** – a collection of test cases
- **Test plan** – A test specification
- **Test harness** – The software, tools, samples of data input and output, and configurations

Testing cycle

- There is a typical cycle for testing: Requirements Analysis, Test Planning, Test Development, Test Reporting, Test Result Analysis, Retesting the Resolved Defects, Regression Testing, Test Closure



Links

- Behavioral Patterns: http://www.oodesign.com/behavioral-patterns/
- Pattern Synopses2,3:
  http://www.mindspring.com/~mgrand/pattern_synopses2.htm
  http://www.mindspring.com/~mgrand/pattern_synopses3.htm
- Software Quality Assurance:
  http://satc.gsfc.nasa.gov/assure/agbsec3.txt
- Software Testing:
  http://en.wikipedia.org/wiki/Software_testing
- GUI Software Testing:
  http://en.wikipedia.org/wiki/GUI_software_testing
- Regression Testing:
  http://en.wikipedia.org/wiki/Regression_testing
- Junit Test Example:
  http://www.cs.unc.edu/~weiss/COMP401/s08-27-JUnitTestExample.doc

Links

- Software Bug: http://en.wikipedia.org/wiki/Software_bug
- http://en.wikipedia.org/wiki/Manual_testing
- http://en.wikipedia.org/wiki/Test_automation
- HP: BTO Software – Download Center https://h10078.www1.hp.com/cda/hpdc/display/main/search_results.jsp?zn=bto&cp=54_4012_100__

Coding Style – Links

- C++:
  - http://www.chris-lott.org/resources/cstyle/
  - http://geosoft.no/development/cppstyle.html
- Java:
  - http://java.sun.com/docs/codeconv/
  - http://geosoft.no/development/javastyle.html

Ingineria Programării

Cursul 12 – 23 mai

Cuprins

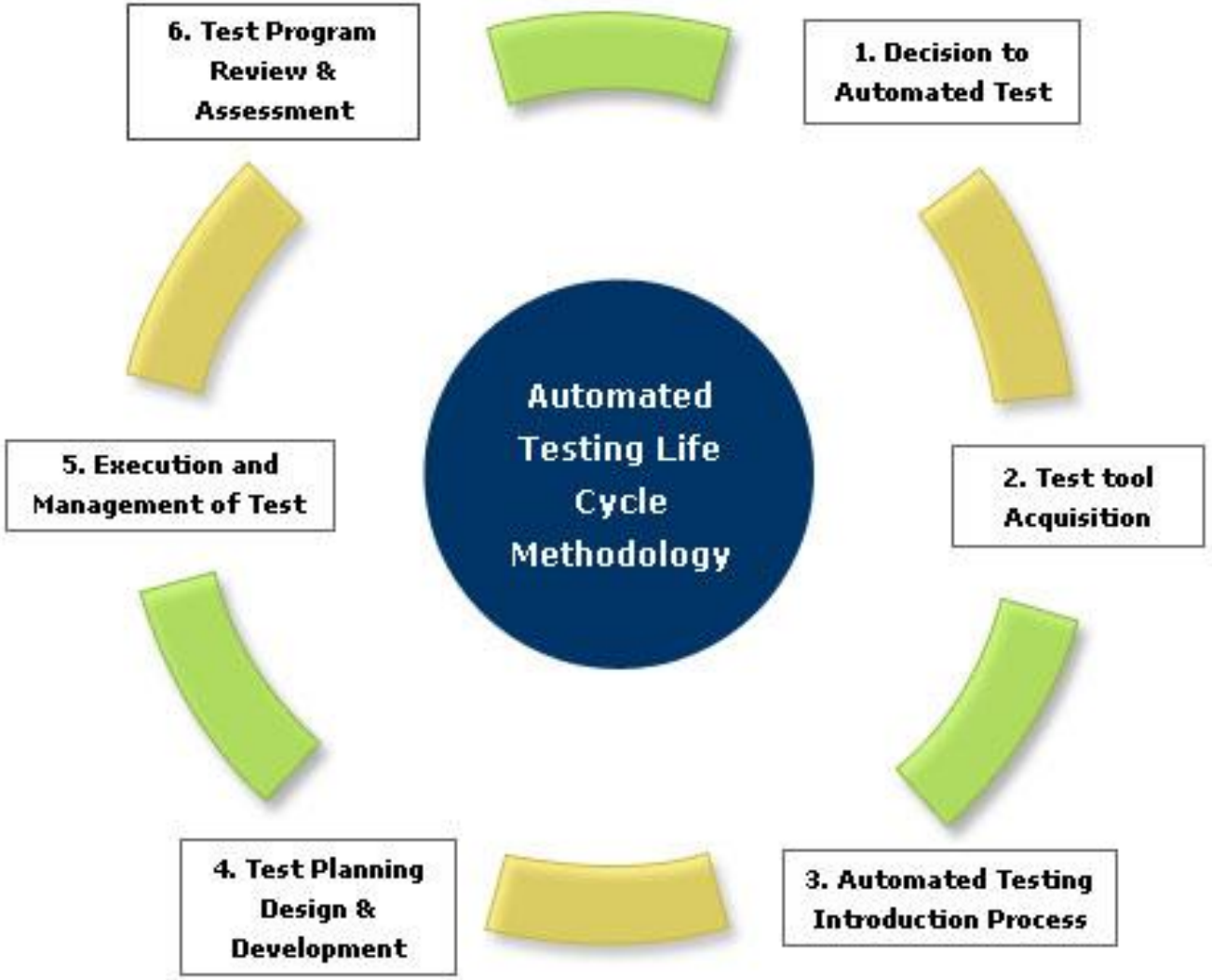
- Previous courses…
  - Testing
  - Test Automation
  - Software Bug
  - Testing cycle
- Program Quality
- Metrics
- Copyright

R: Manual Testing

- *How, Who, When, Where, Results*

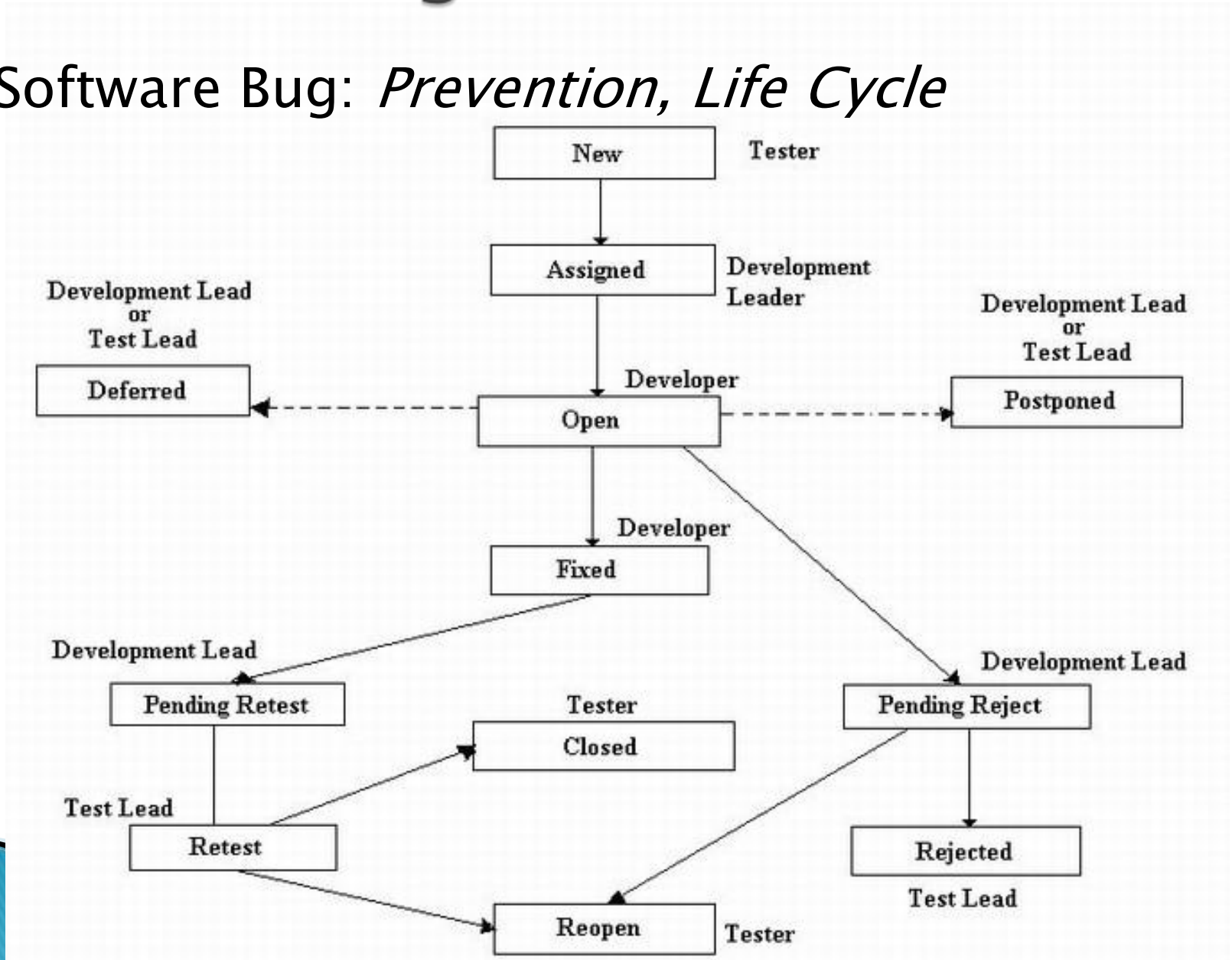
Test Automation

- Test Automation: *How, Who, When, Results*

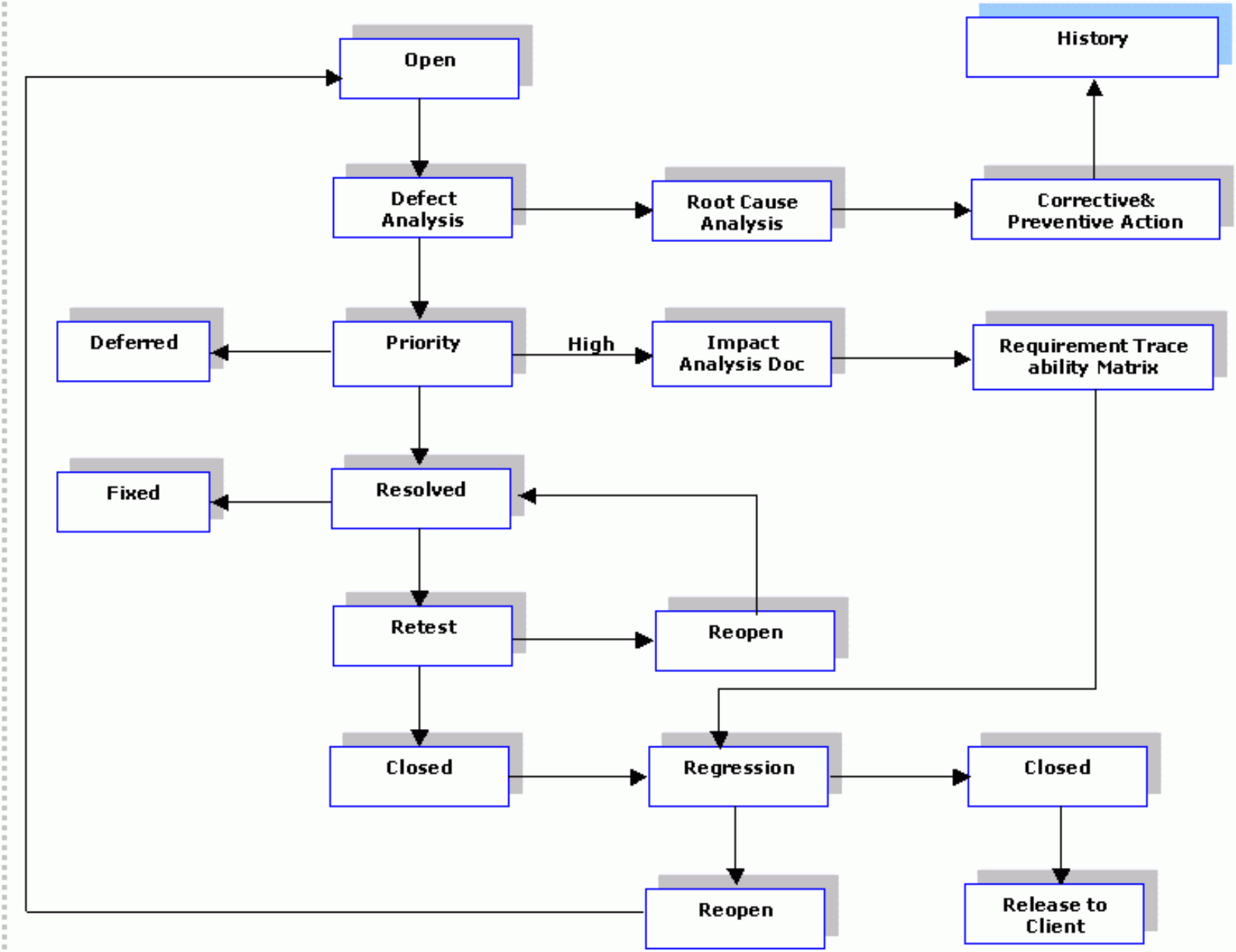


Software Bug

- Software Bug: *Prevention, Life Cycle*



Bug Life Cycle



R: Testing Cycle

- Testing Cycle: *Persons involved, Tasks*

Assessing quality

- How do we measure the quality of an item?
  - **Construction quality** (how well it is built, whether the raw material has flaws, etc ...)
  - **Design quality** (comfort, elegance...)
  - A combination between quality of desogn and construction (sturdiness...)

- In general, we can say that chair A is better than chair B regarding some particular aspect, but it is usually difficult to say by how much.

Calitatea unui program (1)

- Nu se examinează calitatea construcției (=> unicitate între toate disciplinele inginerești)
- Toate atributele calității se referă la design.
- Dacă ne referim la valori estetice:
  - Programele sunt în mare parte invizibile, iar valorile estetice contează doar pentru părțile vizibile
  - În afară de interfață, singurele aspecte observabile la un program sunt:
    - Notațiile folosite pentru design și scrierea codului
    - Comportamentul programului atunci când interacționează cu alte entități.

Calitatea unui program (2)

- Atunci când vorbim despre calitatea unui program trebuie:
  - **Să definim atribute ale calității** care ne interesează;
  - Să căutăm **modalități de măsurare** a lor;
  - Să putem da o **reprezentare a designului**;
  - **Să scriem specificații** care să ghideze programatorii (calitățile designului să fie respectate și de implementare).

Codul sursă

- Codul care implementează un design este o reprezentare a acelui design.
- Asigurarea calității făcută după scrierea codului este un proces costisitor și e posibil să fie inutil.
- De regulă doar se ține cont de modul în care e scris codul (coding style, design patterns, adaptability, maintenance, reuse (cuplaj, coeziune), security)

Calitatea unui program (3)

- Măsoară cât de bine se potriveste un program mediului său.
- Ia în calcul aspecte de tipul:
  - Programul funcționează;
  - Programul face ceea ce trebuie să facă;
  - Programul este sigur;
  - Programul poate fi adaptat pe măsură ce nevoile se schimbă.
- Toate măsurile referitoare la calitate sunt relative!!!

Concepte ale calității programelor

- Siguranță (safety)

- Eficiență (efficiency)

- Întreținere (maintenance)

- Uzabilitate (usability)

Siguranța

- Este programul **complet**, **consistent** și **robust**?
  - **completitudine** – tratează toate intrările posibile;
  - **consistență** – se comportă întotdeauna așa cum este așteptat;
  - **robustețe** – se comportă bine în situații anormale (ex. lipsa resurselor, lipsa conexiunii, etc.)

Eficiența

- Programul utilizează într-un mod eficient resursele? (procesor, memorie, rețea...)
- Eficiența este întotdeauna mai puțin importantă decât siguranța: *Este mai ușor să facem un program sigur să fie eficient decât un program eficient să fie sigur*

Întreținere

- Cât de ușor poate fi modificat design-ul ulterior?

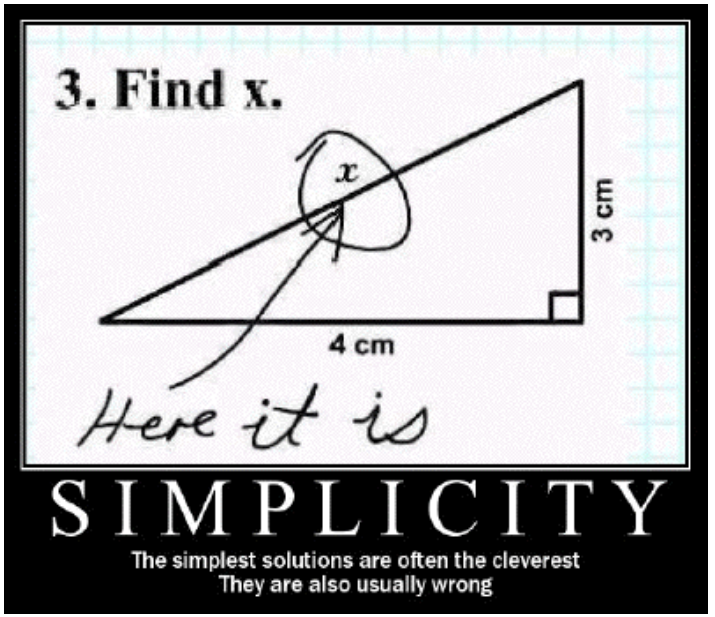
- Tipuri de întreținere:
  - **Corectivă**: eliminarea erorilor;
  - **Perfectivă**: adăugarea de funcționalități care ar fi trebuit să fie oferite;
  - **Adaptivă**: actualizarea programului când se schimbă cerințele.

Uzabilitate

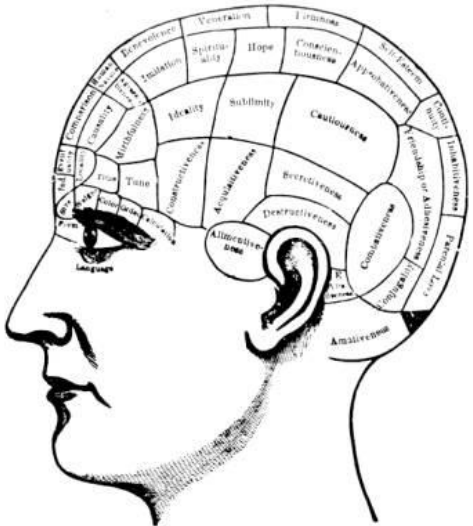
- Cât de usor este programul de învățat și de folosit?

Atribute măsurabile

- Simplitate



- Modularitate



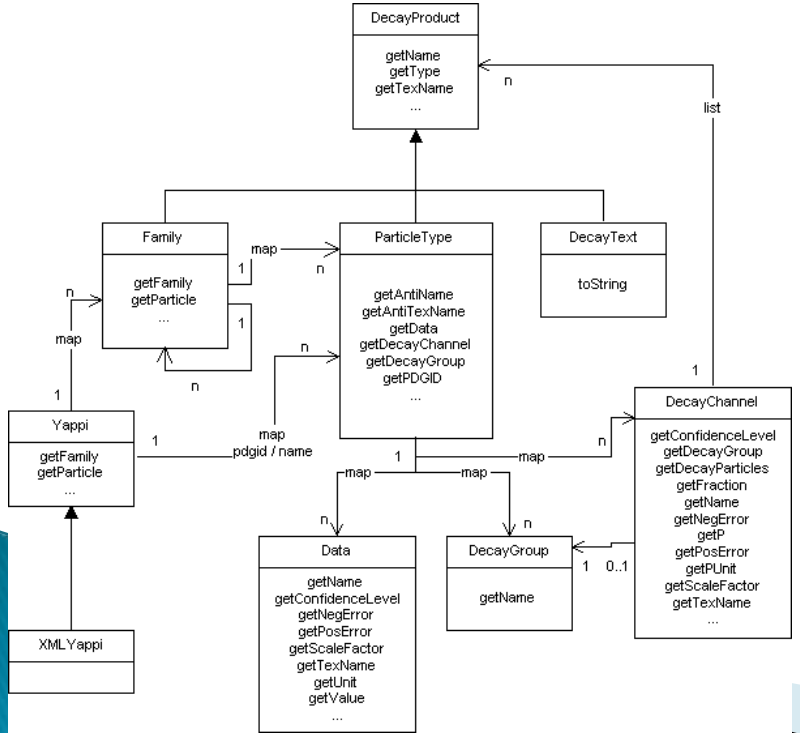
Simplitate

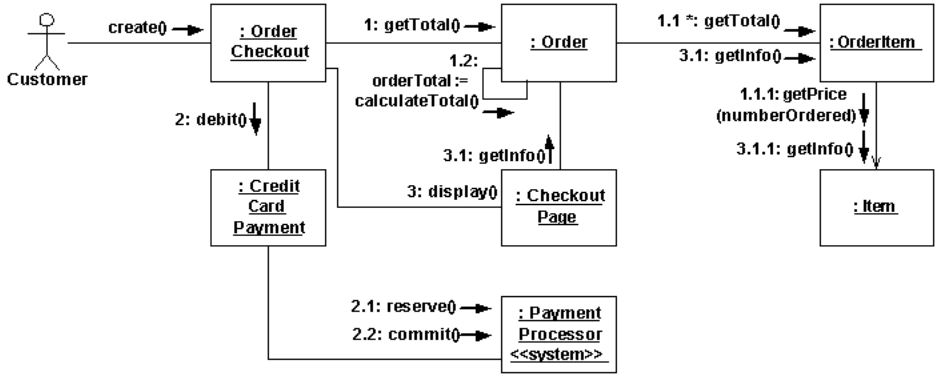
- Este măsura inversă a complexității.

- Aspecte ale complexității:
  - Complexitatea **fluxului de control**: numără căile posibile de execuție ale unui program
  - Complexitatea **fluxului de informații**: numără datele care sunt transmise în program
  - Complexitatea **înțelegerii**: numără identificatorii și operatorii folosiți

Modularitate

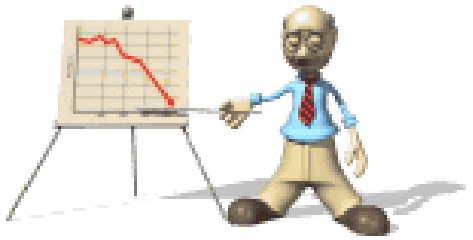
- Poate fi măsurată uitându-ne la:
  - **Coeziune**: cât de bine lucrează împreună componentele unui modul.
  - **Cuplaj**: gradul de interacțiune între module.





Motivații pentru metrici

- Efectuăm măsurători pentru
  - a înțelege
  - a controla
  - a prevedea



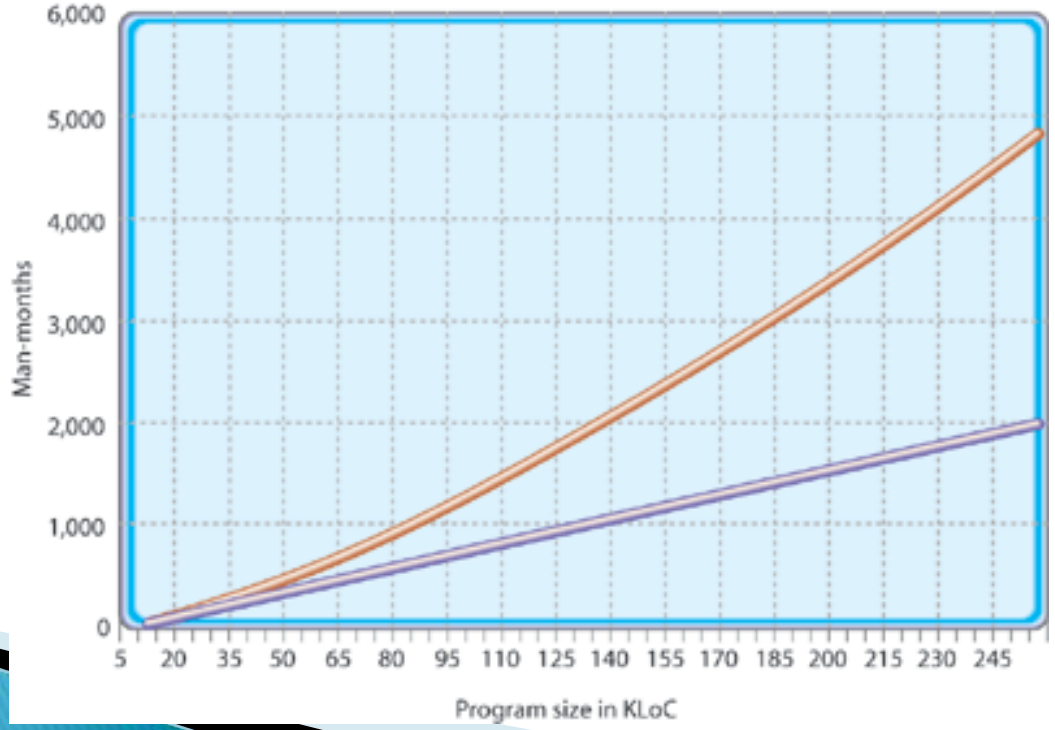
- *Când poți să măsori lucrul despre care vorbesti și să exprimi aceasta în numere, atunci știi ceva despre acel lucru. Dar când nu poți să îl măsori, când nu poți să îl exprimi în numere, cunostințele tale sunt slabe și nesatisfăcătoare; ele pot fi începutul cunoasterii, dar mai nimic nu este încă făcut pentru a ajunge la stadiul de știință. (Lord Kelvin)*

Ce se dorește a fi măsurat

- Dimensiunea unui program
- Complexitatea unui program
- Fiabilitatea unui program
- Timpul necesar dezvoltării unui program
- Alocarea resurselor necesare dezvoltării unui program
- Productivitatea muncii
- Costurile de dezvoltare

Metrici de bază

- KLOC: Kilo Lines Of Code (mii linii de cod)
- Effort, PM: Person – Month (Om – lună).



COCOMO

- COnstructive COst MOdel (Boehm 1981)
- Folosit pentru evaluarea costurilor
- Trei nivele de rafinare a predicțiilor:
  - COCOMO de bază
  - COCOMO intermediar
  - COCOMO detaliat

COCOMO de bază

- Formula pentru efortul necesar dezvoltării în funcție de numărul de linii de cod

- **Effort Applied** = a_b (KLOC)^{bb} [man-months]
- **Development Time** = c_b (Effort Applied)^{db} [months]
- **People required** = Effort Applied / Development Time [count]

- a_b , b_b, c_b, d_b – constante ce depind de tipul proiectului

COCOMO 2

- Propus de Boehm în 1995
- Ia în calcul tehnicile moderne de dezvoltare apărute
  - Prototipizare
  - Dezvoltarea pe componente
  - 4GL (fourth generation language)
- Oferă posibilitatea de a face estimări încă din primele faze ale dezvoltării

COCOMO 2: Prototipizarea inițială

- Surprinde efortul necesar realizării unui prototip al aplicației
- Se bazează pe numărul de puncte obiect (NOP)
- Formula de calcul a efortului:

$$PM = (1 - P_{reuse}) \frac{NOP}{PROD}$$

NOP

- Se inventariază ecranele ce trebuie afișate
  - Simple: 1
  - Complexe: 2
  - Foarte complexe: 3
- Se inventariază rapoartele ce trebuie generate
  - Simple: 2
  - Complexe: 5
  - Foarte complexe: 8
- Fiecare modul în limbaj de nivel mai jos (ex. 3GL): 10
- Suma punctelor obținute reprezintă numărul total de puncte obiect ale programului.

COCOMO 2: după proiectare

- Se estimează numărul total de linii de cod (ESLOC)
- Factori luați în calcul
  - Volatilitatea cerințelor
  - Gradul posibilității de reutilizare a codului

COCOMO 2: influența asupra costurilor

- Atributele produsului
  - Siguranța produsului; Complexitatea modulelor sistemului; Dimensiunea documentației cerute; Dimensiunile bazei de date utilizate; Procentajul cerut de componente reutilizabile

- Atributele sistemului de calcul
  - Constrângeri privind timpul de execuție; Volatilitatea platformei pe care se face dezvoltarea; Constrângeri de memorie

COCOMO 2: influența asupra costurilor (cont.)

- Atributele personalului
  - Capacitatea analiștilor; Capacitatea programatorilor; Continuitatea personalului; Experiența analistului în domeniul problemei; Experiența programatorilor în domeniul problemei; Experiența în limbajele și instrumentele folosite

- Atributele proiectului
  - Utilizarea intrumentelor; Gradul de lucru în echipe aflate la distanță și calitatea comunicării între echipe; Comprimarea planului de dezvoltare

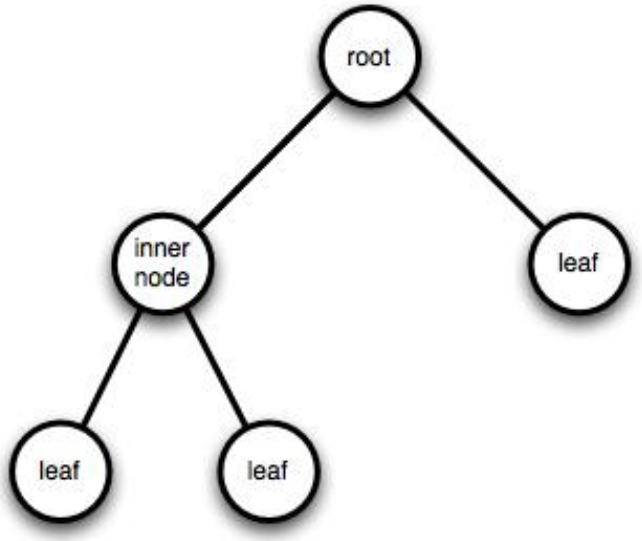
Distribuția forței de muncă în timp

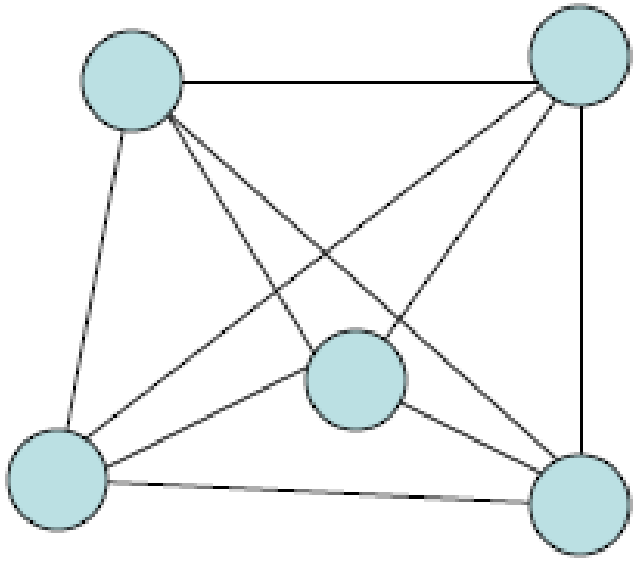
- 20 om-lună. E corect calculul de mai jos???
  - 20 oameni muncesc 1 luna
  - 4 oameni muncesc 5 luni
  - 1 om muncește 20 luni
- Productivitatea individuală scade atunci când echipa de dezvoltare crește
  - Comunicare suplimentară
  - La adăugarea de noi membri, productivitatea inițială scade
- *Creșterea numerică a forței de muncă la un proiect rămas în urmă are ca efect rămânerea și mai în urmă a proiectului.* (Legea lui Brooks)

Distribuția forței de muncă în timp (2)

- Pentru o echipă cu P persoane putem avea între P-1 și P(P-1)/2 canale de comunicație

- Fiecare canal induce o pierdere de eficiență





Cum *NU se calculează costurile* și *NU se fac planificări*

- Avem 12 luni să terminăm treaba, deci treaba va dura 12 luni. Legea lui Parkinson: *munca se întinde pe timpul avut la dispoziție*.
- Știm că competitorul a cerut $1.000.000. Noi cerem $900.000.
- Știm că bugetul clientului pentru produs este de $500.000. Atât ne costă și pe noi dezvoltarea.
- Programul va dura 1 an, dar voi spune că va dura 10 luni. Ce-o să mai conteze 2 luni peste planificare...

Probleme la folosirea metricilor

- Lipsa de acuratețe
- Rezistența din partea angajaților
- Folosirea lor în alte scopuri decât au fost create
- Disensiuni în cadrul echipei de dezvoltare

Copyright

- Drepturile de autor reprezintă ansamblul prerogativelor de care se bucură autorii cu referire la operele create; instituția dreptului de autor este instrumentul de protecție a creatorilor și operelor lor
- *Copyright gives the creator of an original work exclusive right for a certain time period in relation to that work, including its* ***publication, distribution and adaptation****; after which time the work is said to enter the public domain.*

Exclusive rights

- Several exclusive rights typically attach to the holder of a copyright:
  - to produce copies or reproductions of the work and to sell those copies (mechanical rights; including, sometimes, electronic copies: distribution rights)
  - to create derivative works (works that adapt the original work)
  - to perform or display the work publicly (performance rights)
  - to sell or assign these rights to others
  - to transmit or display by radio or video (broadcasting rights)

Evaluare proiecte

Șeful de grupa prezintă ce a lucrat fiecare membru al grupei (clase și module implementate, resurse, documentație, etc.). Evaluare proiect integrat:

- L9 + L10 – 40 puncte – implementare
- L11 – integrare
  - 10 puncte integrare cu modulul anterior si daca functioneaza
  - 20 de puncte daca functioneaza proiectul la nivel de grupa
- L12 – testare (10 puncte de metoda de test de om)
- L13 – evaluare
- **0 puncte fără integrare**

Cei care nu implementează nimic sau încurcă celelalte subgrupe în realizarea proiectului, primesc punctaj negativ…

Examen scris

- Tematici: IP, logică, cunoștințe generale, etc.
- ~15 subiecte, ~30 de minute
- Răspunsurile scurte la obiect, cuvinte cheie

- Atenție! Vor fi subiecte a căror nerezolvare va duce la acumularea unui punctaj negativ! Aceste subiecte vor fi marcate pe teză cu punctajul sub forma [-10, 10] (lucrurile de bază…)
- Data: soon™

Examen scris

Links

- **Bug Life Cycle:** http://www.buzzle.com/editorials/4-6-2005-68177.asp, http://qastation.wordpress.com/2008/06/13/process-for-bug-life-cycle/
- **COCOMO:** http://en.wikipedia.org/wiki/COCOMO
- Curs 12, Ovidiu si Adriana Gheorghies: http://www.info.uaic.ro/~ogh/files/ip/curs-12.pdf